高级语言程序设计进阶

（国豪工科精英班）

开学篇

1.1 教师简介



- 陈宇飞　副教授　博导
- 同济大学电子与信息工程学院
- 研究方向：机器学习，深度学习，医学影像分析
- 联系方式:

https://see.tongji.edu.cn/info/1156/6423.htm

yufeichen@tongji.edu.cn

1.1 教师简介



- 龚晓亮　高级工程师　硕导
- 同济大学电子与信息工程学院
- 研究方向：情感计算、脑认知与智能计算
- 联系方式：

https://eelab.tongji.edu.cn/78/06/c16462a161798/page.htm

gxllshsh@tongji.edu.cn

1.2 背景简介

- C/C++的历史和地位
  - 1965-2019年最流行的编程语言变化趋势（视频）

1.2 背景简介

- C/C++的历史和地位
  - 1965-2019年最流行的编程语言变化趋势（视频）
  - TIOBE评选C++成为2022年度最佳编程语言

Jan 2023	Jan 2022	Change	Programming Language	Ratings	Change
1	1		Python	16.36%	+2.78%
2	2		C	16.26%	+3.82%
3	**4**	↑	**C++**	**12.91%**	**+4.62%**
4	3	↓	Java	12.21%	+1.55%
5	5		C#	5.73%	+0.05%
6	6		Visual Basic	4.64%	-0.10%
7	7		JavaScript	2.87%	+0.78%
8	9	↑	SQL	2.50%	+0.70%
9	8	↓	Assembly language	1.60%	-0.25%
10	11	↑	PHP	1.39%	-0.00%
11	10	↓	Swift	1.20%	-0.21%
12	13	↑	Go	1.14%	+0.10%
13	12	↓	R	1.04%	-0.21%
14	15	↑	Classic Visual Basic	0.98%	+0.01%
15	16	↑	MATLAB	0.91%	-0.05%
16	18	↑	Ruby	0.80%	-0.08%
17	14	↓	Delphi/Object Pascal	0.73%	-0.27%
18	26	⇈	Rust	0.61%	+0.11%
19	20	↑	Perl	0.59%	-0.12%
20	23	↑	Scratch	0.58%	-0.01%

2023 年 1 月编程语言排行榜中，TIOBE评选 C++ 成为 2022 年度最佳编程语言

1.2 背景简介

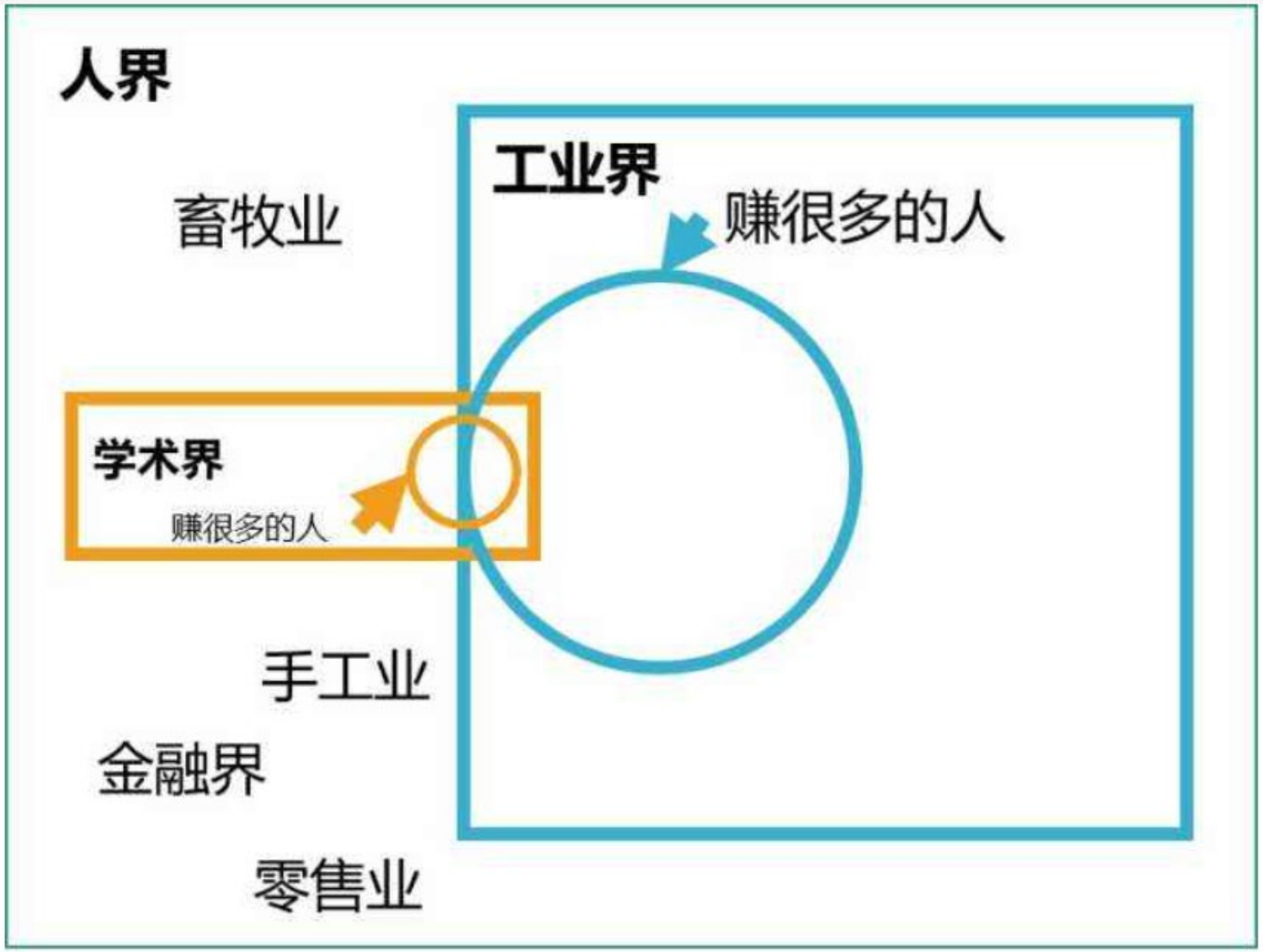
- C/C++的历史和地位
  - 1965-2019年最流行的编程语言变化趋势（视频）
  - TIOBE评选C++成为2022年度最佳编程语言
  - 学生眼中的“编程”

1.2 背景简介

- 程序员职业前景
  - 工业界：企业、政府部门的科研人员或工程师
  - 学术界：科研院所、企业科研部门的科研人员

1.2 背景简介

- 工业界VS学术界



还有三界以外的妖精 …

知乎 @黄远辉

1.2 背景简介

- 工业界VS学术界



//转自网络

1.2 背景简介

- 学术研究

  学术界的攻坚更像恋爱中的男女

  每一点进步都让你们开心无比

  同时还希望不停地有进步，达到新的高度

  看到的全是女孩/男孩好的一面

  你们可以自由地憧憬

  暂没人催你生孩子（产品）

  你们憧憬生一个小孩（产品）会多么美好

  认为孩子一定是世界上最聪明最乖巧的

  因为反正不用真的把孩子生出来

- 工业研发

  工业界研发更像结婚后的男女

  发现生娃（产品）成了你们最首要的任务

  父母（公司老板）天天催着你生娃（产品）

  你们以为孩子生出来会很乖巧

  生出来后才发现一堆的问题一堆的毛病

  社会（用户）也不喜欢他/她

  你不停根据经验和用户反馈来进行调教

  最后孩子强大了，你也头发白了脊椎坏了

  但看着孩子（产品）还是一脸的满足幸福

//颜水成教授报告

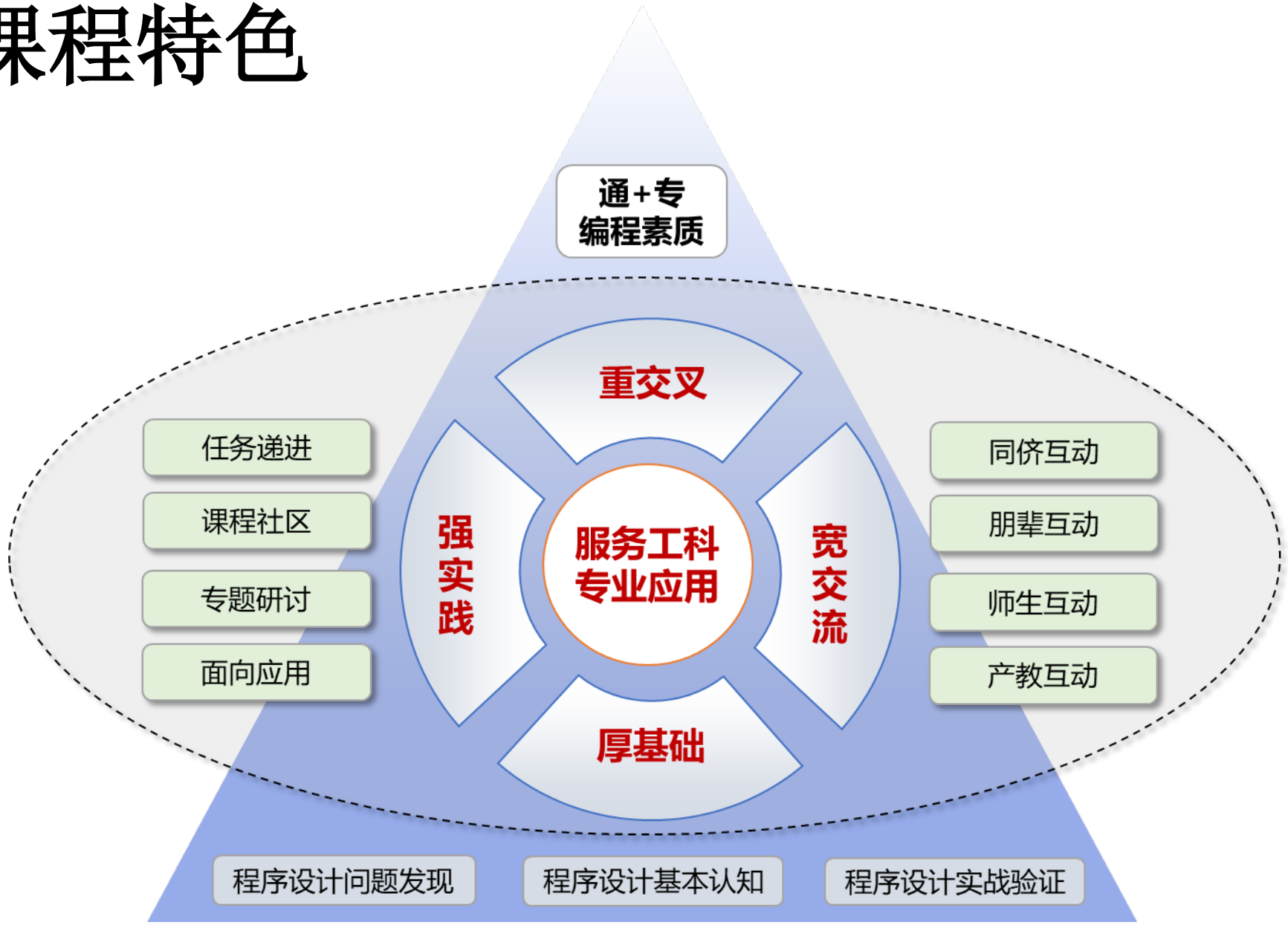
1.3 培养目标

- 区别于通识类程序设计课程：
  - 专：培养学生专业的编程思维能力，使其能够完成专业系统的开发任务；
- 区别于信息类程序设计课程：
  - 通：培养学生综合的编程能力，使其能够完成专题背景的综合开发任务。

1.3 培养目标

- 通过学习(C/C++) 语言的基本概念和方法，培养学生快速学习高级程序设计语言的能力。
- 面向相关专业学生，培养学生更深入的掌握计算机编程知识。
- 侧重于计算机内部原理和工作机制的理解，为计算机类后续课程的学习打基础。

1.4 课程特色


通+专
编程素质
重交叉
强实践
宽交流
厚基础
服务工科
专业应用
任务递进
课程社区
专题研讨
面向应用
同侪互动
朋辈互动
师生互动
产教互动
程序设计问题发现
程序设计基本认知
程序设计实战验证

1.4 课程特色

① 课程教学：

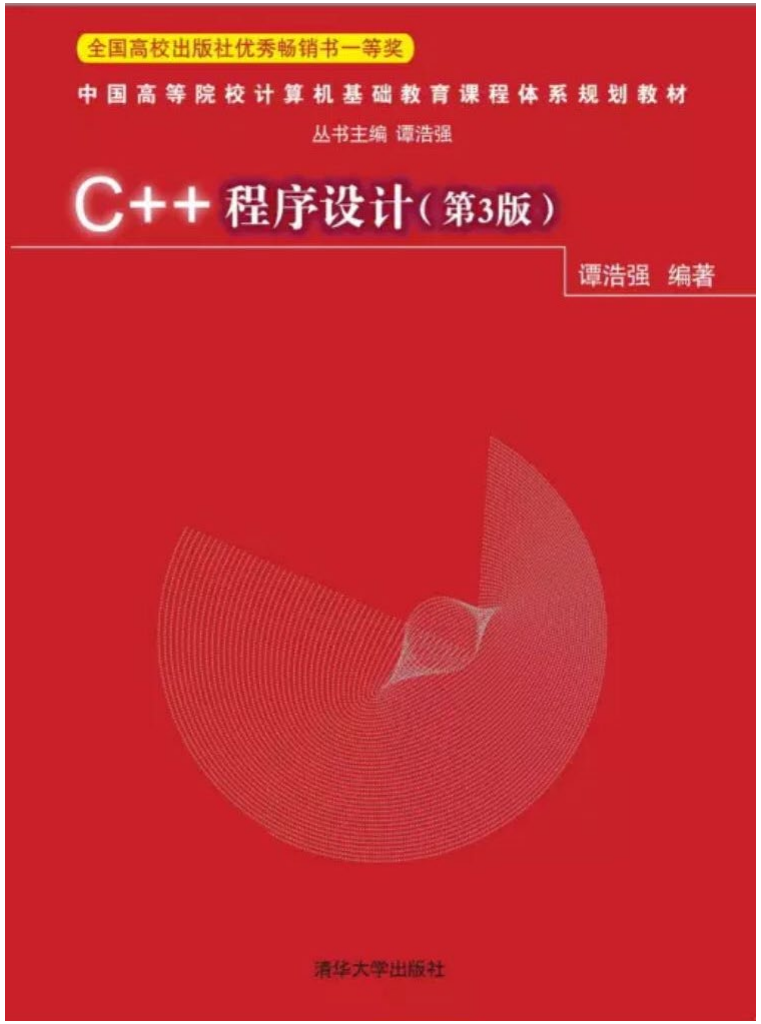
- 高程（上）的内容进阶
- 课程内容教研结合
- 含有AI研究课题相关基础案例和实训
- OJ实时评测

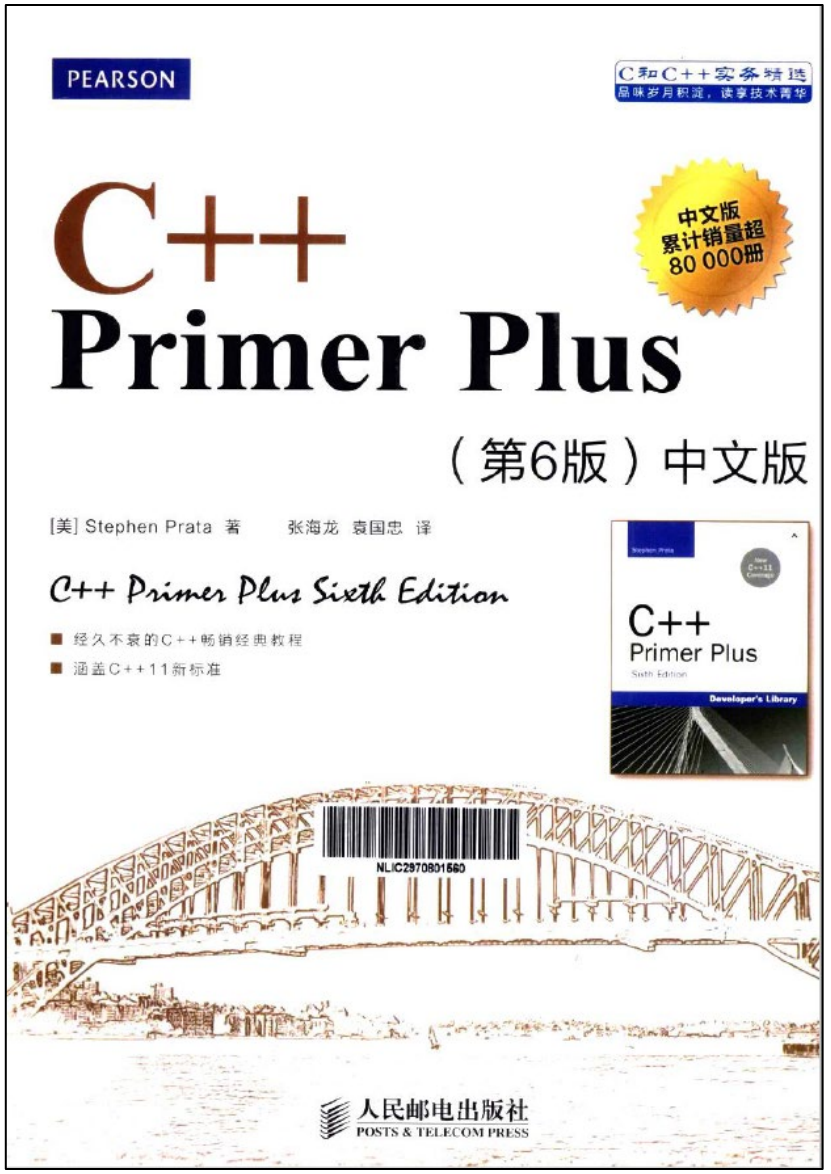
② 实践教学：

- 主题研讨
- 朋辈互动
- 作业讨论
- 学术分享

1.5 课程内容

- 教材说明


全国高校出版社优秀畅销书一等奖
中国高等院校计算机基础教育课程体系规划教材
丛书主编 谭浩强
C++程序设计(第3版)
谭浩强 编著
清华大学出版社


PEARSON
C++
Primer Plus
(第6版)中文版
中文版
累计销量超
80 000册
[美] Stephen Prata 著
张海龙 袁国忠 译
C++ Primer Plus Sixth Edition
经久不衰的C++畅销经典教程
涵盖C++11新标准
人民邮电出版社
POSTS & TELECOM PRESS

1.5 课程内容

	备注课程安排	
1	开学篇	（1）开学篇：课程目标、内容、考核要求、答疑方式等； （2）智能编程与OOP设计 （3）补充概念：共用体、位运算、带参数的main函数； （4）C方式的文件操作
2	C++方式的文件操作	（1）标准输入流 （2）文件操作与文件流 （3）字符串流
3	动态内存申请与释放	（1）动态内存的基本概念 （2）C中的相关函数 （3）C++中的相关运算符 （4）内存的动态申请与释放 （5）定位new运算符
4	链表的实现与应用	（1）链式结构的基本概念 （2）单链表的基本操作 （3）双向链表的基本操作 （4）二叉搜索树
5	包含动态内存申请的类	（1）含动态内存申请的构造函数和析构函数 （2）构造函数与析构函数的调用顺序 （3）对象的动态简历和释放 （4）对象的赋值与复制
6	类与对象的基本使用	（1）对象的初始化方法 （2）构造函数的基本概念与使用 （3）析构函数的基本概念与使用 （4）复制构造函数的调用时机
7	数据共享与参数化类	（1）共用数据的保护 （2）静态成员 （3）类模板
8	继承与派生的基本概念	（1）继承与派生的基本概念 （2）派生类的声明方式及派生类对象的构成 （3）派生类的成员访问属性 （4）派生类的构造与析构函数

1.5 课程内容

9	多重继承的概念与实现	（1）多重继承 （2）虚基类 （3）赋值兼容规则 （4）继承与动态内存分配
10	多态性与虚函数	（1）多态的基本概念 （2）虚函数的定义与使用 （3）虚析构函数 （4）纯虚函数与抽象类
11	运算符重载的基本概念	（1）运算符重载的方法 （2）运算符重载的规则 （3）成员函数与友元函数
12	运算符重载的实例应用	（1）单/双目运算符的重载 （2）流运算符的重载 （3）不同类型间数据的转换
13	模板类	（1）智能指针模板类 （2）标准模板库
14	泛型编程	（1）STL基本概念 （2）使用迭代器 （3）迭代器类型 （4）迭代器结构 （5）容器种类
15	跨平台编译	（1）Cmake编译原理 （2）Makefile的使用
16	答疑	

1.5 课程内容

- 关于作业
  - 全部在线提交，提供OJ实时评测:http://202.120.167.244/
  - 编译环境：VS 2022; VS+Copilot VSCode+Copilot VSCode+DeepSeek
  - 认真阅读帮助中心的使用手册
  - 评测得分以最后一次为准
  - 章节作业发布时间：上课当天晚八点发布（一周时间完成）
  - 大作业发布时间：当周周一中午12点（具体时间提前一周课上通知）
  - 无客观理由，不接受截止后的作业补交
  - 禁止抄袭，全部题目全部代码均查重！！！
  - 认真读题，读要求！！！

4_3/[redacted]cpp (74%)		4_3[redacted].cpp (74%)	
7-27		16-36	
32-39		6-13	



4_3/[redacted].cpp

```
#include<iostream>
#include<cmath>
using namespace std;
double getDouble();
double solve(double a, double b, double c,double d);
```

```
double getDouble()
{
	double x;

	cin >> x;
	if (!cin.good() || x < -1e9 || x>1e9)
	{
		cout << "Input is illegal, program exit.";
		exit(0);
	}

	else return x;
}
double solve(double a, double b, double c,double d)
{
	double x, x0, f1, f2;
	for (x = 1, x0 = 0; fabs(x - x0) > 1e-7;)
	{
		x0 = x;
		f1 = ((a * x0 + b) * x0 + c) * x0 + d;
		f2 = (3 * a * x0 + 2 * b) * x0 + c;
		x = x0 - f1 / f2;
	}
	return x;
}
```

```
int main()
{
	double a, b, c,d;
	a = getDouble();
	b = getDouble();
	c = getDouble();
	d = getDouble();
	cout<<solve(a, b, c,d);
	return 0;
}
```

4_3/[redacted].cpp

```
#include<iostream>
#include<cmath>
using namespace std;
double getDouble();
double solve(double a, double b, double c,double d);
```

```
int main()
{
	double a, b, c,d;
	a = getDouble();
	b = getDouble();
	c = getDouble();
	d = getDouble();
	cout<<solve(a, b, c,d);
	return 0;
}
```

```
double getDouble()
{
	double x;

	cin >> x;
	if (!cin.good() || x < -1e9 || x>1e9)
	{
		cout << "Input is illegal, program exit.";
		exit(0);
	}

	else return x;
}
double solve(double a, double b, double c,double d)
{
	double x, x0, f1, f2;
	for (x = 1, x0 = 0; fabs(x - x0) > 1e-7;)
	{
		x0 = x;
		f1 = ((a * x0 + b) * x0 + c) * x0 + d;
		f2 = (3 * a * x0 + 2 * b) * x0 + c;
		x = x0 - f1 / f2;
	}
	return x;
}
```

1.6 考核方法

- 平时成绩 – 考勤 + 章节作业 + 平时大作业 + 随堂测

  期末成绩 – 期末考试（大作业+汇报展示）

  - 平时成绩、期末成绩分开计算，满分均为100

  总评成绩 = 平时成绩 x 70% + 期末成绩 x 30%

  => 百分制转五分制

  - 若遇特殊情况，以新通知为准

1.7 课程通知

- QQ群：
  - 不鼓励水群
  - 不鼓励开临时对话实时提问
- 知识点疑问课后答疑（45分钟/周）
- 正式的课程联系方式
  - OJ：作业网站、交流讨论
  - Canvas：课程资料、重要通知

1.8 课程建议

- 章节作业：
  - 理论课知识点
  - 题面描述直观
- 大作业：
  - 难点：题面分析 + 设计
  - 需要自学（课外延展）+ 研讨（鼓励交流）

1.8 课程建议

- 完全非码类：
  - 完成大作业基本要求，掌握计算思维
- 不完全非码类：
  - 完成大作业进阶要求，具备专题背景综合开发的能力
- 码类：
  - 课后加强实践（大作业多选、科研项目、程序类竞赛）

1.9 课前问卷

- 做个问卷调查吧~~~

第一章 C语言的补充概念

模块1.1：共用体

1.1 共用体

例：定义一个用于一卡通管理系统的结构，要求包含卡号、余额、消费限额、消费密码等公共信息，此外，若持卡人是学生，要包含学号、姓名、专业等学生特有的信息，若持卡人是教师，则包含工号、姓名、职称等教师特有的信息

```
struct student{
    定义学生信息;
};
struct teacher {
    定义教师信息;
};
struct ykt {
    公共信息;
    student sinfo;
    teacher tinto;
};
int main()
{   ykt y1;//定义变量
}
```

```
对y1的成员的访问：
int main()
{
    ykt y1;
    ...;
    y1.卡号
    y1.sinfo.学号
    y1.tinfo.工号
    ...;
    return 0;
}
```

能否使sinfo/tinfo共用一段空间：

1)当持卡人是学生时，这段空间按student方式访问

2)当持卡人是教师时， 按teacher方式访问

=>(共用体)

1.1 共用体

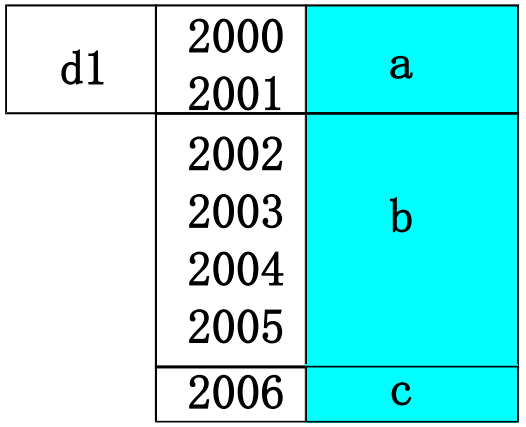
```
union 共用体名 {
    共用体成员1（类型名 成员名）
    ...
    共用体成员n（类型名 成员名）
};
```

```
union data {
    short a;
    long b;
    char c;
};
```

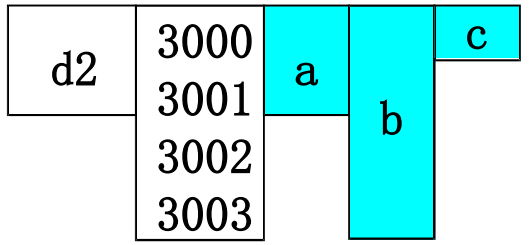
- 所有成员从同一内存开始，共用体的大小为其中占用空间最大的成员的大小
- 给一个共用体成员赋值后，会覆盖其它成员的值，因此只有最后一次存放的成员是有效的
- 其它所有定义、使用方法同结构体

```
#include <iostream>
using namespace std;
struct data1 {
    short a;
    long b;    //12:所有成员所占空间之和(含填充字节)
    char  c;
};
union data2 {
   short a;
   long b;     //4 :所有成员中最大成员所占空间
   char  c;
};
void main()
{    cout << sizeof(data1) << ' ' << sizeof(data2) << endl;  //12 4
}
```

struct data1 d1;

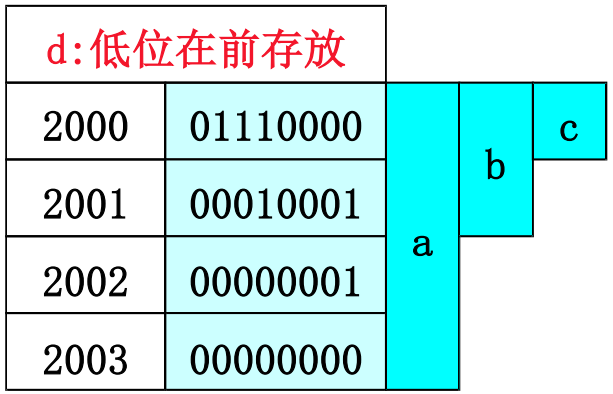


union data2 d2;



```
#include <iostream>
using namespace std;
union data {
    int a;
    short b;
    char  c;
};
int main()
{
     union data d;
     d.a=70000;
     cout << d.a << ' ' << d.b << ' ' << d.c << endl;
     d.b=7000;
     cout << d.a << ' ' << d.b << ' ' << d.c << endl;
     d.c='A';
     cout << d.a << ' ' << d.b << ' ' << d.c << endl;
     return 0;
}
```

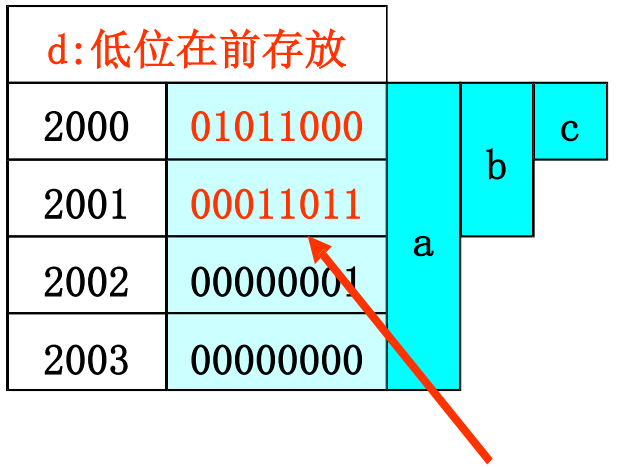
70000=00000000 00000001 00010001 01110000




```
70000 4464 p
72536 7000 X
72513 6977 A
```

```
#include <iostream>
using namespace std;
union data {
    int a;
    short b;
    char  c;
};
int main()
{
    union data d;
    d.a=70000;
    cout << d.a << ' ' << d.b << ' ' << d.c << endl;
    d.b=7000;
    cout << d.a << ' ' << d.b << ' ' << d.c << endl;
    d.c='A';
    cout << d.a << ' ' << d.b << ' ' << d.c << endl;
    return 0;
}
```

72536=00000000 00000001 00011011 01011000



7000=00011011 01011000

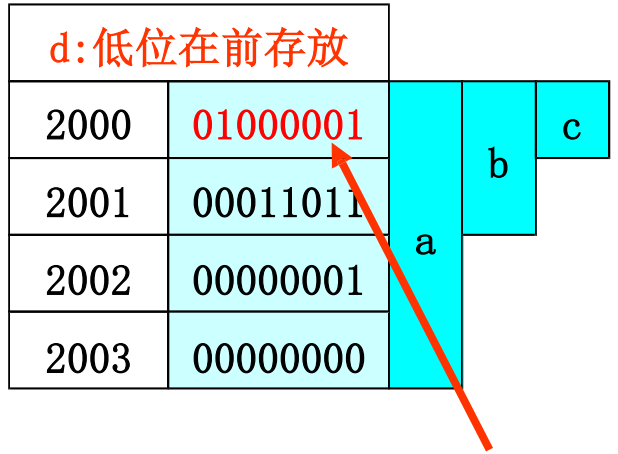
70000 4464 p

72536 7000 X

72513 6977 A

```
#include <iostream>
using namespace std;
union data {
    int a;
    short b;
    char  c;
};
int main()
{
     union data d;
     d.a=70000;
     cout << d.a << ' ' << d.b << ' ' << d.c << endl;
     d.b=7000;
     cout << d.a << ' ' << d.b << ' ' << d.c << endl;
     d.c='A';
     cout << d.a << ' ' << d.b << ' ' << d.c << endl;
     return 0;
}
```

72513=00000000 00000001 00011011 01000001



A=0100 0001

70000 4464 p

72536 7000 X

72513 6977 A

• 所有成员从同一内存开始，共用体的大小为其中占用空间最大的成员的大小

```
#include <iostream>
using namespace std;
union data {
 int a;
 short b;
 char c;
};
int main()
{  union data d;
   d.c='A';
   cout << d.a << ' ' << d.b << ' ' << d.c << endl;  //不确定 不确定 A
   d.b=7000;
   cout << d.a << ' ' << d.b << ' ' << d.c << endl;  //不确定 7000 X
   d.a=70000;
   cout << d.a << ' ' << d.b << ' ' << d.c << endl;  //70000 4464 p
   return 0;
}
```

d.c='A'	
2000	01000001
2001	???
2002	???
2003	???

d.b=7000	
2000	01011000
2001	00011011
2002	???
2003	???

d.a=70000	
2000	01110000
2001	00010001
2002	00000001
2003	00000000

1.1 共用体

```
struct student {
  定义学生信息
};
struct teacher {
  定义教师信息;
};
struct ykt {
  公共信息;
  student sinfo;
  teacher tinto;
};
int main()
{  ykt y1;//定义变量
}
```

空间浪费



```
struct student {
  定义学生信息;
};
struct teacher {
  定义教师信息;
};
union owner {
  student s;
  teacher t;
};
struct ykt {
  公共信息;
  char type; //持卡人类别
  owner info;
};
```

此处保证s/t共用一段空间

```
int main()
{
  ykt y1;//定义变量
  ...;
  y1.卡号;
  if (y1.type=='s'){
     y1.info.s.学号;
   }
  else{
      y1.info.t.工号;
   }
  ...;
  return 0;
}
```

第一章 C语言的补充概念

模块1.2 ： 位运算

1.1 概述

字节和位

字节：byte，计算机中数据表示的基本单位

位 ：bit， 计算机中数据表示的最小单位

1 byte = 8 bits

- 位运算

  以bit为单位进行数据的运算

1.1 概述

- 位运算的基本方法
  - 按位进行（只有0、1）
  - 要求运算数据长度相等，若不等，则右对齐，按最高位补齐左边

char a=0x37;	0000 0000 0011 0111
short b=0x1234;	0001 0010 0011 0100
char a=0xA7;	1111 1111 1010 0111
short b=0x8341;	1000 0011 0100 0001

  - 数在计算机内是用补码表示的

1.2 常用的位运算

运算符	功能	用法
~	位求反	~expr
<<	左移	expr1 << expr2
>>	右移	expr1 >> expr2
&	位与	expr & expr
^	位异或	expr ^ expr
|	位或	expr | expr

1.2 常用的位运算

- 移位运算符
  - 左移运算规则：左移数据，右补0
  - 右移运算规则：右移数据，左补0（逻辑右移）

    右移数据，左补符号位（算术右移）- C/C++的位运算

例： unsigned char bits=0233;

1	0	0	1	1	0	1	1

bits << 8; //bits提升成int类型，然后向左移动8位

0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	1	1	0	1	1	0	0	0	0	0	0	0	0

bits << 31; //向左移动31位，左边超出边界的位丢弃

1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

bits >> 3; //向右移动3位，最右边的3位丢弃

0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	1	1

```
#include <iostream>
#include <iomanip>
using namespace std;
int main()
{
        unsigned char bits = 0233;
        cout << "bits<<8=0x" << hex << (bits << 8) << " "
                             << dec << (bits << 8) << endl;
        cout << "bits<<31=0x"<< hex << (bits << 31)<< " "
                             << dec << (bits << 31)<< endl;
        cout << "bits>>3=0x" << hex << (bits >> 3) << " "
                             << dec << (bits >> 3) << endl;

        return 0;
}
```

```
bits<<8=0x9b00 39680
bits<<31=0x80000000 -2147483648
bits>>3=0x13 19
```

```
#include <iostream>
#include <iomanip>
using namespace std;
int main()
{
    unsigned char bits = 0233;
    cout << "bits<<8=0x" << hex << (int)(char)(bits << 8) << " "
                         << dec << (int)(char)(bits << 8) << endl;
    cout << "bits<<31=0x"<< hex << (int)(char)(bits << 31) << " "
                         << dec << (int)(char)(bits << 31) << endl;
    cout << "bits>>3=0x" << hex << (int)(char)(bits >> 3) << " "
                         << dec << (int)(char)(bits >> 3) << endl;
    return 0;
}
```

```
bits<<8=0x0 0
bits<<31=0x0 0
bits>>3=0x13 19
```

//蓝框部分不必要，可省略

```
#include <iostream>
#include <iomanip>
using namespace std;
int main()
{
        char bits = 0233;  //提升为int类型时，填充的是符号位
        cout << "bits<<8=0x" << hex << (bits << 8) << " "
                             << dec << (bits << 8) << endl;
        cout << "bits<<31=0x"<< hex << (bits << 31) << " "
                             << dec << (bits << 31) << endl;
        cout << "bits>>3=0x" << hex << (bits >> 3) << " "
                             << dec << (bits >> 3) << endl;
        return 0;
}
```

```
bits<<8=0xffff9b00 -25856
bits<<31=0x80000000 -2147483648
bits>>3=0xfffffff3 -13
```

1.2 常用的位运算

- 移位运算符
  - 左移：在不溢出(1不被舍去)的情况下，左移n位等于乘2的n次方(当做无符号数理解)
  - 右移：在不溢出(1不被舍去)的情况下，右移n位等于除2的n次方(当作有符号数理解)

1.2 常用的位运算

- 移位运算符
  - 重载版本--IO运算符
  - 满足左结合律：

```
cout << 42 + 10; //正确：+优先级高，输出求和结果

cout << (10 < 42); //正确：输出1

cout << 10 < 42;  //错误：把数字10写到cout，然后将结果
                  （即cout）与42进行比较
```

1.2 常用的位运算

- 位求反运算符
  - 运算规则：0/1互反

例： unsigned char bits = 0227;

1	0	0	1	0	1	1	1

~bits

//提升成int类型，原来位保持不变，高位添加0：

0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	1	0	1	1	1

//逐位求反：

1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	0	1	1	0	1	0	0	0

1.2 常用的位运算

- 位求反运算符
  - 运算对象：整数类型

例：求表达式的值 ~'q' << 6

//按运算符优先级，先对q按位取反：'q'转换为整数：

0	1	1	1	0	0	0	1

0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	1	1	0	0	0	1

//逐位求反：

1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	0	0	1	1	1	1	0

//移位操作：

1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	0	0	1	1	1	1	0	0	0	0	0	0	0

1.2 常用的位运算

- 位与、位或、位异或运算符
  - 位与运算规则：遇0得0
  - 位或运算规则：遇1得1
  - 位异或运算规则：相同为0，不同为1

例：

```
unsigned char b1 = 0145;
unsigned char b2 = 0257;
b1 & b2
b1 | b2
b1 ^ b2
```

	0	1	1	0	0	1	0	1
	1	0	1	0	1	1	1	1
24个高阶位都是0	0	0	1	0	0	1	0	1
24个高阶位都是0	1	1	1	0	1	1	1	1
24个高阶位都是0	1	1	0	0	1	0	1	0

1.2 常用的位运算

- 位与、位或、位异或运算符
  - 位运算符： & | ^
  - 逻辑运算符：&& || !

例：
```
unsigned long ul1 = 3, ul2 = 7;
(1) ul1 & ul2;          //3，占4个字节
(2) ul1 | ul2;          //7，占4个字节
(3) ul1 && ul2;         //true，占1个字节
(4) ul1 || ul2;         //true，占1个字节
```

1.2 常用的位运算

- 复合位运算符
  - &=　　|=　　^=　　<<=　　>>=

```
…
int main()
{
    char a=0x18;
    int i;
    for(i=1; i<=6; i++) {
        a = a>>1;     //a >>= 1;
        cout << "a>>" << i << "=0x"  << hex << int(a) << " "
                                     << dec << int(a) << endl;
    }
}
```

```
a>>1=0xc 12
a>>2=0x6 6
a>>3=0x3 3
a>>4=0x1 1
a>>5=0x0 0
a>>6=0x0 0
```

1.3 位运算的应用

与(&)的应用

- 清零

```
char a1=0xb6;
cout << "char a=" << hex << (int)a1 << endl
     << " a&0x0=" << dec << (a1&0x0) << endl;
```

例：char a1=0xb6; 现要求将该数清零，则：

1011 0110

& 0?00 ?00?

0000 0000

要清零数为1的位，本数对应位为0

a&0x0 a&0x1 a&0x8 a&0x9

a&0x40 a&0x41 a&0x48 a&0x49

1.3 位运算的应用

- 与(&)的应用
  - 取指定位

例：char a2=0xb6；现要求只保留低4位，而高4位清0，则：

1011 0110

& 0000 1111

0000 0110

要保留的位，本数对应位为1

```
char a2=0xb6;
cout << "char a=0x" << hex << (int)a2
     << "  a&0x0F=" << dec << (a2&0x0F) << endl;
```

1.3 位运算的应用

- 或(|)的应用
  - 设定某些位为1

例：char a=0xb6; 要求1、4位设为1，其它不变

```
  1011 0110
| 0000 1001
-----------
  1011 1111   (0xBF)
```

```
char a=0xb6;
cout << "a=" << hex << (int)a
     << " a|0x9=0x" << (a|0x9) << endl;
```

1.3 位运算的应用

- 异或(^)的应用
  - 特定位翻转（0，1互换）

例：char a1=0xb6; 高4位翻转，低4位不变

1011 0110

^ 1111 0000

0100 0110

要翻转的位，本数对应位为1

```
char a1=0xb6;
cout << "a=" << hex << (int)a1
     << " a^0xF0=0x" << (a1^(char)0xF0) << endl;
```

1.3 位运算的

```
char a2=0xb6, b2=0xc2;
cout << "a=" <<hex<<(int)a2<<" b=" <<(int)b2<< endl;
a2 = a2^b2; b2 = b2^a2; a2 = a2^b2;
cout << "a=" <<hex<<(int)a2<<" b=" <<(int)b2<< endl;
```

- 异或(^)的应用
  - 两数交换

例：char a=0xb6,b=0xc2；要求a，b互换

(1) a=1011 0110

b=1100 0010

a=0111 0100　　a=a^b=0x74

(2) b=1100 0010

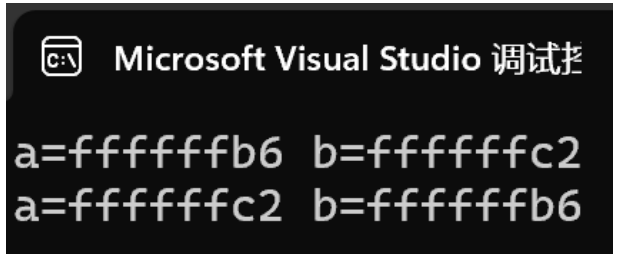
a=0111 0100

b=1011 0110　　b=b^a=0xb6

(3) a=0111 0100

b=1011 0110

a=1100 0010　　a=a^b=0xc2



1.3 位运算的应用

综合应用例：班级有30个学生，老师每周都会对学生进行一次小测验，结果只有通过和不通过两种。为了更好地追踪测验的结果，我们用一个二进制位代表某个学生在一次测验中是否通过，显然全班的测验结果可以用一个无符号整数来表示：

```
unsigned long quiz1 = 0;    //位的集合
```

对序号为27的学生对应的位进行设置，以表示其通过了测验：

```
1UL << 27;    //生成一个值，该值只有第27位为1
quiz1 |= 1UL << 27;  //表示学生27通过了测验
```

1.3 位运算的应用

重新核对发现学生27实际上没有通过测试：

```
quiz1 &= ~(1UL << 27);  //学生27没有通过测验
```

检查学生27的测验情况：

```
bool status = quiz1 & (1UL << 27);  //学生27是否通过了测验?
```

第一章 C语言的补充概念

模块1.3：带参数的main函数

1.1 带参数的main函数

1 引入

- 可执行文件运行时，目前只能简单的运行，如能加上参数，使用中更灵活

2 方法

- 带参数的main函数的定义形式

```
int main(int argc, char **argv)
int main(int argc, char *argv[])
```

两者均可

- 参数解释

  argc：参数的个数，若不带参数，则为1(自身)

  argv：参数的内容，用指针数组表示，每个元素是一个字符串(char *)，最后一个是NULL

- 参数名argc/argv可变，类型不能变（例如：int ac, char **av）

1.1 带参数的main函数

- argv[0]为该可执行文件的文件名（含目录）

```
//集成环境运行
#include<iostream>
using namespace std;
int main(int argc, char* argv[])
{
    int i;
    cout << "当前文件的路径：" << argv[0] << endl;
    for (i = 1; i < argc; i++){
        cout << i << "," << argv[i] << endl;
    }
    return 0;
}
```

只输出该可执行文件的路径，即argv[0]中存储的字符串。

```
当前文件的路径：D:\test\Debug\demo.exe
```

1.1 带参数的main函数

- argv[0]为该可执行文件的文件名（含目录）

```
//命令行运行
#include<iostream>
using namespace std;
int main(int argc, char* argv[])
{
    int i;
    cout << "当前文件的路径：" << argv[0] << en
    for (i = 1; i < argc; i++){
        cout << i << "," << argv[i] << endl;
    }
    return 0;
}
```

假设编译后形成demo.exe，运行：

```
demo
demo hello world
demo who are you
```

```
D:\test\Debug>demo
当前文件的路径：demo

D:\test\Debug>demo hello world
当前文件的路径：demo
1,hello
2,world

D:\test\Debug>demo who are you
当前文件的路径：demo
1,who
2,are
3,you
```

```
//两数交换
#include <iostream>
#include <cstdlib> //atoi函数用到
using namespace std;
int main(int argc, char *argv[])
{   int a, b, t;
    if (argc<3) { /* 参数不足3个则出现提示 */
       cerr << "请带两个整数作为参数"<< endl;
       return -1;
    }
    for (t=0; t<argc; t++) /* 打印所有的参数值 */
       cout << "argv[" << t << "]=" << argv[t] << endl;
    a = atoi(argv[1]); //atoi是将字符串转为整数的函数
    b = atoi(argv[2]);
    cout << "交换前：a=" << a << "  b=" << b << endl;
    t = a; a = b; b = t;
    cout << "交换后：a=" << a << "  b=" << b << endl;
    return 0;
}
```

1.

```
请带两个整数作为参数
```

假设编译后形成demo.exe

1、集成环境运行

2、命令行运行

```
demo
demo 10
demo 10 15
demo 10 15 20
```

2.

```
D:\test\Debug>demo
请带两个整数作为参数

D:\test\Debug>demo 10
请带两个整数作为参数

D:\test\Debug>demo 10 15
argv[0]=demo
argv[1]=10
argv[2]=15
交换前：a=10  b=15
交换后：a=15  b=10

D:\test\Debug>demo 10 15 20
argv[0]=demo
argv[1]=10
argv[2]=15
argv[3]=20
交换前：a=10  b=15
交换后：a=15  b=10
```

1.2 带参数的main函数-例子

- 例：高程大作业-文件压缩小程序

» 要求使用cmdline方式读取参数，参数格式为 压缩文件名 输出文件名 压缩指令(zip/(unzip，选做))

```
int main(int argc, char* argv[]) {
    cout << "Zipper 0.001! Author: root" << endl;
    if (argc != 4) {
        cerr << "Please make sure the number of parameters is correct." << endl;
        return -1;
    }

    if (strcmp(argv[3], "zip"
        cerr << "Unknown para
        return -1;
    }
```

```
命令提示符
Microsoft Windows [版本 10.0.19042.867]
(c) 2020 Microsoft Corporation. 保留所有权利。

C:\Users\april>cd C:\Users\april\source\repos\Gaocheng\Debug

C:\Users\april\source\repos\Gaocheng\Debug>demo C:\Users\april\source
\repos\Gaocheng\ser.log C:\Users\april\source\repos\Gaocheng\ser_comp
ressed.log zip
Zipper 0.001! Author: root
Complete!
```

1.2 带参数的main函数-例子

- 带参数的main函数的扩展形式(仅了解)

形式：　　int main(int argc, char **argv, char **env)

或：　char *env[]

参数解释：

- argc：同前
- argv：同前
- env ：操作系统的环境变量，用指针数组来表示，每个元素是一个字符串(char *)，最后一个元素是NULL

1.2 带参数的main函数-例子

- 带参数的main函数的扩展形式(仅了解)

使用：需要判断/取操作系统的某些设置时才用到

```
//取操作系统的环境变量
#include <iostream>
using namespace std;
int main(int argc, char **argv, char **env)
{
   int i;
   for (i=0; env[i]; i++)
      cout<< "env[" << i << "]=" << env[i] <<endl;
   return 0;
}
```

第二章 文件的操作

模块2.1：C方式的文件操作

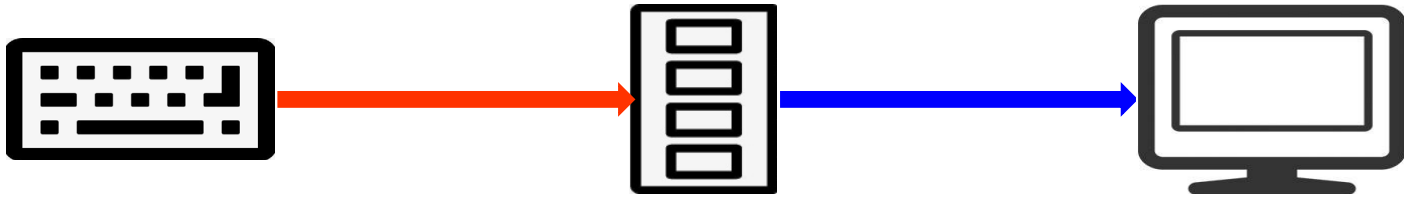
主讲教师：同济大学计算机科学与技术学院 陈宇飞
同济大学计算机科学与技术学院 龚晓亮

目录

- 文件概述
- 文件的打开与关闭
- 文件的读写
- 其他标准I/O函数

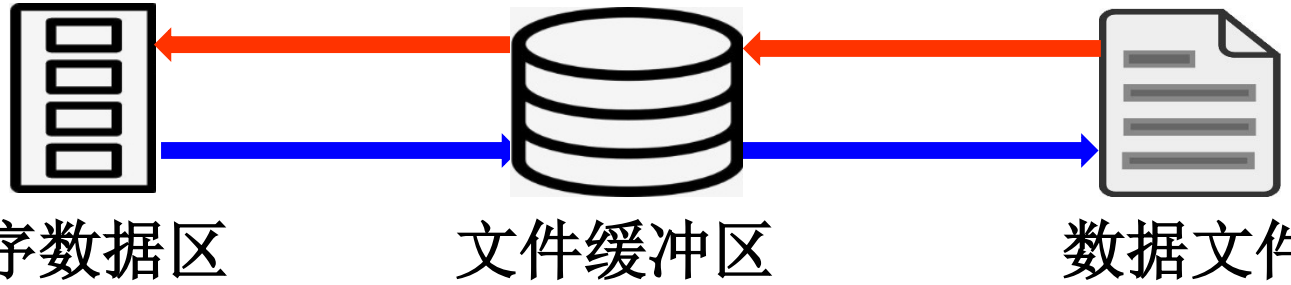
1.1 文件概述

- 标准输入输出



  - 缺点：受限于数据类型，处理数据数量有限，程序结束不保存计算结果。

- 文件输入输出



  - 优点：可以处理大批量的数据，长久的保存计算结果。

1.1 文件概述

- 文件是存储在外存（如磁盘）上的数据集合。
- 每个文件通过唯一的文件名来标识。

  --文件标识，包括三部分：

  - 文件路径：指明文件在外存中的位置
  - 主文件名：要遵循标识符命名规则
  - 文件后缀（扩展名）：表示文件性质

  如：

  D:\document\file1.txt

  表示file1是存放在d盘的document文件夹下的文本文件（txt）

- 计算机按文件名对文件进行读写等操作。

1.1 文件概述

- 文件的物理存储均为二进制，但编码有差异：
  - 文本文件

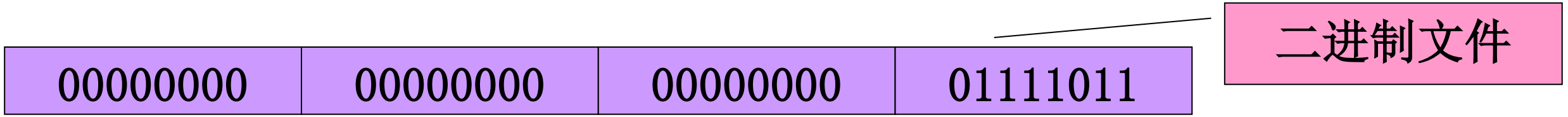
    以ASCII码、Unicode码表示的纯文本文件，只能存储字符信息，不能存储其他信息；文本文件大多是定长编码，每个字符在具体编码中是固定的，如把内存中的数据转换成ASCII码，每个字符用一个ASCII码存储。

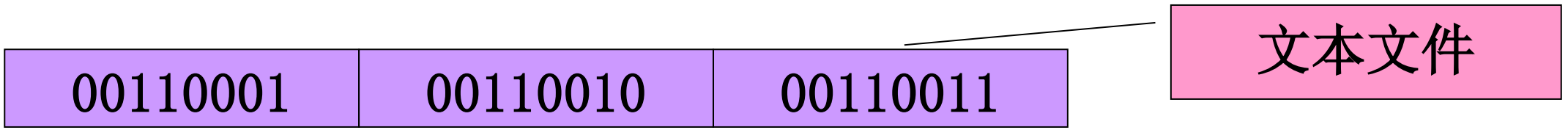
  - 二进制文件

    可存储图形文件和文字处理程序等计算机程序；把内存中的数据按其内存中的存储形式不进行格式转换直接存放在文件上，是变长编码。

1.1 文件概述

如：整数123在内存中占4个字节，其二进制文件也占4个字节，文本文件占3个字节（每个数字以其字符的ASCII码存储）





’1’的ASCII码 ’2’的ASCII码 ’3’的ASCII码

1.2 文件存取方式

➢顺序存取：

每当“打开”文件进行读或写操作时，总是从文件的开头开始，从头到尾顺序地读写；

➢随机存取：

可以通过调用C语言的库函数去指定开始读写的字节号，然后直接对此位置上的数据进行读写操作。

1.3 I/O的级别

- 底层I/O（low-level I/O）：

  使用操作系统提供的基本I/O服务；

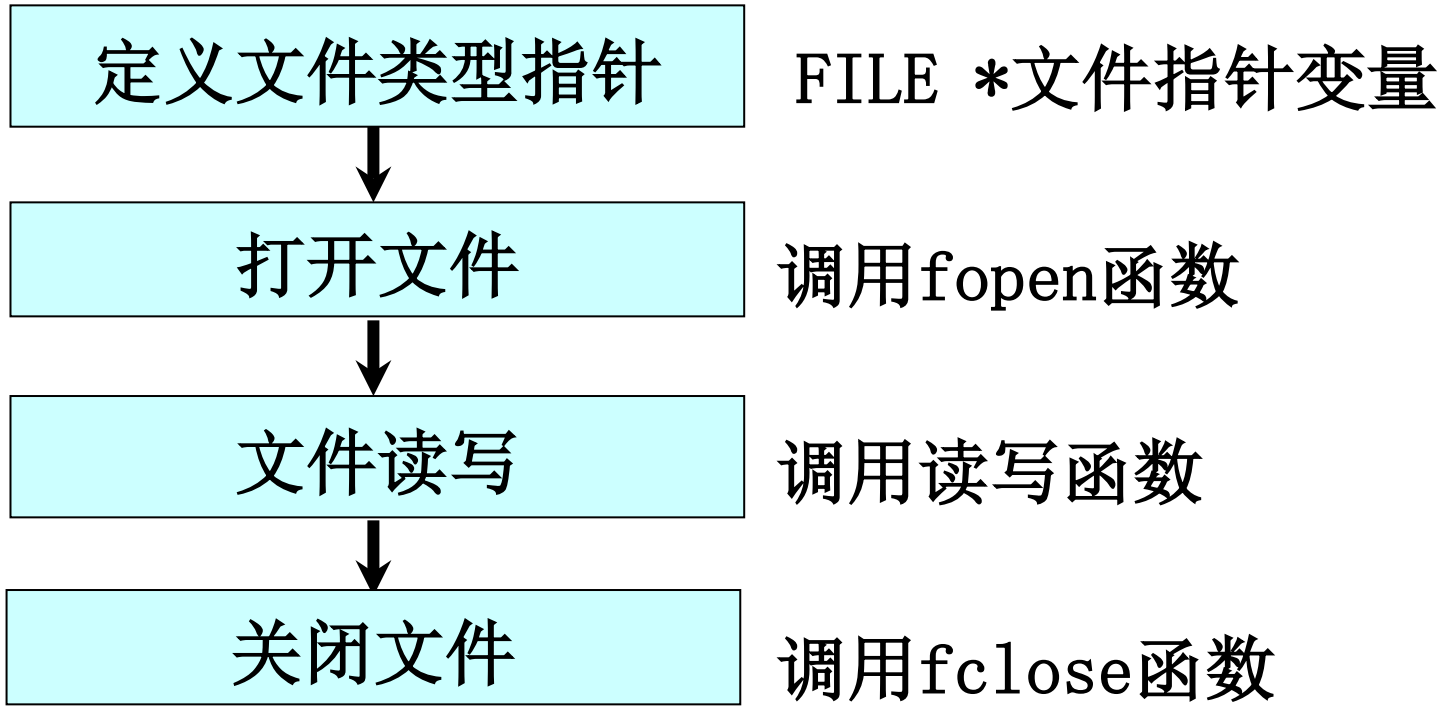
- 标准高级I/O（standard high-level I/O）：

  使用C库的标准包和stdio.h头文件定义，可移植性好。

因为无法保证所有的操作系统都使用相同的底层I/O模型，C标准只支持标准I/O包。

1.3 C语言文件操作

- C语言文件操作步骤

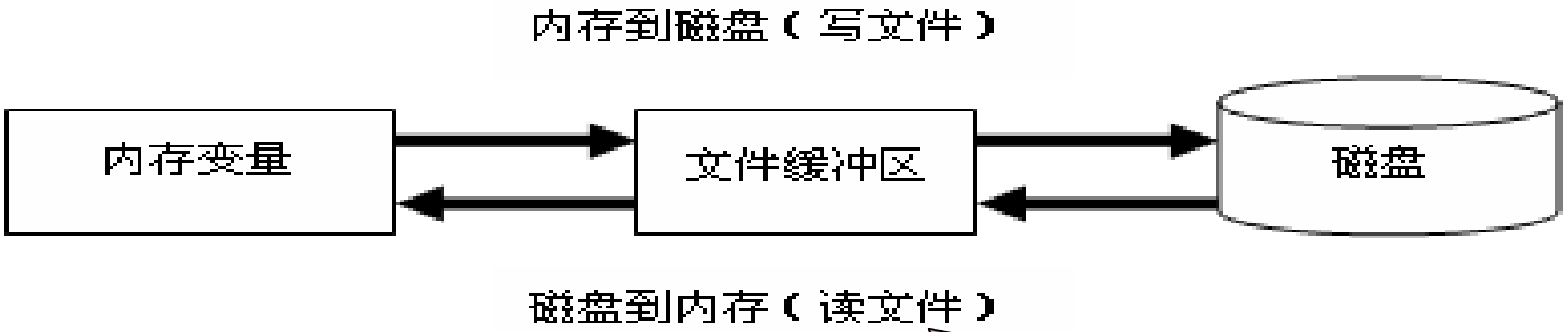


- 标准I/O对文件的处理通过调用标准的输入输出库函数实现。这些专门的函数简化了处理不同I/O的问题。

1.3 C语言文件操作

- 标准I/O的文件操作一般采用“缓冲文件系统”的方式

调用输出函数把程序中变量的值输出到外部文件中，称为“输出”或“写”文件操作



一次转移一大块信息；缓冲在后台处理，让人有逐字符访问的错觉

调用输入函数从外部文件中输入数据赋给程序中的变量，称为“输入”或“读”文件操作

1.3 C语言文件操作

- C语言文件处理的关键是定义一个文件指针，通过该指针对文件进行打开、读写、关闭等操作。

定义：

```
FILE  *文件指针标识符;
```

例如：

```
FILE  *fp1, *fp2;
```

FILE是结构体变量，存放每一个被打开文件的有关信息(如缓冲区的状态、大小，文件当前位置等)。使用FILE类型，需包含stdio.h文件。

1.3 C语言文件操作

- 例如：VS编译环境提供的stdio.h 头文件中有关文件类型的申明：

- 不同的C编译器的FILE类型包含的内容不完全相同，但是大同小异。

```
struct _iobuf
{
    char *_ptr;
    int   _cnt;
    char *_base;
    int   _flag;
    int   _file;
    int   _charbuf;
    int   _bufsiz;
    char *_tmpfname;
};
typedef struct _iobuf FILE;
FILE* pf;//文件指针变量
```

目录

- 文件概述
- 文件的打开与关闭
- 文件的读写
- 其他标准I/O函数

2.1 文件的打开

① 函数原型：FILE *fopen(char *fname, char *mode)

② 功能说明：按照mode规定的方式，打开由fname指定的文件。

③ 参数说明：

fname：字符指针，指向要打开或建立的文件名字符串。

mode：字符指针，指向文件处理方式字符串。

④ 返回值：

正常返回：被打开文件的文件指针。

异常返回：NULL，表示打开操作不成功。

2.1 文件的打开

mode	含义
r	以读模式打开文件，如果文件不存在，则打开文件失败
w	以写模式打开文件，把现有文件的长度截为0，如果文件不存在，则创建一个新文件
a	以写模式打开文件，在现有文件末尾添加内容，如果文件不存在，则创建一个新文件
r+	以更新模式打开文件（即可以读写文件）
w+	以更新模式打开文件（即，读和写），如果文件存在，则将其长度截为0；如果文件不存在，则创建一个新文件
a+	以更新模式打开文件（即，读和写），在现有文件的末尾添加内容，如果文件不存在则创建一个新文件；可以读整个文件，但是只能从末尾添加内容

2.1 文件的打开

续表：

mode	含义
rb, wb, ab, rb+, r+b, wb+, w+b, ab+, a+b	与上一个模式类似，但是以二进制模式而不是文本模式打开文件
wx, wbx, w+x, wb+x, w+bx	（C11）类似非x模式，但是如果文件已存在或以独占模式打开文件，则打开文件失败

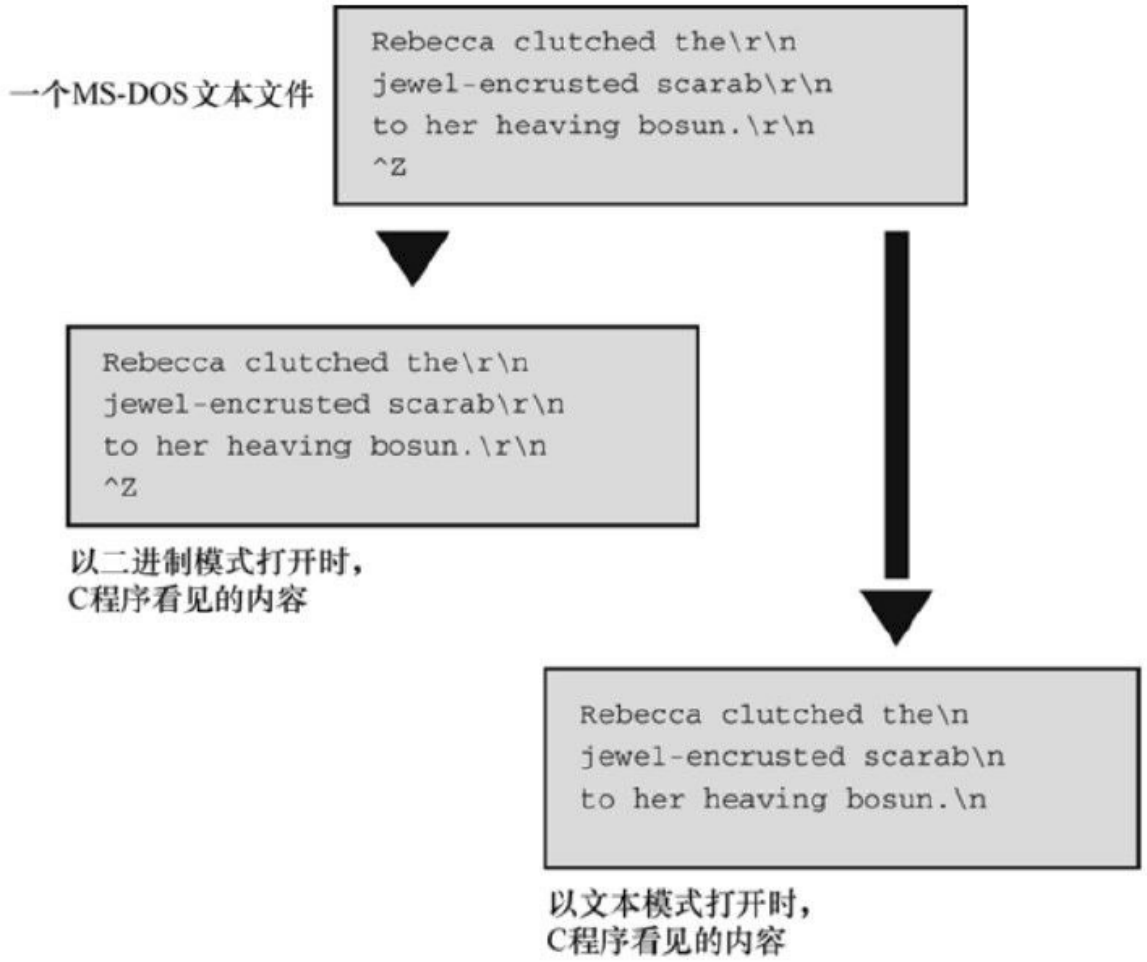
2.1 文件的打开

- 文件访问途径（文件打开模式）
  - ➢ 文本模式

    程序所见的内容与文本实际内容不同。程序会把本地环境表示的行末尾或者文件结尾映射为C模式

  - ➢ 二进制模式

    程序可以访问文件的每个字节，故访问的内容就是文件中存放的内容



```
#define _CRT_SECURE_NO_WARNINGS
#include <iostream>
#include <stdlib.h>
using namespace std;
int main()
{
    FILE* fp1, * fp2;
    char ch;
    int count = 0;
    fp1 = fopen("..\\file1.txt", "r");
    if (fp1 == NULL)
    {
        cout << "can't open file1.txt";
        exit(EXIT_FAILURE);
    }
    while ((ch = fgetc(fp1))!=EOF)
    {
        putchar(ch);
        count++;
    }
    cout << "文本模式打开文件可读字符数："
         << count << endl;
    count = 0;
    fclose(fp1);
    fp2 = fopen("..\\file1.txt", "rb");
    if (fp2 == NULL)
    {
        cout << "can't open file1.txt";
        exit(EXIT_FAILURE);
    }
    while ((ch = fgetc(fp2)) != EOF)
    {
        putchar(ch);
        count++;
    }
    cout << "二进制模式打开文件可读字符数："
         << count << endl;
    fclose(fp2);
    return 0;
}
```

//函数细节后面讲，这里观察分析原因即可

```
test0343fs
文本模式打开文件可读字符数：11
test0343fs
二进制模式打开文件可读字符数：12
```

2.1 文件的打开

- 为了程序的通用性，文件名可在程序运行时输入。如：

```
FILE *fp;
char fname[15];
cout << "Input filename:\n";
cin >> fname;
fp = fopen(fname,"r");
```

注意：

在fopen函数中，如果文件名直接给出，则路径中的"\"应写成"\\"；
如果文件名在程序运行时输入，则路径中的分隔符直接输入字符"\"。

```
fp=fopen("D:\\Doc\\ss.txt","a");
```

```
Input filename:
D:\Doc\ss.txt
Press any key to continue
```

2.1 文件的打开

- 为保证程序正常运行，需对fopen函数的返回值进行检验，以判断文件是否成功地打开。形式如下：

```
 if ((fp = fopen(fname, mode)) == NULL)
{
    cout << "can't open file\n";
    exit(EXIT_FAILURE);
}
```

该段程序使得文件打开失败时，显示提示信息，然后调用exit函数结束程序,使用该函数包含文件"stdlib.h"

2.2 文件的关闭

① 函数原型：int fclose(FILE *fp)

② 功能说明：

关闭由fp所指的文件，释放由fp所指的文件类型结构体变量。

③ 参数说明：fp：一个已打开文件的文件指针。

④ 返回值：

正常返回：0。
异常返回：EOF（-1），表示文件在关闭时发生错误。

注意：应该养成及时关闭文件的习惯，防止误操作或其他原因造成丢失数据的情况发生。

目录

- 文件概述
- 文件的打开与关闭
- 文件的读写
- 其他标准I/O函数

3.1 文件的读写

- C语言提供四种顺序读写函数：

  --字符读写、字符串读写、格式读写和数据块读写。

函数	功能	函数	功能
fputc	向文件写入字符	fputs	向文件写入字符串
fgetc	从文件读取字符	fgets	从文件读取字符串
fprintf	向文件格式化写入数据	fwrite	向文件写入数据块
fscanf	从文件格式化读取数据	fread	从文件读取数据块

3.2 从文件读取字符

① 函数原型：

```
int fgetc(FILE *fp)
```

② 功能说明：

从fp所指文件中读取一个字符，并使文件指针后移一个字符位置。

③ 参数说明：

fp：文件指针，指向要从中读取字符的文件。

④ 返回值：

正常返回：读取的字符。

异常返回：EOF(-1)。

3.2 从文件读取字符

- 从文件读取字符

```
int  fgetc(FILE  *fp)
```

返回类型的说明：用于区分有效数据和输入的结束(EOF)

- 若返回类型为char：一个字节（无法区分）

  EOF： 0xFF

  数据：0xFF

- 若返回类型为int：四个字节（正确区分）

  EOF： 0xFFFFFFFF

  数据：0x000000XX

3.2 从文件读取字符

➢ 文件结束测试函数

① 函数原型：int feof(FILE *fp)

② 功能说明：该函数用来判断文件是否结束。

③ 参数说明：fp：文件指针。

④ 返回值：

0：假值，表示文件未结束。

1：真值，表示文件结束。

⑤ 实例：

```
while(!feof(fp))
{
    ch=fgetc(fp);
    putchar(ch);  //打印到终端
}
```

3.2 向文件写入字符

① 函数原型：int fputc(int ch,FILE *fp)

② 功能说明：把ch中的字符写入fp所指的文件当前位置处，并使文件指针后移一个字符位置。

③ 参数说明：

ch：整型变量，内存要写到文件中的字符。

fp：文件指针，指向要在其中写入字符的文件。

④ 返回值：

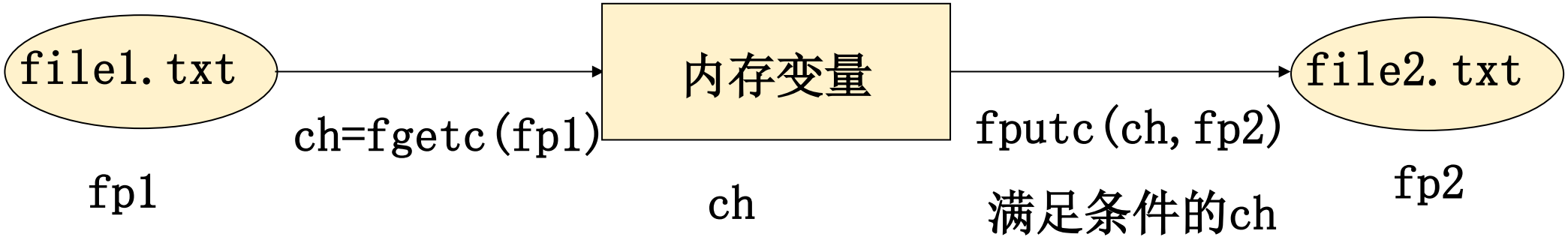
正常返回：写入的字符。

异常返回：EOF(-1)。

3.2 向文件写入字符

例：编写程序将文件file1.txt的内容显示在屏幕上，同时还将该文件中的数字字符复制到文件file2.txt中。

分析：程序涉及文本文件的读写两种操作，操作方式如下：



```
#define _CRT_SECURE_NO_WARNINGS
#include <iostream>
#include <stdlib.h>
using namespace std;
int main()
{   FILE *fp1,*fp2;
    char ch;
    fp1=fopen("..\\file1.txt","r");
    if(fp1==NULL)
    {
       cout<<"can't open file1.txt";
       exit(EXIT_FAILURE);
    }
    if((fp2=fopen("..\\file2.txt","w"))==NULL)
    {
       cout<<"can't open file2.txt";
       fclose(fp1);
       exit(EXIT_FAILURE);
    }
    while(!feof(fp1))
    {
       ch=fgetc(fp1);
       putchar(ch);
       if(ch>='0'&&ch<='9')
          fputc(ch,fp2);
    }
    fclose(fp1);
    fclose(fp2);
    return 0;
}
```

..\\
表示与.sln在同一目录下

```
//例：把文件压缩成原来的1/3！
#define _CRT_SECURE_NO_WARNINGS
#include <iostream>
#include <stdlib.h>
int main()
{
  FILE *in, *out;
  int ch;
  int count = 0;
  if ((in=fopen("..\\eddy.txt", "r")) == NULL)
  {
     printf("can't open eddy.txt\n");
     exit(EXIT_FAILURE);
  }
  if ((out=fopen("..\\eddy.red", "w"))== NULL)
  {
     printf("can't open eddy.red\n");
     fclose(in);
     exit(EXIT_FAILURE);
  }
  while((ch=fgetc(in))!=EOF)
     if (count++ % 3 == 0)
        fputc(ch, out);
  fclose(in);
  fclose(out);
  return 0;
}
```

该程序以保留每3个字符中的第1个字符的方式压缩文件。压缩后的文本存入新文件，名称是原文件名加上.red后缀（red代表reduced）

3.3 向文件写入字符串

① 函数原型：int fputs(char *str,FILE *fp)

② 功能说明：将str指向的字符串的内容输出到fp所指向文件的当前位置，同时将fp自动向前移动strlen(str)个字符位置。

③ 参数说明：str：可以是字符串常量、字符串指针或字符数组名等。

fp：文件指针，指向字符串要写入其中的文件。

④ 返回值：正常返回：非负整数。

异常返回：EOF（-1）。

⑤ 说明：
- 字符串结束符'\0'不输出到文件
- 不自动在字符串末尾添加换行符

3.4 从文件读取字符串

① 函数原型：char *fgets(char *str, int n, FILE *fp)

② 功能说明：从由fp指出的文件中读取n-1个字符，并把它们存放到由str指出的字符数组中去，最后自动加上一个字符串结束符'\0'。

③ 参数说明：str：接收字符串的内存地址，可以是数组名或指针。

n： 指出要读取字符的个数。

fp：文件指针，指向要从中读取字符的文件。

④ 返回值：正常返回：字符串的内存首地址，即str的值。

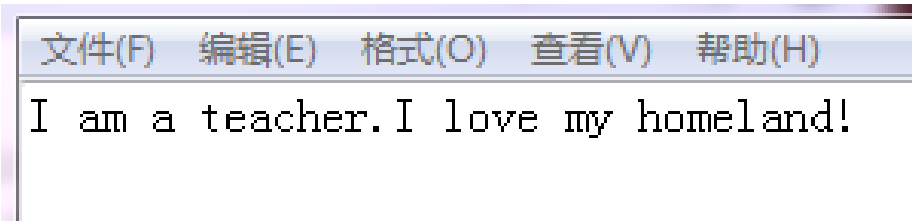
异常返回：NULL。

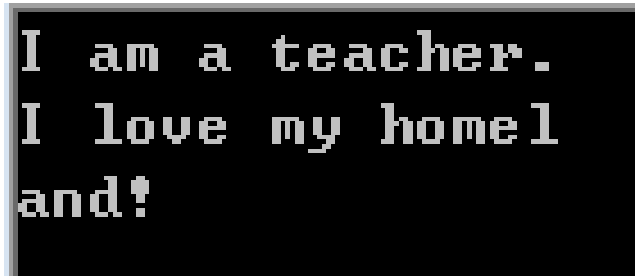
⑤ 说明：
- 读入n-1个字符到文件，遇到换行符或文件结束符则提前结束
- 读入结束后，系统将自动在最后加'\0'

```
#define _CRT_SECURE_NO_WARNINGS
#include <iostream>
#include <stdlib.h>
using namespace std;
int main()
{   FILE *fp;
    char s[100];
    if((fp=fopen("file1.txt", "w"))==NULL)
    {     printf("can't open file1.txt\n");
          return -1;
    }
    fputs("I am a teacher.",fp);
    fputs("I love my homeland!", fp);
    fclose(fp); //必须先关闭，再以读的方式打开。
    if((fp=fopen("file1.txt", "r"))==NULL)
    {     printf("can't open file1.txt\n");
          return -1;
    }
```

表示与.cpp在同一目录下

```
    while(!feof(fp))
    {     fgets(s,16,fp);
          puts(s);
    }
    fclose(fp);
    return 0;
}
```





3.5 向文件格式化写入数据

① 函数原型：int fprintf(FILE *fp,const char *format,arg_list)

② 功能说明：将变量表列（arg_list）中的数据，按照format指出的格式，写入由fp指定的文件。

③ 参数说明：fp：文件指针，指向将数据写入的文件。
format：指向写出数据格式字符串的指针。
arg_list：要写入文件的变量表列，各变量之间用逗号分隔。

④ 返回值：正常返回：输出的字节数。
异常返回：负值。

⑤ 示例：fprintf(fp,"%d%s",4,"China");
表示将整数4和字符串"China"写入fp所指的文件中。

3.6 从文件格式化读取数据

① 函数原型：int fscanf(FILE *fp,const char *format,add_list)

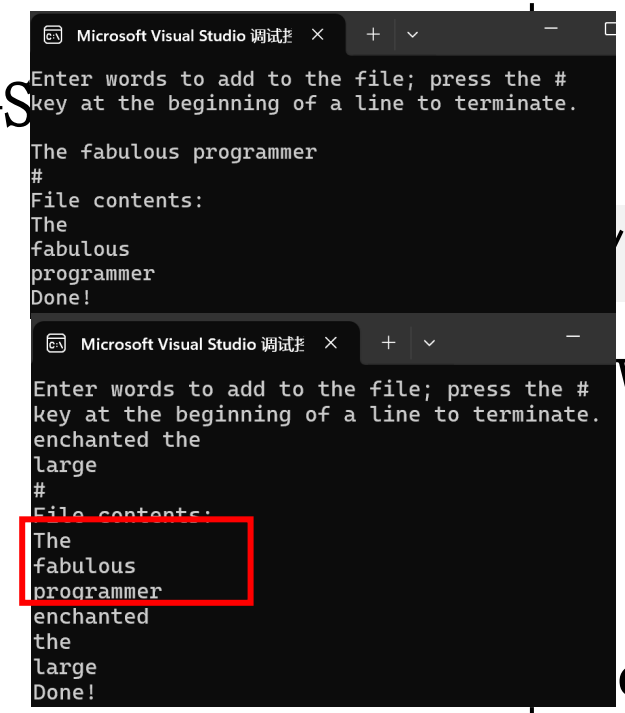
② 功能说明：按照format指出的格式，从fp指定的文件中读取数据存放至地址表列（add_list）的变量中。

③ 参数说明：fp：文件指针，指向要从中读取数据的文件。
format：指向读取数据格式字符串的指针。
add_list：存放读取数据的变量的地址表列。

④ 返回值：正常返回：成功读取的参数的个数。类似scanf，如果%d读到了"abc"，返回0。异常返回：EOF（-1）。

⑤ 示例：fscanf(fp,"%d%d",&x,&y);
表示从fp所指的文件中顺序读取两个整数给变量x和y。

```
#define _CRT_SECURE_NO_WARNINGS
#include <iostream>
#include <stdlib.h>
using namespace std;
#define MAX 41
int main()
{
  FILE* fp;
  char words[MAX];
  if ((fp = fopen("..\\wordy", "a+")) == NULL)
  {
    cout << "Can't open wordy file.\n";
    exit(EXIT_FAILURE);
  }
  cout << "Enter words to add to the file;
press the #\nkey at the beginning of a line
to terminate.\n";
```



运行多次观察结果

```
while((fscanf(stdin,"%40s",
       words) == 1) &&
       (words[0] != '#'))
  fprintf(fp,"%s\n",words);
cout << "File contents:\n";
rewind(fp); //返回文件开始
while (fscanf(fp, "%s",
       words) == 1)
    cout << words << endl;
cout << "Done!\n";
fclose(fp);
return 0;
}
```

3.7 从文件读取数据

- 上述标准I/O函数都是面向文本的，用于处理字符和字符串。

  思考：如何要在文件中保存数值数据？

  例：double num = 1./3.;

  ```
  fprintf(fp,"%f", num);  //储存为8个字符：0.333333
  ```

- 一般而言，用 fprintf()函数和%f转换说明只是把数值保存为字符串。fprintf()把数值转换为字符数据，这种转换可能会改变值。

- 以二进制形式储存数据，不存在从数值形式到字符串的转换过程，保证数值在储存前后一致。

- fread和fwrite函数用于以二进制形式处理数据。

3.7 从文件读取数据块

① 函数原型：

```
size_t fread(void *buffer, size_t size, size_t count,FILE *fp)
```

② 功能说明：从文件指针fp所指的文件的当前位置读取字节数为size大小的数据块共count个，存到buffer所指的内存储区中。

③ 参数说明：

buffer：指向存放读入数据存储区的首地址的指针。

size：数据块的字节数，即一个数据块的大小。

count：一次读入数据块的数量。

fp：文件指针，指向要从其中读出数据的文件。

④ 返回值：正常返回：<=count。正常情况下，该返回值就是count，但如果出现读取错误或读到文件结尾，该返回值就会比count小；异常返回：0。

3.8 向文件写入数据块

① 函数原型：

```
size_t fwrite(void *buffer, size_t size, size_t count,FILE *fp)
```

② 功能说明：将以buffer为起始地址的长度为size的count个数据块输出到文件指针fp所指的位置去。

③ 参数说明：

buffer：指向存放输出数据存储区的首地址的指针。

size：数据块的字节数，即一个数据块的大小。

count：一次输出数据块的数量。

fp：文件指针，指向要从其中写入数据的文件。

④ 返回值：

正常返回：实际输出数据块的个数，即count。

异常返回：0。

3.8 向文件写入数据块

- fread和fwrite常用于读写二进制文件

例如，假设fp指向以二进制形式打开的可读写文件，并有如下的说明：

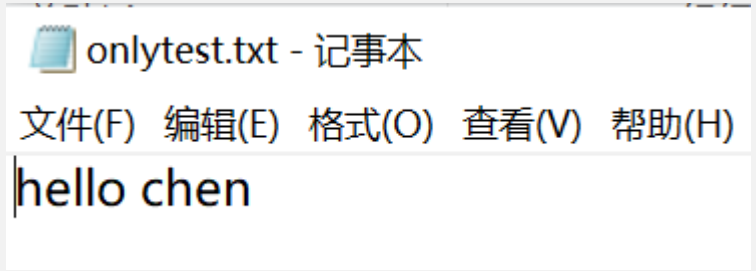
```
float f;  double d[10];
```

常见的块读写应用示例：

```
fwrite(&f,sizeof(float),1,fp);     //把浮点数f写入文件
fwrite(d,sizeof(double),10,fp);    //把数组d中所有数写入文件
fread(&f,sizeof(float),1,fp); //从文件中以块形式读一浮点数到变量f中
fread(d,sizeof(d),1,fp); //从文件中一次性读一个d大小的数据块到数组d中
fwrite(&d[0],sizeof(float),1,fp);
                          //参数类型不匹配会导致数据错误（编译不错）
```

```
#define _CRT_SECURE_NO_WARNINGS
#include <iostream>
#include <stdlib.h>
using namespace std;
int main()
{
    FILE* fp; char ch; int count = 0;
    fp = fopen("..\\onlytest.txt", "w");
    if (fp == NULL)
    {
        cout << "can't open onlytest";
        exit(EXIT_FAILURE);
    }
    char d[] = "hello\0chen";
    fwrite(d, sizeof(d), 1, fp); //fprintf(fp,"%s",d);
    return 0;
}
```

分别观察运行后的文件结果：



例：从键盘上读入5个学生的成绩信息，将它们以文件的形式保存在磁盘上。然后再将从文件中读取学生信息，并在屏幕上显示出来。假设学生成绩信息包括学生姓名、学号、总分。

具体步骤如下：

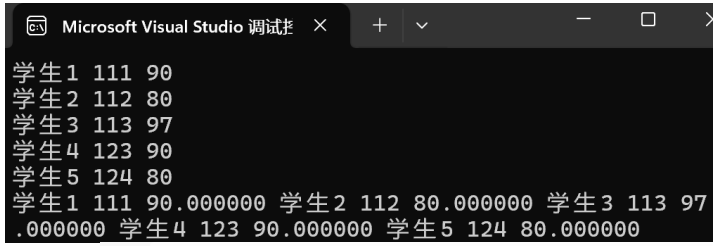
- 声明学生成绩结构类型
- 定义学生成绩结构数组
- 以块读写的方式对学生信息进行文件读写

```
struct student
{
    char name[20];
    char num[8];
    double score;
};
```

```
#define _CRT_SECURE_NO_WARNINGS
#define N 5
#include <cstdio>
using namespace std;
struct student
{  char name[20]; char num[8]; double score;};
int main()
{
   struct student s[N], t[N];
   int i;
   FILE* fp;
   if ((fp = fopen("student.dat", "wb")) == NULL)
   {
      printf("can't open student.dat\n");
      return -1;
   }
//接下页
```

```
    for (i = 0; i < N; i++)
    {
        scanf("%s%s%lf", s[i].name, s[i].num, &s[i].score);
        fwrite(&s[i], sizeof(student), 1, fp);//逐元素向文件写数据
    }
    fclose(fp); //写文件结束先关闭，再以读的方式打开。
    if ((fp = fopen("student.dat", "rb")) == NULL)
    {
        printf("can't open student.dat\n");
        return -1;
    }
    fread(t, sizeof(t), 1, fp); //一次从文件读出整个数组
    for (i = 0; i < N; i++)
        printf("%s %s %lf ", t[i].name, t[i].num, t[i].score);
    fclose(fp);
    return 0;
}
```

//本示例受限篇幅，没有输入输出提示及输入数据合理性检验等





3.9 随机读写

- 顺序读写文件只能从头开始，顺序读写各个数据。
- 随机读写可按需要只读写文件中某些指定的部分。
- 随机读写的关键是要按要求移动位置指针，即进行文件的定位。
- 实现文件定位、移动文件内部位置指针的函数主要有rewind、ftell、fseek、fgetpos和fsetpos等函数。

3.9 随机读写

1.rewind 函数

① 函数原型：void rewind(FILE *fp)

② 功能说明：使指示文件位置的指针重新返回到文件开始。

③ 参数说明：fp是文件指针

3.9 随机读写

2. ftell 函数

① 函数原型：long ftell(FILE *fp)

② 功能说明：求文件指针的当前位置。

③ 参数说明：fp是文件指针。

④ 返回值：距文件开始处的字节数。

3.9 随机读写

3. fseek 函数

① 函数原型：int fseek(FILE *fp, long offset, int whence)

② 功能说明：使文件指针fp移到基于whence的相对位置offset处。

③ 参数说明：offset是相对whence的字节位移量，用长整型表示。

whence是移动的基准，常用符号常量表示。

④ 返回值：正常返回：0。

异常返回：-1，表示定位操作出错。

3.9 随机读写

- 参数whence的意义

符号常量	值	基准位置
SEEK_SET	0	文件开头
SEEK_CUR	1	当前读写的位置
SEEK_END	2	文件尾部

- 示例

```
fseek(fp, 0L, SEEK_SET);     // 定位至文件开始处
fseek(fp, 10L, SEEK_SET);    // 定位至文件中的第10个字节
fseek(fp, 2L, SEEK_CUR);     // 从文件当前位置前移2个字节
fseek(fp, 0L, SEEK_END);     // 定位至文件结尾
fseek(fp, -10L, SEEK_END);   // 从文件结尾处回退10个字节
```

3.9 随机读写

- fseek()和ftell()潜在的问题是，它们都把文件大小限制在long 类型能表示的范围内。
- ANSI C新增了两个处理较大文件的新定位函数：fgetpos()和fsetpos()。
- 这两个函数不使用long类型的值表示位置，它们使用一种新类型：fpos_t（代表file position type，文件定位类型）。

3.9 随机读写

4. fgetpos 函数

① 函数原型：int fgetpos(FILE * fp, fpos_t * pos)

② 功能说明：获取流fp的当前文件位置，并把该fpos_t类型的值放在pos指向的位置上。

③ 参数说明：fp是文件指针；pos是fpos_t类型的指针。

④ 返回值：如果成功，函数返回0；

如果失败，返回非0。

3.9 随机读写

5. fsetpos 函数

① 函数原型：int fsetpos(FILE *fp, const fpos_t *pos)

② 功能说明：使用pos指向位置上的fpos_t类型值来设置文件指针fp指向该值指定的位置。fpos_t类型的值应通过之前调用fgetpos()获得。

③ 参数说明：fp是文件指针；pos是fpos_t类型的指针。

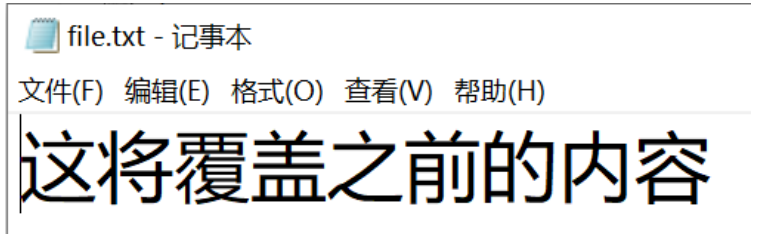
④ 返回值：如果成功，函数返回0；如果失败，则返回非0。

```c
#define _CRT_SECURE_NO_WARNINGS
#include <stdio.h>
int main()
{
    FILE* fp;
    fpos_t position;

    fp = fopen("file.txt", "w+");
    fgetpos(fp, &position);
    fputs("Hello, World!", fp);

    fsetpos(fp, &position);
    fputs("这将覆盖之前的内容", fp);
    fclose(fp);

    return 0;
}
```



4.1 其他标准I/O函数

- ungetc 函数

① 函数原型：int ungetc(int c, FILE *fp)

② 功能说明：把c指定的字符放回输入流中，下次调用标准输入函数时将读取该字符。

③ 参数说明：c是要被放回的字符，以其对应的int值进行传递；

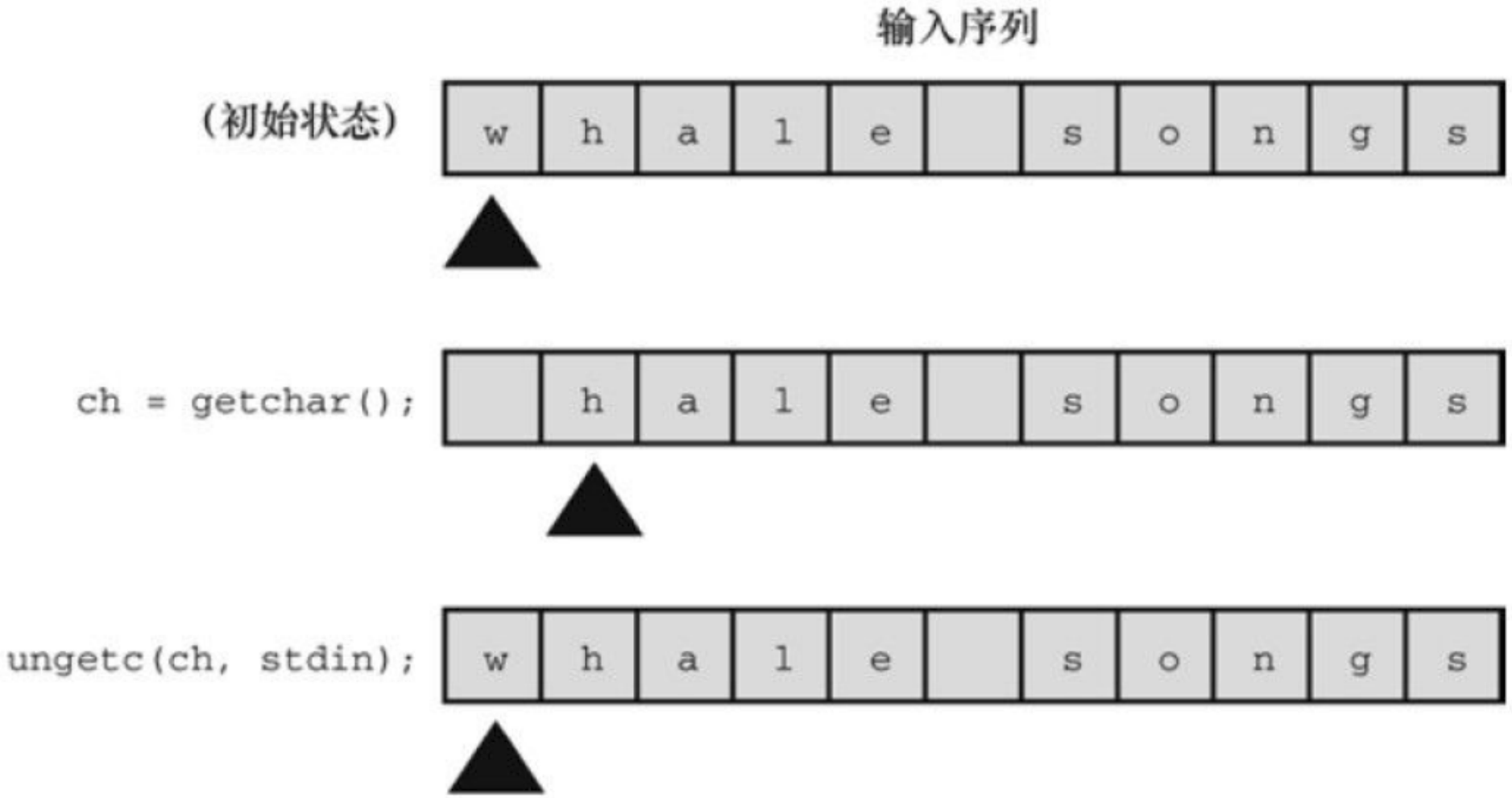
fp是文件指针。

① 返回值：如果成功，则返回被放回的字符；

否则返回EOF。

4.1 其他标准I/O函数

- ungetc 函数



4.1 其他标准I/O函数

- fflush 函数

① 函数原型：int fflush(FILE *fp)

② 功能说明：会强迫将缓冲区内的数据写回参数fp指定的文件中；如果参数fp为NULL，fflush()会将所有打开的文件数据更新。

③ 参数说明：fp是文件指针。

④ 返回值：如果成功刷新，则返回0。指定的流没有缓冲区或者只读打开时也返回0值；否则返回EOF。

4.1 其他标准I/O函数

- setvbuf 函数

① 函数原型：

```
int setvbuf(FILE *fp, char *buf, int mode, size_t size)
```

② 功能说明：设置文件的缓冲区。

③ 参数说明：指针fp识别待处理的流；buf指向待使用的存储区(期望缓冲区地址)，如果buf的值不是NULL，则必须创建一个缓冲区；变量size告诉setvbuf()数组的大小；

4.1 其他标准I/O函数

- setvbuf 函数（续）

mode的选择如下：

_IOFBF表示完全缓冲（在缓冲区满时刷新）；

_IOLBF表示行缓冲（在缓冲区满时或写入一个换行符时）；

_IONBF表示无缓冲（没有缓冲区，直接从流中读入/写入数据）。

④ 返回值：如果操作成功，函数返回0；否则返回非零值。

注意：打开文件之后且未对流进行其它操作之前调该函数

4.1 其他标准I/O函数

- ferror 函数

① 函数原型：int ferror(FILE *fp)

② 功能说明：判断文件是否出错。

③ 参数说明：fp是文件指针。

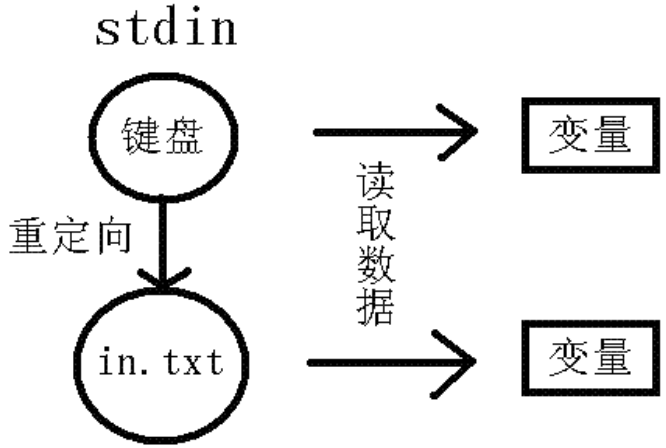
④ 返回值：如果读写未出错，则返回0；否则返回一个非零值。

注意：

- 同一文件每次调用输入输出函数，均产生一个新的ferror函数值。
- 在执行fopen函数时，ferror函数的初始值自动置为0。

4.2 重定向输入输出流

- 调试往往不能一次成功，每次运行时，都要重新输入一遍测试数据，对于有大量输入数据的题目，输入数据需要花费大量时间。
- 使用freopen函数可以将标准IO与文件关联起来，解决测试数据输入问题，避免重复输入。　　例如：



4.2 重定向输入输出流

① 函数原型：

FILE *freopen(const char *path, const char *mode, FILE *stream)

② 功能说明：把预定义的标准流文件重定向到由path指定的文件中。

③ 参数说明：

path：文件名，用于存储输入输出的自定义文件名。

mode：文件打开的模式。和fopen中的模式（如r-只读，w-写）相同。

stream：通常使用标准流文件。

④ 返回值：正常返回：path所指定文件的指针；

异常返回：NULL。

```
#define _CRT_SECURE_NO_WARNINGS
#include <iostream>
#include <stdio.h>
using namespace std;
int main() {
    int a, b;
    if(freopen("in.txt", "r", stdin)==NULL) return -1; //输入数据从文件中读取
    if(freopen("out.txt", "w", stdout)==NULL) return -1; //输出数据保存在文件中
    while (cin >> a >> b)
        cout << a + b << endl;
    fclose(stdin);//关闭文件
    fclose(stdout);//关闭文件

    if (freopen("CON", "w", stdout) == NULL) return -1; //定向输出到控制台
    cout << "hello world" << endl;
    fclose(stdout);//关闭文件
    return 0;
}
```

总结

功能	函数名	函数原型
打开文件	fopen	FILE *fopen(char *fname, char *mode)
关闭文件	fclose	int fclose(FILE *fp)
判断文件是否出错	ferror	int ferror(FILE *fp)
字符输入函数	fgetc	int fgetc(FILE *fp)
字符输出函数	fputc	int fputc(int ch,FILE *fp)
文本行输入函数	fgets	char *fgets(char *str,int n,FILE *fp)
文本行输出函数	fputs	int fputs(char *str,FILE *fp)
格式化输入函数	fscanf	int fscanf(FILE *fp,const char *format, add_list)
格式化输出函数	fprintf	int fprintf(FILE *fp,const char *format, arg_list)
二进制输入	fread	size_t fread(void *buffer, size_t size, size_t count,FILE *fp)
二进制输出	fwrite	size_t fwrite(void *buffer, size_t size, size_t count,FILE *fp)
判断文件是否结束	feof	int feof(FILE *fp)

总结

功能	函数名	函数原型
求文件指针的当前位置	ftell	long ftell(FILE *fp)
移动文件指针	fseek	int fseek(FILE *fp, long offset, int whence)
获取当前文件位置	fgetpos	int fgetpos(FILE * fp, fpos_t * pos)
设置文件指针的位置	fsetpos	int fsetpos(FILE *fp, const fpos_t *pos)
使指示文件位置的指针重新返回到文件开始	rewind	void rewind(FILE *fp)
把字符放回输入流中	ungetc	int ungetc(int c, FILE *fp)
冲洗流中的信息	fflush	int fflush(FILE *fp)
设置文件的缓冲区	setvbuf	int setvbuf(FILE *fp, char *buf, int mode, size_t size)
流文件重定向	freopen	FILE *freopen(const char *path, const char *mode, FILE *stream)

总结

- C语言文件操作相关函数：fopen()、fclose()、exit()、feof()、fgetc()、fputc()、fgets()、fputs()、fprintf()、fscanf()、fread()、fwrite()、rewind()、fseek()、ftell()、fgetpos()、fsetpos()、ungetc()、fflush()、setvbuf()、ferror()、freopen()
- 如何使用C标准I/O系列的函数处理文件
- 文件模式和二进制模式、文本文件和二进制文件
- 文件的顺序访问与随机访问

第二章 文件的操作

模块2.2：C++方式的文件操作

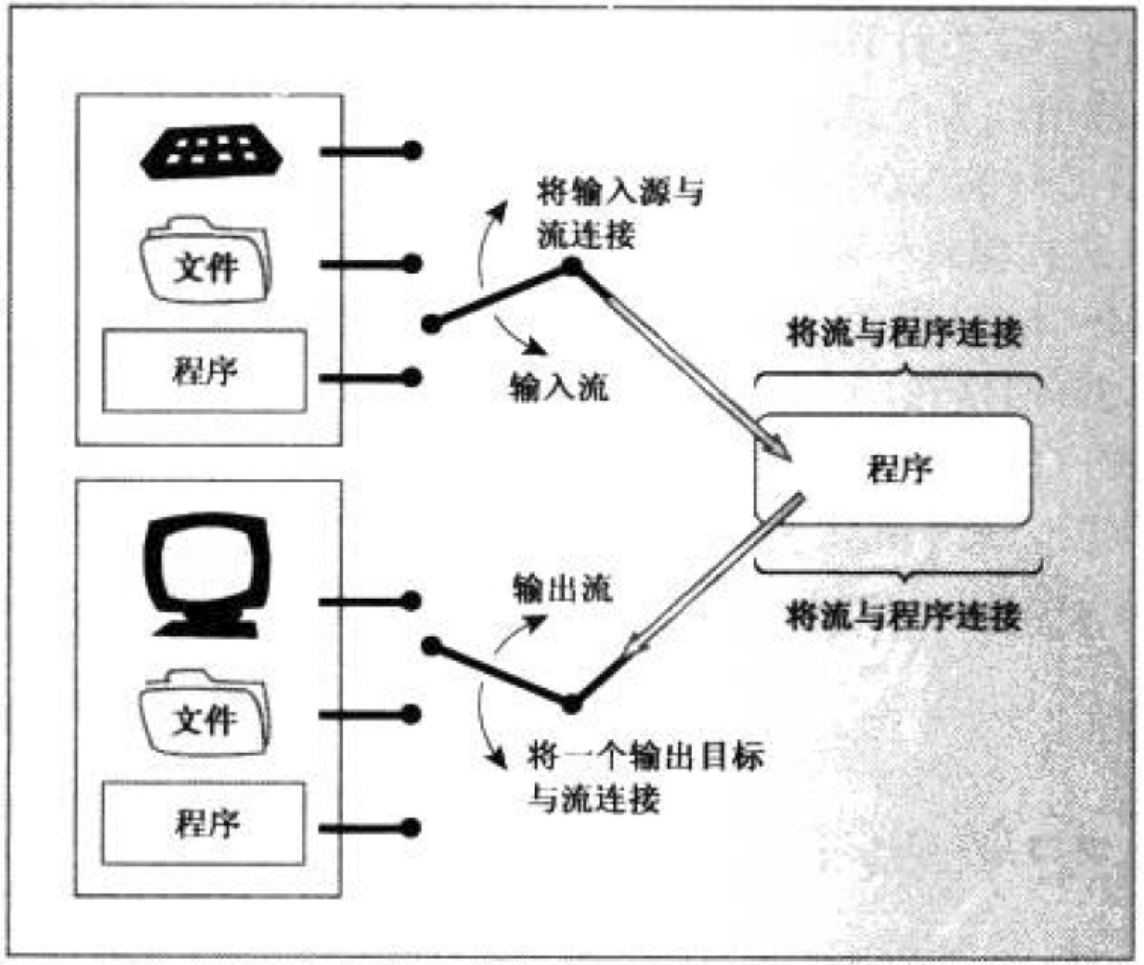
主讲教师：同济大学电子与信息工程学院 陈宇飞
同济大学电子与信息工程学院 龚晓亮

目录

- 标准输入流
- C++方式的文件操作与文件流
- C++方式的字符串流
- C方式类似字符串流的操作

1.1 标准输入流：流和缓冲区

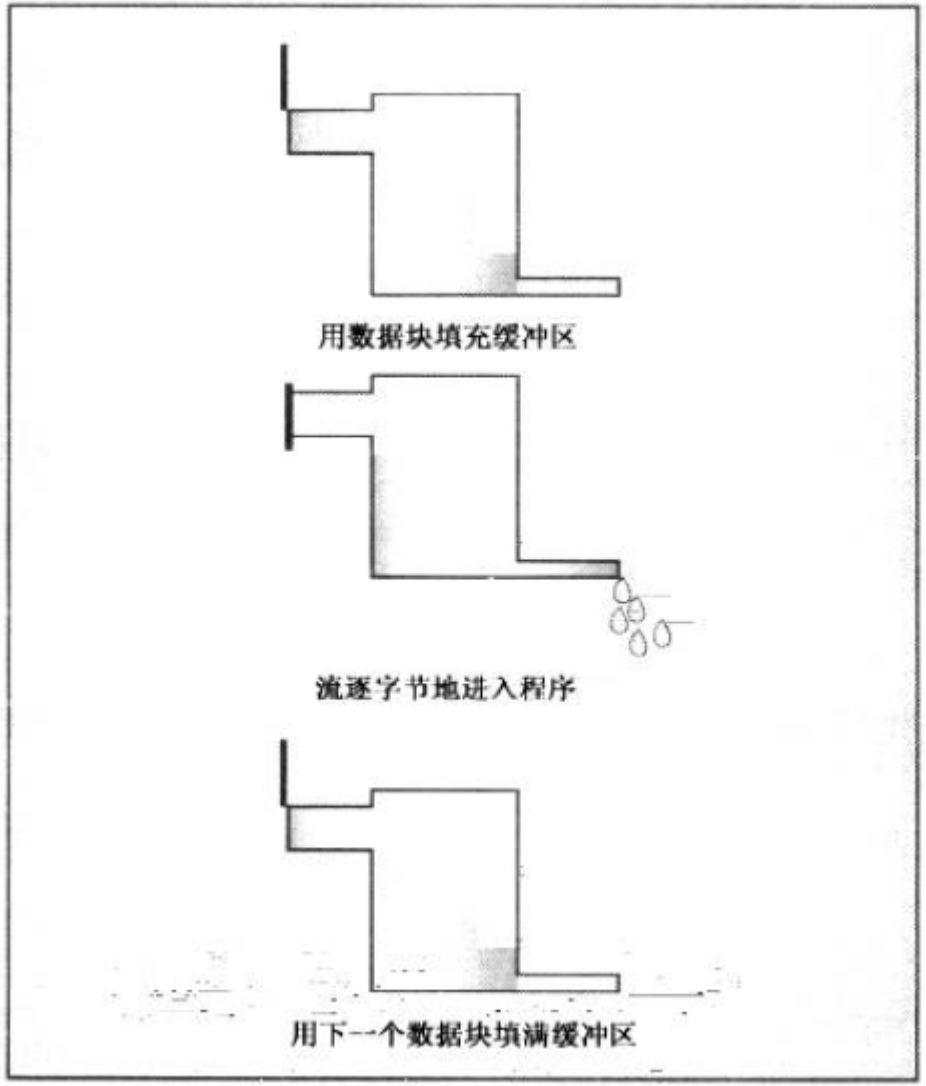
- c++将输入和输出看作字节流。输入时，程序从输入流中抽取字节；输出时，程序将字节插入输出流。
- 输入流中的字节可能来自键盘以及存储设备或其他程序
- 输出流中的字节可以流向屏幕、打印机、存储设备或其他程序
- 管理输入包括：1）将流与输入去向的程序关联起来；2）将流与文件端连接起来。



1.1 标准输入流：流和缓冲区

- 流充当了程序和流源或流目标之间的桥梁：水滴就是字节，水滴汇成的水流就是流，从水流中取水（从流中提取输出的内容），往水流中加水（插入输入的内容）。
- 缓冲区：用作中介的内存块，将信息从设备传输到程序或从程序传输到设备的临时存储工具，可以提高数据的读取速率。
- 输出时，程序首先填满缓冲区，然后把整块的数据传输给硬盘，并清空缓冲区，这被称为刷新缓冲区。

  --缓冲区满；换行符；flush…



1.1 标准输入流：流和缓冲区

- 为了管理流，C++提供了一系列头文件：

➢streambuf类为缓冲区提供了内存，并提供了填充缓冲区、访问缓冲区、刷新缓冲区和管理缓冲区内存的类方法；

➢ios_base类表示流的一般特征，如是否可读取，是二进制流还是文本流等；

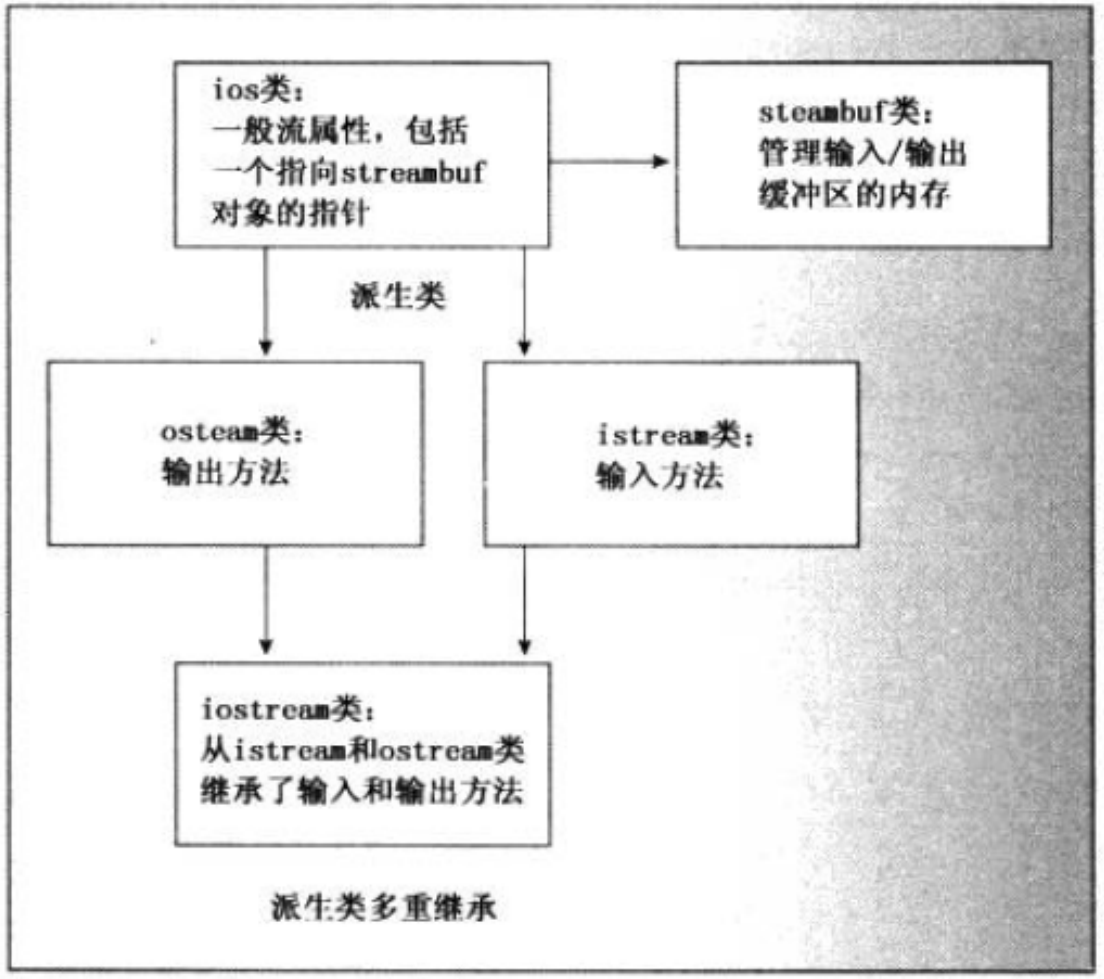
➢ios类基于ios_base；

➢ostream类提供了输出方法；

➢istream类提供了输入方法；

➢iostream类提供了输入和输出方法。

//后续课程讲解



1.1 标准输入流：流和缓冲区

- C++为了能够处理需要16位国际字符集或更宽的字符类型

  传统的8位char（“窄”）类型　　新添加wchar_t（“宽”）类型

  有专门的输入输出对象用于处理宽字符如wcin/wcout（了解即可）。

- iostream头文件会自动生成八个流对象（四个是宽类型的），重点掌握窄类型。

  - cin标准输入流：
  - cout标准输出流：
  - cerr标准错误流：
  - clog标准错误流：

1.1 标准输入流：流和缓冲区

➢cin标准输入流：

- 输入各种数据类型，也可以通过重载使其输入用户自定义的类；

➢cout标准输出流：

- 输出各种数据类型，也可以通过重载使其输出用户自定义的类；

➢cerr标准错误流：

- 用于显式错误信息，默认情况下，这个流被关联到标准输出设备（显示器），这个流没有被缓冲，这意味着信息将直接发送给屏幕；

➢clog标准错误流：

- 用于显式错误信息，默认情况下，这个流被关联到标准输出设备（显示器），与cerr不同的是，clog有缓冲区。

1.2 标准输入流：使用cin进行输入

- cin提取数据后，会根据数据类型是否符合要求而返回逻辑值

```
#include <iostream>
using namespace std;
int main()
{
    int a=-9;
    cin >> a;
    cout << a << " " << (cin ? 1 : 0) << endl;
    return 0;
}
```

输入	结果
10	10 1
ab	0 0
12ab	12 1
很大的数字	2147483647 0

1.2 标准输入流：使用cin进行输入

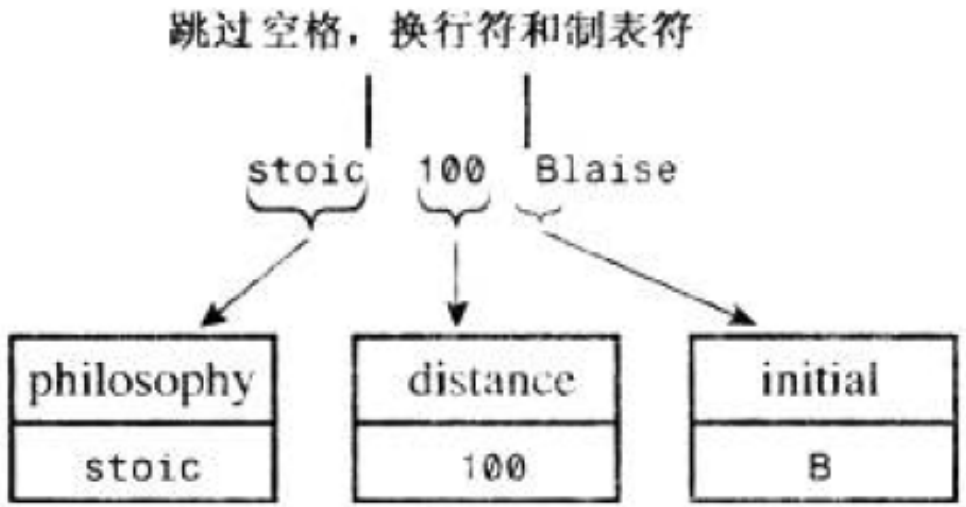
```
#include <iostream>
using namespace std;
int main()
{
   float grade;
   cout << "enter grade:";
   while( cin >> grade ) {
     if ( grade >= 85 && grade <= 100)
          cout << "Good!" << endl;
     if ( grade < 60 )
          cout << "fail!" << endl;
   }                    //循环一直执行，直到输入为非数字格式
   return 0;
}
```

1.2 标准输入流：使用cin进行输入

- cin>>如何检查输入：跳过空白（空格、换行符和制表符），直到遇到非空白字符。

```
char philosophy[20];
int distance;
char initial;

cin >> philosophy >> distance >> initial;
```




```
//注意此例：
int elevation;
cin >> elevation;

假设键盘输入：-123Z
则：elevation = -123
Z留在输入流中，下一个cin语句
从这里开始读取。
```

```
//check_it.cpp -- checking for valid input
#include <iostream>
using namespace std;
int main()
{
      cout << "Enter numbers:";
      int sum = 0;
      int input;
      while ( cin >> input )
      {
          sum += input;
      }
      cout << "Last value entered  = " << input << endl;
      cout << "sum = " << sum << endl;
      return 0;
}
```

```
Enter numbers:200
10 -50 -123Z 60
Last value entered  = 0
sum = 37
```

1.2 标准输入流：使用cin进行输入

使用cin进行输入

- 流状态：

  //primer书P756 表17.4

成员	描述
eofbit	如果到达文件尾，则设置为1
badbit	如果流被破坏，则设置为1
failbit	如果输入操作未能读取预期的字符或输出操作没有写入预期的字符，则设置为1
goodbit	另一种表示0的方法
good()	如果流可以使用（所有的位都被清除），则返回true
eof()	如果eofbit被设置，则返回true
bad()	如果badbit被设置，将返回true
fail()	如果badbit或failbit被设置，则返回true
rdstate()	返回流状态
exceptions()	返回一个位掩码，指出哪些标记导致异常被引发
exceptions(iostate ex)	设置哪些状态将导致clear()引发异常
clear(iostate s)	将流状态设置为s；s的默认值为0；如果(rdstate() & exceptions()) != 0，则引发异常basic_ios::failure
setstate(iostate s)	调用clear(rdstate() | s)。这将设置与s中设置的位对应的流状态位，其他流状态位保持不变

一位
0或1

判断

设置

1.2 标准输入流：使用cin进行输入

- 流状态--设置状态
  - clear()方式将状态设置为它的参数：

```
clear(); //默认参数0，清除全部3个状态位（eofbit，badbit，failbit）
clear(eofbit);        //将状态设置为eofbit，另外两个状态位清除
```

  - setstate()方法只影响其参数中已设置的位，而不会影响其他位：

```
setstate(eofbit);    //只设置eofbit，不影响其他位
```

1.2 标准输入流：使用cin进行输入

- 流状态—I/O和异常
  - exceptions() 方法用来控制异常如何被处理
  - exceptions()方法返回一个位字段，它包含3位，分别对应于eofbit、failbit和badbit，指出哪些标记导致异常被引发
  - 修改流状态后，clear()方法将当前的流状态与exceptions()返回的值进行比较。如果在返回值中某一位被设置，而当前状态中的对应位也被设置，则clear()将引发ios_base::failure异常，该异常包含what()方法
  - 如果exceptions()返回goodbit，则不会引发任何异常

1.2 标准输入流：使用cin进行输入

- 流状态—I/O和异常
  - exceptions()的默认设置为goodbit，不会引发异常
  - 重载的exceptions(iostate)函数可以控制其行为：

```
cin.exceptions(badbit);    //setting badbit causes exception
```

  - 位运算符|使得能够指定多位：

```
cin.exceptions(badbit | eofbit);
```

```
// cinexcp.cpp -- having cin throw an exception
#include <iostream>
#include <exception>
using namespace std;
int main()
{
    // have failbit cause an exception to be thrown
    cin.exceptions(ios_base::failbit);
    cout << "Enter numbers: ";
    int sum = 0;
    int input;
    try {
        while (cin >> input)
        {
                sum += input;
        }
    } catch (ios_base::failure& bf)
    {
        cout << bf.what() << endl;
        cout << "0! the horror!\n";
    }
    cout << "Last value entered = "
         << input << endl;
    cout << "Sum = " << sum << endl;
    return 0;
}
```

```
Enter numbers: 20 30 40 pi 6
ios_base::failbit set: iostream stream error
0! the horror!
Last value entered = 40
Sum = 90
```

1.2 标准输入流：使用cin进行输入

- 流状态的影响

```
…
while ( cin >> input )
{
    sum += input;
}
…
cout << "Now enter a new number:";
cin >> input;  //won't work
```

//流状态位被设置

方法1：一直读取字符，直到到达空白

```
…
while ( cin >> input )
{
    sum += input;
}
…
cout << "Now enter a new number:";
cin.clear();  //reset stream state
while (!isspace(cin.get()))
    continue; //get rid of bad input
cin >> input;  //will work now
```

//将流状态重置，但不匹配输入仍在输入队列，必须跳过

1.2 标准输入流：使用cin进行输入

- 流状态的影响

```
…
while ( cin >> input )
{
    sum += input;
}
…
cout << "Now enter a new number:";
cin >> input;  //won't work
```

//流状态位被设置

```
…  方法2：丢弃行中的剩余部分
while ( cin >> input )
{
    sum += input;
}
…
cout << "Now enter a new number:";
cin.clear();  //reset stream state
while (cin.get() != '\n')
    continue; //get rid rest of line
cin >> input;  //will work now
```

//将流状态重置，但不匹配输入仍在输入队列，必须跳过

1.2 使用cin进行输入

- 流状态的影响

循环结束的条件：

1)不恰当输入而终止；//已讨论

2)到达文件尾或者由于硬件故障。

```
…
while ( cin >> input )
{ sum += input;}
…
if (cin.fail() && !cin.eof())
{  //failed because of mismatched input
    cin.clear();  //reset stream state
    while (!isspace(cin.get()))
        continue;  //get rid of bad input
}
else //else fail out
{
    cout << "I cannot go on !\n";
    exit(1);
}
cout << "Now enter a new number:";
cin >> input;  //will work now
```

1.3 cin的成员函数

- 单字符输入

(1)成员函数get(char &)

(2)成员函数get(void)

特征	cin.get(ch)	ch=cin.get()
传输输入字符的方法	赋给参数ch	将函数返回值赋给ch
字符输入时函数的返回值	指向istream对象的引用	字符编码（int值）
达到文件尾时函数的返回值	转换为false	EOF

```
int ct = 0;
char ch;
cin >> ch;
while (ch != '\n')  //Fails
{
    cout << ch;
    ct++;
    cin >> ch;
}
cout << ct <<endl;
//输入 I love C++
//输出 IloveC++  （跳过空格）
//循环不终止！！！
（提取运算符跳过了换行符）
```

```
int ct = 0;
char ch;
cin.get(ch);
while (ch != '\n')
{
    cout << ch;
    ct++;
    cin.get(ch);
}
cout << ct <<endl;
//输入 I love C++
//输出 I love C++10
//程序运行成功！
```

```
char c1,c2,c3;
cin.get(c1).get(c2) >> c3;
```

返回调用对象cin

```
cin.get(c2) >> c3;
```

返回调用对象cin

```
cin >> c3;
```

非空白字符赋给c3

```
char ch;
while (cin.get(ch))
{
    //process input
}
```

特征	cin.get(ch)
传输输入字符的方法	赋给参数ch
字符输入时函数的返回值	指向istream对象的引用
达到文件尾时函数的返回值	转换为false

```
int ct = 0;
char ch;
cin.get(ch);
while (ch != '\n')
{
    cout << ch;
    ct++;
    cin.get(ch);
}
cout << ct <<endl;
//输入 I love C++
//输出 I love C++10
//程序运行成功!
```

```
int ct = 0;
char ch;
ch = cin.get();
while (ch != '\n')
{
    cout << ch;
    ct++;
    ch = cin.get();
}
cout << ct <<endl;
//输入 I love C++
//输出 I love C++10
//程序运行成功!
```

```
char c1,c2,c3;
cin.get().get() >> c3;
返回int: not valid

char c1;
cin.get(c1).get();  //valid
```

```
int ch;
while ((ch=cin.get())!=EOF)
{
    //process input
}
```

特征	cin.get(ch)	ch=cin.get()
传输输入字符的方法	赋给参数ch	将函数返回值赋给ch
字符输入时函数的返回值	指向istream对象的引用	字符编码（int值）
达到文件尾时函数的返回值	转换为false	EOF

1.3 cin的成员函数

- 字符串输入

(1) cin.get(…)　　//读取一行输入，直到到达换行符，将换行符保留在输入序列

(2) cin.getline(…)　　//读取一行输入，直到到达换行符，并丢弃换行符

```
istream & get(char *, int, char);
                    //第三个参数是分界符，遇到分界符输入停止，并保留在输入序列
istream & get(char *, int);
istream & getline(char *, int, char);
                    //第三个参数是分界符，遇到分界符输入停止，并丢弃分界符
istream & getline(char *, int);
```

1.3 cin的成员函数

- 意外字符串输入

方法	行为
getline(char *, int)	如果没有读取任何字符(换行符视为读取一个字符)，则设置failbit 如果读取了最大数目的字符，且行中还有其他字符，则设置failbit
get(char *, int)	如果没有读取任何字符，则设置failbit

```
char temp[80];

while (cin.get(temp, 80)) //terminates on empty line

while (cin.getline(temp, 80) && temp[0] != '\0')

//terminates on empty line，getline仍将抽取换行符，虽然不存储它
```

1.3 cin的成员函数

- istream类的其它成员函数

```
cin.eof()              //文件结束

cin.peek()             //观测下一个字符

cin.putback(ch)        //将字符ch返回到输入流

cin.ignore(…)          //跳过字符
```

```
//get_fun.cpp—using get() and getline()
#include <iostream>
using namespace std;
const int Limit = 255;
int main()
{
      char input[Limit];
      cout << "Enter a string for getline() processing:\n";
      cin.getline(input, Limit, '#');   //读取直到#，并丢弃#
      cout << "Here is your input:\n";
      cout << input << "\nDone with phase 1\n";
```

//接下页

```
Enter a string for getline() processing:
Please pass
me a #3 melon!
Here is your input:
Please pass
me a
Done with phase 1
```

```
    char ch;
    cin.get(ch); //保留换行符在输入序列
    cout << "The next input character is " << ch << endl;
    if (ch != '\n')
        cin.ignore(Limit, '\n');      // discard rest of line
    cout << "Enter a string for get() processing:\n";
    cin.get(input, Limit, '#');   //读取直到#，并保留#在输入序列
    cout << "Here is your input:\n";
    cout << input << "\nDone with phase 2\n";
    cin.get(ch);  //读取#
    cout << "The next input character is " << ch << endl;
    return 0;
}
```

```
The next input character is 3
Enter a string for get() processing:
I still
want my #3 melon!
Here is your input:
I still
want my
Done with phase 2
The next input character is #
```

```
//peeker.cpp—some istream methods
…
while (cin.get(ch))    // terminates on EOF
{      if (ch != '#') cout << ch;
       else
       {   cin.putback(ch);      // reinsert character
           break;
       }
}
if (!cin.eof())
{      cin.get(ch);
       cout << endl << ch << " is next input character.\n";
}
else
{      cout << "End of file reached.\n";
       exit(0);
}
//未完接下页
```

```
I used a #3 pencil when I should have used a #2.
I used a
# is next input character.
```

```
//接上页
while (cin.peek() != '#')      // look ahead
{      cin.get(ch);
       cout << ch;
}
if (!cin.eof())
{      cin.get(ch);
       cout << endl << ch << " is next input character.\n";
}
else
       cout << "End of file reached.\n";
…
```

```
I used a #3 pencil when I should have used a #2.
I used a
# is next input character.
3 pencil when I should have used a
# is next input character.
```

目录

- 标准输入流
- C++方式的文件操作与文件流
- C++方式的字符串流
- C方式类似字符串流的操作

2.1 文件操作与文件流

- C++对文件的访问：

  低级I/O：字符流方式输入/输出（以字节为单位）

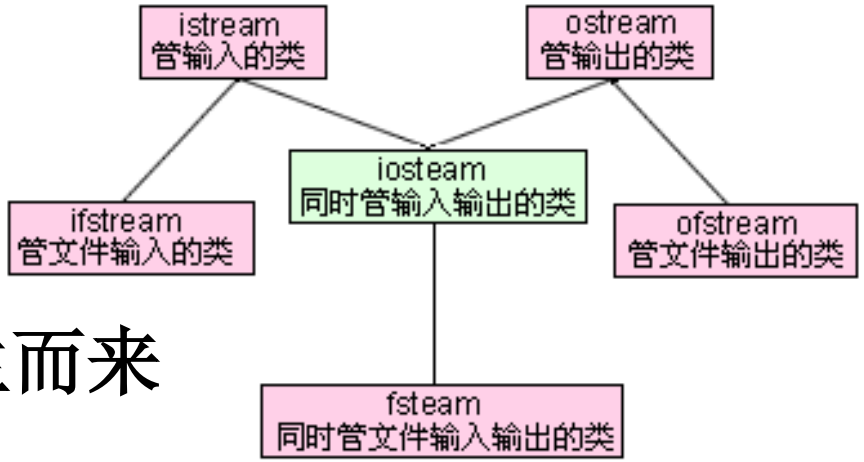
  高级I/O：转换为数据指定形式的输入/输出（若干个字节，有意义）

- 与磁盘文件有关的流类：

  输入：ifstream类，从istream类派生而来

  输出：ofstream类，从ostream类派生而来

  输入/输出：fstream类，从iostream类派生而来



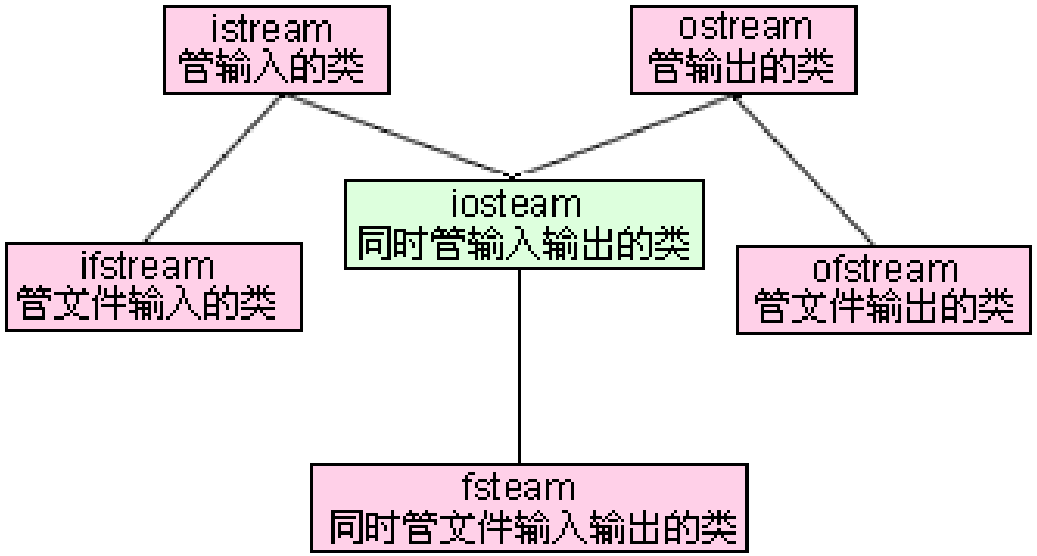
2.1 文件操作与文件流

- 流对象的建立：

ifstream 流对象名：用于输入文件的操作

ofstream 流对象名：用于输出文件的操作

fstream 流对象名：用于输入/输出文件的操作



2.1 文件操作与文件流

- 文件的打开：文件流对象名.open(文件名，打开方式)；
- 加#include <fstream>
- 打开方式

常量	含义
ios::in	打开文件，以便读取
ios::out	打开文件，以便写入。若文件存在，内容全部清除
ios::ate	打开文件，并移到文件尾
ios::app	追加到文件尾
ios::trunc	如果文件存在，则截短文件；如不存在，建立新文件
ios::binary	二进制方式打开文件
ios::_Nocreate	打开已有文件，不建立新文件，如文件不存在，打开失败
ios::_Noreplace	如文件不存在则建立新文件，如文件存在则操作失败

2.1 文件操作与文件流

- 各个打开方式可用"位或运算符|"进行组合，但不能组合互相排斥的方式

```
ios::in|ios::out                //以输入和输出的方式打开文件，文件可读可写
ios::in|ios::binary             //以二进制方式打开一个输入文件
ios::_Nocreate|ios::_Noreplace  //不允许
```

- 文件名允许带全路径，若不带路径，则表示与可执行文件同目录

```
ofstream out;
out.open("aa.dat", ios::out);
out.open("..\\C++\\aa.dat", ios::out);  ..表示父目录
out.open(".\\C++\\aa.dat", ios::out);   . 表示当前目录
out.open("\\C++\\aa.dat", ios::out| ios::app);
out.open("c:\\C++\\aa.dat", ios::out);
```

2.1 文件操作与文件流

- 文件的打开与关闭

➢ 可在声明文件流对象时直接打开

```
ofstream out("aa.dat", ios::out);
```

➢ 打开方式与文件流对象之间要兼容,否则无意义

```
ifstream in;

in.open("aa.dat", ios::out); //in对象用out打开，无意义
```

➢ 每个文件被打开后，都有一个文件指针，初始指向开始/末尾的位置

(根据打开方式决定)

2.1 文件操作与文件流

- 文件的打开与关闭

➢ 判断文件是否成功打开的方法　//注意不同编译器有差别

if (outfile.open("f1.dat",ios::app)==0) if (!outfile.open("f1.dat",ios::app))	open函数返回值
if (outfile==NULL) if (!outfile)	流对象
if (outfile.is_open()==0) if (!outfile.is_open())	打开后判断 is_open函数返回值

➢ 文件的关闭：文件流对象名.close();

```
#include <iostream>
#include <fstream>
using namespace std;
int main()
{
    ofstream out;
    out.open("aa.dat", ios::out);
    if (out.is_open()==0)
       cout << "open failed." << endl;
    else
       cout << "open success." << endl;
    out.close();
    return 0;
}
```

不存在时：成功(创建)

存在时：成功(覆盖)

存在并只读：失败

```
#include <iostream>
#include <fstream>
using namespace std;
int main()
{
    ifstream in;
    in.open("bb.dat", ios::in);
    if (!in.is_open()==0)
        cout << "open failed." << endl;
    else
        cout << "open success." << endl;
    in.close();
    return 0;
}
```

不存在时：失败

存在时：成功

```
#include <iostream>
#include <fstream>
using namespace std;
int main()
{
    ofstream out;
    out.open("bb.dat", ios::out|ios::_Nocreate);
    if (out.is_open()==0)
        cout << "open failed." << endl;
    else
        cout << "open success." << endl;
    out.close();
    return 0;
}
```

不存在时：失败

存在时：成功(覆盖)

存在并只读：失败

2.2 对ASCII文件的操作

基本方法：将文件流对象名当作cin/cout对象，用>>和<<进行格式化的输入和输出

>>和<<使用时的注意事项与cin、cout时相同

```
cin  >> 变量   => infile  >> 变量
cout << 变量   => outfile << 变量
```

成员函数的使用方法与前面相同

```
cout.put('A') => outfile.put('A')
```

```
#include <iostream>
#include <fstream>
#include <cstdlib>
using namespace std;
int main()
{  int a[10];
   ofstream outfile("f1.dat", ios::out);
   if (!outfile.is_open()) {
      cerr << "open error!" << endl; exit(1);
   }
   cout << "enter 10 integer numbers:" << endl;
   for(int i=0; i<10; i++) {
      cin >> a[i];
      outfile << a[i] << " "; //int型输出到文件
   }
   outfile.close();  return 0;
}
```

打开方式若修改方式为：

```
ios::out | ios::app
```

//运行两次（观察文件内容）：

```
1 3 5 2 4 6 10 8 7 9
```

```
1 3 5 2 4 6 10 8 7 9 1 3 5 2 4 6 10 8 7 9
```

//运行两次（观察文件内容）：

```
1 3 5 2 4 6 10 8 7 9
```

```
1 3 5 2 4 6 10 8 7 9
```

```
#include <iostream>
#include <fstream>
#include <cstdlib>
using namespace std;
int main()
{    int a[10], max, i, order;
     ifstream infile("f1.dat", ios::in);
     if (!infile.is_open()) {
        cerr << "open error!" << endl; exit(1);
     }
     for(i=0; i<10; i++) {
        infile >> a[i];      //从文件中读10个int放入a数组
        cout << a[i] << " "; //int型输出到屏幕
     }
     //找最大值...
     infile.close();  return 0;
}
```

```
//打开文件并将内容输出到屏幕上
#include <iostream>
#include <fstream>
using namespace std;
int main()
{ ifstream in;
  char ch;
  in.open("d:\\test\\data.txt", ios::in);
  if (in.is_open()==0) {
       cout << "文件打开失败\n";        //cerr
       return -1;                       //exit(1)
  }
  while(!in.eof()) {
       ch = in.get();          //in.get(ch);      while((ch=in.get())!=EOF)
       putchar(ch);            //cout.put(ch);        cout.put(ch);
  }
  in.close();return 0;
}
```

```
//复制文件内容至另一个文件
#include <iostream>
#include <fstream>
using namespace std;
int main()
{   ifstream in;  ofstream out;  char ch;
    in.open("d:\\test\\data.txt", ios::in);
    if (!in.is_open()) {
       cout << "无法打开源文件" << endl;  return -1;
    }
    out.open("d:\\demo\\data2.txt", ios::out);
    if (!out.is_open()) {
       cout << "无法打开目标文件" << endl; in.close(); return -1;
    }
    while(in.get(ch))     //从输入文件中一次读取
       out.put(ch);   //写入输出文件中
    in.close();out.close(); return 0;
}
```

2.3 对二进制文件的操作

- 用ASCII文件的字符方式进行操作(按字节读写)
  - 用read/write进行操作

文件流对象名.read(内存空间首指针，长度);

从文件中读长度个字节，放入从首指针开始的空间中

文件流对象名.write(内存空间首指针，长度);

将从首指针开始的连续长度个字节写入文件中

```
//ASCII方式
#include <iostream>
#include <fstream>
#include <cstdlib>
using namespace std;
struct student {
   char name[20];int num;int age;char sex;
};
int main()
{  student stud[3]={"Li",1001,18,'f', "Fun",1002,19,'m',"Wang",1004,17,'f'};
   ofstream outfile("stud.dat", ios::out);
   if (!outfile.is_open()) {
      cerr << "open error!" << endl;  exit(-1);
   }
   for(int i=0; i<3; i++)
      outfile<<stud[i].name<<stud[i].num<<stud[i].age<<stud[i].sex<<endl;
   outfile.close();
   return 0;
}
```

生成的stud.dat文件共36字节：

```
Li100118f
Fun100219m
Wang100417f
```

```
//二进制文件方式
#include <iostream>
#include <fstream>
#include <cstdlib>
using namespace std;
struct student {
   char name[20];int num;int age;char sex;
};
int main()
{  student stud[3]={"Li",1001,18,'f', "Fun",1002,19,'m',"Wang",1004,17,'f'};
   ofstream outfile("stud.dat", ios::binary);
   if (!outfile.is_open()) {
      cerr << "open error!" << endl;
      exit(-1);
   }
   for(int i=0; i<3; i++)
      outfile.write((char *)&stud[i], sizeof(stud[i]));
   outfile.close();  return 0;
}
```

生成的stud.dat文件共96字节：

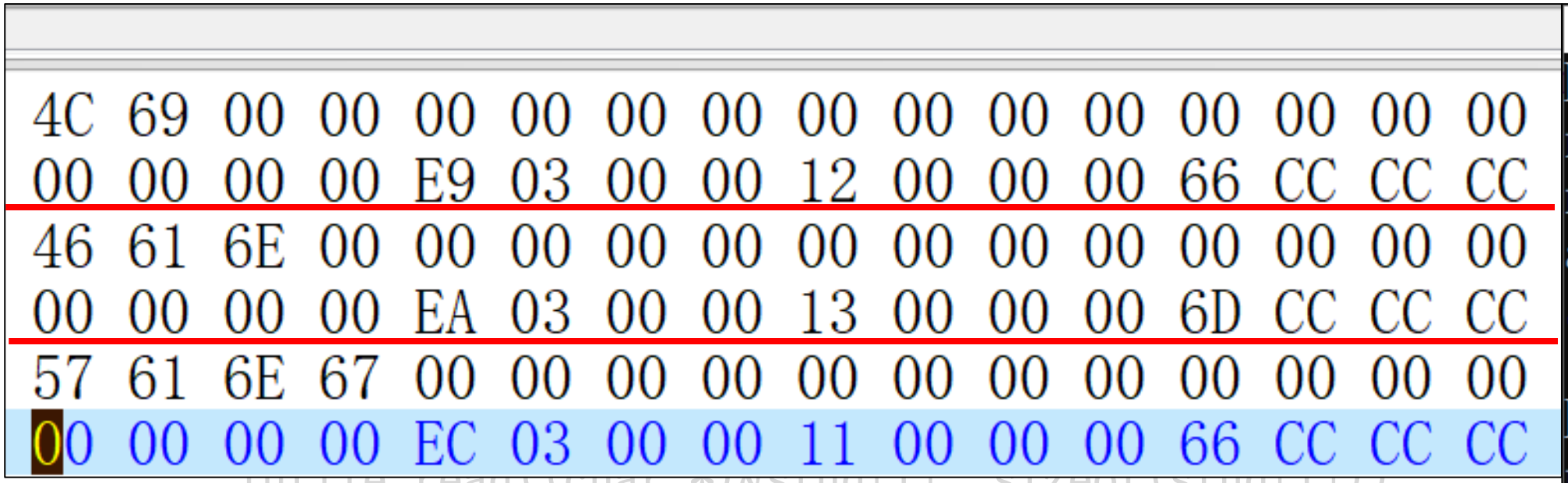
```
4C 69 00 ?? ?? ?? ?? ??
?? ?? ?? ?? ?? ?? ?? ??
?? ?? ?? ?? E9 03 00 00
12 00 00 00 66 ?? ?? ??
…
```

前32字节

```
4C6900   => "Li"(含尾零)
E9030000 => 0x000003E9 => 1001
12000000 => 0x00000012 => 18
66       => 'f'(后3个是填充字节)
```

```
//二进制文件方式
int main()
{   student stud[3];
    int i;
    ifstream infile("stud.dat", ios::binary);
    if (!infile.is_open()) {
        cout << "文件打开失败" << endl; return -1;
    }
    for(i=0; i<3; i++)  //一次读入一个数组元素（32字节）
        infile.read((char *)&stud[i], sizeof(stud[i]));
    infile.close();
    for(i=0; i<3; i++) {
        cout << "No." << i+1 << endl;
        cout << "name:" << stud[i].name << endl;
        cout << "num:" << stud[i].num << endl;
        cout << "age:" << stud[i].age << endl;
        cout << "sex:" << stud[i].sex << endl << endl; //多空一行
    }
    return 0;
}
```

//stud.dat的内容是前例生成的二进制文件




```
    infile.close();
    for(i=0; i<3; i++) {
        cout << "No." << i+1 << endl;
        cout << "name:" << stud[i].name << endl;
        cout << "num:" << stud[i].num << endl;
        cout << "age:" << stud[i].age << endl;
        cout << "sex:" << stud[i].sex << endl << endl; //多空
    }
    return 0;
}
```

Microsoft Visual Studio 调试控制台

```
No.1
name:Li
num:1001
age:18
sex:f

No.2
name:Fan
num:1002
age:19
sex:m

No.3
name:Wang
num:1004
age:17
sex:f
```

2.4 对二进制文件的操作

- 与文件指针有关的流成员函数
  - 适用于输入文件的：

    gcount()：返回最后一次读入的字节数

    tellg()：返回输入文件的当前指针

    seekg(位移量，参照位置)：移动输入文件指针
  - 适用于输出文件的：

    tellp()：返回输出文件的当前指针

    seekp(位移量，参照位置)：移动输出文件指针

2.4 对二进制文件的操作

- 与文件指针有关的流成员函数
  - 参照位置：

    ios::beg：从文件头部移动，位移量必须为正

    ios::cur：从当前指针处移动，位移量可正可负

    ios::end：从文件尾部移动，位移量必须为负

  - 随机访问二进制数据文件

    在文件的读写过程中，可前后移动文件指针，达到按需读写的目的

```
//随机访问二进制文件
int main()
{   student stud[3]={"Li",1001,18,'f',
                "Fan",1002,19,'m',"Wang",1004,17,'f'};
    ofstream outfile("stud.dat", ios::binary);
    if (!outfile.is_open()) {
        cout << "文件打开失败" << endl;
        return -1;
        }
    for(int i=0; i<3; i++) { //一次写入一个数组元素
        outfile.write((char *)&stud[i],sizeof(stud[i]));//从当前位置写32字节
        cout << outfile.tellp() << endl;  //写完32字节后，文件指针为32
        outfile.seekp(0, ios::beg);       //文件指针移回0
        cout << outfile.tellp() << endl;  //文件指针为0
        }
    outfile.close();
    return 0;
}
```

```
#include <iostream>
#include <fstream>
using namespace std;
struct student {
    char name[20];
    int  num;
    int  age;
    char sex;
}; //含填充共32字节
```

Microsoft Visual Studio 调试控制台

```
32
0
32
0
32
0
```

2.5 关于二进制访问的几个注意事项

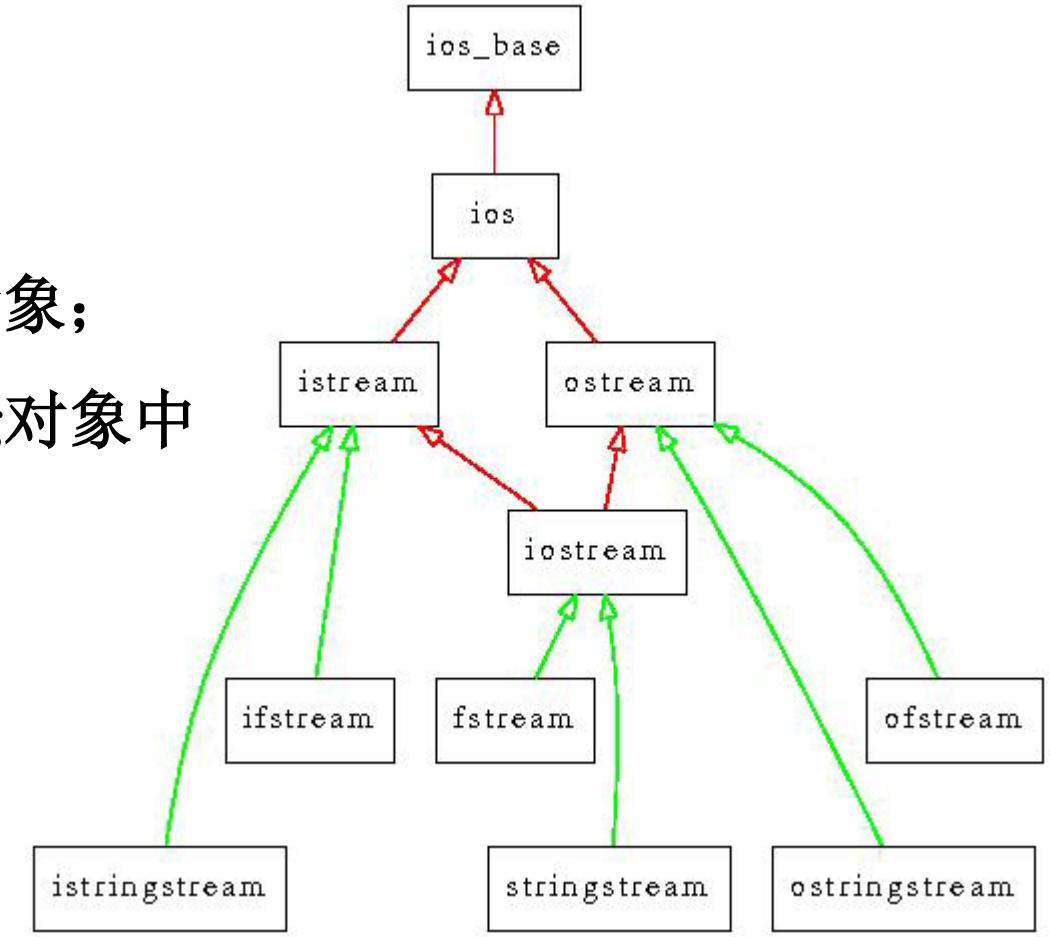
- read参数中的长度是最大读取长度，不是实际读取长度，因此read后要用gcount()返回真实读到的字节数
- 如果读写方式打开(ios::in | ios::out)，则只有一个文件指针，seekg()和seekp()是同步的，tellg()和tellp()也是同步的
- 在文件的操作超出正常范围后(例：read()已到EOF、seekg()/seekp()超文件首尾范围等)，再次对文件进行seekg()/seekp()/tellg()/tellp()等操作都可能会返回与期望不同的值，建议在文件操作过程中勤用good()/fail()/eof()/clear()等函数

目录

- 标准输入流
- C++方式的文件操作与文件流
- C++方式的字符串流
- C方式类似字符串流的操作

3.1 基本概念

- iostream族：程序与终端之间的I/O；
- fstream族：程序与文件之间的I/O；
- sstream族：程序与string对象之间的I/O。
  - ➢ostream方法将格式化信息写入到string对象；
  - ➢istream方法（如getline()）来读取string对象中的信息。
    - ⟹内核格式化（incore formatting）



3.1 基本概念

- 以内存中的string类型变量为输入/输出对象
  - 可以存放各种类型的数据
  - 与标准输入输出流相同，进行文本和二进制之间的相互转换

    向string存数据　　cout：二进制 => ASCII

    从string取数据　　cin ：ASCII => 二进制

    推论：可用于不同数据类型的转换
  - 不是文件，不需要打开和关闭

3.2 字符串流对象的使用-写入

- 加 #include <sstream>
- 创建ostringstream类对象(输出流对象)

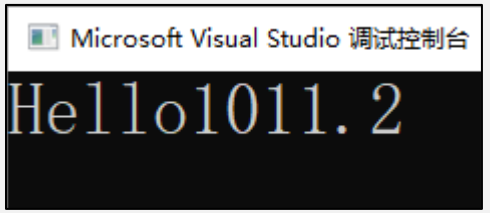
```
ostringstream outstr;
double price = 380.0;
const char* ps = " for a copy of the ISO/EIC C++ standard!";
outstr.precision(2);
outstr << fixed;
outstr << "Play only CHF" << price << ps << endl;
```

- ostringstream类的成员函数str()，返回一个被初始化为缓冲区内容的字符串对象

```
string mesg = outstr.str();
```

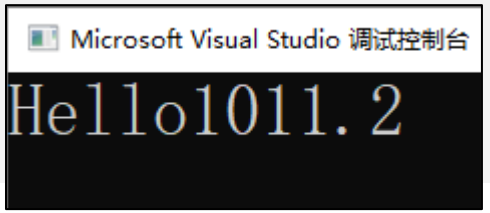
```
Play only CHF380.00 for a copy of the ISO/EIC C++ standard!
```

```
//例1：输出流对象的基本使用：将多个格式化内容拼在一起，集中输出
#include <iostream>
#include <sstream>
using namespace std;
int main()
{   ostringstream out;
    out << "Hello" << 10 << 11.2 << endl;
    string s1 = out.str();
    cout << s1 << endl;
    return 0;
}
```



成员函数str()的作用：
将ostringstream的内容转换为string格式

```
ostringstream out;
out << "Hello" << 10 << 11.2 << endl;
cout << out.str() << endl; //跟上例等价
```



```
//例2：内核格式化 strout.cpp  -  incore formatting (output)
#include <iostream>
#include <sstream>
#include <string>
using namespace std;
int main()
{   ostringstream outstr;
    string hdisk;
    cout << "What's the name of your hard disk? ";
    getline(cin, hdisk);              // cin -> hdisk

    int cap;
    cout << "What's its capacity in GB? ";
    cin >> cap;
    // write formatted information to string stream   //(格式化信息 -> string对象)
    outstr << "The hard disk " << hdisk << " has a capacity of " << cap << " gigabytes.\n";
    string result = outstr.str();  //save result
    cout << result;                //show contents
    return 0;
}
```

读取string对象中的格式化信息或将格式化信息写入string对象中被称为内核格式化(incore formatting)

```
What's the name of your hard disk? YChen
What's its capacity in GB? 20
The hard disk YChen has a capacity of 20 gigabytes.
```

3.3 字符串流对象的使用-读取

- 加 #include <sstream>
- 创建istringstream类对象(输入流对象)，并使用string对象初始化

```
istringstream instr(facts);      //use facts to initialize stream
int n;
int sum = 0;
while (instr >> n)
      sum += n;
```

```
//例1：内核格式化 strin.cpp  -  formatted reading from a char array
#include <iostream>
#include <sstream>
#include <string>
using namespace std;

int main()
{
    string lit = "It was a dark and stormy day, and "
                 " the full moon glowed brilliantly. ";
    istringstream instr(lit); //use buf for input lit->instr
    string word;
    while (instr >> word)  //read a word a time
        cout << word << endl;

    return 0;
}
```

```
It
was
a
dark
and
stormy
day,
and
the
full
moon
glowed
brilliantly.
```

```
//例2：输入流对象-流状态
int main()
{
    istringstream in("Hello 10 11.2");
    cout << in.str() << endl;

    char s[10];
    short i;
    float f;
    in >> s >> i >> f;
    cout << s << '-' << i << '-' << f << endl;

    cout << in.good() << ' ' << in.fail() <<endl; //good和fail均为0
    //如果现有内容全部读完，goodbit会置0，此时good和fail均为0，表示EOF
    cout << in.str() << endl;
    return 0;
}
```



```
//例3：输入流对象-流状态
int main()
{
    istringstream in("Hello 10 11.2 xyz");
    cout << in.str() << endl;

    char s[10];
    short i;
    float f;
    in >> s >> i >> f; //读之后内容仍在，可用str()查看
    cout << s << '-' << i << '-' << f << endl;

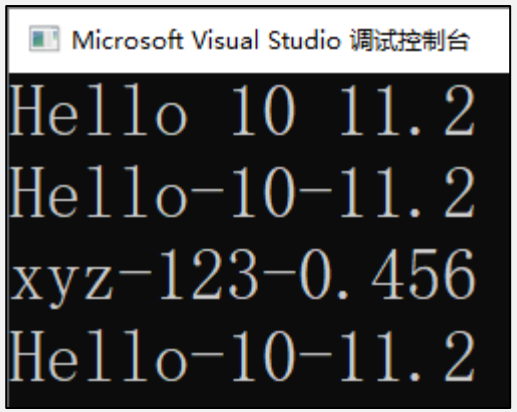
    cout << in.good() << ' ' << in.fail() <<endl; //good为1，fail为0
    //现有内容尚未读完，goodbit为1，failbit为0
    cout << in.str() << endl;   //可用str()打印现有内容
    return 0;
}
```



```
//例4：输入流对象-重复读取
int main()
{
    istringstream in("Hello 10 11.2");
    cout << in.str() << endl;

    char s1[10], s2[10]="xyz";
    short i1, i2=123;
    float f1, f2=0.456F;
    in >> s1 >> i1 >> f1;
    cout << s1 << '-' << i1 << '-' << f1 << endl;
    cout << s2 << '-' << i2 << '-' << f2 << endl;

    in.clear();
    in.seekg(0, ios::beg);
    in >> s2 >> i2 >> f2;  // istringstream的内容可重复读取
    cout << s2 << '-' << i2 << '-' << f2 << endl;
    return 0;
}
```



```
//例5：输入流对象-数据超范围
int main()
{   istringstream in("Hello 70000 11.2");
    char  s[10];
    short i;
    float f;
    in >> s;
    cout << in.good() << endl;
    in >> i;
    cout << in.good() << endl;
    //in.clear();
    in >> f;
    cout << s << '-' << i << '-' << f << endl;
    return 0;
}
```



加入in.clear();



如果数据超范围，后续会错，可clear恢复

```
//例6：输入流对象-使用带参str()再次赋新内容
int main()
{
    istringstream in("Hello 10 11.2");
    char s[10];
    short i;
    float f;
    in >> s >> i >> f;
    cout << s << '-' << i << '-' << f << endl;
    cout << in.good() << endl;
    in.str("tongji 123 0.123");
    in >> s >> i >> f;
    cout << s << '=' << i << '=' << f << endl;
    return 0;
}
```



"=10=11.2"是因为读s出错，s置空，i/f未变

```
//例7：输入流对象-使用带参str()再次赋新内容
int main()
{
    istringstream in("Hello 10 11.2 12345");
    char s[10];
    short i;
    float f;
    in >> s >> i >> f;
    cout << s << '-' << i << '-' << f << endl;
    cout << in.good() << endl;
    in.str("tongji 123 0.123");
    in >> s >> i >> f;
    cout << s << '=' << i << '=' << f << endl;
    return 0;
}
```



可用带参str()再次赋新内容，但注意goodbit

3.4 字符串流对象的使用-读取与写入

- 加 #include <sstream>
- 创建stringstream类对象（输入/输出流对象）
- 使用总结：
  - 存储形式为string，不需要用户考虑空间
  - 使用方式同iostream/fstream基本相似(部分细节可能不同)
  - 如果结果与预期不同，多判断good()/fail()

```
//例1：字符串流对象
int main()
{   stringstream ss("Hello 10 11.2");
    char  s[10];
    short i;
    float f;
    ss >> s >> i >> f;
    cout << s << '-' << i << '-' << f << endl;
    cout << ss.tellg() << endl;

    ss << "xyz 123 0.456";
    cout << ss.str() << endl;
    cout << ss.tellg() << endl;



    return 0;
}
```

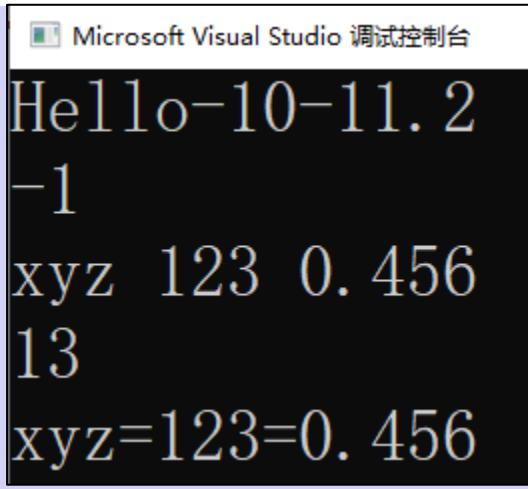


stringstream可读可写，但注意goodbit

```
//左侧相应位置加入下述程序：
ss.clear();



ss.seekg(0, ios::beg);
ss >> s >> i >> f;
cout << s << '=' << i << '=' << f << endl;
```

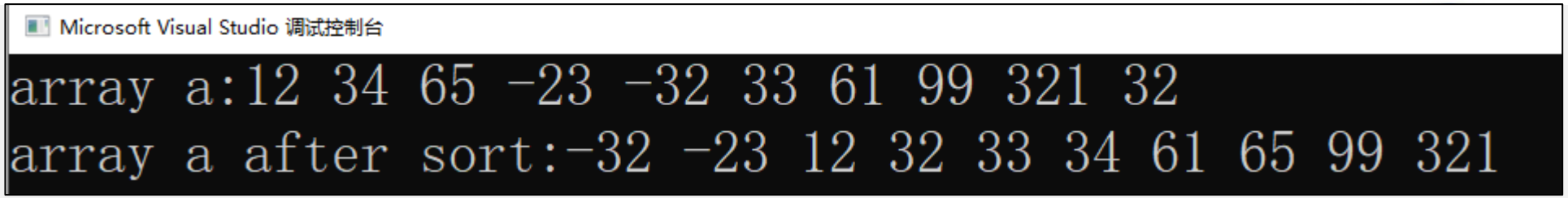


```
//例2：从sstream中读入10个数并排序
#include <iostream>
#include <sstream>
using namespace std;
#define N  10
int main()
{
    stringstream ss("12 34 65 -23 -32 33 61 99 321 32");
    int a[N], i, j, t;

    for (i = 0; i < N; i++)
        ss >> a[i];  //ss中的内容逐个读入int a[10]中

    cout << "array a:";
    for (i = 0; i < N; i++)  //输出int a[10]的内容
        cout << a[i] << " ";
    cout << endl;
```

```
int main()
{   ...
    //冒泡排序
    for (i = 0; i < N-1; i++)
        for (j = 0; j < N - 1 - i; j++)
            if (a[j] > a[j + 1]) {
                t = a[j];
                a[j] = a[j + 1];
                a[j + 1] = t;
            }
    //输出到ss中（ss刚才用做了输入流）
    ss.clear();
    ss.seekg(0, ios::beg);
    for (i = 0; i < N; i++)
        ss << a[i] << " ";
    ss << endl;
    cout << "array a after sort:" << ss.str() << endl;
}
```

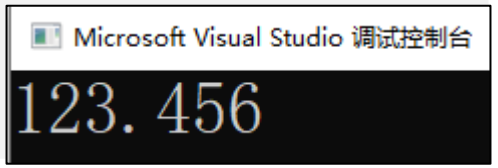


3.5 字符串流对象的应用

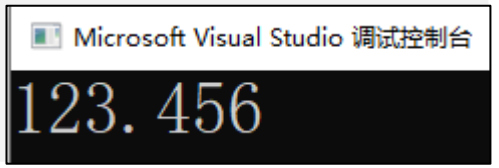
- 实现不同数据类型的转换
  - 字符串转double
  - double转字符串
  - 多种类型转字符串（重载方式）
- 实现字符串串接

- 应用1：实现不同数据类型的转换

```
//字符串转double
#include <iostream>
#include <sstream>
using namespace std;
int main()
{
    istringstream in("123.456");
    double d;
    in >> d;
    cout << d << endl;
    return 0;
}
```



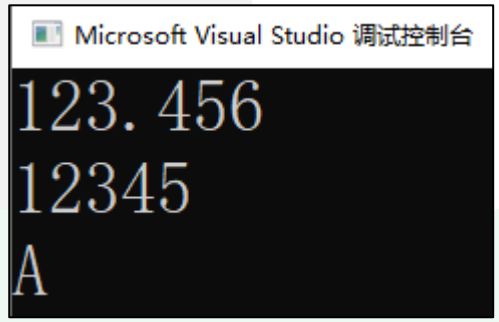

```
//double转字符串
#define _CRT_SECURE_NO_WARNINGS
#include <iostream>
#include <sstream>
using namespace std;
int main()
{   ostringstream out;
    double d = 123.456;
    char str[10];
    out << d;
    strcpy(str, out.str().c_str());
    cout << str << endl;
    return 0;
}
```



- 应用1：实现不同数据类型的转换

```
//多种类型转字符串(重载方式)
string tj_to_string(const double d)
{   ostringstream out;
    out << d;
    return out.str();
}
string tj_to_string(const int i)
{   ostringstream out;
    out << i;
    return out.str();
}
string tj_to_string(const char ch)
{   ostringstream out;
    out << ch;
    return out.str();
}
```

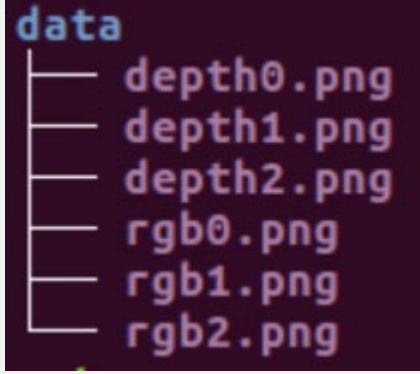
```
int main()
{
    string s1 = tj_to_string(123.456);
    string s2 = tj_to_string(12345);
    string s3 = tj_to_string('A');
    cout << s1 << endl;
    cout << s2 << endl;
    cout << s3 << endl;
}
```



- 应用2：实现字符串串接

```
//stringstream.cpp
#include <iostream>
#include <fstream>
#include <sstream>
#include <opencv2/core/core.hpp>
#include <opencv2/highgui/highgui.hpp>
using namespace std;
using namespace cv;
void readFrame(int index);
int main()
{
    for (int i = 0; i < 3; i++)
    {   readFrame(i);
    }
    return 0;
}
```

```
void readFrame(int index)
{   string dir = "./data/";
    string rgb = "rgb";
    string dep = "depth";
    string Ext = ".png";
    stringstream ss;
    ss << dir << rgb << index << Ext;  // ./data/rgb1.png
    string filename;
    ss >> filename;
    Mat color = imread(filename,1);    //rgb
    imshow(filename, color);
    ss.clear();
    ss << dir << dep << index << Ext;  // ./data/depth1.png
    filename.clear();
    ss >> filename;
    Mat depth = imread(filename,1);    //depth
    imshow(filename, depth);
    waitKey();
}
```



将字符串类型（rgb）和整数类型(index)拼起来,index不断更新，从而读取不同下标的文件

目录

- 标准输入流
- C++方式的文件操作与文件流
- C++方式的字符串流
- C方式类似字符串流的操作

4.1 向字符串输出格式化的数据

int sprintf(字符数组，"格式串"，输出表列);

① 返回值是输出字符的个数

指不同类型数据按格式串的要求转换为文本方式后字符的个数

② 与printf相同，完成二进制向ASCII的转换

③ VS下需加 #define _CRT_SECURE_NO_WARNINGS

4.1 向字符串输出格式化的数据

int sprintf(字符数组，"格式串"，输出表列);

```
#define _CRT_SECURE_NO_WARNINGS
#include <iostream>
using namespace std;
int main()
{   char c[80];
    int ret;
    ret = sprintf(c, "Hello%d%.1f", 10, 11.2);
    printf("%s\n", c);
    printf("ret=%d\n", ret);
    return 0;
}
```




```
//C++方式
int main()
{   ostringstream out;
    out << "Hello" << 10 << 11.2 << endl;
    cout << out.str() << endl;
    return 0;
}
```

```
//多次向字符数组输出
#define _CRT_SECURE_NO_WARNINGS
#include <iostream>
#include <strstream>
using namespace std;
struct student {
    int num; char name[20]; float score;
};
int main()
{
    student stud[3]={1001,"Li",78,1002,"Wang",89.5,1004,"Fun",90};
    char c[50], *s = c;
    for (int i=0; i<3; i++)
        s+=sprintf(s, "%d %s %.1f", stud[i].num , stud[i].name, stud[i].score);
    cout << "array c:" << c << endl;
    return 0;
}
```

```
array c:1001 Li 78.01002 Wang 89.51004 Fun 90.0
```

4.2 从字符串中输入格式化的数据

int sscanf(字符数组，"格式串"，输入表列);

① 返回值是正确读入的输入数据的个数

② 与scanf相同，完成ASCII向二进制的转换

③ VS下需加 #define _CRT_SECURE_NO_WARNINGS

4.2 从字符串中输入格式化的数据

int sscanf(字符数组，"格式串"，输入表列);

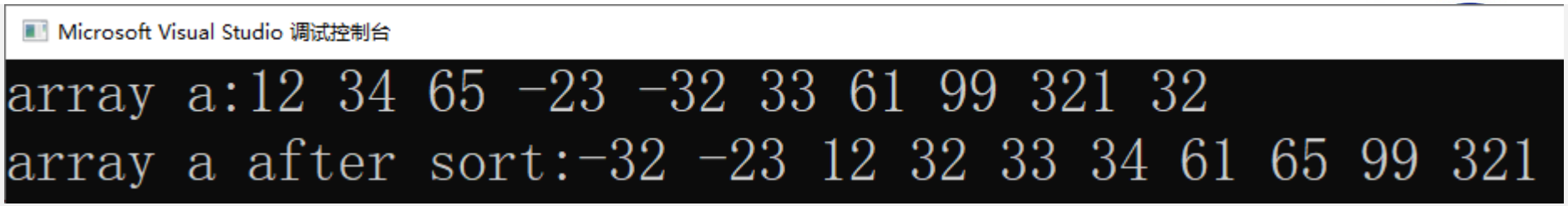
```
#define _CRT_SECURE_NO_WARNINGS
#include <iostream>
using namespace std;
int main()
{   char c[80] = "Hello 10 11.2";
    char s[10];
    int i, ret;
    float f;
    ret = sscanf(c, "%s %d %f", s, &i, &f);
    printf("%s-%d-%.1f\n", s, i, f);
    printf("ret=%d\n", ret);
    return 0;
}
```

```
//C++方式
int main()
{   istringstream in("Hello 10 11.2");
    char s[10];
    int i; float f;
    in >> s >> i >> f;
    cout << s << '-' << i << '-' << f << endl;
    return 0;
}
```



```
//例：从字符串中读入10个数并排序
#define _CRT_SECURE_NO_WARNINGS
#include <stdio.h>
#include <string.h>
#define N 10
int main()
{   char ss[80] = "12 34 65 -23 -32 33 61 99 321 32", *s = ss;
    int a[N], i, j, t;
    //ss中的内容逐个读入int a[10]中
    for (i = 0; i < N; i++) {
        sscanf(s, "%d", &a[i]);
        s = strchr(s, ' '); //在串s中查找给定字符' '的第一个匹配之处
        s++; //指向空格后的字符
    }
    printf("array a:");
    for (i = 0; i < N; i++)  //输出int a[10]的内容
        printf("%d ", a[i]);
    printf("\n");
```

```
int main()
{   ...
    //进行排序
    for (i = 0; i < N - 1; i++)
        for (j = 0; j < N - 1 - i; j++)
            if (a[j] > a[j + 1]) {
                t = a[j];
                a[j] = a[j + 1];
                a[j + 1] = t;
            }
    //输出到ss中（ss刚才用做了输入流）
    s = ss; //重新指向ss[0]
    for (i = 0; i < N; i++)
        s+=sprintf(s, "%d ", a[i]);
    s+=sprintf(s, "\n");
    printf("array a after sort:%s\n", ss);
    return 0;
}
```



总结

- 标准输入流
- C++方式的文件操作与文件流（熟练）
- C++方式的字符串流（熟练）
- C方式类似字符串流的操作（了解）

第三章 动态内存申请与释放

主讲教师：同济大学计算机科学与技术学院 陈宇飞
同济大学计算机科学与技术学院 龚晓亮

目录

- 动态内存的基本概念
- C中的相关函数
- C++中的相关运算符
- 内存的动态申请与释放
- 定位new运算符

1.1 动态内存的基本概念

- 编译器使用三块独立的内存
  - 静态变量
  - 自动变量
  - 动态存储
- 内存的分配
  - 静态存储区：编译时已分配好，内存在程序运行期间都存在（全局变量）
  - 栈：执行函数时，函数内局部变量的存储单元在栈上创建，函数执行结束时自动被释放，效率高，但内存容量有限（局部变量）
  - 堆：动态内存分配，可申请任意大小，用后自己负责释放。生存期自己决定，使用灵活，容易出错！

1.1 动态内存的基本概念

- 使用C函数malloc()等，或者C++运算符分配的内存
  - 不受作用域和链接性规则控制，因此可以在一个函数中分配动态内存，在另一个函数中将其释放
  - 动态内存不是LIFO，其分配和释放由new/delete或malloc()/free()完成

目录

- 动态内存的基本概念
- C中的相关函数
- C++中的相关运算符
- 内存的动态申请与释放
- 定位new运算符

2.1 C中的相关函数

- void *malloc(unsigned size);
  - 申请size字节的连续内存空间，返回该空间首地址，对申请到的空间不做初始化操作
  - 如果申请不到空间，返回NULL
  - 申请0字节内存，函数并不返回NULL，而是返回一个正常的内存地址。注意：无法使用这块大小为0的内存，而且此时无法用if（NULL！=p）校验
- void *calloc(unsigned n, unsigned size);
  - 申请n*size字节的连续内存空间，返回该空间首地址，对申请到的空间做初始化为 0（\0）
  - 如果申请不到空间，返回NULL

2.1 C中的相关函数

- void *realloc(void *ptr, unsigned newsize);
  - 表示为指针ptr重新申请newsize大小的空间
  - ptr必须是malloc/calloc/realloc返回的指针
  - 新老空间可重合，也可能不重合，若不重合，原空间原有内容会被复制到新空间，再释放原空间
  - 对申请到的空间不做初始化操作
  - 若申请不到，则返回NULL

2.1 C中的相关函数

- void free(void *p);
  - 释放p所指的内存空间

(p必须是malloc/calloc/realloc返回的首地址)

用malloc/calloc等申请的空间，用free释放

因为是系统库函数，需要包含头文件

```
#include <stdlib.h> //C方式
#include <cstdlib> //C++方式
```

目录

- 动态内存的基本概念
- C中的相关函数
- C++中的相关运算符
- 内存的动态申请与释放
- 定位new运算符

3.1 C++中的相关运算符

- 用 new 运算符申请空间

  如果申请不到空间，new缺省会抛出bad_alloc异常，需要使用try-catch方式处理异常；也可以在new时加nothrow来强制禁用抛出异常并返回NULL

  - try-throw-catch称为C++的异常处理机制，后面再介绍

- 用 delete 运算符释放空间

  用new申请的空间，用delete释放

  因为是运算符，不需要包含头文件

3.1 C++中的相关运算符

- 运算符new和new[]分别调用如下函数（分配函数）：

```
void * operator new(std::size_t); //used by new
void * operator new[](std::size_t); //used by new[]
```

- 运算符delete和delete[]分别调用如下函数（释放函数）：

```
void * operator delete(void *);
void * operator delete[](std::size_t);
```

- 使用运算符重载（后续内容），std::size_t是一个typedef，对应于合适的整型

3.1 C++中的相关运算符

- 举例：

  int *pi=new int; 实际被转换成int *pi=new(sizeof(int))

  int *pa=new int[40]; 实际被转换成int *pa=new(40*sizeof(int))

  delete pi; 实际被转换成delete (pi)函数调用

- 说明：

new和delete可以根据需要进行定制（因为是可替换函数）。例如：定义作用域为类的替换函数，满足该类的内存分配需求。代码实现时仍然使用new运算符，但实际调用自定义的new()函数

目录

- 动态内存的基本概念
- C中的相关函数
- C++中的相关运算符
- 内存的动态申请与释放
- 定位new运算符

4.1 内存的动态申请与释放（普通变量）

申请对象	C的函数方式	C++的运算符

普通变量	形式1：先定义指针变量，再申请
	int *p;
	p = (int *)malloc(sizeof(int));
	p = (int *)calloc(1,sizeof(int));
		形式1：先定义指针变量，再申请
		int *p;
		p=new int;

	形式2：定义指针变量的同时申请
	int *p = (int *)malloc(sizeof(int));
	int *p = (int *)calloc(1, sizeof(int));
		形式2：定义指针变量的同时申请
		int *p=new int;

	说明：虽然初次申请时也可以用 p = (int *)realloc(NULL, sizeof(int)); 但一般不用	形式3：申请空间时赋初值 int *p; p=new int(10); 或 int *p=new int(10);

4.2 内存的动态申请与释放（一维数组）

申请对象	C的函数方式

一维数组	形式1：先定义指针变量，再申请
	int *p;
	p = (int *)malloc(10*sizeof(int));
	p = (int *)calloc(10, sizeof(int));

	形式2：定义指针变量的同时申请空间
	char *name = (char *)malloc(10*sizeof(char));
	char *name = (char *)calloc(10, sizeof(char));

	说明： 虽然初次申请时也可以用 p = (int *)realloc(NULL, 10*sizeof(int)); 但一般不用

4.2 内存的动态申请与释放（一维数组）



申请对象	C++的运算符
一维数组	形式1：先定义指针变量，再申请 int *p; p = new int[10]; //申请10个int型空间
	形式2：定义指针变量的同时申请空间 char *name=new char[10]; //申请10个char
	形式3：申请空间时赋初值 • 动态申请的一维数组可以在申请时赋初值，方法为后面跟{} {}前不要加=，且[]内必须有数，其余规则同一维数组定义时初始化

4.2 内存的动态申请与释放（一维数组）

- 动态申请的一维数组可以在申请时赋初值，方法为后面跟{}，{}前不要加=，且[]内必须有数，其余规则同一维数组定义时初始化

```
例：int *p;
p = new int[5] {1,2,3,4,5}; //正确
p = new int[5] {1,2,3,4,5,6}; //错误
p = new int[5] {1,2}; //后面自动为0
p = new int[5]={1,2,3,4,5}; //错误
p = new int[] {1,2,3,4,5}; //错误
char *s;
s = new char[5]{'H','e','l','l','o' };//正确
s = new char[5] {"Hello"}; //错误
s = new char[6] {"Hello"}; //正确
```

4.3 内存的动态申请与释放（二维数组）



申请对象	C的函数方式

二维数组	形式1：先定义指针变量，再申请
	int (*p)[4];
	p = (int (*)[4])malloc(3*4*sizeof(int));
	p = (int (*)[4])calloc(3*4, sizeof(int));

	形式2：定义指针变量的同时申请空间
	int (*p)[4] = (int (*)[4])malloc(3*4*sizeof(int));
	int (*p)[4] = (int (*)[4])calloc(3*4, sizeof(int));

	说明： 虽然初次申请时也可以用 p = (int (*)[4])realloc(NULL, 3*4*sizeof(int));但一般不用

4.3 内存的动态申请与释放（二维数组）

<table>
<tr><th>申请对象</th><th>C++的运算符</th></tr>
<tr><td rowspan="3">二维数组</td><td>形式1：先定义指针变量，再申请
int *p;
p = new int[3][4]; //申请3行4列，错
int (*p)[4];
p = new int[3][4]; //申请3行4列，对</td></tr>
<tr><td>形式2：定义指针变量的同时申请空间
float (*p)[4]=new float[3][4];</td></tr>
<tr><td>形式3：申请空间时赋初值
• 动态申请的二维数组可以在申请时赋初值，方法为后面跟双层{}，{}前不要加=,且[]内必须有数,其余规则同二维数组定义时初始化</td></tr>
</table>

4.3 内存的动态申请与释放（二维数组）



- 动态申请的二维数组可以在申请时赋初值，方法为后面跟{}，{}前不要加=，且[]内必须有数，其余规则同二维数组定义时初始化

```
例：int (*p)[3];
    p = new int[2][3] {1,2,3,4,5,6};      //错误
    p = new int[2][3] {{1,2,3},{4,5,6}};//正确
    p = new int[2][3] {{1,2},{3,4,5,6}};//错误
    p = new int[2][3] {1,4};              //错误
    p = new int[2][3] {{1},{4}};          //正确
例：char (*p)[6];  //注：字符型在使用字符串方式初始化时，一层{}
    p = new char[2][6] {'A','B','C' };     //错误
    p = new char[2][6] {{'A'},{'B','C' }};//正确
    p = new char[2][6] {"Hello","China"}; //正确
    p = new char[2][6]{"Hello1","China"}; //错误
```

小结：内存的动态申请与释放（C方式）

释放对象	C的函数方式

普通变量	int *p = (int *)malloc(sizeof(int));
	int *p = (int *)calloc(1, sizeof(int));
	free(p);

一维数组	int *p = (int *)malloc(10*sizeof(int));
	int *p = (int *)calloc(10, sizeof(int));
	free(p);

二维数组	int (*p)[4] = (int (*)[4])malloc(3*4*sizeof(int));
	int (*p)[4] = (int (*)[4])calloc(3*4, sizeof(int));
	free(p);

小结：内存的动态申请与释放（C++方式）

释放对象	C++的运算符
普通变量	int *p = new int; delete p;
一维数组	int *p = new int[10]; delete []p; • 某些资料上说可以 delete name，因为一维数组地址可理解为首元素地址，不加[] • 对于int/char等基本类型的数组，加不加均正确
二维数组	int (*p)[4] = new int[3][4]; delete []p; • 二维以上必须加一个[]，否则编译警告

- 静态数据区、动态数据区、动态内存分配区(堆空间)的地址各不相同

```
#include <iostream>
#include <cstdlib>
using namespace std;
int a;
int main()
{    int b;
     int *c;
     c = (int *)malloc(sizeof(int));
     cout << &a <<endl;
     cout << &b <<endl;
     cout << &c << ' ' << c <<endl;
     free(c);
     return 0;
}
```

```
00BEC13C
0034F970
0034F964 006DB298
```

说明：

1、虽然申请一个int空间不可能申请失败，但从程序规范角度出发，要求每次申请后均需要判断申请是否成功

2、本课程课件中，为了节约空间，大部分示例程序省略了判断

实际写程序时要判断！！！

作业中会人工审核并扣分！！！

后续不再强调，自行补充所有ppt示例程序

- C：通过强制类型转换将void型的指针转为其它类型

```
int main()
{    int *p;
     p=(int *)malloc(10*sizeof(int));
       //或 p=(int *)calloc(10, sizeof(int));
     if (p == NULL) {
         cout << "No Memory" << endl;
         return 0;
     }
     ...
     free(p);
     ...
     return 0;
}
```

申请10个int型的变量空间不建议直接写成malloc(40)/calloc(10,4)
因为程序的适应性差

- C++：申请时自动确定类型

```
int main()
{
    int *p;
    p = new(nothrow) int[10];
    if (p == NULL) {
        cout << "No Memory" << endl;
        return 0;
    }
    ...
    delete p;
    ...
    return 0;
}
```

- 动态申请返回的指针可以进行指针运算，但释放时必须给出申请返回时的首地址，否则释放时会出错

```
#include <iostream>
#include <cstdlib>
using namespace std;
int main()
{
    int i, *p;
    p = &i;      //错误原因：p不是动态申请的空间
    free(p);
    return 0;
}
```

编译不错执行错

- 动态申请返回的指针可以进行指针运算，但释放时必须给出申请返回时的首地址，否则释放时会出错

```
#include <iostream>
#include <stdlib.h>
using namespace std;
int main()
{   int *p;
    p = (int*)malloc(sizeof(int));
    if (p != NULL) {
        p++;      //错误原因：p已不指向动态申请的空间
        free(p);
    }
    return 0;
}
```

编译不错执行错

- 动态申请返回的指针可以进行指针运算，但释放时必须给出申请返回时的首地址，否则释放时会出错

```
#include <iostream>
//不需要包含头文件stdlib
using namespace std;
int main()
{
    int i, *p;
    p = &i;     //错误原因：p不是动态申请的空间
    delete p;
    return 0;
}
```

编译不错执行错

- 动态申请返回的指针可以进行指针运算，但释放时必须给出申请返回时的首地址，否则释放时会出错

```
#include <iostream>
//不需要包含头文件stdlib
using namespace std;
int main()
{
    int *p;
    p = new(nothrow) int;
    if (p != NULL)
    {   p++;   //错误原因：p已不指向动态申请的空间
        delete p;
    }
    return 0;
}
```

编译不错执行错

- new申请失败会抛出异常bad_alloc，需要使用 try-catch来处理异常

```
#include <iostream>
using namespace std;
int main()
{  char *p; int count = 0;
   while (1) {
     try {
         p = new char[1024 * 1024];
     }
     catch (const bad_alloc &mem_fail) {
         cout << mem_fail.what() << endl;//打印原因
         break;
     }
     count++;
   }
   cout << count << " MB" << endl;  return 0;
}
```

```
//或者:
try {
    while (1) {
    p = new char[1024 * 1024];
    count++;}
}
catch (const bad_alloc &mem_fail) {
    cout << mem_fail.what() << endl;
    break;
}
```

- 动态内存申请的空间若不释放，则会造成内存泄露，这种情况不会导致即时错误，但最终会耗尽内存

```
#include <iostream>
using namespace std;
int main()
{
    char *p;
    int num = 0;
    while(1) {
        p = (char *)malloc(1024*1024*sizeof(char));
        if (p == NULL)
            break;
        num ++;
    }
    cout << num << " MB" << endl;
    return 0;
}
```

循环内：每次均申请空间，完成后不释放，且p不再指向，导致内存泄露。循环至内存耗尽

- 动态内存申请的空间若不释放，则会造成内存泄露，这种情况不会导致即时错误，但最终会耗尽内存

```
#include <iostream>
using namespace std;
int main()
{
    char *p;
    int num = 0;
    while(1) {
        p = new(nothrow) char[1024*1024];
        if (p == NULL)
            break;
        num ++;
    }
    cout << num << " MB" << endl;
    return 0;
}
```

循环内：每次均申请空间，完成后不释放，且p不再指向，导致内存泄露。循环至内存耗尽

- 动态内存申请的空间若不释放，则程序退出时操作系统会自动回收

```
…
int main()
{
    …
    p = (int *)malloc(...);
    …
    return 0;
}
```

```
…
int main()
{
    …
    p = new ...;
    …
    return 0;
}
```

p所申请的空间可能：
1）在程序运行结束后由操作系统回收
2）逐渐耗尽内存

坚决反对此种用法，且不是所有的操作系统都支持，作业中重点审核！

- realloc申请时新老空间可重合，也可能不重合，若不重合，原空间原有内容会被复制到新空间，再释放原空间，因此不需要再释放老空间

```
#include <iostream>
#include <cstdlib>
using namespace std;
int main()
{
    int *p;
    p = (int *)malloc(10 * sizeof(int));
    cout << p << endl;
    p = (int *)realloc(p, 20 * sizeof(int));
    cout << p << endl;
    free(p);
    return 0;
}
```

realloc的常规用法：传入指针和返回指针用同一个

//再次注意此例未判断申请是否成功，自行补充，后续例子类似

- realloc申请时新老空间可重合，也可能不重合，若不重合，原空间原有内容会被复制到新空间，再释放原空间，因此不需要再释放老空间



```
#include <iostream>
#include <cstdlib>
using namespace std;
int main()
{
    int *p;
    p = (int *)malloc(10 * size
    cout << p << endl;
    p = (int *)realloc(p, 20 *
    cout << p << endl;
    free(p);
    return 0;
}
```

```
//非常规用法
int *p, *q;
p  = (int *)malloc(10 * sizeof(int));
cout << p << endl;
q = (int *)realloc(p, 20 *sizeof(int));
cout << p << ' ' << q << endl;
free(p); //需注释掉此行才正确
free(q);
```

realloc的常规用法：传入指针和返回指针用同一个

//再次注意此例未判断申请是否成功，自行补充，后续例子类似

• realloc申请时，原空间原有内容会被复制到新空间

```
#include <iostream>
#include <cstdlib>
using namespace std;
int main()
{   int i, *p;
    p = (int *)malloc(10*sizeof(int));
    cout << "地址:" << p << endl;
    for(i=0; i<10; i++)
        p[i] = (i+1)*(i+1);   //[0-9]的值为1-10的平方

    p = (int *)realloc(p, 20*sizeof(int));
    cout << "新地址:" << p << endl;
    for(i=0; i<20; i++)
        cout << *(p+i) << endl;
    free(p);
    return 0;
}
```

```
地址:006CAEA8
新地址:006CB3C0
1
4
9
16
25
36
49
64
81
100
-842150451
-842150451
-842150451
-842150451
-842150451
-842150451
-842150451
-842150451
-842150451
-842150451
-842150451
```

//申请一个int型空间：

```
#include <iostream>
#include <cstdlib>
using namespace std;
int main()
{
   int *p;
   p = (int *)malloc(sizeof(int));
   *p = 10;
   cout << *p << endl;
   free(p); //记得释放
   return 0;
}
```

```
#include <iostream>
using namespace std;

int main()
{
   int *p;
   p = new(nothrow) int;
   *p = 10;
   cout << *p << endl;
   delete p; //记得释放
   return 0;
}
```

//申请10个int型空间当一维数组用：指针法/下标法均可

```
#include <iostream>
#include <cstdlib>
using namespace std;
int main()
{
   int i, *p;
   p = (int *)malloc(10*sizeof(int));
   for(i=0; i<10; i++)
      p[i] = (i+1)*(i+1); //赋初值
   for(i=0; i<10; i++)
      cout << *(p+i) << endl;//打印
   free(p);
   return 0;
}
```

```
#include <iostream>
using namespace std;

int main()
{
   int i, *p;
   p = new(nothrow) int[10];
   for(i=0; i<10; i++)
      p[i] = (i+1)*(i+1); //赋初值
   for(i=0; i<10; i++)
      cout << *(p+i) << endl; //打印
   delete []p;
   return 0;
}
```

//申请10个int型空间当一维数组用：
用head记住申请的首地址，便于复位和释放，工作指针p可++/--

```
#include <iostream>
#include <cstdlib>
using namespace std;
int main()
{
   int i, *p, *head;
   p = (int *)malloc(10*sizeof(int));
   head = p;
   for(i=0; i<10; i++)
      *p++ = (i+1)*(i+1); //赋初值
   for(p=head; p-head<10; p++)
      cout << *p << endl; //打印
   free(head);
   return 0;
}
```

```
#include <iostream>
using namespace std;

int main()
{
   int i, *p, *head;
   p = new(nothrow) int[10];
   head = p;
   for(i=0; i<10; i++)
      *p++ = (i+1)*(i+1); //赋初值
   for(p=head; p-head<10; p++)
      cout << *p << endl; //打印
   delete []head;
   return 0;
}
```

//申请12个int型当二维数组使用：
p为行指针，p_element为元素指针，head记住首址



```
#include <iostream>
#include <cstdlib>
using namespace std;
int main()
{  int i, j, (*p)[4], (*head)[4], *p_element;
   head = p = (int (*)[4]) malloc(3*4*sizeof(int));
   for(i=0; i<3; i++)
      for(j=0; j<4; j++)
         p[i][j] = i*4+j; //赋初值
   for(p=head; p-head<3; p++) {
      for(p_element=*p; p_element-*p<4; p_element++)
          cout << *p_element << ' '.
      cout << endl; //每行加回车
   }
  free(head);  return 0;
}
```

```
//换成C++方式程序其余相同
head = p = new(nothrow) int[3][4];
…
delete []head;
```

- 嵌套进行动态内存申请：按从外到内的顺序进行申请，反序进行释放

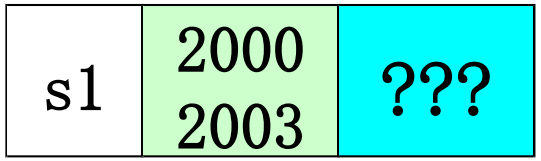
```
#include <iostream>
#include <cstdlib>
using namespace std;
struct student {
    int num;  char *name;
};
int main()
{  student *s1;
   s1 = (student *)malloc(sizeof(student)); //申请8字节
   s1->name = (char *)malloc(6*sizeof(char));//申请6字节
   s1->num = 1001;  strcpy(s1->name, "zhang");
   cout << s1->num << ":" << s1->name << endl;
   free(s1->name);//释放6字节
   free(s1);//释放8字节
   return 0;
}
```

申请：先student变量，再name成员
释放：先name变量，再student成员

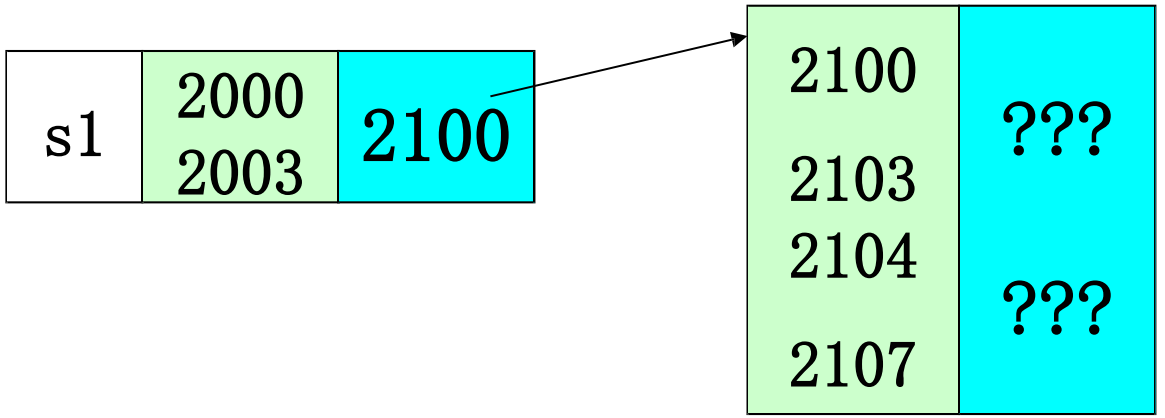
- 嵌套进行动态内存申请：按从外到内的顺序进行申请，反序进行释放

```
#include <iostream>
using namespace std;
struct student {
    int num;  char *name;
};
int main()
{
    student *s1;
    s1 = new(nothrow) student; //申请8字节
    s1->name = new(nothrow) char[6];//申请6字节
    s1->num = 1001;  strcpy(s1->name, "zhang");
    cout << s1->num << ":" << s1->name << endl;
    delete s1->name;//释放6字节
    delete s1;//释放8字节
    return 0;
}
```

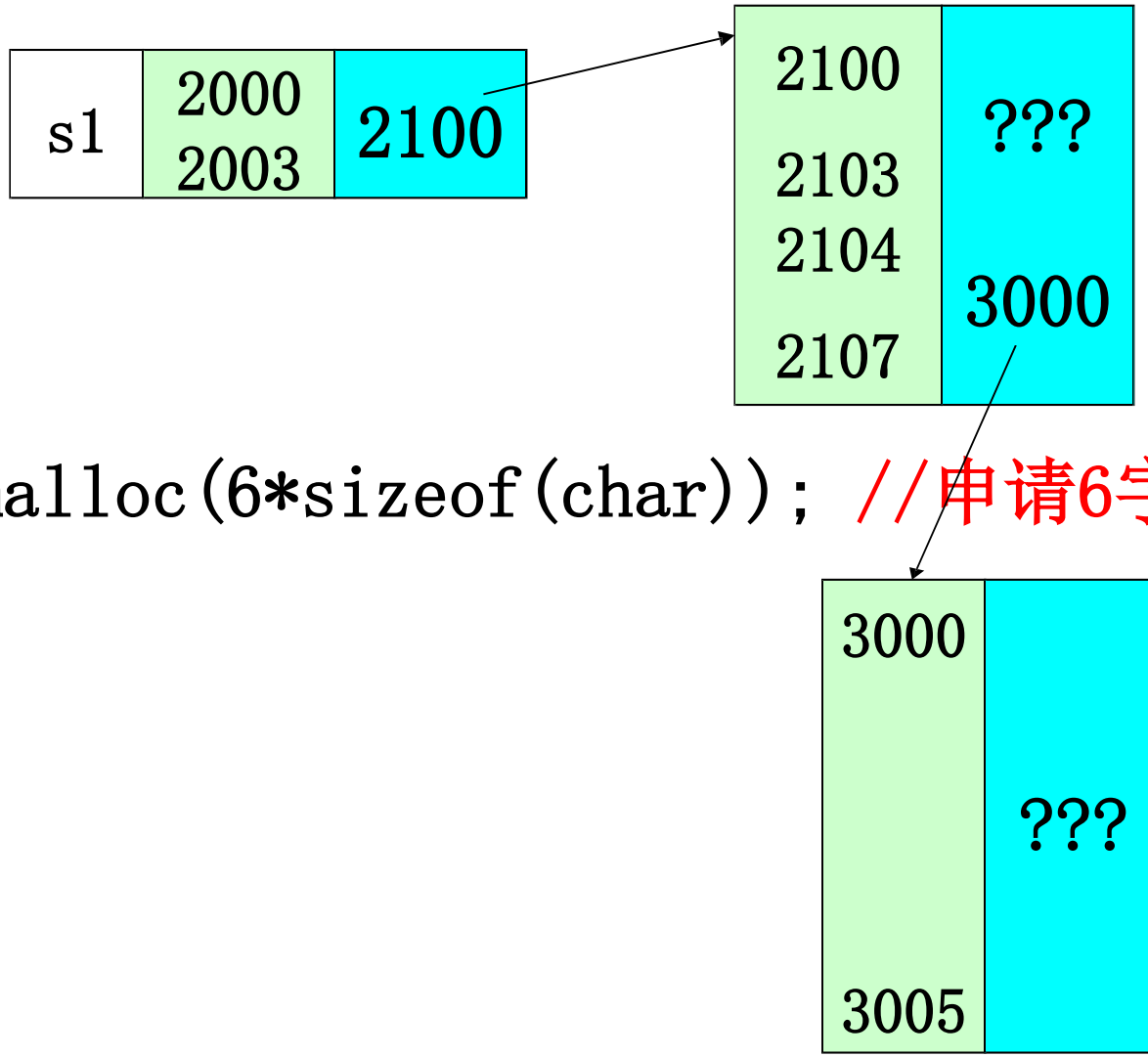
申请：先student变量，再name成员
释放：先name变量，再student成员



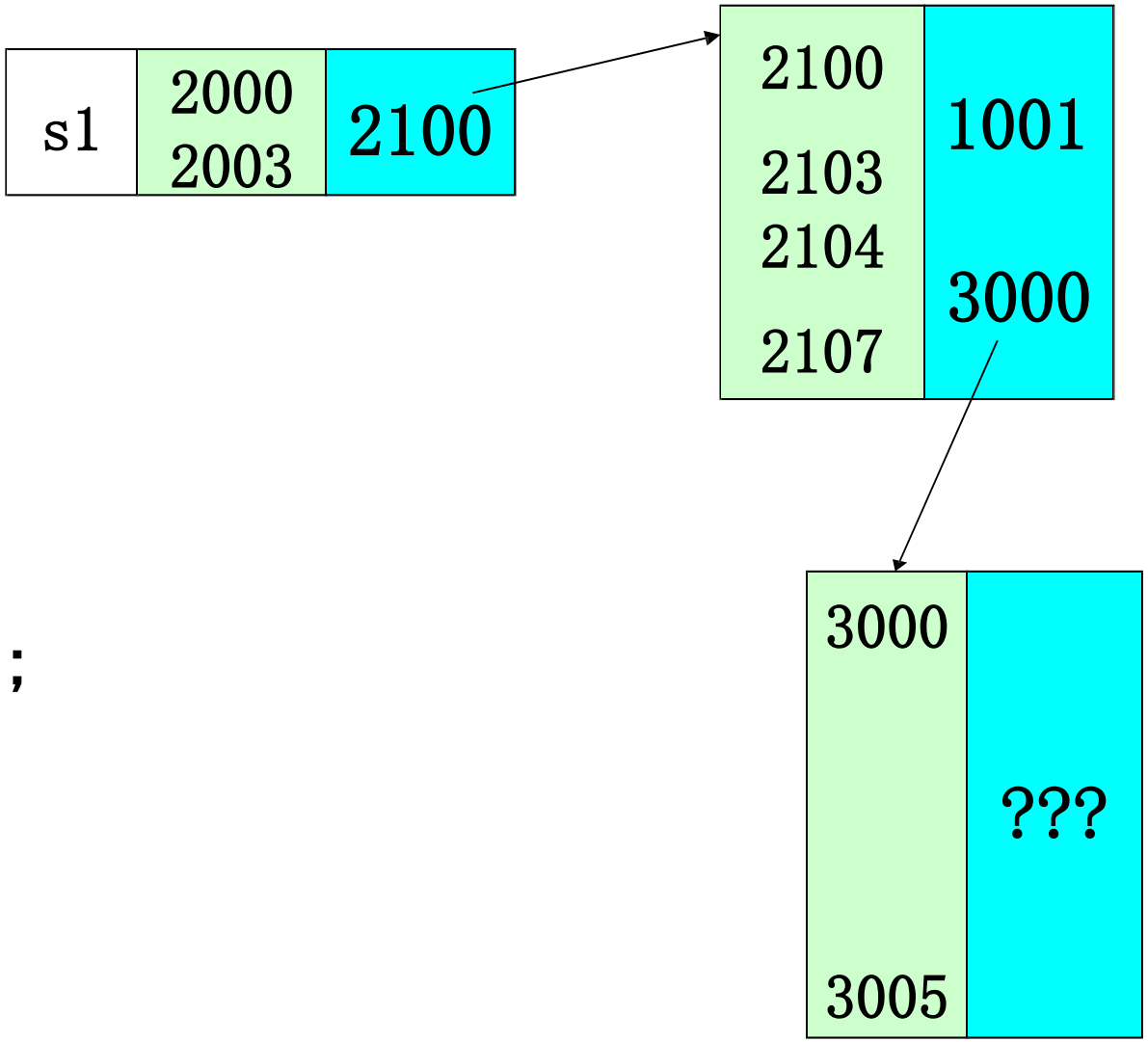

```
student *s1;
```



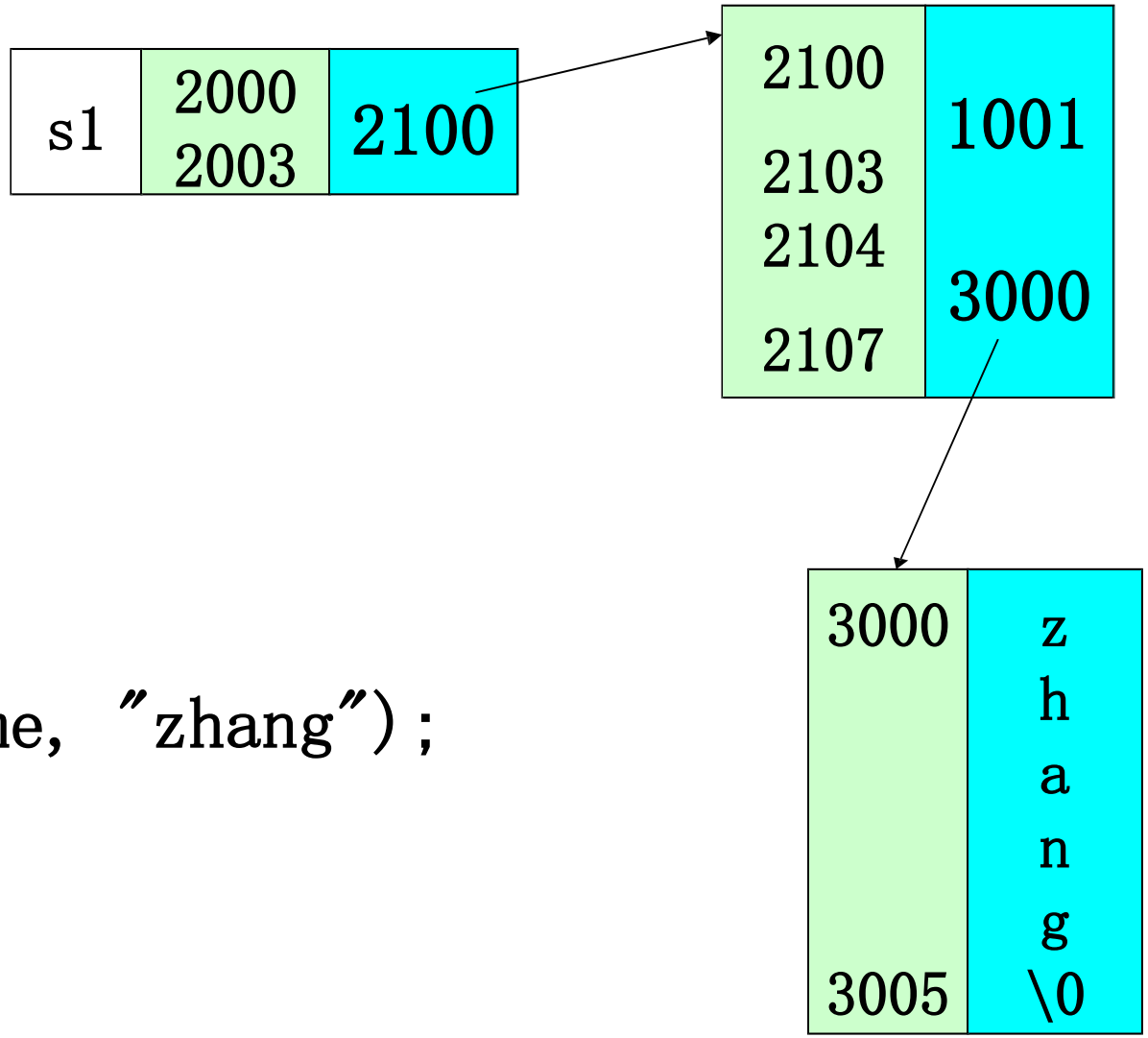

```
s1 = (student *)malloc(sizeof(student)); //申请8字节
```



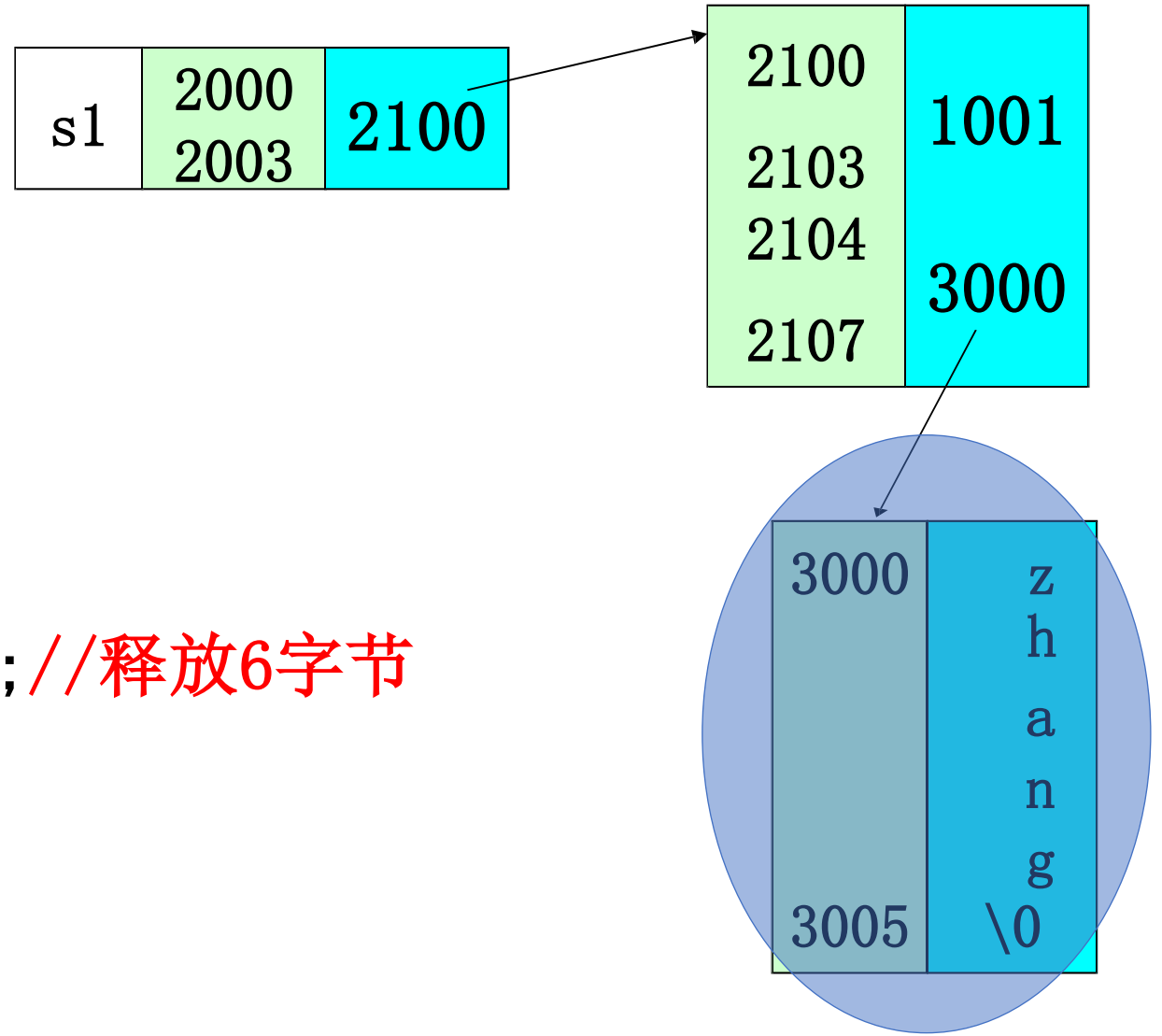

```
s1->name = (char *)malloc(6*sizeof(char)); //申请6字节
```



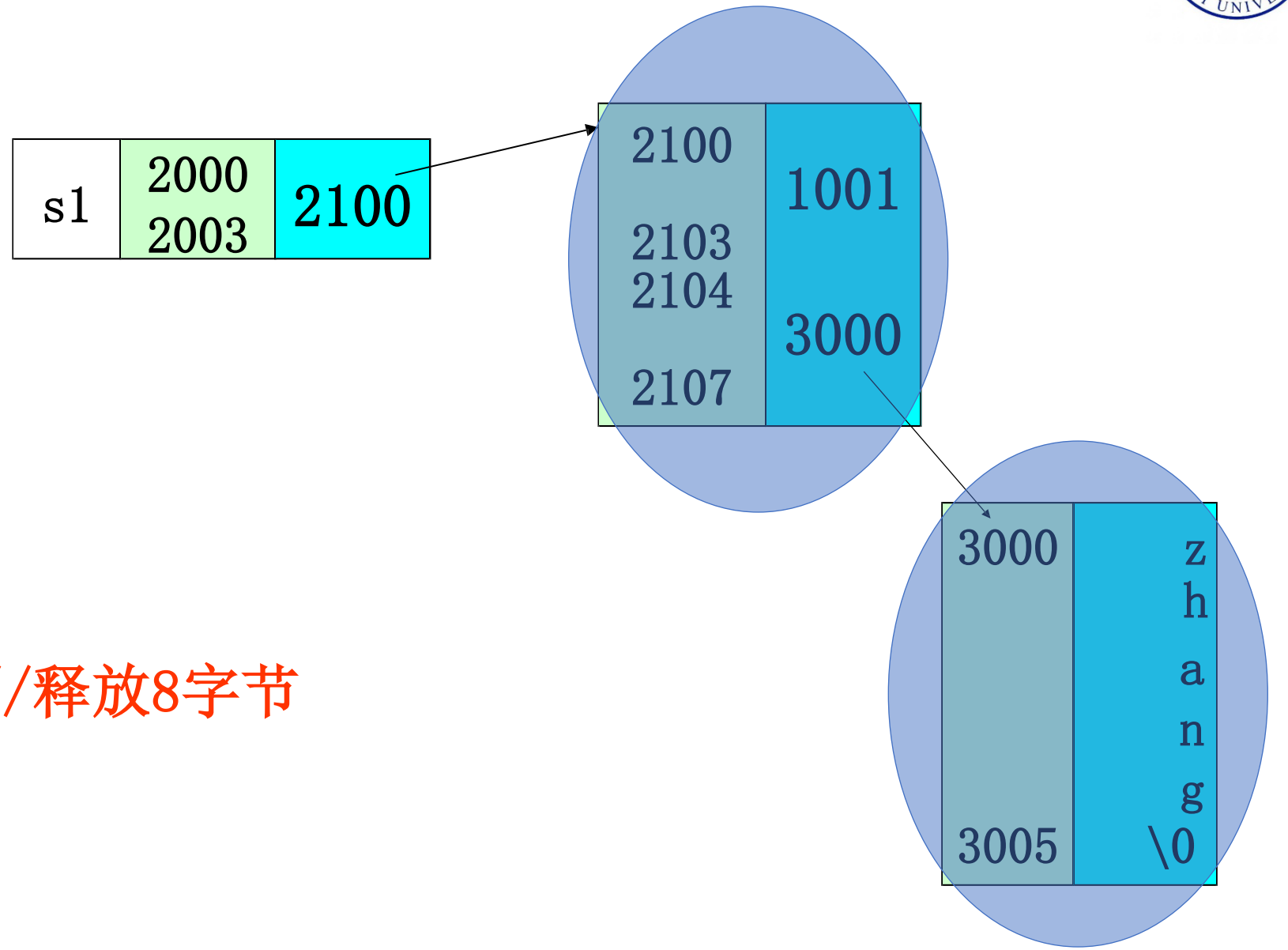

```
s1->num = 1001;
```



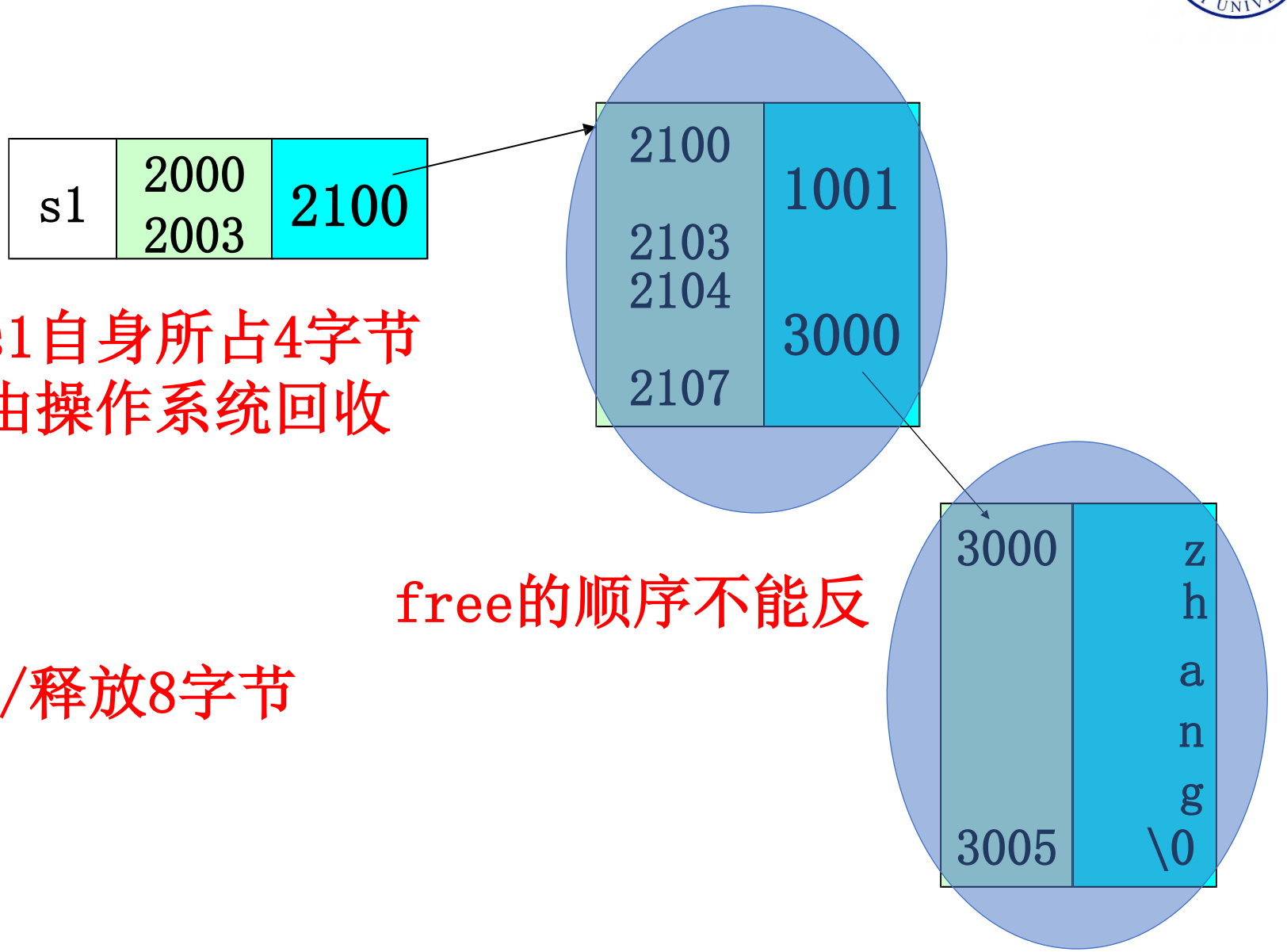
strcpy(s1->name, "zhang");



free(s1->name);//释放6字节



free(s1);//释放8字节



s1自身所占4字节
由操作系统回收

free的顺序不能反

free(s1);//释放8字节

- 动态申请的内存，只能通过首指针释放一次，若重复释放，则会导致运行出错

```
#include <iostream>
#include <cstdlib>
using namespace std;
int main()
{
    int *p;
    p = (int *)malloc(sizeof(int));
    *p = 10;
    cout << *p << endl;
    free(p); //释放
    free(p); //再次释放，致运行出错
}
```

```
#include <iostream>

using namespace std;
int main()
{
    int *p;
    p = new(nothrow) int;
    *p = 10;
    cout << *p << endl;
    delete p; //释放
    delete p; //再次释放，致运行出错
}
```

```
#include <iostream>
#include <string>
using namespace std;
struct student
{
    string name;
    int num;
    char sex;
};
```

```
int main()                                          //错误
{
  student *p;
  p = (student *)malloc(sizeof(student));
  if (p == NULL)
     return -1;
  p->name = "Wang Fun";
  p->num = 10123;
  p->sex = 'm';
  cout << p->name << endl << p->num << endl
       << p->sex << endl;
  free(p);
  return 0;
}
```

对于非内部数据类型的对象而言，malloc/free无法满足动态对象的要求

malloc/free是库函数，不在编译器控制权限之内，不能执行构造函数和析构函数

```
#include <iostream>
#include <string>
using namespace std;
struct student
{
    string name;
    int num;
    char sex;
};
```

```
int main()                                          //正确
{
  student *p;
  p = new(nothrow) student;
  if (p == NULL)
      return -1;
  p->name = "Wang Fun";
  p->num = 10123;
  p->sex = 'm';
  cout << p->name << endl << p->num << endl
       << p->sex << endl;
  delete p;
  return 0;
}
```

运算符new能完成动态内存分配和初始化工作
运算符delete能完成清理与释放内存的工作

//malloc/free和new/delete的本质区别

```
#include <iostream>
using namespace std;
class Obj
{
public:
    Obj(){cout << "Initialization" << endl;}
    ~Obj(){cout << "Destroy" << endl;}
};
void UseMallocFree()
{
    cout << "in UseMallocFree()..." << endl;
    Obj* a = (Obj*)malloc(sizeof(Obj));
    free(a);
}
```

```
void UseNewDelete()
{
    cout << "in UseNewDelete()
             ..." << endl;
    Obj* a = new Obj;
    delete a;
}
int main()
{
    UseMallocFree();
    UseNewDelete();
    return 0;
}
```

```
in UseMallocFree()...
in UseNewDelete()...
Initialization
Destroy
```

非内部数据类型的对象而言：
malloc/free和new/delete有区别！！

```
// newstruct.cpp -- using new with a structure
#include <iostream>
struct inflatable   // structure definition
{
    char name[20];
    float volume;
    double price;
};
```

提示：C++在指定结构成员时，如果结构标识符是结构名，则使用句点运算符；如果标识符是指向结构的指针，则使用箭头运算符

```
int main()
{
    using namespace std;
    inflatable * ps = new inflatable; // allot memory for structure
    cout << "Enter name of inflatable item: ";
    cin.get(ps->name, 20);            // method 1 for member access
    cout << "Enter volume in cubic feet: ";
    cin >> (*ps).volume;              // method 2 for member access
    cout << "Enter price: $";
    cin >> ps->price;
    cout << "Name: " << (*ps).name << endl;              // method 2
    cout << "Volume: " << ps->volume << " cubic feet\n"; // method 1
    cout << "Price: $" << ps->price << endl;             // method 1
    delete ps;                        // free memory used by structure
    return 0;
}
```

```
#define _CRT_SECURE_NO_WARNINGS
#include <iostream>     // delete.cpp -- using the delete operator
#include <cstring>      // or string.h
using namespace std;
char * getname(void);   // function prototype
int main()
{
    char * name;        // create pointer but no storage

    name = getname();   // assign address of string to name
    cout << name << " at " << (int *) name << "\n";
    delete [] name;     // memory freed

    name = getname();   // reuse freed memory
    cout << name << " at " << (int *) name << "\n";
    delete [] name;     // memory freed again
    return 0;
}
```

```
Enter last name: Tongji
Tongji at 00FDD390
Enter last name: CS
CS at 00FD4978
```

```
char * getname()    // return pointer to new string
{   char temp[80];      // temporary storage
    cout << "Enter last name: ";
    cin >> temp;
    char * pn = new char[strlen(temp) + 1];
    strcpy(pn, temp);   // copy string into smaller space
    return pn; // temp lost when function ends
}
```

```
#include <iostream>      // use_new.cpp -- using the new operator
int main()
{
    using namespace std;
    int nights = 1001;
    int * pt = new int;          // allocate space for an int
    *pt = 1001;                  // store a value there

    cout << "nights value = ";
    cout << nights << ": location " << &nights << endl;
    cout << "int ";
    cout << "value = " << *pt << ": location = " << pt << endl;

    double * pd = new double;    // allocate space for a double
    *pd = 10000001.0;            // store a double there

    cout << "double ";
    cout << "value = " << *pd << ": location = " << pd << endl;
    cout << "location of pointer pd: " << &pd << endl;
    cout << "size of pt = " << sizeof(pt);
    cout << ": size of *pt = " << sizeof(*pt) << endl;
    cout << "size of pd = " << sizeof pd;
    cout << ": size of *pd = " << sizeof(*pd) << endl;
    return 0;
}
```

```
nights value = 1001: location 0118FDF8
int value = 1001: location = 013E8CC0
double value = 1e+07: location = 013ED3D0
location of pointer pd: 0118FDE0
size of pt = 4: size of *pt = 4
size of pd = 4: size of *pd = 8
```

目录

- 动态内存的基本概念
- C中的相关函数
- C++中的相关运算符
- 内存的动态申请与释放
- 定位new运算符

5.1 定位new运算符

- new运算符的变体，指定要使用的位置
- 程序员可通过其设置内存管理规程，处理需要通过特定地址进行访问的硬件或在特定位置创建对象
- 需要包含头文件new

```
#include <new>
using namespace std;
struct chaff {
    char dross[20];
    int slag;
};
char buffer1[50];
char buffer2[500];
```

```
int main()
{   chaff *p1, *p2; int *p3, *p4;
    //the regular forms of new
    p1 = new chaff;    //place structure in heap
    p3 = new int[20]; //place int array in heap
    //the two forms of placement new
    p2 = new (buffer1) chaff;    //place structure in buffer1
    p4 = new (buffer2) int[20]; //place int array in buffer2
    …
}
```

//书-程序清单9.10：三次内存分配以及其对应的地址的输出



```
#include <new>
const int BUF = 512;
char buffer[BUF]; // chunk of memory
…
int main()
{
  …
  pd1 = new double[N]; // use heap
  pd2 = new (buffer) double[N]; // use buffer array
  …
  for (i = 0; i < N; i++)
  {
      cout << pd1[i] << " at " << &pd1[i] << "; ";
      cout << pd2[i] << " at " << &pd2[i] << endl;
  }
  …接下页
```

- pd1使用的是常规的new，从堆中申请空间并返回首地址
- pd2使用的是定位new运算符，他们都是new但是参数数量上有区别，功能上也有区别，它会在指定空间（buffer）中分配N个double空间

```
Calling new and placement new:
Memory addresses:
 heap:00AACC10 static:0087D2D8
Memory contents:
1000 at 00AACC10; 1000 at 0087D2D8
1020 at 00AACC18; 1020 at 0087D2E0
1040 at 00AACC20; 1040 at 0087D2E8
1060 at 00AACC28; 1060 at 0087D2F0
1080 at 00AACC30; 1080 at 0087D2F8
```

```
…
pd3 = new double[N];          // find new address
pd4 = new (buffer) double[N];// overwrite old data
…//接下页
```

- pd3使用的是常规的new运算符，它负责从堆中申请一块空间，并返回首地址
- pd4仍然从buffer这里开始分配地址，因为定位new运算符是在传入的那个地址处分配指定大小的空间而不去管这个空间之前有没有被占用。所以会出现两次地址一样（新写入的数据会复盖之前的）

```
Calling new and placement new:
Memory addresses:
 heap:00AACC10 static:0087D2D8
Memory contents:
1000 at 00AACC10; 1000 at 0087D2D8
1020 at 00AACC18; 1020 at 0087D2E0
1040 at 00AACC20; 1040 at 0087D2E8
1060 at 00AACC28; 1060 at 0087D2F0
1080 at 00AACC30; 1080 at 0087D2F8

Calling new and placement new a second time:
Memory contents:
1000 at 00AA48C0; 1000 at 0087D2D8
1040 at 00AA48C8; 1040 at 0087D2E0
1080 at 00AA48D0; 1080 at 0087D2E8
1120 at 00AA48D8; 1120 at 0087D2F0
1160 at 00AA48E0; 1160 at 0087D2F8

Calling new and placement new a third time:
Memory contents:
1000 at 00AACC10; 1000 at 0087D300
1060 at 00AACC18; 1060 at 0087D308
1120 at 00AACC20; 1120 at 0087D310
1180 at 00AACC28; 1180 at 0087D318
1240 at 00AACC30; 1240 at 0087D320
```

```
    ...
    delete[] pd1;
    pd1 = new double[N];
    pd2 = new (buffer + N * sizeof(double)) double[N];
    ...
}
```

- 第三次的pd1和第一次的地址一样，它在new的时候重新占据了被delete之前所占据的空间
- pd2通过计算，避开了正在被使用的内存空间，这里的定位new运算符被传入了指向buffer后面的地址。所以不会复盖之前存在buffer里的数据

```
Calling new and placement new:
Memory addresses:
 heap:00AACC10 static:0087D2D8
Memory contents:
1000 at 00AACC10; 1000 at 0087D2D8
1020 at 00AACC18; 1020 at 0087D2E0
1040 at 00AACC20; 1040 at 0087D2E8
1060 at 00AACC28; 1060 at 0087D2F0
1080 at 00AACC30; 1080 at 0087D2F8

Calling new and placement new a second time:
Memory contents:
1000 at 00AA48C0; 1000 at 0087D2D8
1040 at 00AA48C8; 1040 at 0087D2E0
1080 at 00AA48D0; 1080 at 0087D2E8
1120 at 00AA48D8; 1120 at 0087D2F0
1160 at 00AA48E0; 1160 at 0087D2F8

Calling new and placement new a third time:
Memory contents:
1000 at 00AACC10; 1000 at 0087D300
1060 at 00AACC18; 1060 at 0087D308
1120 at 00AACC20; 1120 at 0087D310
1180 at 00AACC28; 1180 at 0087D318
1240 at 00AACC30; 1240 at 0087D320
```

5.1 定位new运算符

- 定位new运算符创建的指针，不需要delete，这不同于常规的new运算符
- 常规new调用接收一个参数的new()函数，标准定位new调用接收两个参数的new()函数：

```
int *p1 = new int;                  //invokes new(sizeof(int))
int *p2 = new(buffer) int;          //invokes new(sizeof(int),buffer)
int *p3 = new(buffer) int[40]; //invokes new(40*sizeof(int),buffer)
```

- 定位new函数不可替换，但可以重载。它至少需要接收两个参数。（了解）

总结

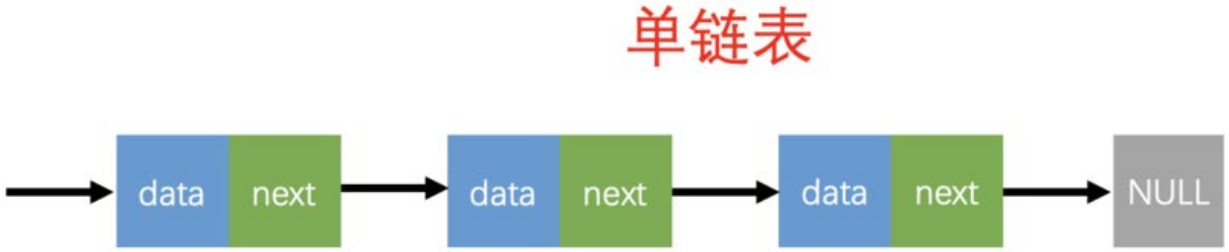
- 动态内存的基本概念
- C中的相关函数
  - void *malloc(unsigned size);
  - void *calloc(unsigned n, unsigned size);
  - void *realloc(void *ptr, unsigned newsize);
  - void free(void *p);
- C++中的相关运算符
  - new delete
- 内存的动态申请与释放（熟练）
- 定位new运算符（了解）

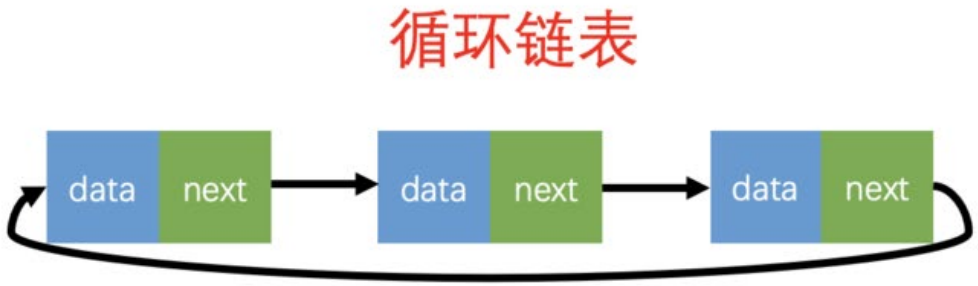
第四章 链表的实现与应用

主讲教师：同济大学计算机科学与技术学院 陈宇飞
同济大学计算机科学与技术学院 龚晓亮

目录

- 链式结构的基本概念
- 单链表的基本操作
- 双向链表的基本操作
- 二叉搜索树







1.1 链式结构的基本概念

- 数组的不足
  - 大小必须在定义时确定，导致空间浪费

    是否可以按需分配空间
  - 占用连续空间，导致小空间无法充分利用

    是否可以充分利用不连续的空间
  - 在插入/删除元素时必须前后移动元素

    插入/删除时能否不移动元素
- 链表

  不连续存放数据，用指针指向下一数据的存放地址

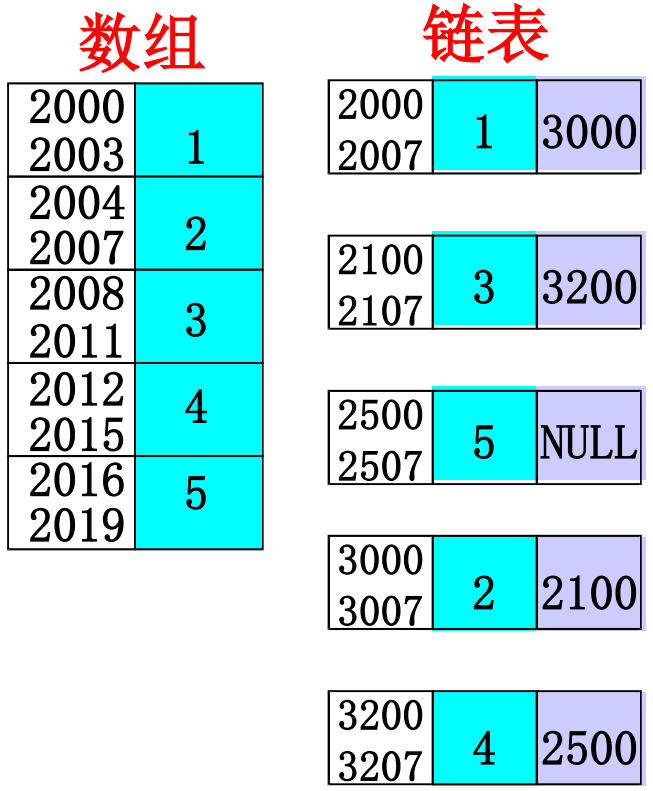
1.1 链式结构的基本概念

例：数据1，2，3，4，5，分别存放在数组和链表中

存放5个元素：

数组：连续的20字节

链表：非连续的40字节(每个结点的8字节连续)



1.1 链式结构的基本概念

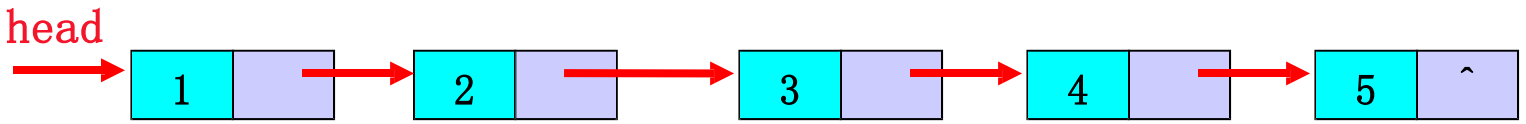
- 链表与数组的比较

数组	链表
大小在声明时固定	大小不固定
处理的数据个数有差异时，须按最大值声明	根据需要随时增加/减少结点
内存地址连续，可直接计算得到某个元素的地址	内存地址不连续，必须依次查找
逻辑上连续，物理上连续	逻辑上连续，物理上不连续

1.1 链式结构的基本概念

- 结点：存放数据的基本单位
  - 数据域：存放数据的值
  - 指针域：存放下一个同类型结点的地址
- 链表：由若干结点构成的链式结构
- 表头结点：第一个结点
- 表尾结点：链表的最后一个结点，指针域为NULL(空)
- 头指针：指向链表的表头结点的指针

链表的表示方法：



- 建立一个有5个结点的链表，学生的基本信息从键盘进行输入，假设键盘输入为：

```
Zhang    1001    m
Li       1002    f
Wang     1003    m
Zhao     1004    m
Qian     1005    f
```

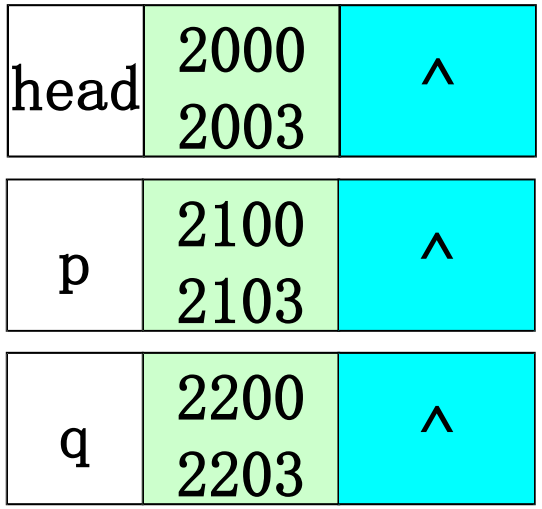
```
struct student {
    string name;
    int num;
    char sex;
    struct student *next;
};
```

//指向结构体自身的指针成员类型不允许是自身的结构体类型，但可以是指针(因为指针占用空间已知)

```
int main()
{   student *head=NULL, *p=NULL, *q=NULL;
    int i;
    for(i=0; i<5; i++) {
       if (i > 0)  q = p;
       p = new(nothrow) student;
       if (p == NULL)  return -1;
       if (i == 0)  head = p;
       else  q->next = p;
       cout <<"请输入第"<< i+1 <<"个人的基本信息"<< endl;
       cin >> p->name >> p->num >> p->sex;
       p->next = NULL;
     }
}    //是否完整?
```

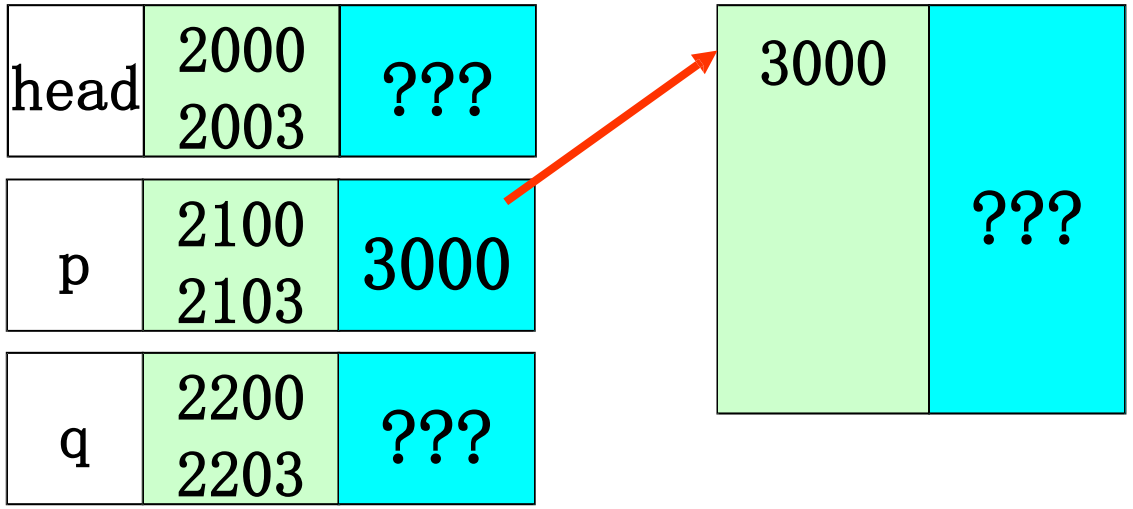
```
student *head=NULL, *p=NULL, *q=NULL; int i;
```

初始状态：



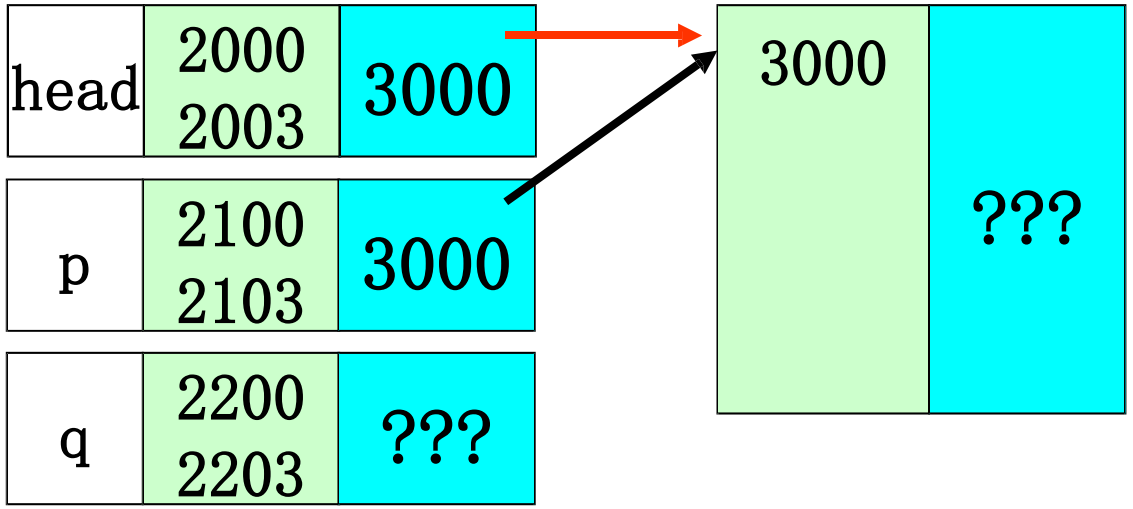
```
p = new(nothrow) student;
```

i=0的循环：



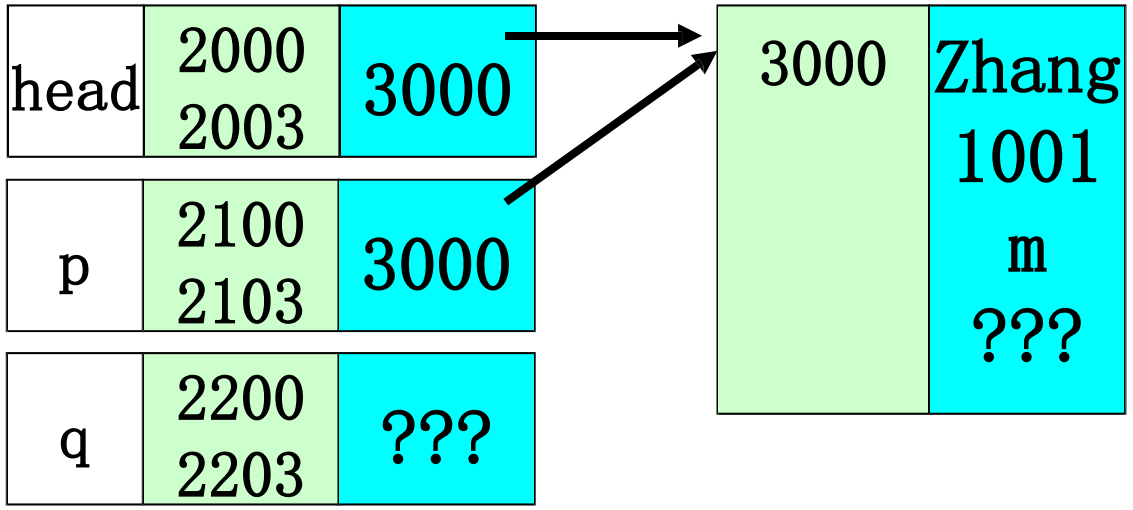
```
head = p; //head指向第1个结点
```

i=0的循环：



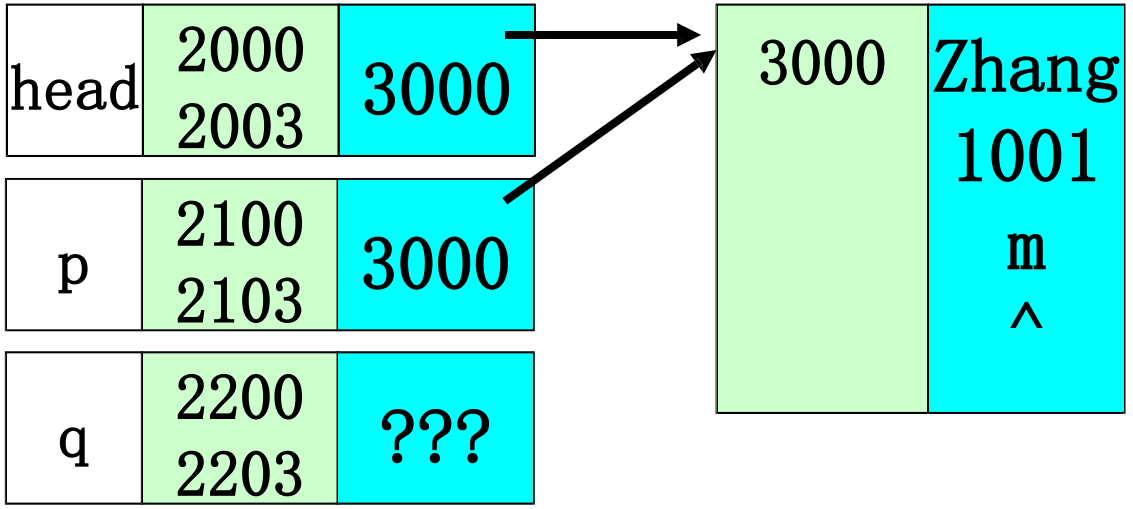
```
cin >> p->name >> p->num >> p->sex; //键盘输入基本信息
```

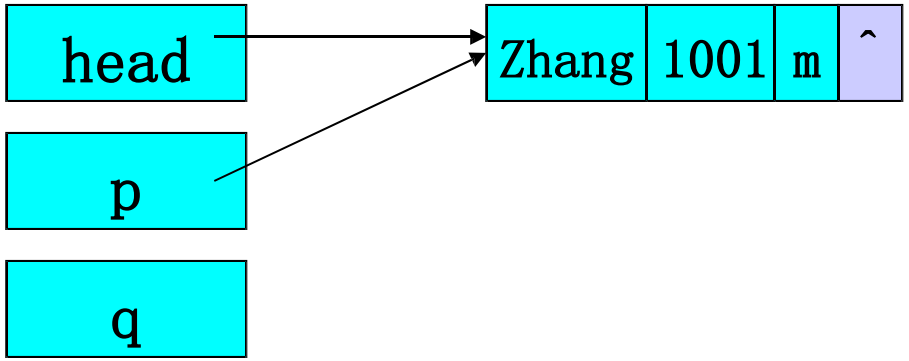
i=0的循环：



```
p->next = NULL;
```

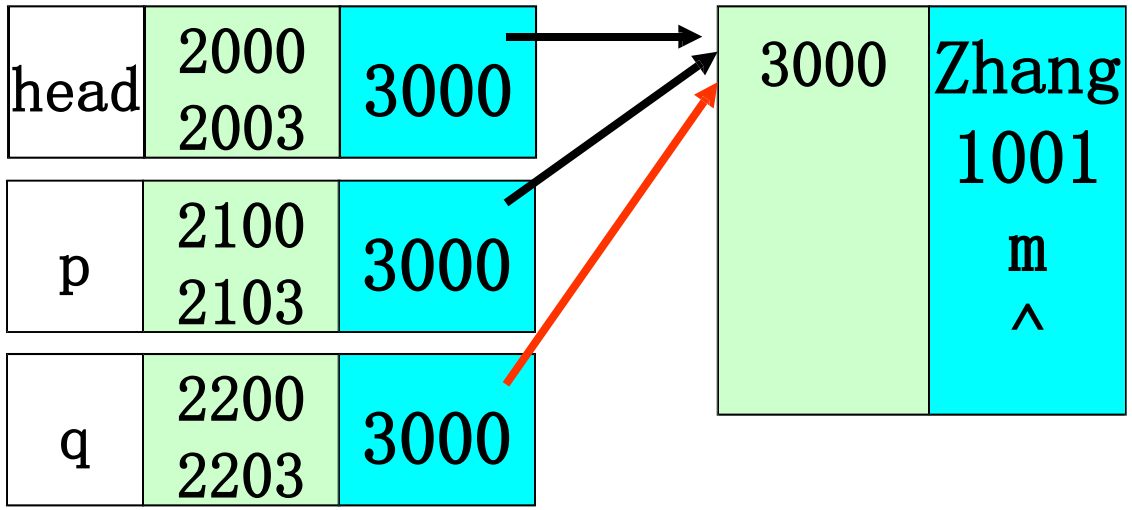
i=0的循环结束：





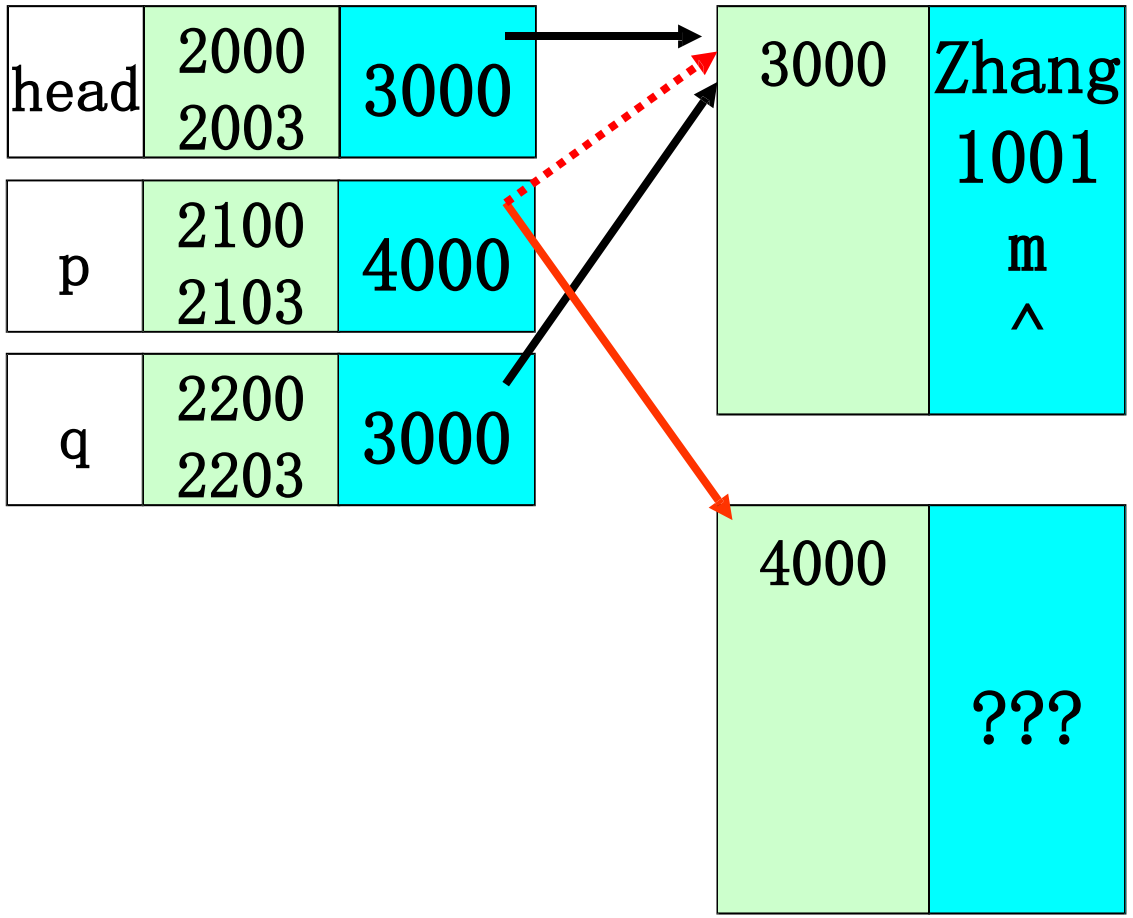
```
if (i > 0)   q = p;
```

i=1的循环：



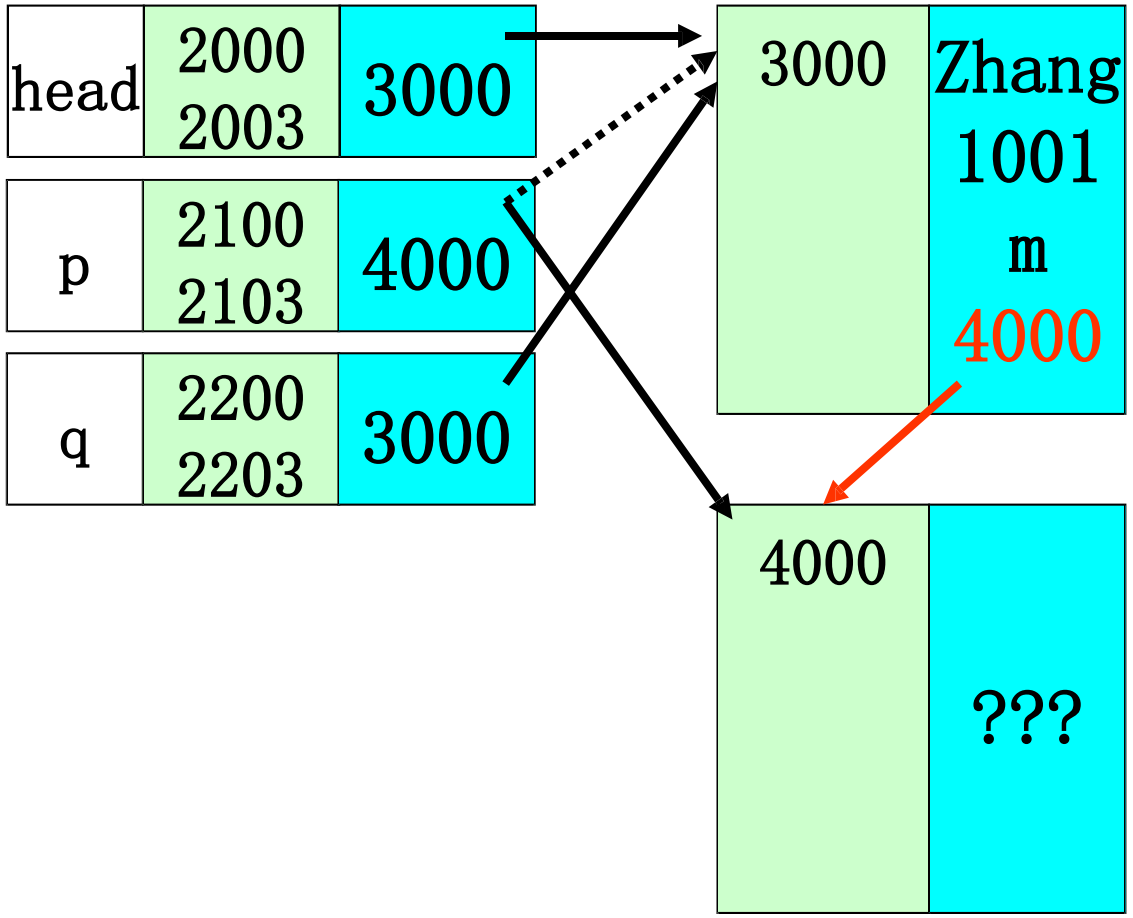
```
p = new(nothrow) student;
```

i=1的循环：



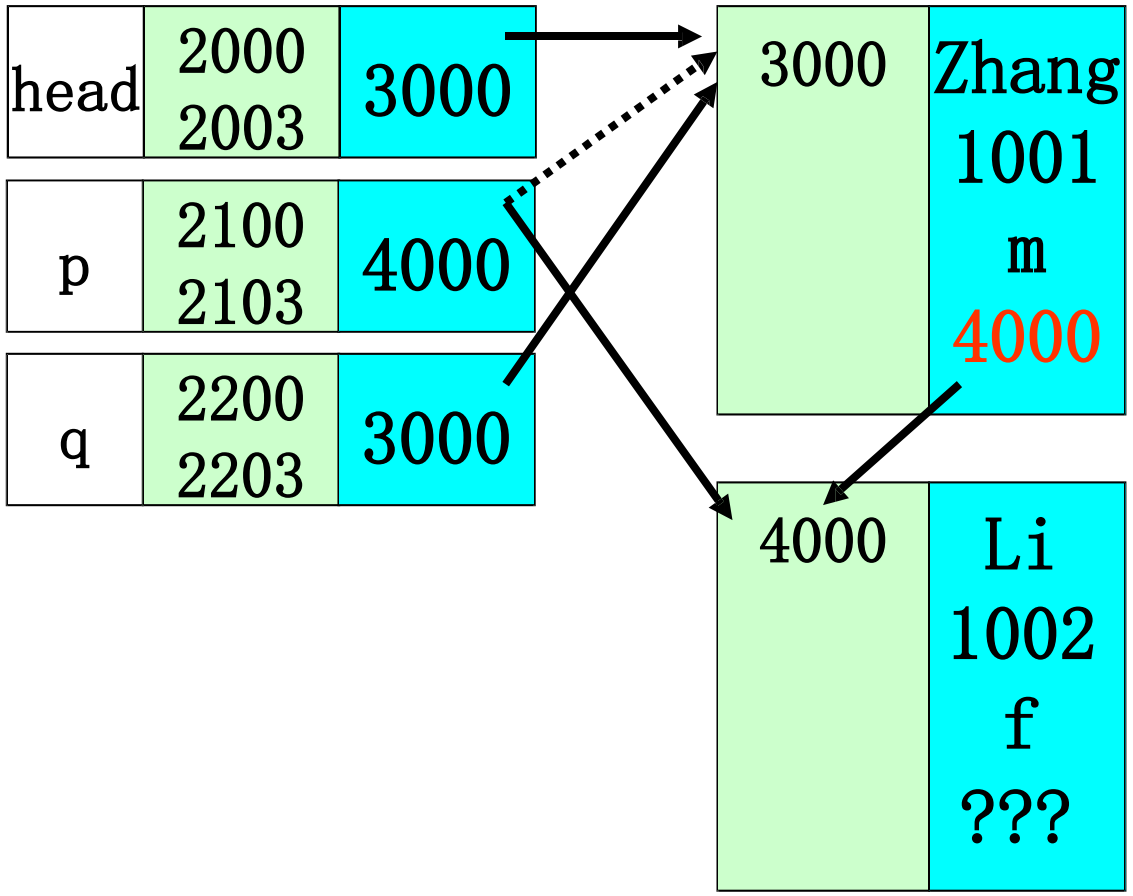
q->next = p;

i=1的循环：



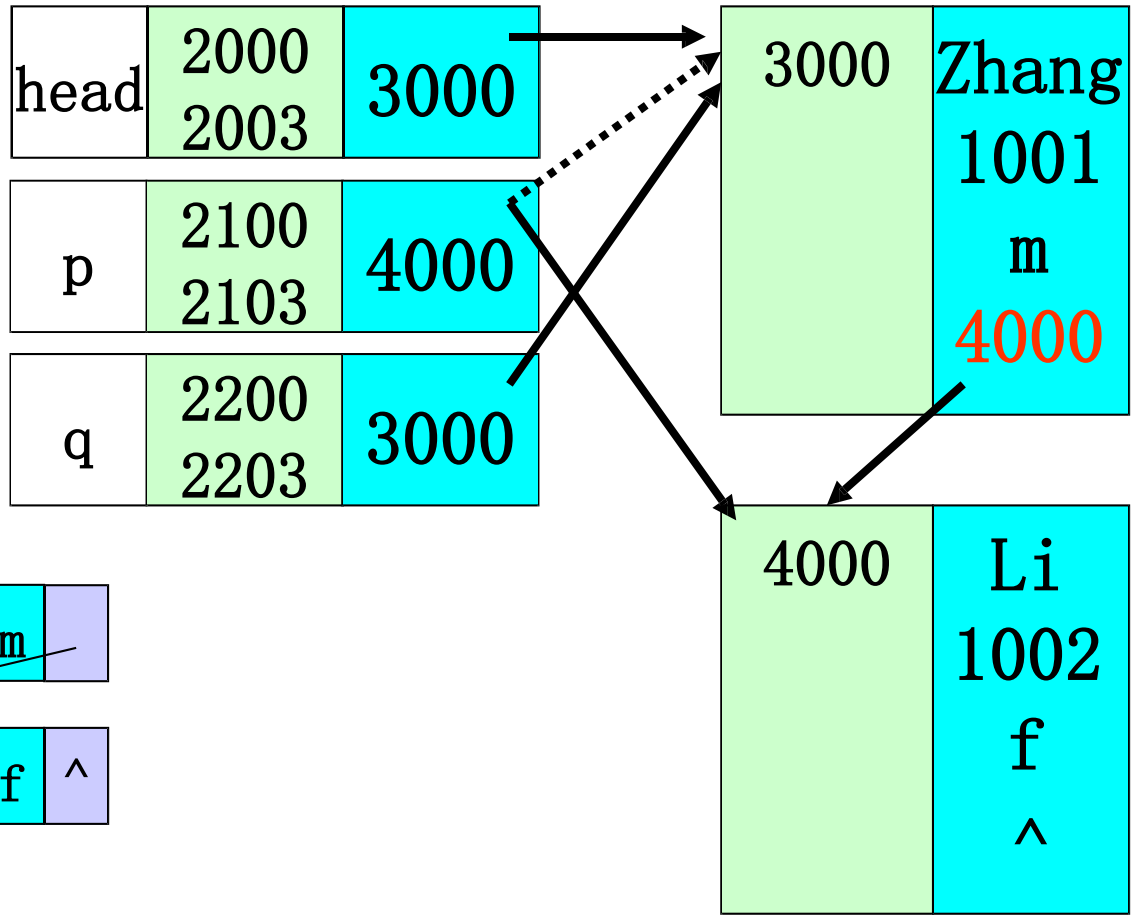
```
cin >> p->name >> p->num >> p->sex; //键盘输入基本信息
```

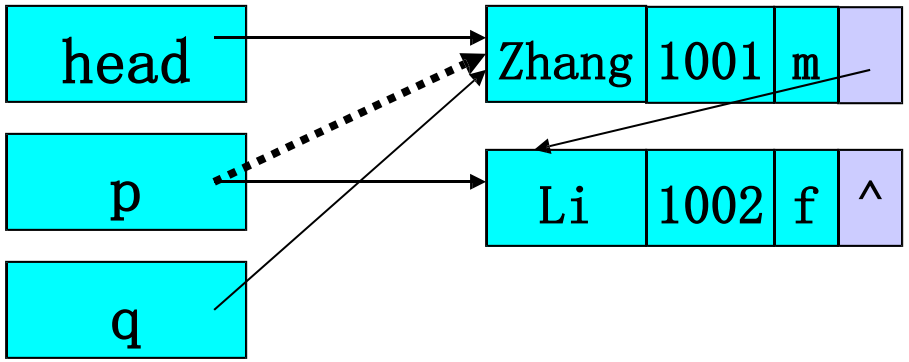
i=1的循环：



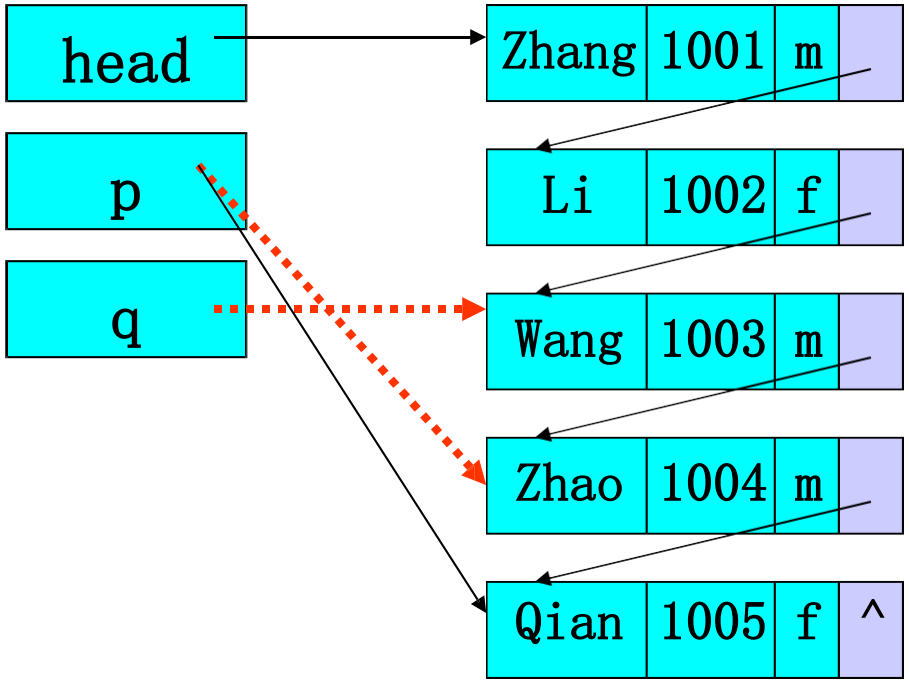
p->next = NULL;

i=1的循环结束：

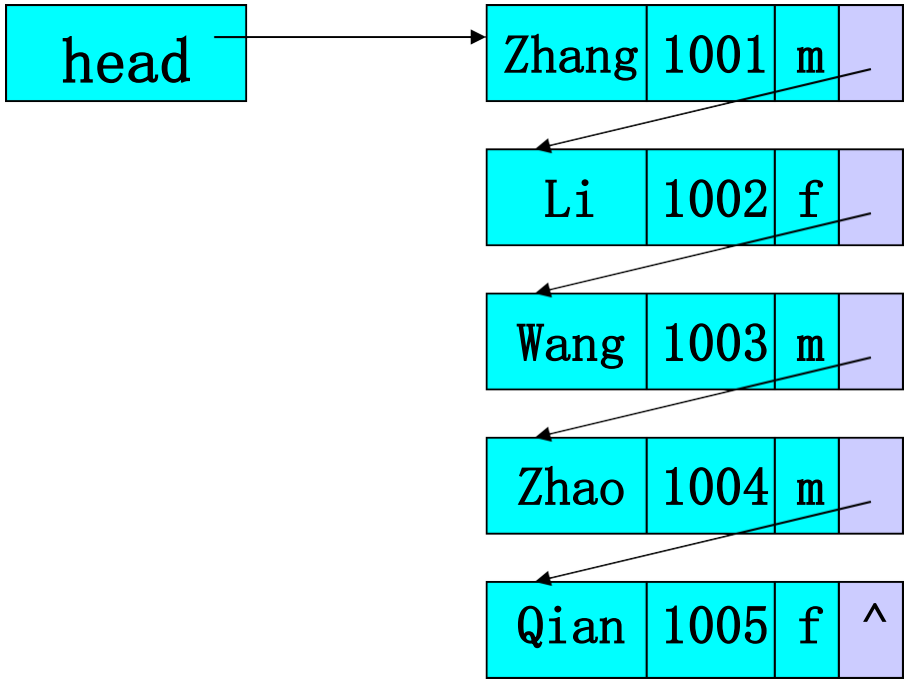




i=4的循环结束：



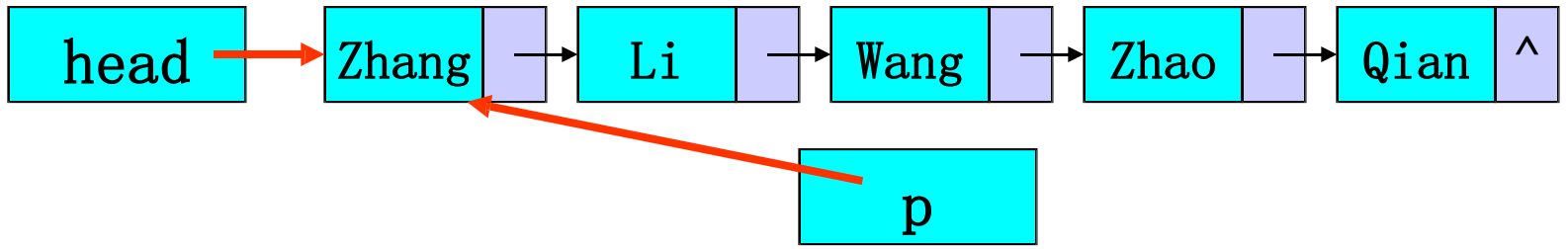
循环完成：



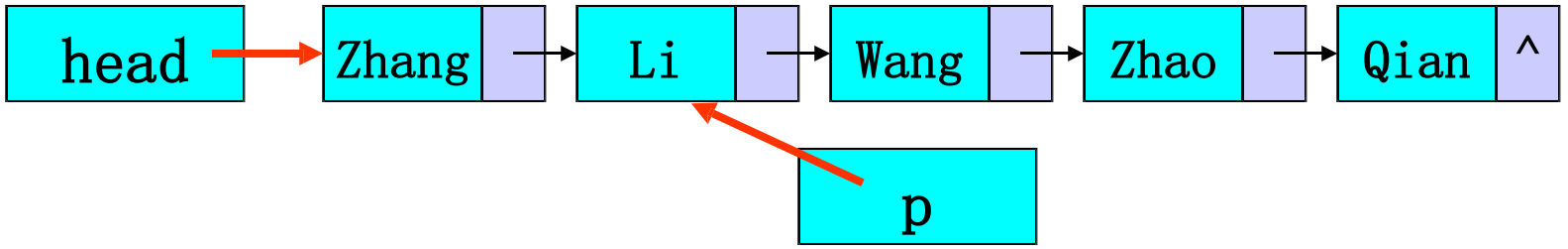
//在前例基础上完整程序

```
int main()
{   student *head = NULL, *p = NULL, *q = NULL; int i;
    for(i = 0; i < 5; i++) { …//刚才建立链表的循环}
    p = head; //p复位，指向第1个结点
    while(p != NULL) { //循环进行输出
        cout << p->name << " " << p->num << " " << p->sex << endl;
        p = p->next;
    }
    p = head; //p复位，指向第1个结点
    while(p) { //循环进行各结点释放
        q = p->next;
        delete p;
        p = q;
    }
    return 0;
}
```

```
p = head; //p复位，指向第1个结点
```



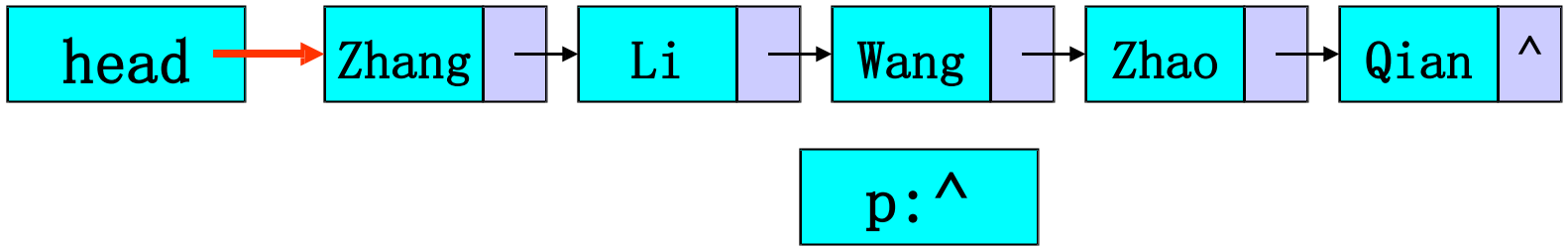
```
p = p->next;
```



Zhang	1001	m

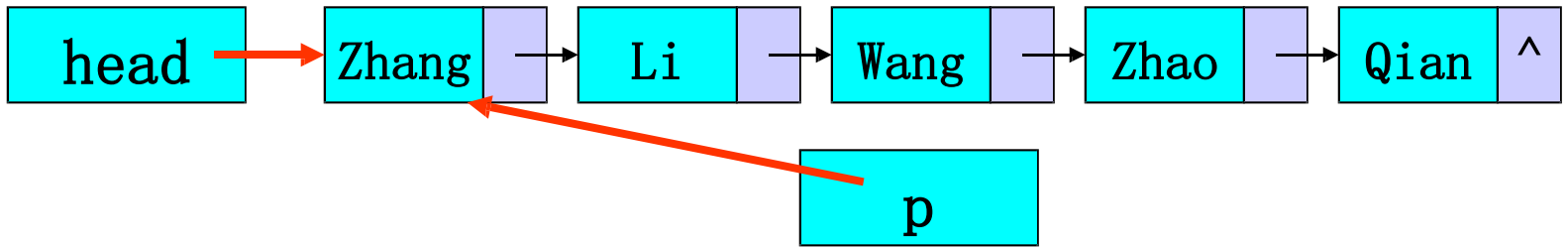
while(p!=NULL)

循环结束：

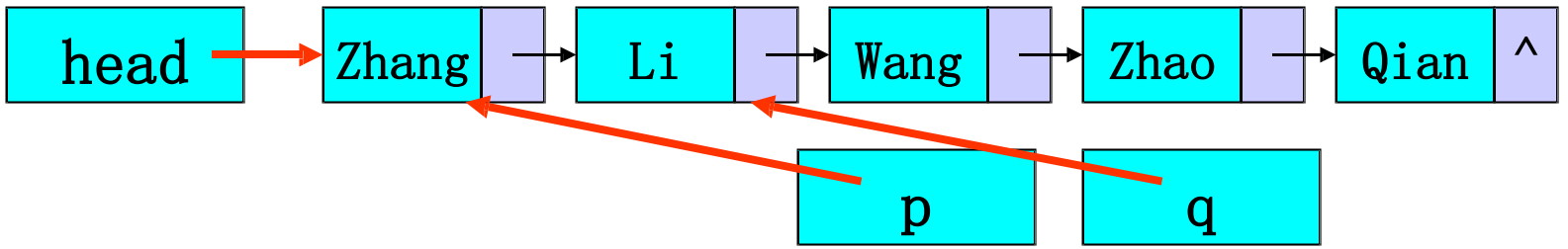



```
Zhang 1001 m
Li 1002 f
Wang 1003 m
Zhao 1004 m
Qian 1005 f
```

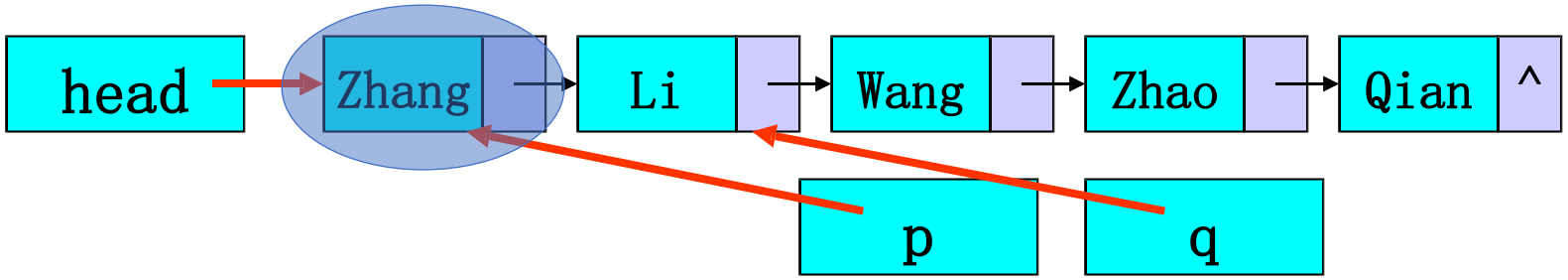
```
p = head; //p复位，指向第1个结点
```



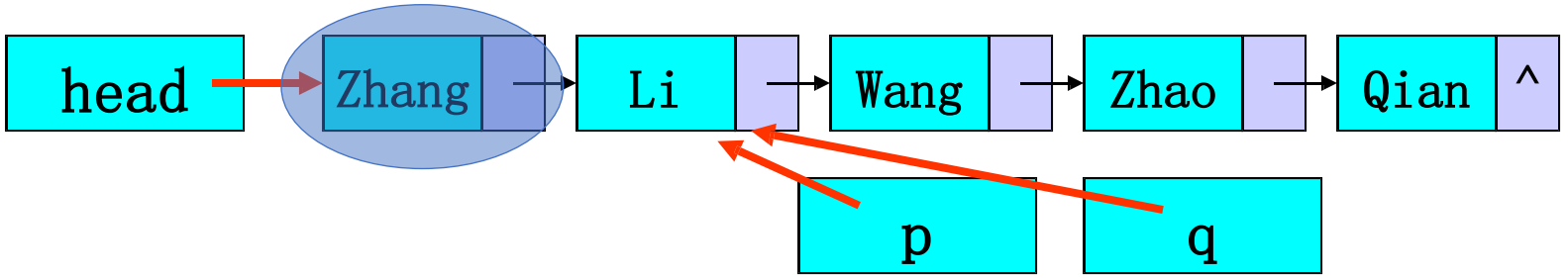
```
q = p->next;
```



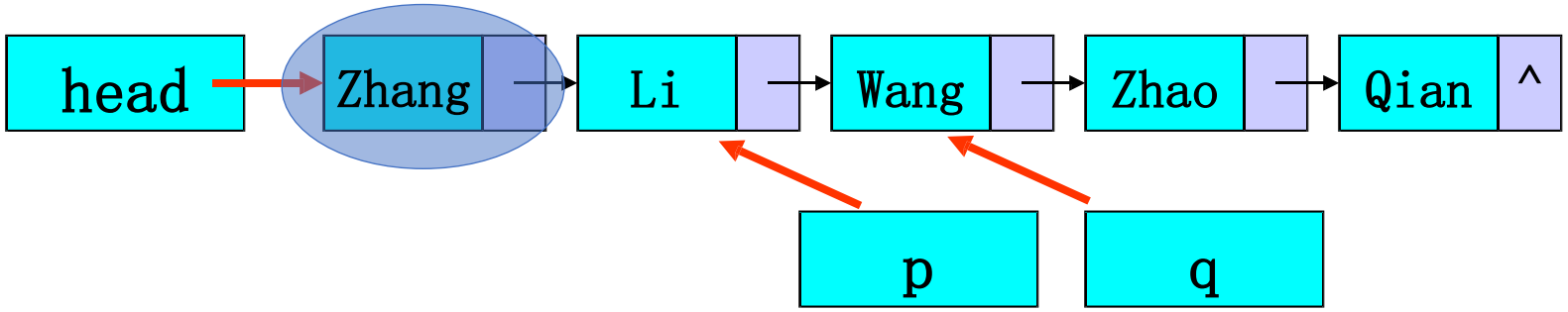
```
delete p;
```



```
p = q;
```



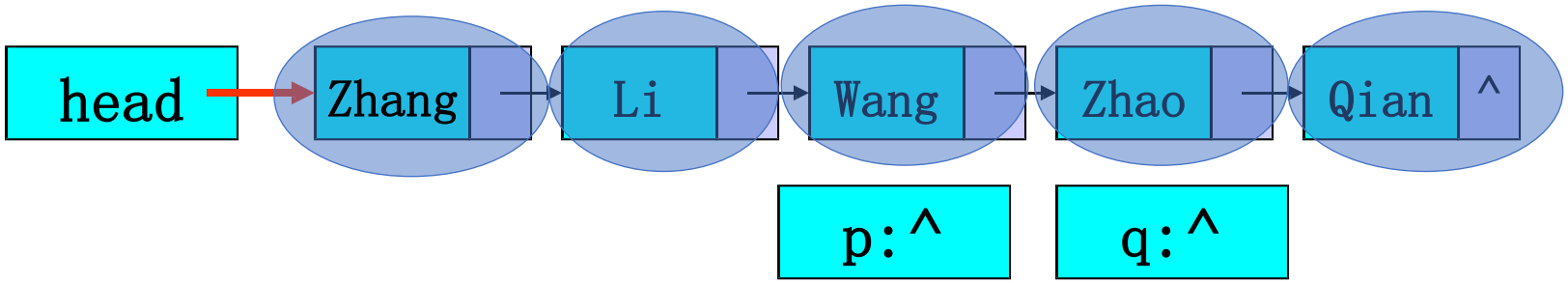
```
q = p->next;
```



while(p){…}

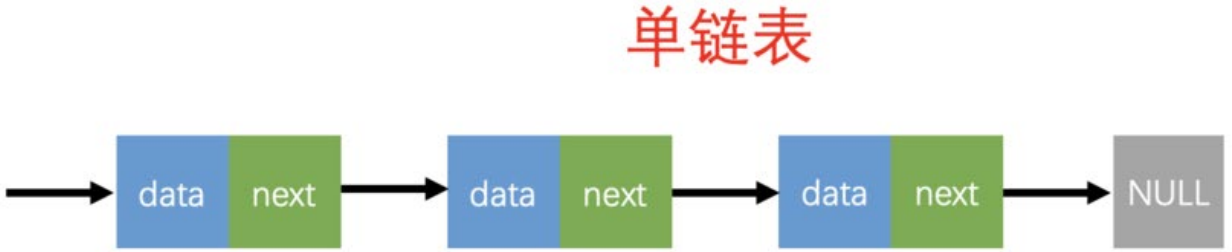
循环结束：new申请的5个空间已被释放

指针变量head/p/q自身不是动态申请空间，由操作系统回收



目录

- 链式结构的基本概念
- 单链表的基本操作
- 双向链表的基本操作
- 二叉搜索树



2.1 链表的遍历

- 由于链表的指针域中包含了后继结点的存储地址，所以只要知道该链表的头指针，即可依次对每个结点进行访问
- 假设已创建包含一个数据域，且其类型为整型的单链表，head为表头指针

结点结构说明：

```
struct node
{
    int data;
    struct node *next;
};
```

头指针定义：struct node *head;

```
//输出已建立单链表的各结点的值
void print(struct node *head)
{
    struct node *p = head;
    while(p != NULL){
        cout << p->data << '\t';
        p = p->next;
    }
}
```

2.2 统计结点个数

- 设置工作指针从表头结点开始，每经过一个结点，计数器值加1

```
int count(struct node *head)
{
    struct node  *p = head;
    int n = 0;
    while (p != NULL){
       n++;
       p = p->next;
   }
    return(n);
}
```

2.3 查找结点

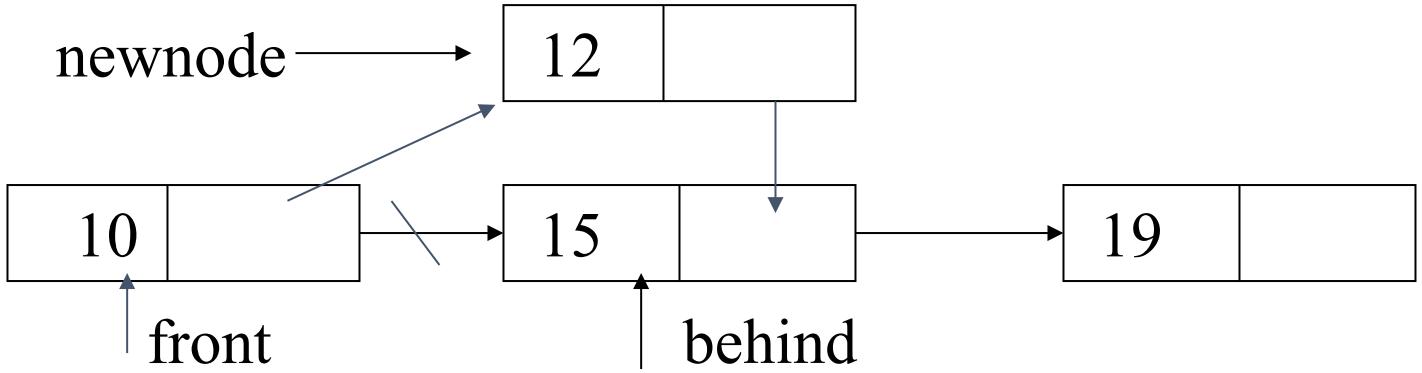
- 设置一个序号计数器j和一个工作指针p，从表头结点开始，顺着链表进行查找。仅当j==i并且p!= NULL时查找成功，否则查找不成功

```
void search(struct node *head, int i)
{   int j = 1;
    struct node *p = head;
    if(i < 0)
        cout << "illegal index\n";
    else
    {
        while(j != i && p != NULL){
            j++;
            p = p->next ;
        }
```

```
//接左边
  if(j == i && p != NULL)
     cout<<"index"<<i<<":"<<p>data;
  else
     cout<<"illegal index \n";
  }
}
```

2.4 在链表中插入结点

- 假定有一个指针behind指向链表中的某个结点，newnode指向待插入结点



- 如果有一个指针front指向behind的前驱，则：

```
front->next = newnode;
newnode->next = behind;
```

- 如果没有behind指针，则：

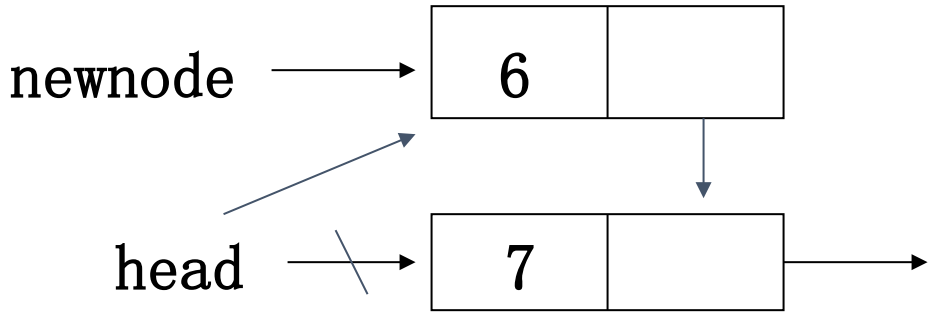
```
newnode->next = front->next;
front->next = newnode;        //不可交换次序
```

2.5两种特殊情况：

- 在表头结点之前插入：

```
newnode->next = head;

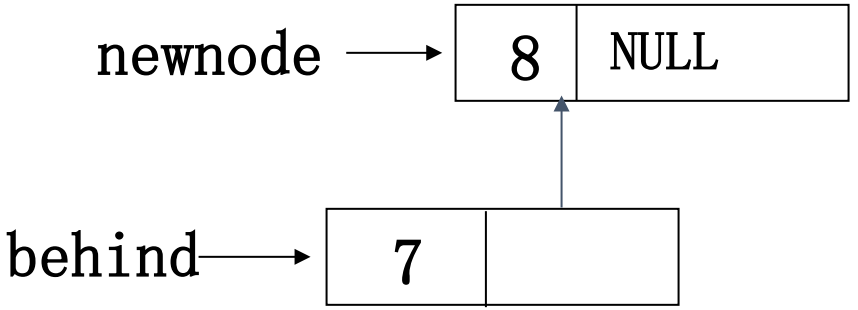
head = newnode;
```



- 在尾结点之后插入：

```
behind->next = newnode;

newnode->next = NULL;
```



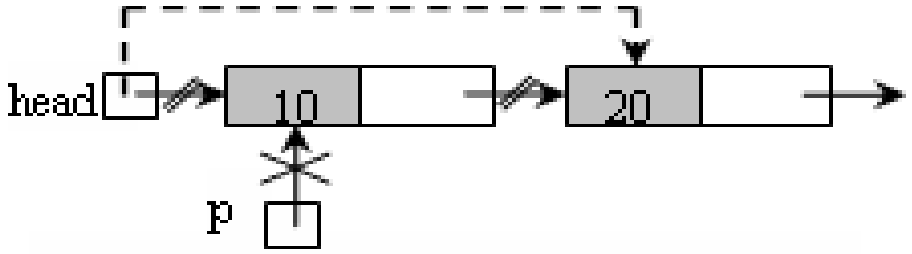
```
//实现在头结点为head的链表中插入值为x的结点（已按数据递增排序）。
struct node * insert(node *head,int x)
{  struct node *behind, *front, *newnode;
   newnode=new(nothrow) node; //未判断newnode是否为NULL，后续课程内容
   newnode->data = x;      behind = head;
   if(head == NULL) {head = newnode; newnode->next = NULL;} //空表
   else                    //非空表
   {   while(behind != NULL && x > behind->data)  //找插入位置
       {   front = behind; behind = behind->next; }
       if(behind == head)                  //插到第一个结点前
       {   newnode->next = head; head = newnode; }
       else if(behind == NULL)             //插到最后一个结点后
       {   front->next = newnode; newnode->next = NULL; }
       else                                //插到front之后,behind之前
       {   front->next = newnode; newnode->next = behind; }
   }
   return head;
}
```

2.6 删除链表中的某个结点

- 把被删除结点的后继结点的地址,赋给其前趋结点的指针域或表头指针head，无后继结点时，则赋NULL

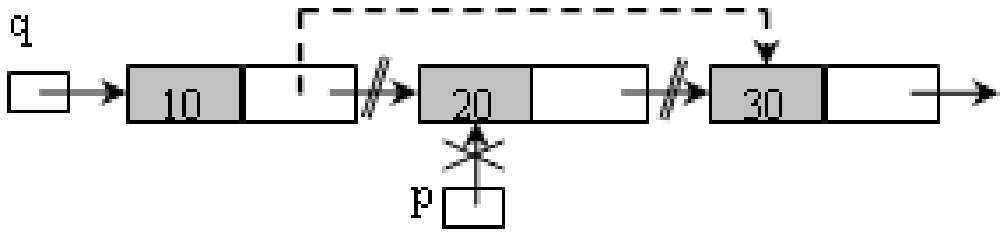
假定p为指向要删除结点的指针，q为指向删除结点前趋的指针

- 如删除第一个结点




```
head = p->next;
delete p;
```

- 如删除链表的中间结点




```
q->next = p->next;
delete p;
```

```
//实现在头结点为head的链表中删除值为x的结点。
struct node *delnode(node *head,int x)
{   struct node *p,*q;  //p为工作指针,q为p的前驱
    p = head;
    if(head == NULL)          //空表
       cout<<"The list is null!\n";
    else{                     //非空表
       while(p != NULL && p->data != x)  //找删除的结点
       {   q = p;  p = p->next; }
       if(p == head)                          //删除第一个结点
       {   head = p->next;  delete p; }
       else if(p != NULL)                     //删除非表头结点
       {   q->next = p->next ;  delete p; }
       else                                   //未找到要删除的元素
           cout << x <<"does not exist in the list!\n";
     }
     return head;
 }
```

2.7 带表头结点的单链表

- 表头结点（又称伪结点）位于表的最前端，本身不带数据，仅标志表头
- 设置表头结点的目的是统一空表与非空表、表头和表中位置的操作形式，简化链表操作的实现



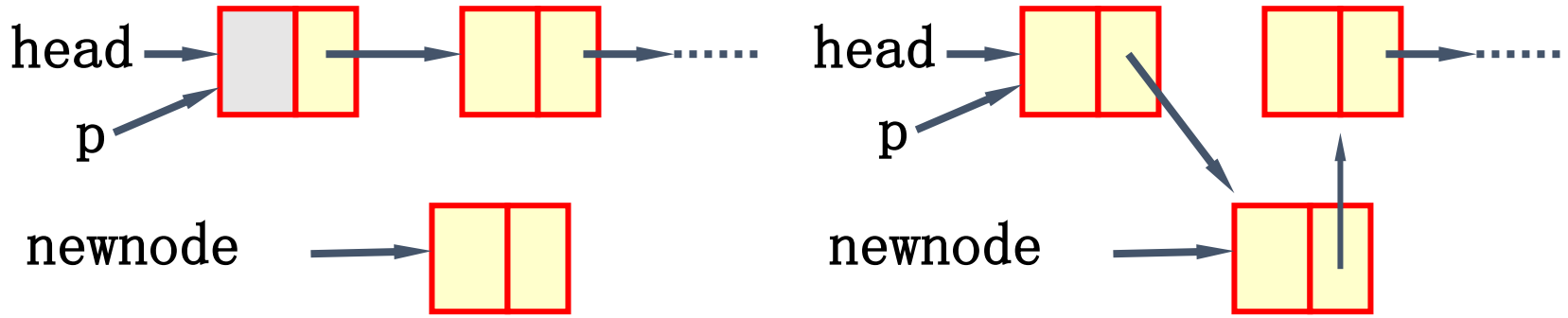
非空表

空表

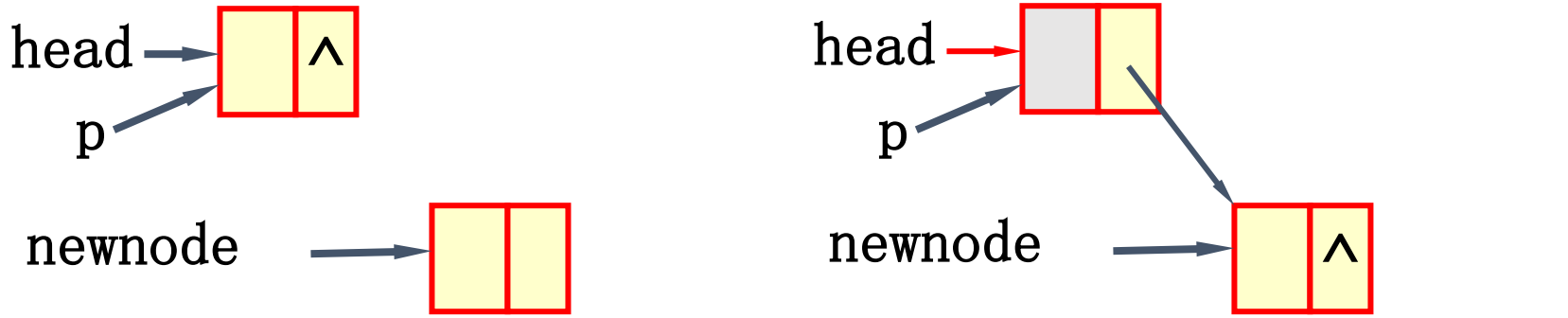
2.8 单链表的基本操作应用

- 例：在带表头结点的单链表最前端插入新结点

非空表



空表

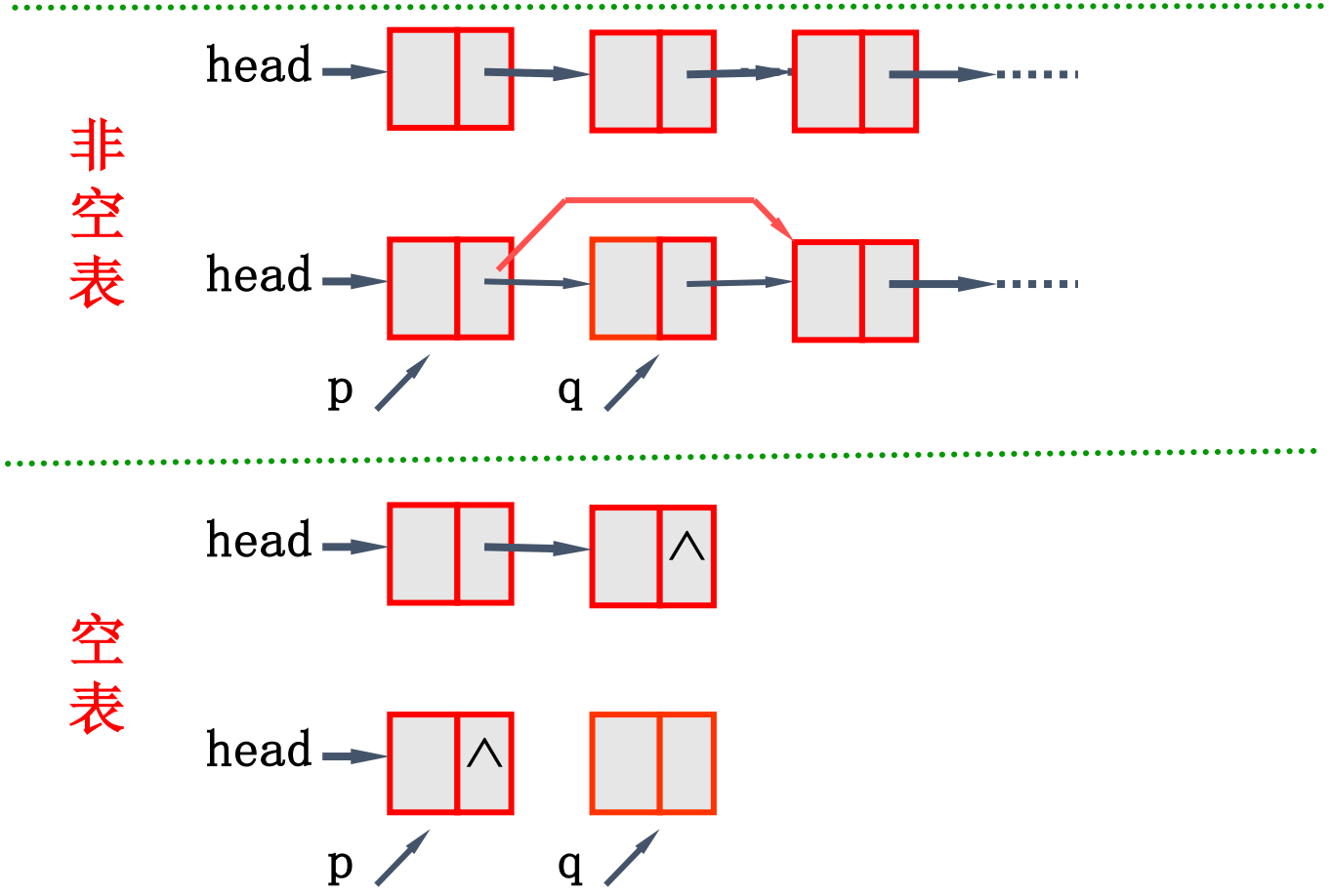


- 空表和非空表的操作是一致的

```
newnode->next = p->next;
p->next = newnode;
```

2.8 单链表的基本操作应用

- 例：从带表头结点的单链表中删除最前端的结点



- 即使删除后为空表，也无需修改head，空表和非空表的操作是一致的

```
q = p->next;
p->next = q->next;
delete q;
```

目录

- 链式结构的基本概念
- 单链表的基本操作
- 双向链表的基本操作
- 二叉搜索树

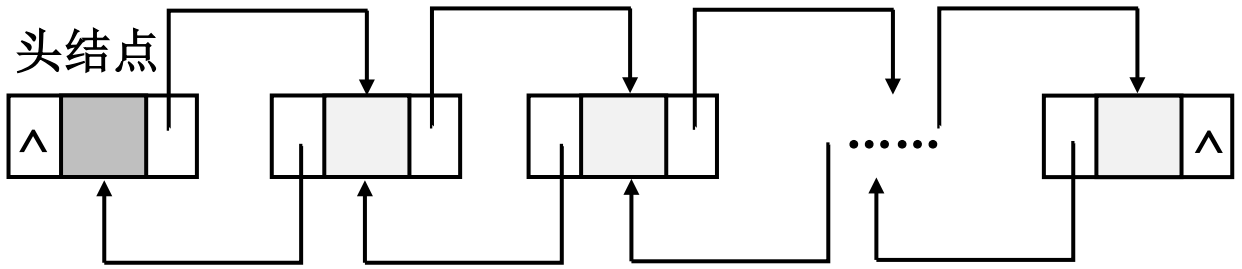


3.1 双向链表的基本操作

- 单链表的缺陷
  - 只能从头结点开始访问链表中的数据元素，如果需要逆序访问单链表中的数据元素将极其低效
- 双向链表的引入
  - 每个数据结点中都有两个指针，分别指向直接后继和直接前驱

双向链表的定义：

```
typedef struct DbNode
{
    int data;  //结点数据
    DbNode *left; //前驱结点指针
    DbNode *right; //后继结点指针
}DbNode;
```



```
#include <iostream>
using namespace std;
typedef struct DbNode
{
    int data;//结点数据
    DbNode *left;//前驱结点指针
    DbNode *right;//后继结点指针
}DbNode;
//根据数据创建结点
DbNode *CreateNode(int data)
{
    DbNode *pnode = (DbNode *)malloc(sizeof(DbNode));
    if (pnode != NULL) //判断malloc是否成功申请到存储空间，后续课程内容
    {    pnode->data = data;
         pnode->left = pnode->right = NULL;
    }
    return pnode;
}
```

```
//创建链表头
DbNode *CreateList(int hdata)//头结点数据
{
    DbNode *pnode = (DbNode *)malloc
                    (sizeof(DbNode));
    if (pnode != NULL)
    {   pnode->data = hdata;
        pnode->left = pnode->right = NULL;
    }
    return pnode;
}
```

```
//插入新结点，总是在表尾插入
DbNode *AppendNode(DbNode *head, int data)
{
    DbNode *node = CreateNode(data);
    if (node != NULL)
    {
        DbNode *p = head, *q = NULL;

        while (p != NULL)
        {
            q = p;
            p = p->right;
        }
        if (q != NULL) q->right = node;
        node->left = q;
    }
    return head;
}
```

```
int main()
{
    DbNode *head = CreateList(0);
            //生成链表头结点，数据为0
    for (int i = 1; i < 10; i++)
            //添加9个结点，数据为1到9
    {
        head = AppendNode(head, i);
    }

    return 0;
}
```

3.2 双向链表的遍历

- 使用right指针进行遍历，直到NULL为止

```
//双向链表的测长
int GetLength(DbNode *head)
{
    int count = 1;
    DbNode *pnode = NULL;
    if (head == NULL) //链表空
    {
        return 0;
    }
    pnode = head->right;
    //接左侧
    while (pnode != NULL)
    {
        pnode = pnode->right;
                //使用right指针遍历
        count++;
    }
    return count;
}
```

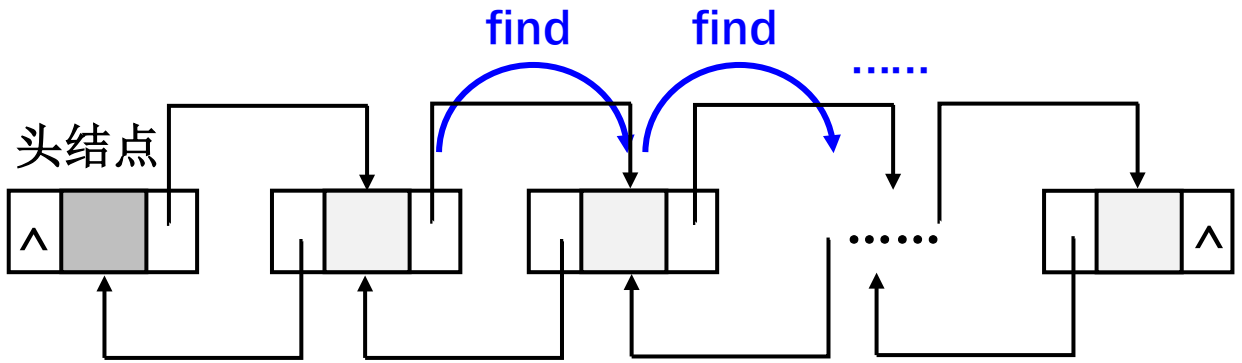
3.2 双向链表的遍历

- 使用right指针进行遍历，直到NULL为止

```
//打印整个链表
void PrintList(DbNode *head)
{
    DbNode *pnode = NULL;
    if (head == NULL)//链表为空
    {
        return;
    }
    pnode = head;
    //接左侧
    while (pnode != NULL)
        {
            printf("%d ", pnode->data);
            pnode = pnode->right;
                    //使用right指针遍历
        }
        printf("\n");
}
```

3.3 双向链表结点的查找

- 使用right指针进行遍历，直到找到数据为data的结点。如果找到则返回结点，否则返回NULL

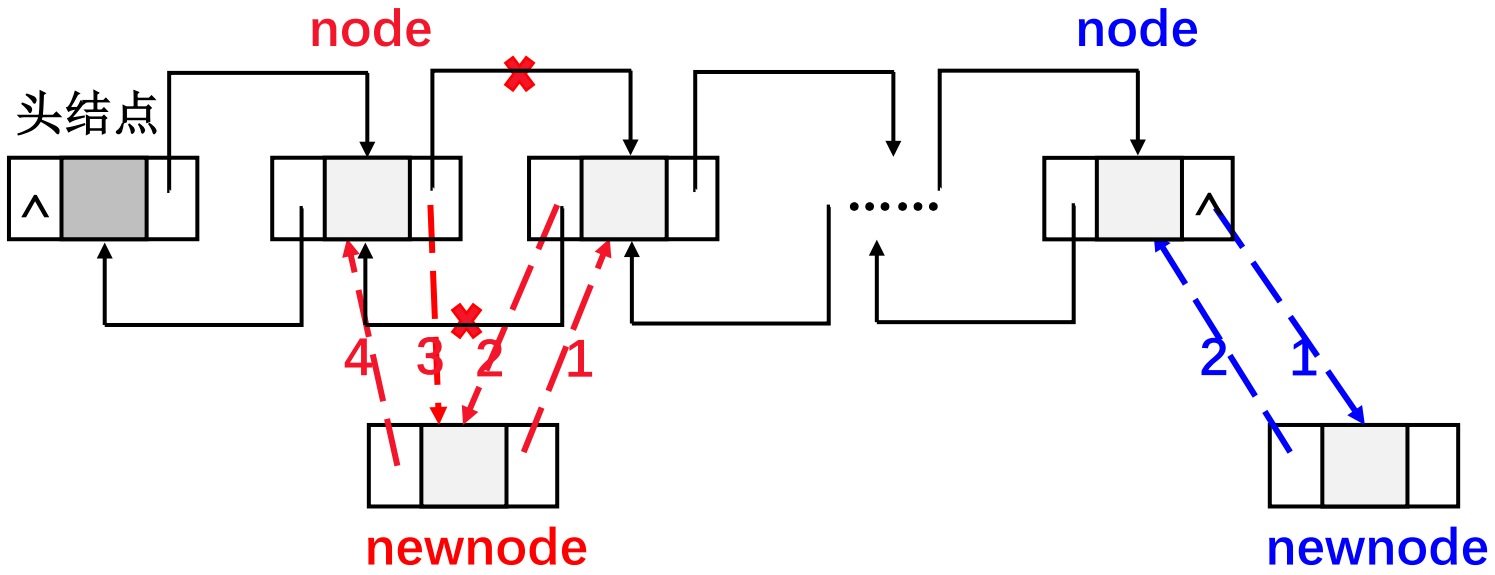


```
//查找数据为data的结点，如果找到则返回结点，否则返回NULL
DbNode *FindNode(DbNode *head, int data)
{
    DbNode *pnode = head;
    if (head == NULL) return NULL;    //链表空

    //找到数据或者到达链表末尾，退出while循环
    while (pnode != NULL && pnode->data != data)
    {
        pnode = pnode->right;          //使用right指针遍历
    }
    //没有找到数据为data的结点，返回NULL
    if (pnode == NULL)
    {
        return NULL;
    }
    return pnode;
}
```

3.4 双向链表结点的插入

- 插入位置在中间：原后继结点的前驱指针指向新插入结点
- 插入位置在末尾：直接插入

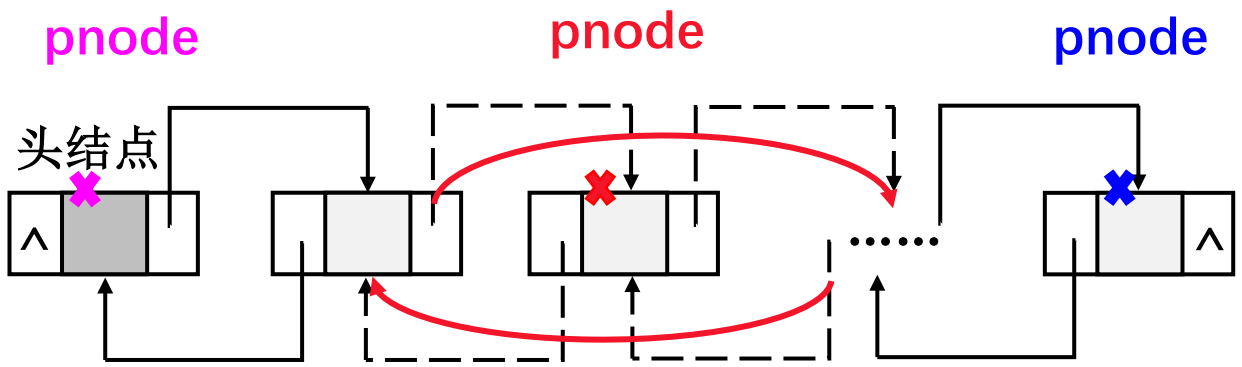


```
//在node结点之后插入新结点
void InsertNode(DbNode *node, int data)
{
    DbNode *newnode = CreateNode(data);

    if (node == NULL) return;  //node为NULL时返回
    if (node->right == NULL)  //node为最后一个结点
    {
        node->right = newnode; newnode->left = node;
    }
    else   //node为中间结点
    {
        newnode->right = node->right;
        node->right->left = newnode;
        node->right = newnode;
        newnode->left = node;
    }
}
```

3.4 双向链表结点的删除

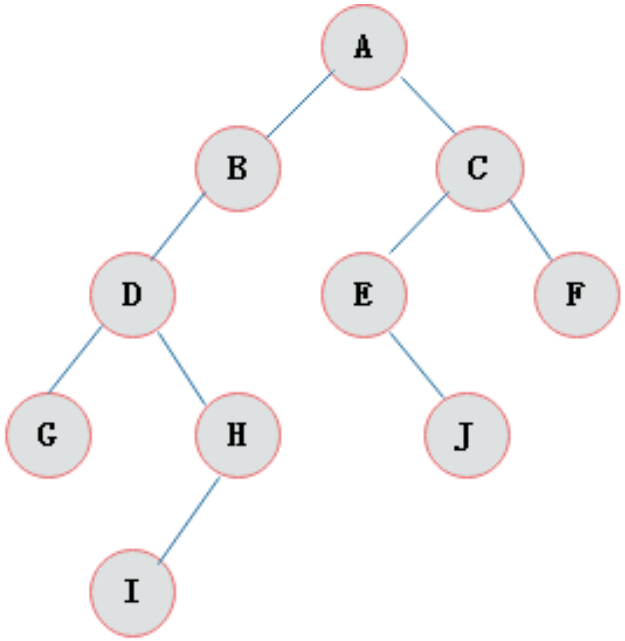
- 删除头结点
- 删除中间结点
- 删除尾结点



```
//删除满足指定条件的结点，返回表头指针，删除失败（结点不存在）返回NULL
DbNode *DeleteNode(DbNode *head, int data)
{
    DbNode *pnode = FindNode(head, data); //查找结点
    if (pnode == NULL) return NULL;  //结点不存在时返回NULL
    else if (pnode->left == NULL)  //pnode为第一个结点
    {
        head = pnode->right;
        if (head != NULL)  head->left = NULL;  //链表不为空
    }
    else if (pnode->right == NULL) //node为最后一个结点
    {
        pnode->left->right = NULL;
    }
    else   //node为中间的结点
    {
        //接右侧
            pnode->left->right = pnode->right;
            pnode->right->left = pnode->left;
        }
        free(pnode);
        return head;
}
```

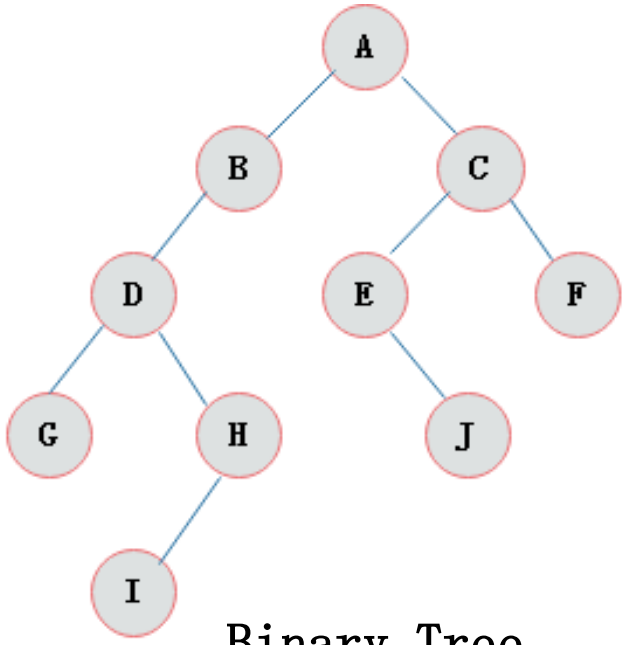
目录

- 链式结构的基本概念
- 单链表的基本操作
- 双向链表的基本操作
- 二叉搜索树



4.1 二叉搜索树（Binary Search Tree）

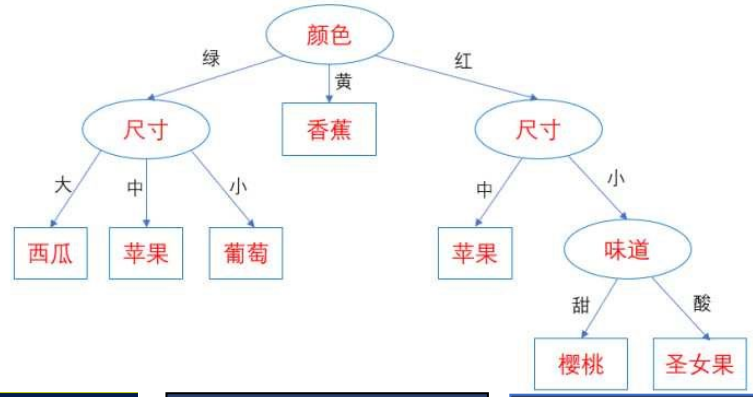
- 每个结点最多有两个子结点的树
- 叶子结点有0个子结点，根结点或者内部结点有一个或两个子结点



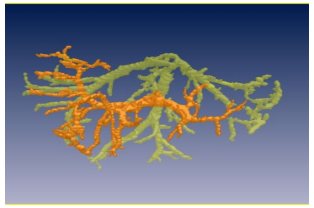
Binary Tree

如何存储？

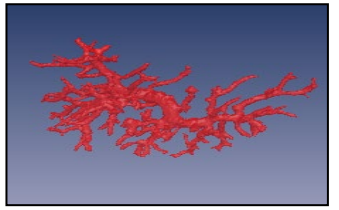
树的应用案例1：



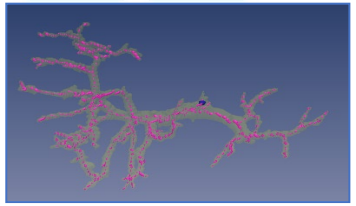
案例2：



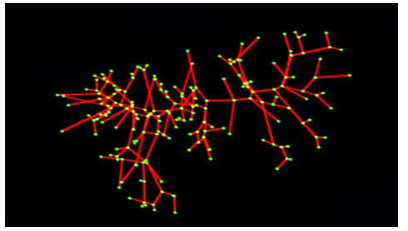
(a)：肝静脉与门静脉



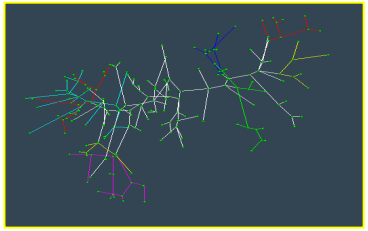
(b)：门静脉



(c)：门静脉骨架化



(d)：门静脉血管树图

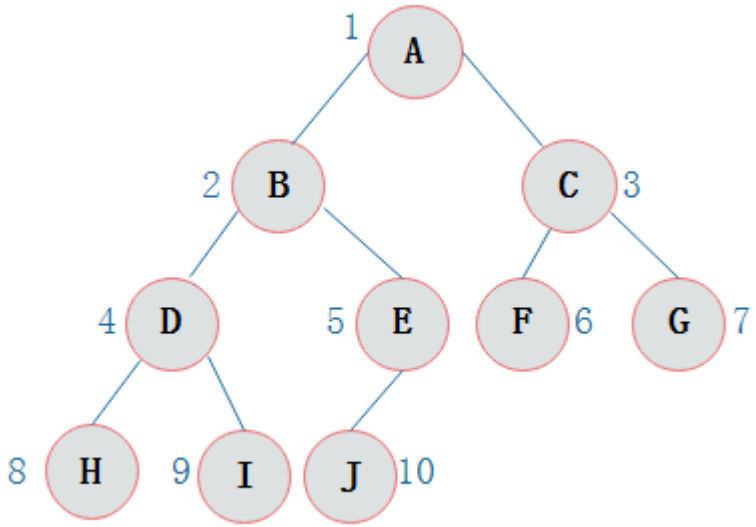


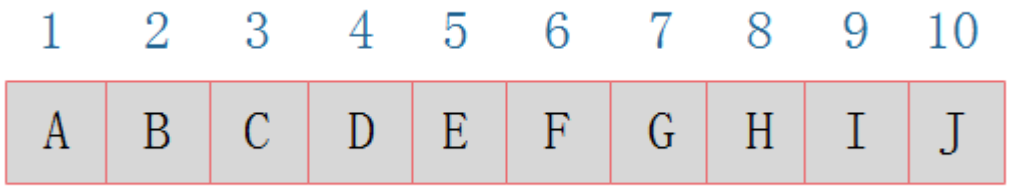
(e)：血管树分支归类

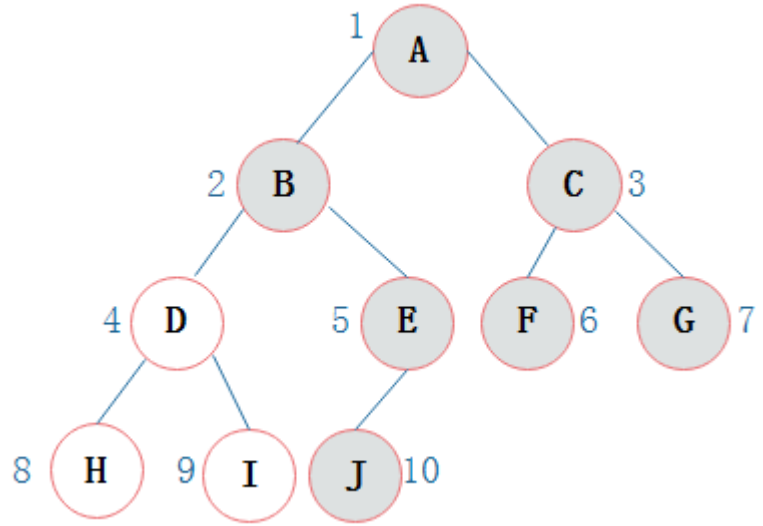
4.2 二叉树存储

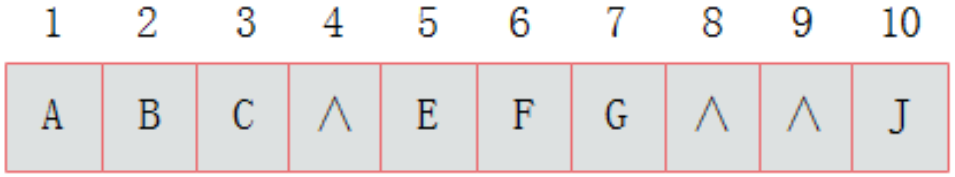
① 顺序存储：使用一维数组存储二叉树中的结点，并且结点的存储位置，就是数组的下标索引

缺点：一般只适合完全二叉树，否则采用顺序存储的方式十分浪费空间



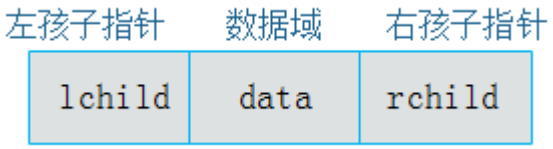




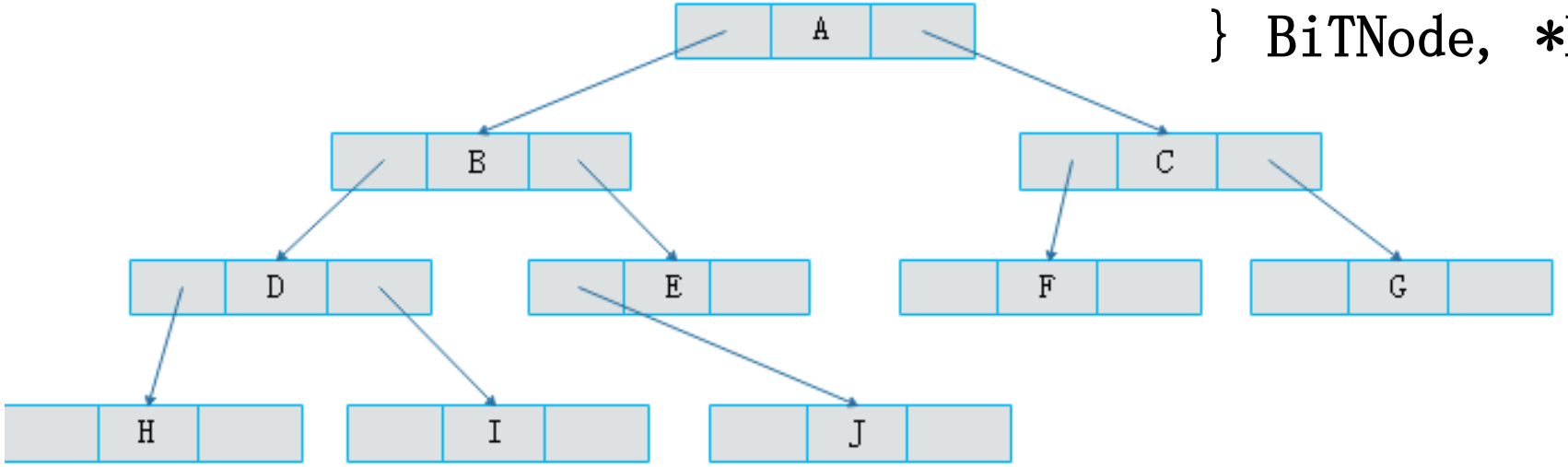


4.2 二叉树存储

② 链式存储：二叉树的每个结点最多有两个子结点，因此可以将结点数据结构定义为一个数据和两个指针域

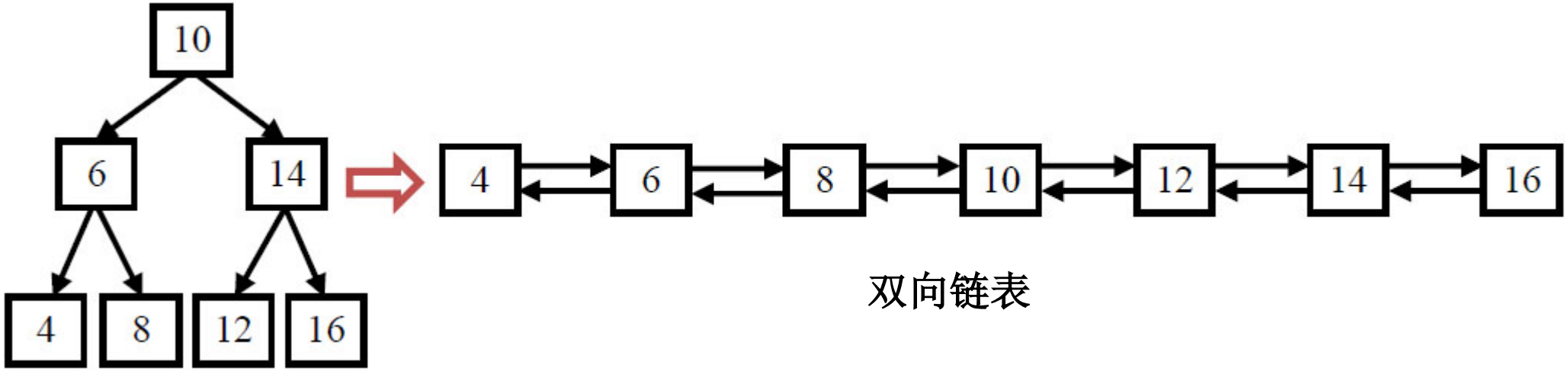



```
typedef struct BiTNode{
    TElemType data;//数据
    struct BiTNode *lchild, *rchild;
                        //左右孩子指针
} BiTNode, *BiTree;
```



4.3 二叉搜索树（Binary Search Tree）

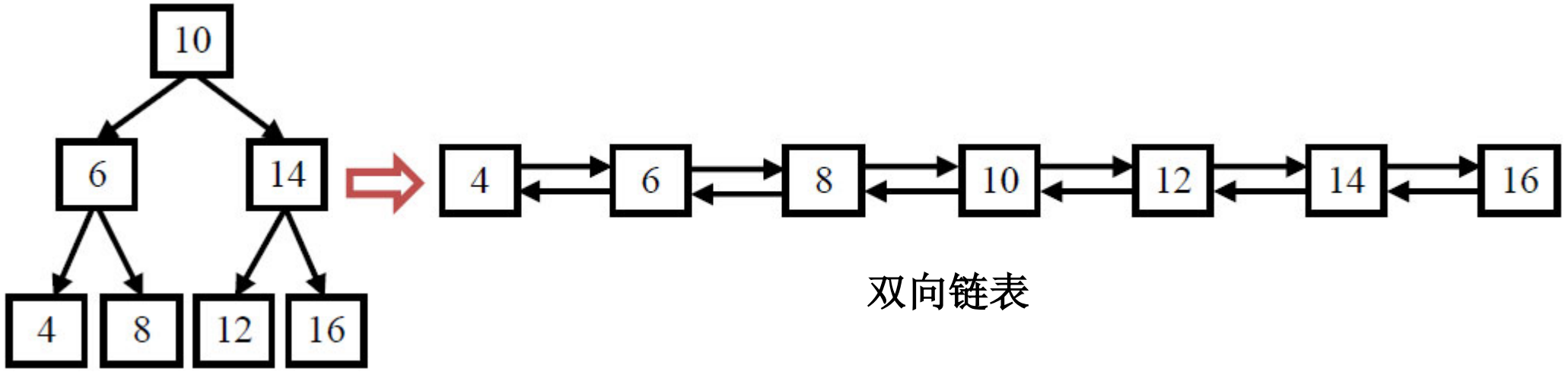
- 若它的左子树非空，则左子树上所有结点的值均小于根结点的值
- 若它的右子树非空，则右子树上所有结点的值均大于根结点的值
- 左、右子树本身又各是一颗二叉搜索树



二叉搜索树

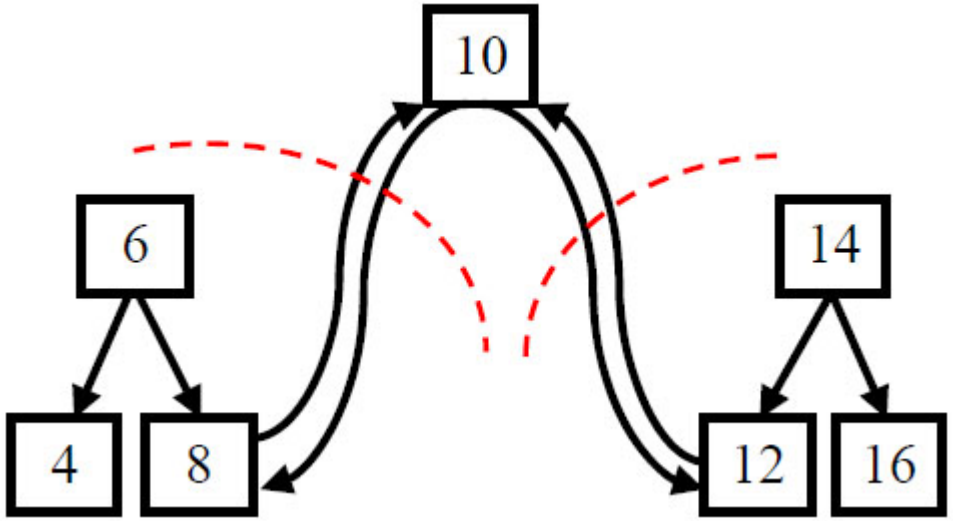
双向链表

- 输入一棵二叉搜索树，将该二叉搜索树转换成一个排序的双向链表。要求不能创建任何新的结点，只能调整树中结点指针的指向。



二叉搜索树

双向链表



中序遍历
递归

数据结构课程，本课程了解应用即可

总结

- 链式结构的基本概念
- 单链表的基本操作（熟练掌握）
  - 遍历、统计、查找、插入、删除等
  - 带表头结点的链表
- 双向链表的基本操作（了解）
- 二叉搜索树（了解）

类和对象的使用

构造函数和析构函数

目录

- 对象的初始化方法
- 构造函数的基本概念
- 构造函数的使用
- 析构函数的基本概念
- 析构函数的使用

1 对象的初始化方法

- 对象的初始化方法

(1) 若全部成员都是公有,可按结构体的方式进行初始化

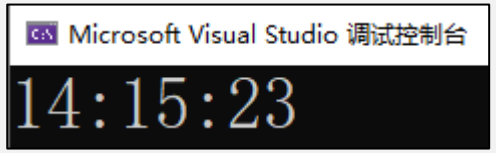
(若有私有成员，不能用此方法)

(2) 写一个赋初值的公有成员函数，在其它成员被调用之前进行调用

(3) 声明类时对数据成员进行初始化（C++11标准支持）

(1)若全部成员都是公有,可按结构体的方式进行初始化

```
#include <iostream>
using namespace std;
class Time {
   public:
      int hour;
      int minute;
      int sec;
      int f2(){ return 0; }; //成员函数不占空间
};
int main()
{
   Time t1={14,15,23};
   cout << t1.hour << ':' << t1.minute << ':' << t1.sec << endl;
}
```



```
(13,27): error C2440: “初始化”: 无法从“int”转换为“Time”
(13,27): message : 无构造函数可以接受源类型，或构造函数重载决策不明确
```

cpp-demo.
cpp-dem

```cpp
#include <iostream>
using namespace std;

class Time {
public:
    int hour;
    int minute;
    int sec;
    int f2() { return 0; }; //成员函数不占空间
};
int main()
{
    Time t1 = (14, 15, 23);
    cout << t1.hour << ':' << t1.minute << ':' << t1.sec << endl;
}
```

以下两种形式均报错：

```cpp
Time t1=(14,15,23);
Time t1(14,15,23);
```

```
(13,23): error C2440: “初始化”: 无法从“initializer list”转换为“Time”
(13,23): message : 无构造函数可以接受源类型，或构造函数重载决策不明确
```

```cpp
#include <iostream>
using namespace std;

class Time {
public:
    int hour;
    int minute;
    int sec;
    int f2() { return 0; }; //成员函数不占空间
};
int main()
{
    Time t1(14, 15, 23);
    cout << t1.hour << ':' << t1.minute << ':' << t1.sec << endl;
}
```

以下两种形式均报错：

```cpp
Time t1=(14,15,23);
Time t1(14,15,23);
```

error C2440: “初始化”: 无法从“initializer list”转换为“Time”
message : 无构造函数可以接受源类型，或构造函数重载决策不明确

```
2   using namespace std;
3   class Time {
4   public:
5       int hour;
6       int minute;
7   private:
8       int sec;
9       int f2() { return 0; }; //成员函数不占空间
10  };
11  int main()
12  {
13      Time t1 = { 14, 15, 23 };
14      cout << t1.hour << ':' << t1.minute << endl;
15  }
```

缺点：若有私有数据成员，不能用此方法

(2)写一个赋初值的公有成员函数，在其它成员被调用之前进行调用

```
class Time {
    private:
        int hour;
        int minute;
        int sec;
     public:
        void set(int h, int m, int s)
        {
            hour=h;
            minute=m;
            sec=s;
        }
};
```

```
int main()
{
    Time t;
    t.set(14,15,23);
    t.其它
}
```

(3) 声明类时对数据成员进行初始化（C++11标准支持）

```
class Time {
    public:
        int hour   = 0;
        int minute = 0;
        int sec    = 0;
};
```

缺点：不同对象的值被统一初始化，无法个性化

⟹ 需要引入合适的初始化方式！！！ ⟹ 构造函数！！！

目录

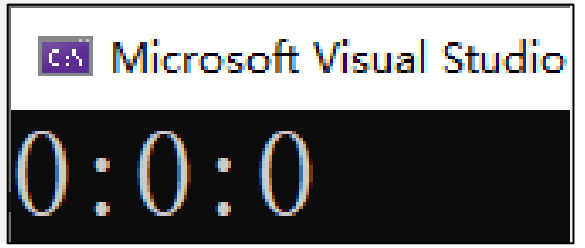
- 对象的初始化方法
- 构造函数的基本概念
- 构造函数的使用
- 析构函数的基本概念
- 析构函数的使用

2 构造函数的基本概念

- 构造函数的功能：
  - 完成对象的初始化工作，对象建立时被自动调用
- 构造函数的形式：
  - 与类同名，无返回类型(非void，也不是缺省int)
  - 构造函数必须公有
  - 构造函数既可以体内实现，也可以体外实现

```
class Time {
    private:
        int hour, minute, sec;  //相同类型可以写在一行上
    public:
        Time()  无返回值(非void，也不是缺省int)
        {   hour   = 0;
            minute = 0;                    体内实现
            sec    = 0;
        }
        void display()
        {   cout << hour << ':' <<minute << ':' << sec << endl;
        }
};
```

```
int main()
{
    Time t;
    t.display();
}
```

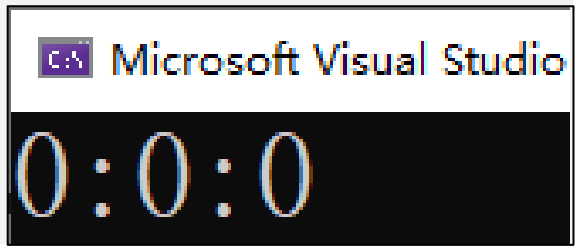


```
class Time {
   private:
      int hour, minute, sec;  //相同类型可以写在一行上
   public:
      Time(); //函数声明
      void display()
      {   cout << hour << ':' <<minute << ':' << sec << endl;
      }
};
```

```
Time::Time()  无返回值(非void，也不是缺省int)
{   hour=0;
    minute=0;
    sec=0;
}
```

体外实现

```
int main()
{
    Time t;
    t.display();
}
```



目录

- 对象的初始化方法
- 构造函数的基本概念
- 构造函数的使用
- 析构函数的基本概念
- 析构函数的使用

3 构造函数的使用

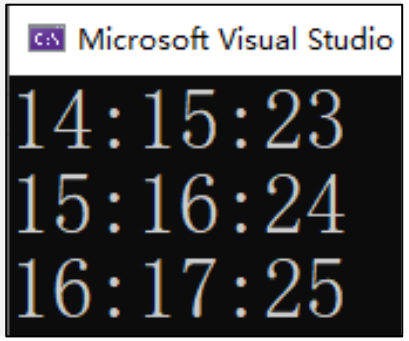
- 构造函数的使用
  - 若不指定构造函数，则系统缺省生成一个无参空体构造函数
  - 若用户定义了构造函数，则缺省构造函数不再存在
  - 允许定义带参数的构造函数，以解决无参构造函数初始化各对象的值相同的情况

```
class Time {
    private:
        int hour, minute, sec;
    public:
        Time(int h, int m, int s)
        {   hour   = h;
            minute = m;
            sec    = s;
        }
        void display()
        {   cout << hour << ':' <<minute << ':' << sec << endl;
        }
};
```

体内实现

```
int main()
{
    Time t1 (14,15,23);
    Time t2 {15,16,24};
    Time t3={16,17,25};
    t1.display();
    t2.display();
    t3.display();
}
```

三种形式均可



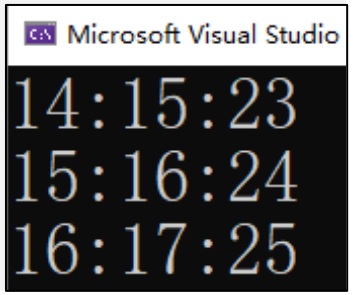
```
class Time {
   private:
      int hour, minute, sec;
   public:
      Time(int h, int m, int s);
      void display()
      {   cout << hour << ':' <<minute << ':' << sec << endl;
      }
};
```

```
Time::Time(int h, int m, int s)
{   hour   = h;
    minute = m;
    sec    = s;
}
```

体外实现

```
int main()
{
    Time t1 (14,15,23);
    Time t2 {15,16,24};
    Time t3={16,17,25};
    t1.display();
    t2.display();
    t3.display();
}
```

三种形式均可



```
(22,19): error C2661: “Time::Time”: 没有重载函数接受 2 个参数
(23,10): error C2512: “Time”: 没有合适的默认构造函数可用
(4,7): message : 参见“Time”的声明
```

```
    public:
        Time(int h, int m, int s);
        void display()
        {   cout << hour << ':' <<minute << ':' << sec << endl;
        }
};
```

```
Time::Time(int h, int m, int s)
{   hour   = h;
    minute = m;
    sec    = s;
}
```

体外实现

```
//仅三参构造，则不符合会报错
int main()
{
    Time t1(14,15);
    Time t2;
}
```

3 构造函数的使用

- 构造函数的使用（续）
  - 有参构造函数可以使用参数初始化表来对数据成员进行初始化

    （仅适用于简单的赋值）

```
//无法通过参数初始化表实现的案例
Time::Time(int h, int m, int s)
{
    if (h>=0 && h<=23)
        hour = h;
    else
        hour = 0;
}
```

（构造函数初始值列表的引入原因等具体分析为后续课程内容，此处掌握基本概念即可）

```
class Time {
    private:
        int hour, minute, sec;
    public:
        Time(int h, int m, int s): hour(h),minute(m),sec(s)
        {
        }
        void display()
        {   cout << hour << ':' <<minute << ':' << sec << endl;
        }
};
int main()
{  Time t1(14,15,23);
   t1.display();
}
```

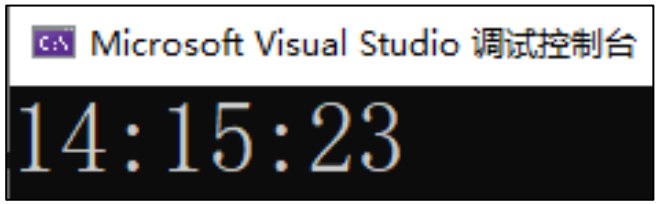
成员名

参数名

h初始化hour
m初始化minute
s初始化sec

函数体为空

体内实现

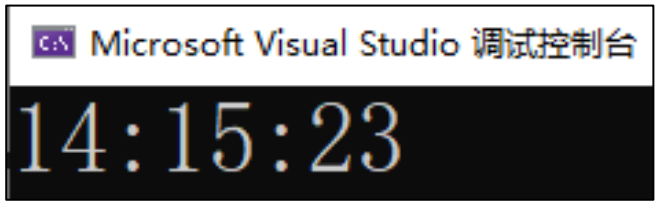


```
class Time {
    private:
        int hour, minute, sec;
    public:
        Time(int h, int m, int s); //函数声明不允许带初始化表
        void display()
        {   cout << hour << ':' <<minute << ':' << sec << endl;
        }
    };
Time::Time(int h, int m, int s): hour(h),minute(m),sec(s)
{
}
int main()
{  Time t1(14,15,23);
   t1.display();
}
```

函数体为空　　体外实现

成员名　参数名

h初始化hour
m初始化minute
s初始化sec



3 构造函数的使用

- 构造函数的使用（续）
  - 构造函数允许重载
  - 构造函数允许带默认参数，但要注意可能与重载产生二义性冲突
  - 构造函数也可以显式调用，一般用于带参构造函数

- 构造函数允许重载

```
class Time {
    ...
    public:
        Time();
        Time(int h, int m, int s);
};
Time::Time()
{  hour   = 0;
   minute = 0;
   sec    = 0;
}
Time::Time(int h, int m, int s)
{  hour   = h;
   minute = m;
   sec    = s;
}
```

```
int main()
{
    Time t1(14,15,23); //正确
    Time t2;           //正确
    ...
}
```

- 构造函数允许带默认参数，但要注意可能与重载产生二义性冲突

```
class Time {
    ...
    public:
        Time(int h, int m, int s=0);
};

Time::Time(int h, int m, int s)
{
    hour   = h;
    minute = m;
    sec    = s;
}
```

```
int main()
{
    Time t1(14,15,23); //正确
    Time t2(14,15);    //正确
}
```

- 构造函数允许带默认参数，但要注意可能与重载产生二义性冲突

```
class Time {
    ...
    public:
        Time();
        Time(int h, int m, int s=0);
};
Time::Time()
{  hour   = 0;
   minute = 0;
   sec    = 0;
}
Time::Time(int h, int m, int s)
{  hour   = h;
   minute = m;
   sec    = s;
}
```

无参与带缺省参数的重载，不冲突
适应带0/2/3个参数的情况

```
int main()
{
    Time t1(14,15,23); //正确
    Time t2(14,15);    //正确
    Time t3;           //正确
}
```

- 构造函数允许带默认参数，但要注意可能与重载产生二义性冲突

```
class Time {
    ...
    public:
        Time(int h=0, int m=0, int s=0);
};

Time::Time(int h, int m, int s)
{
    hour   = h;
    minute = m;
    sec    = s;
}
```

一个函数带三个缺省参数，
适应0/1/2/3个参数等4种情况

```
int main()
{   Time t1(14,15,23); //正确
    Time t2(14,15);    //正确
    Time t3(14);       //正确
    Time t4;           //正确
}
```

- 构造函数允许带默认参数，但要注意可能与重载产生二义性冲突

```
class Time {
   private:
      int hour, minute, sec;
   public:
      Time()
      {   hour = 0;
          minute = 0;
          sec = 0;
      }
      Time(int h=0, int m=0, int s=0) : hour(h), minute(m), sec(s)
      {
      }
      void display()
      {   cout << hour << ':' <<minute << ':' << sec << endl;
      }
};
```

```
int main()
{
    Time t1(14,15,23);//3参正确
    Time t2(14,15);   //2参正确
    Time t3(14);      //1参正确
    Time t4;  //无参错误
}
```

```
(25): error C2668: “Time::Time”: 对重载函数的调用不明确
(14,5): message : 可能是“Time::Time(int,int,int)”
(8,5): message : 或    “Time::Time(void)”
(25,12): message : 尝试匹配参数列表“()”时
```

无参与带缺省参数的重载，冲突!!!

- 构造函数也可以显式调用，一般用于带参构造函数

```
class Test {
  private:
    int a;
  public:
    Test(int x) {
      a=x;
      }
};
```

```
Test fun()
{  ...
   return Test(10);  //显式
}

int main()
{
   Test t1(10);        //隐式
   Test t2=Test(10); //显式
   Test t3=Test{10}; //显式
}
```

目录

- 对象的初始化方法
- 构造函数的基本概念
- 构造函数的使用
- 析构函数的基本概念
- 析构函数的使用

4 析构函数的基本概念

- 析构函数的功能：
  - 在对象被撤销时(生命期结束)时被自动调用
  - 完成一些善后工作(主要是内存清理)，但不是撤销对象本身
- 析构函数的形式：

  ~类名();
  - 无返回值(非void,也不是int)，无参，不允许重载
  - 析构函数必须公有
  - 析构函数既可以体内实现，也可以体外实现

- 构造函数与析构函数的形式：

```
class Time {
    ...
    public:
        Time()  //构造体内实现
        {  ...
        }
        ~Time() //析构体内实现
        {  ...
        }
};
```

```
class Time {
    ...
    public:
        Time();  //构造声明
        ~Time(); //析构声明
    };
Time::Time()  //构造体外实现
{  ...
}
Time::~Time() //析构体外实现
{  ...
}
```

目录

- 对象的初始化方法
- 构造函数的基本概念
- 构造函数的使用
- 析构函数的基本概念
- 析构函数的使用

5 析构函数的使用

- 析构函数的使用
  - 对象撤销时被自动调用，用户不能显式调用
  - 若不指定析构函数，则系统缺省生成一个无参空体析构函数
  - 若用户定义了析构函数，则缺省析构函数不再存在
  - 在数据成员没有动态内存申请需求的情况下，一般不需要定义析构函数(动态内存申请为后续课程内容，此处不再展开)

总结

- 对象的初始化方法
- 构造函数的基本概念（熟练掌握）
- 构造函数的使用（熟练掌握）
- 析构函数的基本概念（熟练掌握）
- 析构函数的使用

第五章 包含动态内存申请的类

主讲教师：同济大学计算机科学与技术学院 陈宇飞
同济大学计算机学科与技术学院 龚晓亮

目录

- 含动态内存申请的构造与析构函数
- 构造函数与析构函数的调用时机
- 对象的动态建立和释放
- 对象的赋值与复制

1. 含动态内存申请的构造与析构函数

- 使用：
  - 有数据成员需要动态内存申请的情况下，可在构造函数中申请空间，在析构函数中释放空间
  - 在没有数据成员需要动态内存申请的情况下，一般不需要定义析构函数
  - 在有数据成员需要动态内存申请的情况下，也可以不定义析构函数而通过其他方法释放（但不提倡）

- 常规做法：系统自动调用析构函数（非显式）

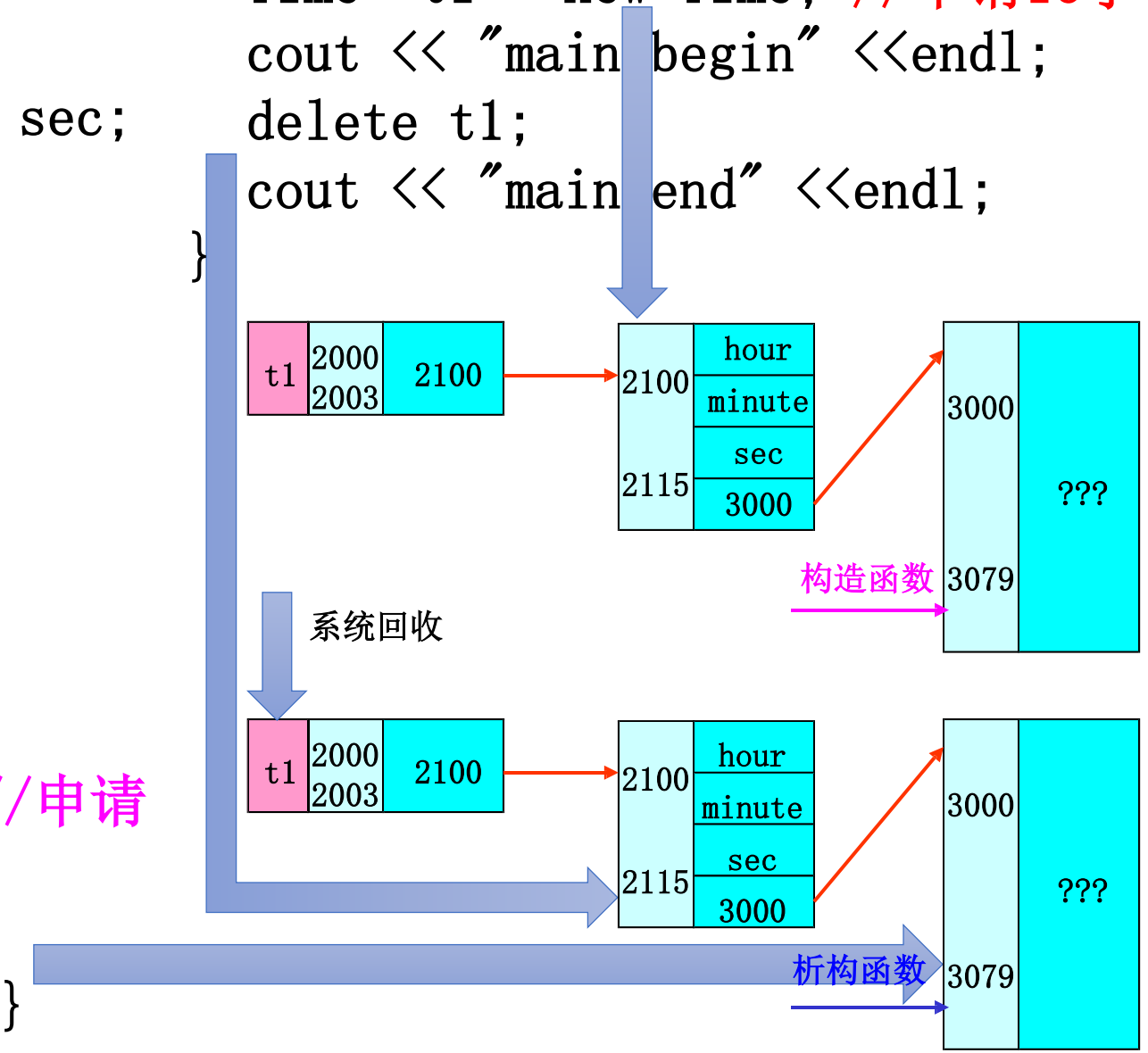
```
class Time {
  private:
    int hour;
    int minute;
    int sec;
    char *s;
  public:
    Time(); //构造函数
    ~Time();//析构函数
};
Time::Time()
{
    hour = 0;
    minute = 0;
    sec = 0;
    //接左侧
    s = new char[80];//动态申请
}
Time::~Time()
{
    delete s; //释放
}
int main()
{
    Time t1;
    ...
}
```

- 不提倡：调用Release函数（显式）

```
class Time {
  private:
    int hour;
    int minute;
    int sec;
    char *s;
  public:
    Time(); //构造函数
    Release(); //定义成员函数
};
Time::Time()
{
    hour = 0;
    minute = 0;
    sec = 0;
    //接左侧
    s = new char[80];//动态申请
}
Time::Release()
{
    delete s; //释放
}
int main()
{
    Time t1;
    ...
    t1.Release();
}
```

```
#include <iostream>
using namespace std;
class Time {
  private:
      int hour, minute, sec;
      char *s;
  public:
     Time();
     ~Time();
};
Time::Time(){
     hour = 0;
     minute = 0;
     sec = 0;
     s = new char[80];//申请
}
Time::~Time()
{   delete s;  //释放 }
```

```
int main()
{
  Time *t1 = new Time; //申请16字节
  cout << "main begin" <<endl;
  delete t1;
  cout << "main end" <<endl;
}
```




```
main begin
main end
```

目录

- 含动态内存申请的构造与析构函数
- 构造函数与析构函数的调用时机
- 对象的动态建立和释放
- 对象的赋值与复制

2. 构造函数与析构函数的调用时机

- 构造函数：

  自动对象(形参) ：函数中变量定义时

  静态局部对象 ：第一次调用时

  静态全局/外部全局对象 ：程序开始时

  动态申请的对象 ：new时

- 析构函数：

  自动对象(形参) ：函数结束时

  静态局部对象 ：程序结束时

  静态全局/外部全局对象 ：程序结束时

  动态申请的对象 ：delete时

```
#include <iostream>
using namespace std;
class Time {
   private:
     int hour;
     int minute;
     int second;
   public:
     Time(int h=0, int m=0, int s=0);
     ~Time();
};
Time::Time(int h, int m, int s)
{
    hour = h;
    minute = m;
    second = s;
    cout << "Time Begin" << endl;
}
```

```
main begin
Time Begin
fun
Time End
continue
Time Begin
fun
Time End
main end
```

```
Time::~Time()
{   cout << "Time End" << endl;
}
void fun()
{   Time t1;
    cout << "fun" <<endl;
}
int main()
{
    cout << "main begin" <<endl;
    fun();
    cout << "continue" << endl;
    fun();
    cout << "main end" << endl;
}
```

1、函数调用时分配空间结束时回收空间
2、函数多次调用则多次分配/回收空间

```
#include <iostream>
using namespace std;
class Time {
   private:
     int hour;
     int minute;
     int second;
   public:
     Time(int h=0, int m=0, int s=0);
     ~Time();
};
Time::Time(int h, int m, int s)
{
    hour = h;
    minute = m;
    second = s;
    cout << "Time Begin" << endl;
}
```

```
main begin
Time Begin
fun
continue
fun
main end
Time End
```

```
Time::~Time()
{    cout << "Time End" << endl;
}
void fun()
{    static Time t1;
     cout << "fun" <<endl;
}
int main()
{
     cout << "main begin" <<endl;
     fun();
     cout << "continue" << endl;
     fun();
     cout << "main end" << endl;
}
```

1、函数第1次调用时分配
2、后续函数调用不分配
3、全部程序结束后回收

```
#include <iostream>
using namespace std;
class Time {
   private:
     int hour;
     int minute;
     int second;
   public:
     Time(int h=0, int m=0, int s=0);
     ~Time();
};
Time::Time(int h, int m, int s)
{
    hour = h;
    minute = m;
    second = s;
    cout << "Time Begin" << endl;
}
```

```
Time Begin
main begin
fun begin
fun end
main end
Time End
```

```
Time::~Time()
{   cout << "Time End" << endl;
}
Time t1;
void fun()
{
    cout << "fun begin" <<endl;
    cout << "fun end" <<endl;
}
int main()
{
    cout << "main begin" <<endl;
    fun();
    cout << "main end" <<endl;
}
```

1、main开始前分配
2、main结束后回收

```
#include <iostream>
using namespace std;
class Time {
   private:
     int hour;
     int minute;
     int second;
   public:
     Time(int h=0, int m=0, int s=0);
     ~Time();
};
Time::Time(int h, int m, int s)
{
    hour = h;
    minute = m;
    second = s;
    cout << "Time Begin" << endl;
}
```

```
main begin
Time Begin
new end
Time End
main end
```

```
Time::~Time()
{    cout << "Time End" << endl;
}
int main()
{
     cout << "main begin" <<endl;
     Time *t1 = new Time;
     cout << "new end" <<endl;
     delete t1;
     cout << "main end" <<endl;
}
```



1、new时分配
2、delete时回收

```
main begin
new end
main end
```

```
#include <iostream>
using namespace std;
class Time {
   private:
     int hour;
     int minute;
     int second;
   public:
     Time(int h=0, int m=0, int s=0);
     ~Time();
};
Time::Time(int h, int m, int s)
{
    hour = h;
    minute = m;
    second = s;
    cout << "Time Begin" << endl;
}
Time::~Time()
{  cout << "Time End" << endl;
}
int main()
{
    cout << "main begin" <<endl;
    Time *t1 =(Time *)malloc
                (sizeof(Time));
    cout << "new end" <<endl;
    free(t1);
    cout << "main end" <<endl;
}
```

malloc仅仅是分配内存，不会调用构造函数

free不会调用析构函数

```
#include <iostream>
using namespace std;
class Time {
  private:
    int hour,minute,second;
  public:
    Time(int h=0, int m=0, int s=0);
    ~Time();
};
Time::Time(int h, int m, int s)
{   hour = h;
    minute = m;
    second = s;
    cout<< "Time Begin" << hour <<endl;
}
Time::~Time()
{   cout << "Time End" << hour << endl;
}
```

```
void fun()
{   Time t1(15), t2(16);
    cout << "fun" <<endl;
}
int main()
{   cout << "main begin" <<endl;
    fun();
    cout << "main end" <<endl;
}
```

```
main begin
Time Begin15
Time Begin16
fun
Time End16
Time End15
main end
```

t1,t2都是自动变量
构造：t1,t2
析构：t2,t1

```
#include <iostream>
using namespace std;
class Time {
  private:
    int hour,minute,second;
  public:
    Time(int h=0, int m=0, int s=0);
    ~Time();
};
Time::Time(int h, int m, int s)
{   hour = h;
    minute = m;
    second = s;
    cout<< "Time Begin" << hour <<endl;
}
Time::~Time()
{   cout << "Time End" << hour << endl;
}
```

```
void fun()
{   static Time t1(15), t2(16);
    cout << "fun" <<endl;
}
int main()
{   cout << "main begin" <<endl;
    fun();
    cout << "main end" <<endl;
}
```

```
main begin
Time Begin15
Time Begin16
fun
main end
Time End16
Time End15
```

t1,t2都是静态局部变量

构造：t1,t2

析构：t2,t1

```
#include <iostream>
using namespace std;
class Time {
  private:
    int hour,minute,second;
  public:
    Time(int h=0, int m=0, int s=0);
    ~Time();
};
Time::Time(int h, int m, int s)
{   hour = h;
    minute = m;
    second = s;
    cout<< "Time Begin" << hour <<endl;
}
Time::~Time()
{   cout << "Time End" << hour << endl;
}
```

```
Time t1(15), t2(16);

int main()
{
  cout << "main" <<endl;
}
```

```
Time Begin15
Time Begin16
main
Time End16
Time End15
```

t1,t2都是全局变量
构造：t1,t2
析构：t2,t1

```
#include <iostream>
using namespace std;
class Time {
  private:
    int hour,minute,second;
  public:
    Time(int h=0, int m=0, int s=0);
    ~Time();
};
Time::Time(int h, int m, int s)
{   hour = h;
    minute = m;
    second = s;
    cout<< "Time Begin" << hour <<endl;
}
Time::~Time()
{   cout << "Time End" << hour << endl;
}
```

```
void fun()
{   Time t1(15);
    static Time t2(16);
    cout << "fun" <<endl;
}
int main()
{   cout << "main begin" <<endl;
    fun();
    cout << "main end" <<endl;
}
```

```
main begin
Time Begin15
Time Begin16
fun
Time End15
main end
Time End16
```

t1,t2是不同性质的变量

不遵循栈规则

```
#include <iostream>
using namespace std;
class Time {
  private:
    int hour,minute,second;
  public:
    Time(int h=0, int m=0, int s=0);
    ~Time();
};
Time::Time(int h, int m, int s)
{   hour = h;
    minute = m;
    second = s;
    cout<< "Time Begin" << hour <<endl;
}
Time::~Time()
{   cout << "Time End" << hour << endl;
}
```

```
int main()
{   Time *t1, *t2;
    t2=new Time(16);
    t1=new Time(15);
    cout << "main begin" <<endl;
    delete t2;
    cout << "main end" <<endl;
    delete t1;
}
```

```
Time Begin16
Time Begin15
main begin
Time End16
main end
Time End15
```

动态申请的变量，按new的顺序调构造， 按delete的顺序调析构，不遵循栈规则

目录

- 含动态内存申请的构造与析构函数
- 构造函数与析构函数的调用时机
- 对象的动态建立和释放
- 对象的赋值与复制

3. 对象的动态建立和释放

- C++方法：

```
Time *p1, *p2;
申请：p1 = new(nothrow) Time; if (p1==NULL) { ... }
      p2 = new(nothrow) Time(); if (p2==NULL) { ... }
      //new Time()创建对象时候，系统除了执行默认构造函数会
      执行的那些操作外，还会为基本数据类型和指针类型的成员
      用0赋初值
      p3 = new(nothrow) Time[2]; if (p3==NULL) { ... }
释放：delete p1;
      delete p2;
      delete []p3;
```

```
#include<iostream>
using namespace std;
class Time
{
private:
  int hour;
  int minute;
  int second;
public:
  void display();
};
void Time::display()
{
  cout << hour << endl;
  cout << minute << endl;
  cout << second << endl;
}
```

```
int main()
{  Time* p1 = new(nothrow) Time;
   if (p1 == NULL) { return -1; }
   p1->display();  //随机值
   delete p1;
   Time* p2 = new(nothrow) Time();
   if (p2 == NULL) { return -1; }
   p2->display();  //0, 0, 0
   delete p2;
   Time* p3 = new(nothrow) Time[2];
   if (p3 == NULL) { return -1; }
   p3[0].display();  //随机值
   p3[1].display();  //随机值
   delete []p3;
   return 0;
}
```

```
-842150451
-842150451
-842150451
0
0
0
-842150451
-842150451
-842150451
-842150451
-842150451
-842150451
-842150451
```

3. 对象的动态建立和释放

- C++中一般不建议使用C方法动态申请
  - C方式动态内存申请和释放时不会调用构造和析构函数
  - 模块02例中，struct中有string类，则malloc/free会出错

```
#include<iostream>
using namespace std;
class Time
{
private:
  int hour;
  int minute;
  int second;
public:
  Time() { cout << "called!\n"; }
  void display();
};
void Time::display()
{
  cout << hour << endl;
  cout << minute << endl;
  cout << second << endl;
}
```

```
int main()
{  Time* p1 = new(nothrow) Time;
                    //调用构造函数
   if (p1 == NULL) { return -1; }
   p1->display();
   delete p1;

   Time* p2 = (Time*)malloc(sizeof(Time));
                    //未调用构造函数
   if (p2 == NULL) { return -1; }
   p2->display();
   free(p2);

   retuen 0;
}
```

```
called!
-842150451
-842150451
-842150451
-842150451
-842150451
-842150451
-842150451
```

目录

- 含动态内存申请的构造与析构函数
- 构造函数与析构函数的调用时机
- 对象的动态建立和释放
- 对象的赋值与复制

4. 对象的赋值与复制

- 基本概念

	赋值	复制
含义	将一个对象的所有数据成员的值对应赋值给另一个对象的数据成员	建立一个新对象,其值与某个已有对象完全相同
形式	//执行语句 类名 对象名1，对象名2; 对象名1=对象名2;	//定义语句 类名 对象名(已有对象名) 类名 对象名=已有对象名
实现	将对象2的全部数据成员的值对应赋给对象1的全部数据成员，不包括成员函数 (整体内存拷贝)	建立新对象时自动调用拷贝构造函数 浅拷贝：缺省拷贝构造函数（内存拷贝） 深拷贝：拷贝构造函数重载（动态分配）

思考：1）若对象数据成员无动态分配的数据，结果是否与预期一致？若不一致如何解决？

2）若对象数据成员是指针及动态分配的数据呢？

4. 对象的赋值与复制

- 对象的赋值
  - 有动态内存申请

```
int main()
{
   test t1("hello"), t2;
   t1.display();
   t2.display();
   t2=t1;
   t2.display();
   t1.set("china");
   t1.display();
   t2.display();
}
```

```
#define _CRT_SECURE_NO_WARNINGS
#include <iostream>
#include <cstring>
using namespace std;
class test {
 private:
    int a;
    int b;
    char *c;
 public:
    test(const char *s="A")
    {   a=0; b=0;
        c = new char[20];
        strcpy(c, s);
     }
 ~test() {
   delete c;
 };
 void set(const char *s)
 {
   strcpy(c, s);
 }
 void display()
 {
   cout << c << endl;
 }
};
```

4. 对象的赋值与复制

- 对象的赋值
  - 有动态内存申请　　//上例运行结果：

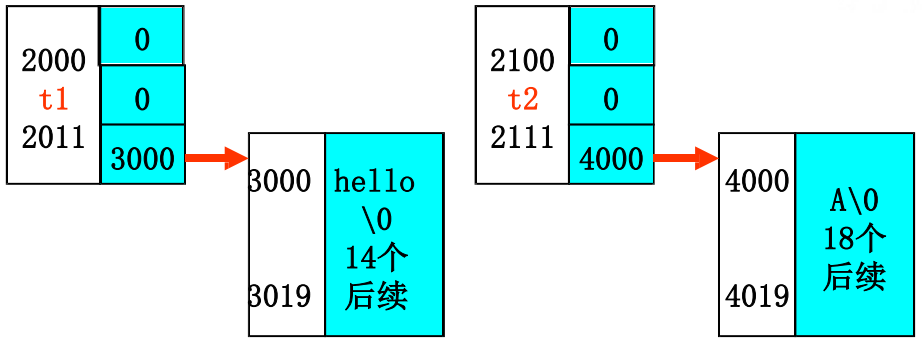


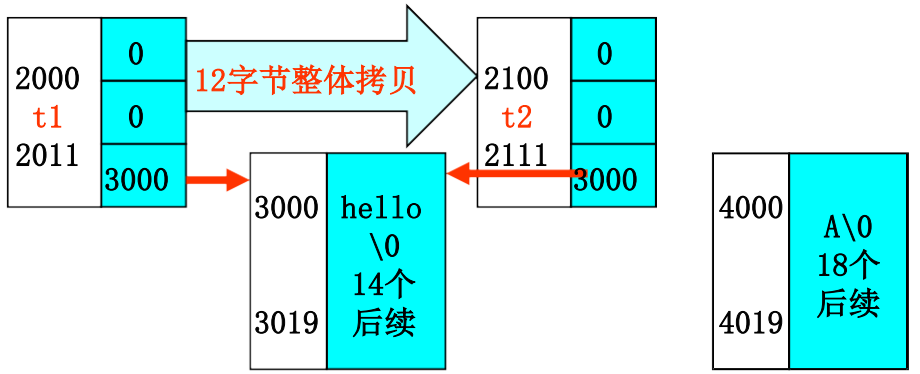
//有动态内存申请时，执行结果错且有错误弹窗

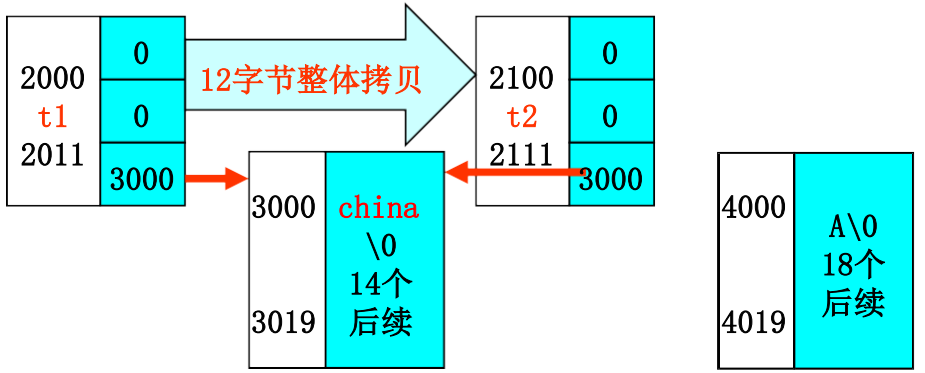
4. 对象的赋值与复制

- 对象的赋值
  - 有动态内存申请

```
int main()
{   test t1("hello"), t2;
    t1.display();       hello
    t2.display();       A
    t2=t1;
    t2.display();       hello
    t1.set("china");
    t1.display();       china
    t2.display();       china
}
```



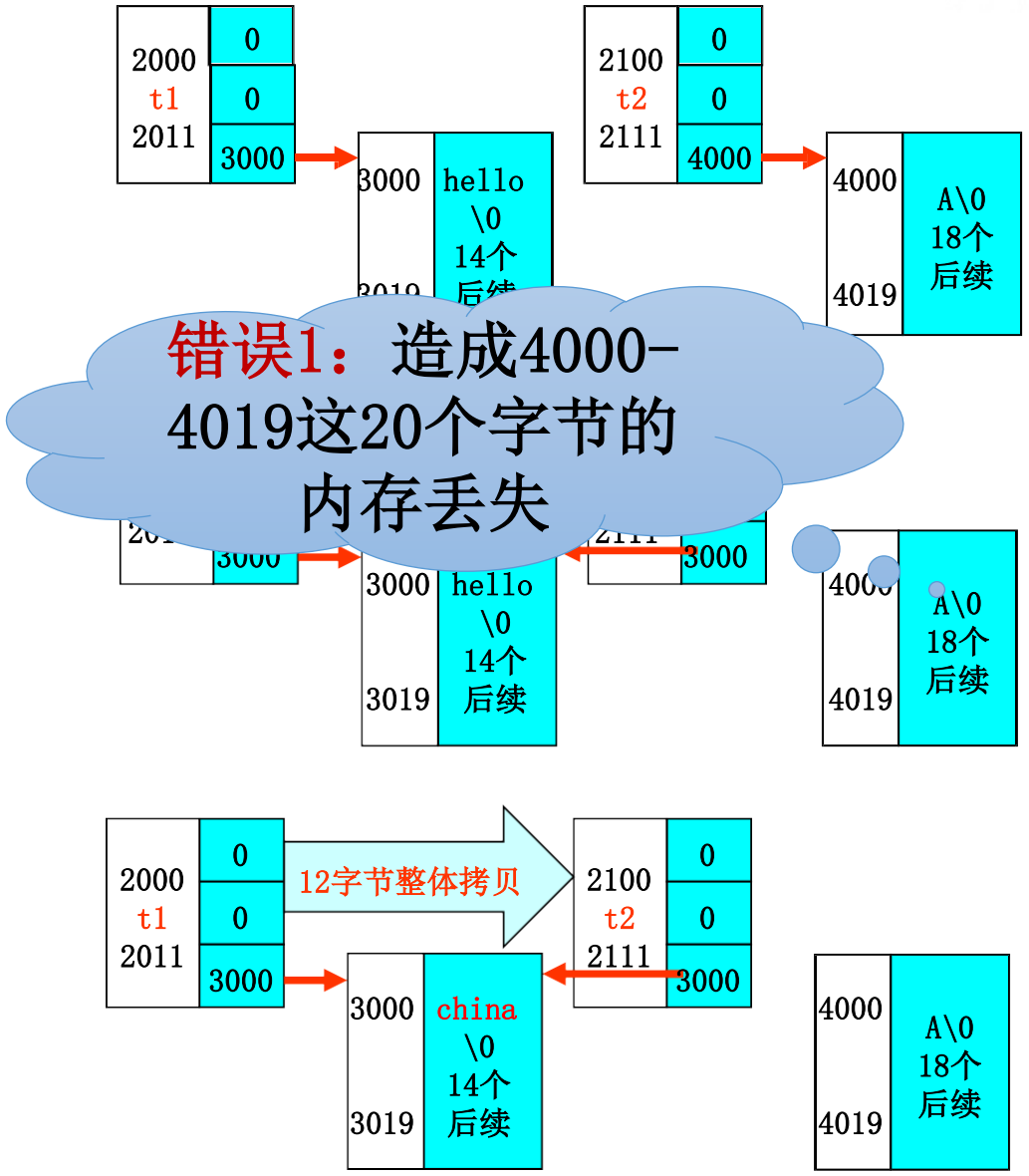




4. 对象的赋值与复制

- 对象的赋值
  - 有动态内存申请

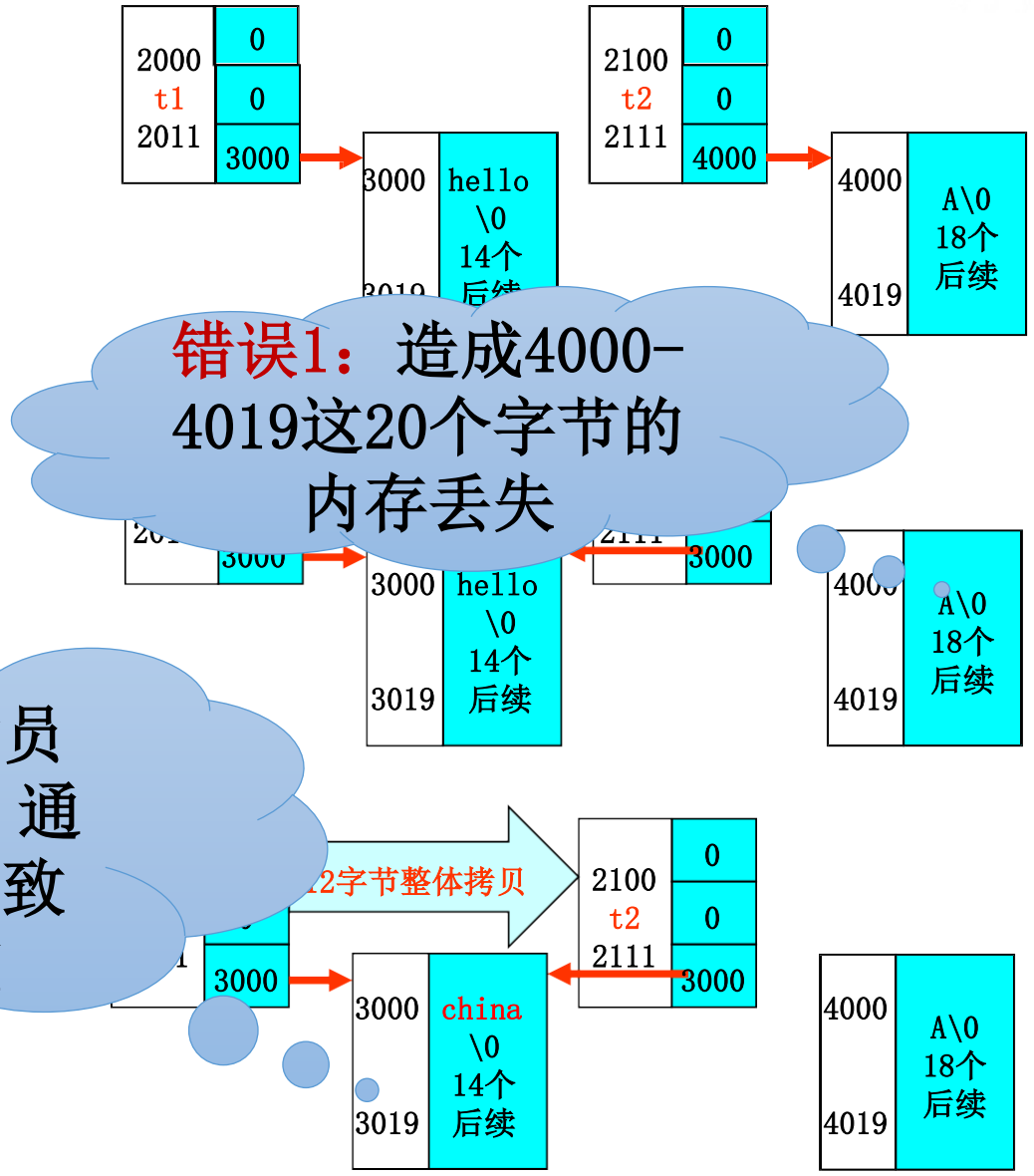
```
int main()
{   test t1("hello"), t2;
    t1.display();       hello
    t2.display();       A
    t2=t1;
    t2.display();       hello
    t1.set("china");
    t1.display();       china
    t2.display();       china
}
```



4. 对象的赋值与复制

- 对象的赋值
  - 有动态内存申请

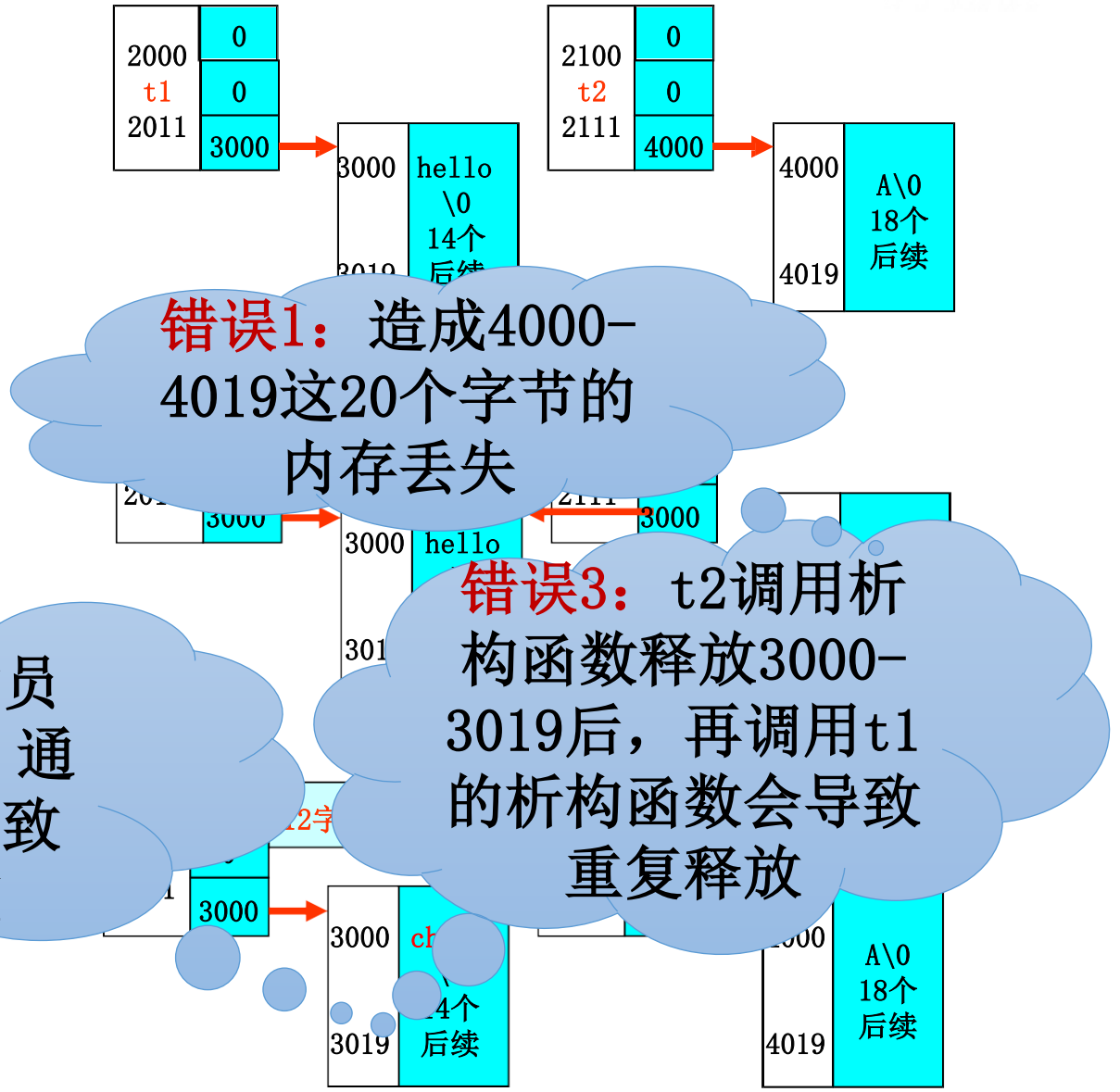
```
int main()
{   test t1("hello"), t2;
    t1.display();       hello
    t2.display();       A
    t2=t1;
    t2.display()
    t1.set("
    t1.di
    t2.dis
}
```



4. 对象的赋值与复制

- 对象的赋值
  - 有动态内存申请

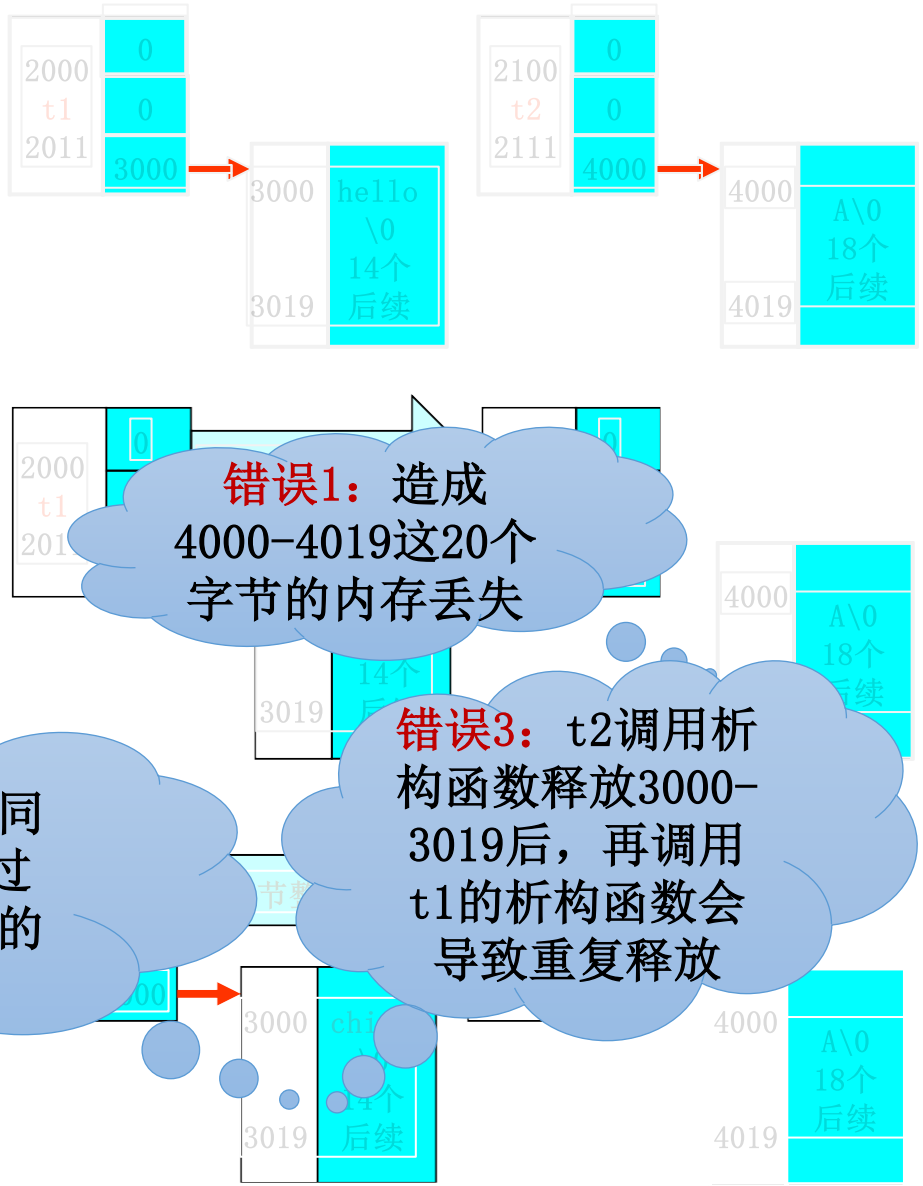
```
int main()
{   test t1("hello"), t2;
    t1.display();       hello
    t2.display();       A
    t2=t1;
    t2.display()
    t1.set("
    t1.di
    t2.dis
}
```



4. 对象的赋值与复制

- 对象的赋值
  - 有动态内存申请

```
int main()
{   test t1("hello"), t2;
    t1.display();
    t2.display();
    t2=t1;
    t2.display();
    t1.set("china");
    t1.display();
    t2.display();
}
```



• 解决方法：运算符重载！！（后续内容，此处了解即可）

```
#define _CRT_SECURE_NO_WARNINGS
#include <iostream>
#include <cstring>
using namespace std;
class test {
 private:
    int a; int b; char *c;
 public:
    test(const char *s="A")
    {    a=0; b=0;
         c=new char[20];
         strcpy(c, s);
    }
    ~test()
    { delete c; };
    void set(const char *s)
    { strcpy(c, s); }
    void display()
    { cout << c << endl; }
    test &operator=(const test &t);
         //重载=的声明
};
test &test::operator=(const test &t)
         //重载=体外实现
{  a = t.a;   b = t.b;
   delete c;          //释放原空间
   c=new char[20];    //申请新空间
   strcpy(c, t.c);
   return *this;      //返回对象自身
}
int main()
{  …
}
```

4. 对象的赋值与复制

- 对象的赋值
  - 有动态内存申请 //上例运行结果(运算符重载为后续内容，此处仅了解)

test1.cpp

Project1 | test | operator=(const test & t)

```
1  #define _CRT_SECURE_NO_WARNINGS
2  #include <iostream>
3  #include <cstring>
4  using namespace std;
5  class test {
6  private:
7      int a; int b; char* c;
8  public:
9      test(const char* s = "A")
10     {
11         a = 0; b = 0;
12         c = new char[20];
13         strcpy(c, s);
14     }
15     ~test() { delete c; };
16     void set(const char* s) { strcpy(c, s); }
17     void display() { cout << c << endl; }
18     test& operator=(const test& t); //重载=的声明
19 };
20 test& test::operator=(const test& t)//重载=体外实现
```

Microsoft Visual Studio 调试控制台

```
hello
A
hello
china
hello

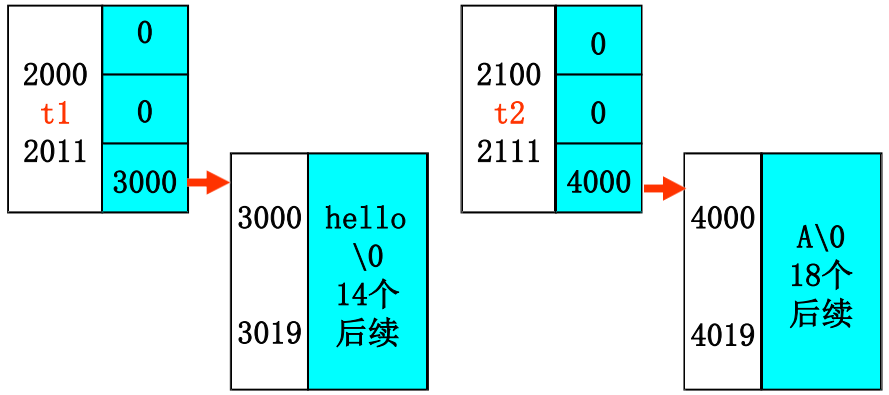
C:\Users\april\source\repos\Project1\Debug\Project1.exe (进程 14356)已退出，代码为 0。
要在调试停止时自动关闭控制台，请启用“工具”->“选项”->“调试”->“调试停止时自动关闭控制台”。
按任意键关闭此窗口. . .
```

4. 对象的赋值与复制

- 对象的赋值
  - 有动态内存申请

```
int main()
{   test t1("hello"), t2;
    t1.display();    hello
    t2.display();    A
    t2=t1;
    t2.display();
    t1.set("china");
    t1.display();
    st2.display();
}
```

```
//解决方法：运算符重载！！
test &test::operator=(const test &t)
{  a = t.a;    b = t.b;
   delete c;                  //释放原空间
   c=new char[20];            //申请新空间
   strcpy(c, t.c);
   return *this;              //返回对象自身
}
```

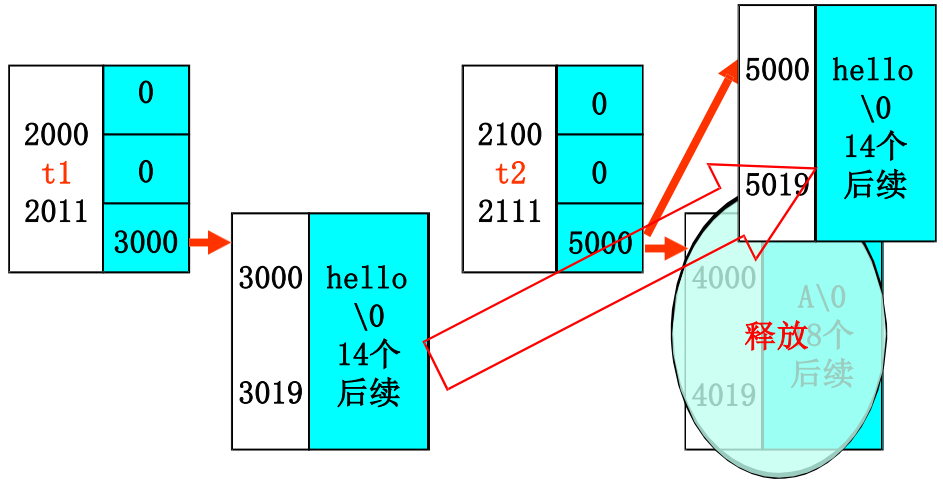


4. 对象的赋值与复制

- 对象的赋值
  - 有动态内存申请

```
int main()
{   test t1("hello"), t2;
    t1.display();    hello
    t2.display();    A
    t2=t1;
    t2.display();    hello
    t1.set("china");
    t1.display();
    t2.display();
}
```

```
//解决方法：运算符重载！！
test &test::operator=(const test &t)
{  a = t.a;    b = t.b;
   delete c;                  //释放原空间
   c=new char[20];            //申请新空间
   strcpy(c, t.c);
   return *this;          //返回对象自身
}
```

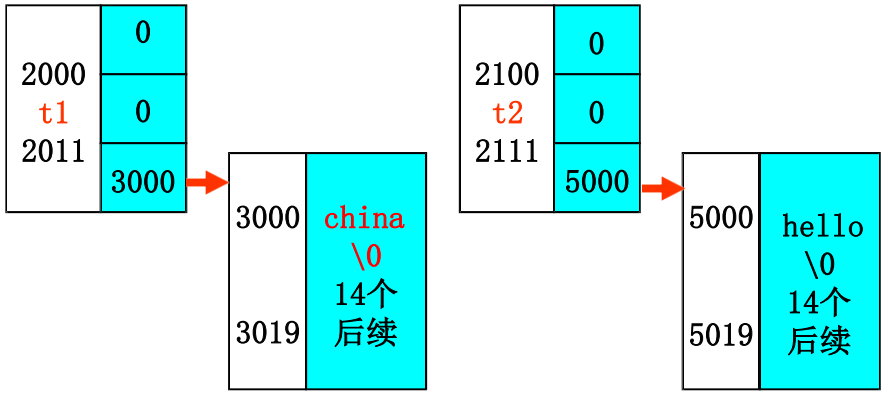


4. 对象的赋值与复制

- 对象的赋值
  - 有动态内存申请

```
int main()
{   test t1("hello"), t2;
    t1.display();    hello
    t2.display();    A
    t2=t1;
    t2.display();    hello
    t1.set("china");
    t1.display();    china
    t2.display();    hello
}
```

```
//解决方法：运算符重载！！
test &test::operator=(const test &t)
{  a = t.a;    b = t.b;
   delete c;                  //释放原空间
   c=new char[20];            //申请新空间
   strcpy(c, t.c);
   return *this;              //返回对象自身
}
```



4. 对象的赋值与复制

- 对象的复制
  - 拷贝构造函数/复制构造函数
  - 形式：类名(const 类名 &引用名)
    - 用一个对象的值去初始化另一个对象
    - 允许体内实现或体外实现
    - 复制构造函数和普通构造函数（可能多个）的地位平等，调用其中一个后就不再调用其它构造函数
    - 若不定义复制构造函数，则系统自动定义一个，参数为const型引用，函数体为对应成员内存拷贝（浅拷贝）
    - 若定义了复制构造函数，则系统缺省定义的消失（可做深拷贝）

4. 对象的赋值与复制

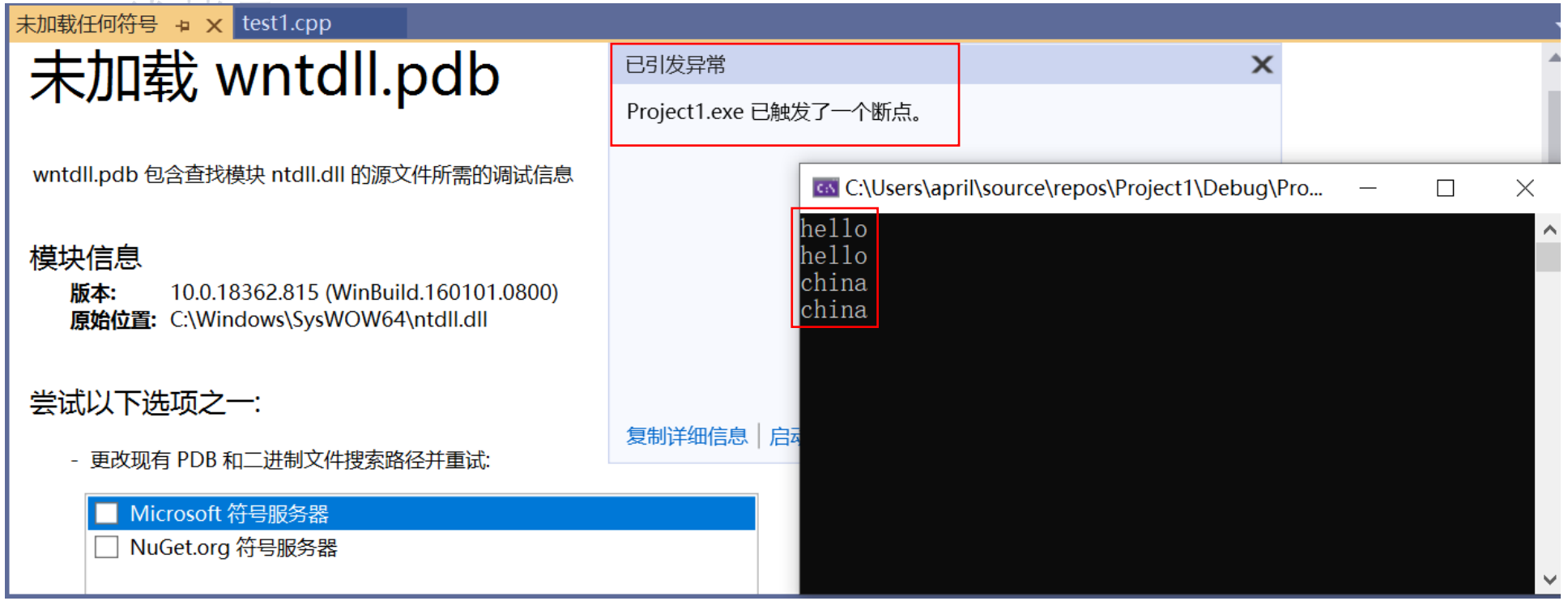
- 对象的复制
  - 浅拷贝

```
#define _CRT_SECURE_NO_WARNINGS
#include <iostream>
#include <cstring>
using namespace std;
class test {
 private:
   int a;
   int b;
   char *c;
 public:
   test(const char *s="A")
   {   a=0; b=0;
       c=new char[20];
       strcpy(c, s);
   }
 ~test(){
       delete c;
 }
 void set(const char *s)
 {    strcpy(c, s);
 }
 void display() {
       cout << c << endl;
 }
};
```

```
int main()
{
   test t1("hello"),t2(t1);
   t1.display();
   t2.display();
   t1.set("china");
   t1.display();
   t2.display();
}
```

4. 对象的赋值与复制

- 对象的复制
  - 浅拷贝　　//上例运行结果：



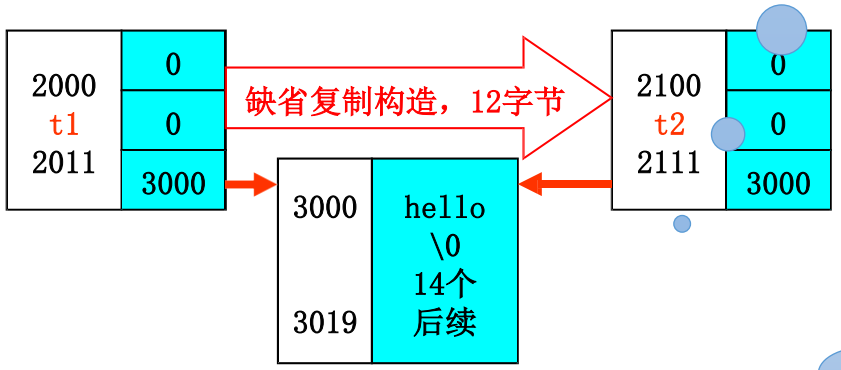
//有动态内存申请时，执行结果错且有错误弹窗

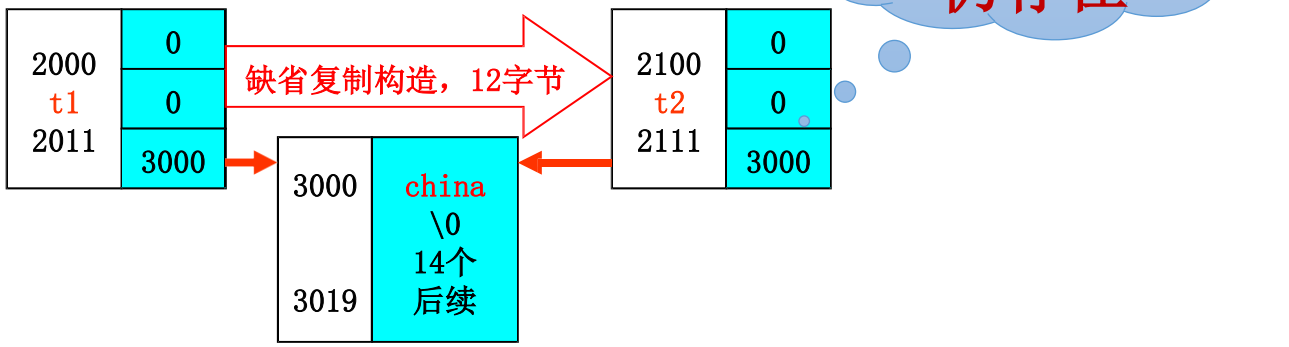
4. 对象的赋值与复制

- 对象的复制
  - 浅拷贝

```
int main()
{   test t1("hello"), t2(t1);
    t1.display();    hello
    t2.display();    hello
    t1.set("china");
    t1.display();    china
    t2.display();    china
}
```







4. 对象的赋值与复制

- 对象的复制
  - 浅拷贝

如何解决？

```
int main()
{   test t1("hello"), t2(t1);
    t1.display();          hello
    t2.display();          hello
    t1.set("china");
    t1.display();          china
    t2.display();          china
}
```



• 深拷贝 --动态成员不是简单赋值，而是重新动态分配空间

```
#define _CRT_SECURE_NO_WARNINGS
#include <iostream>
#include <cstring>
using namespace std;
class test {
 private:
     int a;
     int b;
     char *c;
 public:
    test(const char *s="A")
    {    a=0; b=0;
         c=new char[20];
         strcpy(c, s);
    }
    test(const test &t);
       //复制构造函数的声明
    ~test(){  delete c; }
    void set(const char *s)
    { strcpy(c, s); }
    void display()
    { cout << c << endl; }
};
test::test(const test &s)
     //复制构造的体外实现
{    a=s.a; b=s.b;
     c=new char[20];
     strcpy(c, s.c);
}
int main()
{…}
```

4. 对象的赋值与复制

- 对象的复制
  - 深拷贝　　//上例运行结果：

test1.cpp

Project1　　(全局范围)

```
1  #define _CRT_SECURE_NO_WARNINGS
2  #include <iostream>
3  #include <cstring>
4  using namespace std;
5  class test {
6  private:
7      int a;
8      int b;
9      char* c;
10 public:
11     test(const char* s = "A")
12     {
13         a = 0; b = 0;
14         c = new char[20];
15         strcpy(c, s);
16     }
17     test(const test& t); //复制构造函数的声明
18     ~test() { delete c; }
19     void set(const char* s) { strcpy(c, s); }
20     void display() { cout << c << endl; }
```

Microsoft Visual Studio 调试控制台

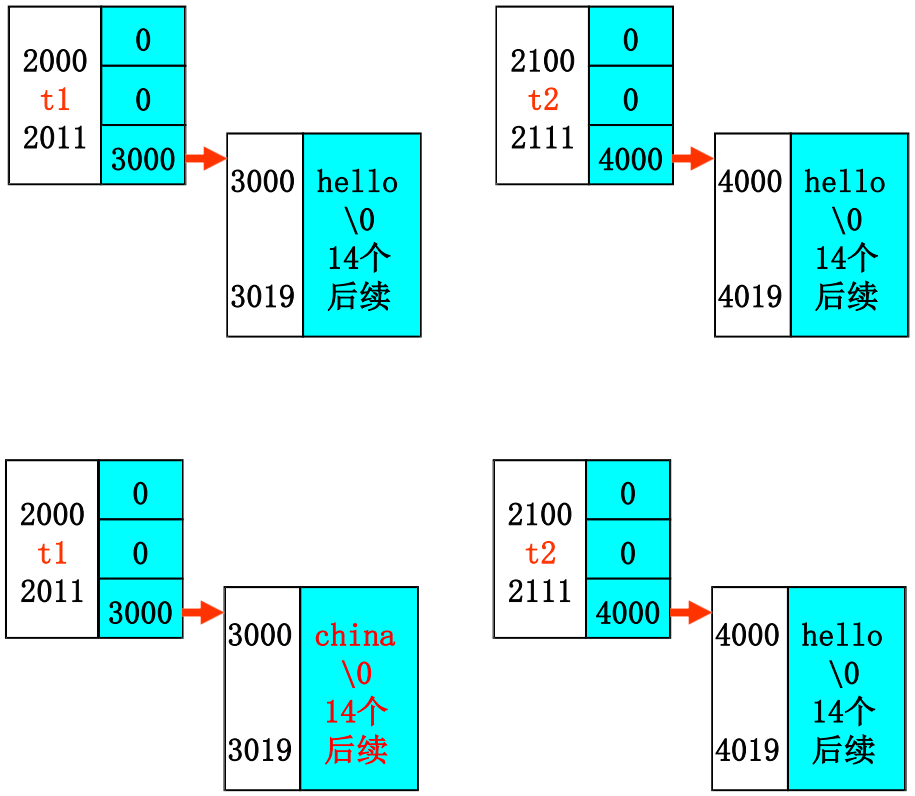
```
hello
hello
china
hello

C:\Users\april\source\repos\Project1\Debug\Project1.exe (进程 20248)已退出，代码为 0。
要在调试停止时自动关闭控制台，请启用“工具”->“选项”->“调试”->“调试停止时自动关闭控制台”。
按任意键关闭此窗口. . .
```

4. 对象的赋值与复制

- 对象的复制
  - 深拷贝

```
int main()
{   test t1("hello"), t2(t1);
    t1.display();      hello
    t2.display();      hello
    t1.set("china");
    t1.display();      china
    t2.display();      hello
}
```



4. 对象的赋值与复制

- 对象的赋值与复制（小结）
  - 对象的赋值发生在执行语句时，对象的复制发生在定义语句时
  - 赋值的操作是整体内存拷贝，复制的操作是自动调用拷贝构造函数
  - 若对象数据成员是指针及动态分配的数据，则可能导致不可预料的后果甚至报错
    - 解决方法：

      赋值：运算符重载（后续章节）

      复制：拷贝构造函数重载（深拷贝）

总结

- 含动态内存申请的构造与析构函数
- 构造函数与析构函数的调用时机
- 对象的动态建立和释放
- 对象的赋值与复制

第六章 类和对象的使用进阶

模块6.1：构造函数进阶

主讲教师：同济大学计算机科学与技术学院 陈宇飞
同济大学计算机科学与技术学院 龚晓亮

目录

- 构造函数初始值列表
- 委托构造函数
- 默认构造函数的作用
- 隐式的类类型转换
- 复制构造函数的调用时机

1.1 构造函数初始值列表

- 初始化：直接初始化数据成员；赋值：先初始化再赋值
- 构造函数的初始值有时必不可少

```
class ConstRef {
public:
      ConstRef(int ii);
private:
      int i;
      const int ci;
      int &ri;
};
```

```
// 错误：ci和ri必须被初始化
ConstRef::ConstRef(int ii)
{ // 赋值
      i = ii;    // 正确
      ci = ii;   // 错误：不能给const赋值
      ri = i;    // 错误：ri没被初始化
}
```

1.1 构造函数初始值列表

- 初始化：直接初始化数据成员；赋值：先初始化再赋值
- 构造函数的初始值有时必不可少

```
class ConstRef {
public:
      ConstRef(int ii);
private:
      int i;
      const int ci;
      int &ri;
};
```

```
// 正确：显式的初始化引用和const成员
ConstRef::ConstRef(int ii): i(ii),
ci(i), ri(ii) { }
```

结论：如果成员是const、引用，或者属于某种未提供默认构造函数的类类型，必须通过构造函数初始值列表为这些成员提供初值。

1.1 构造函数初始值列表

- 成员初始化的顺序：与类定义中出现的顺序一致，跟初始化列表中的顺序无关
  - 建议1：构造函数初始值的顺序与成员声明的顺序保持一致
  - 建议2：尽量避免使用某些成员初始化其它成员

```
class X {
   int i;
   int j;
public:
   //实际上是i先被初始化！
   X(int val): j(val),i(j){};
};
```

```
class X {
   int i;
   int j;
public:
   //正确
   X(int val): i(val),j(val){};
};
```

1.1 构造函数初始值列表

- 默认实参和构造函数：

➢若构造函数为所有参数都提供了默认实参，则相当于定义了默认构造函数

```
class Sales_data{
      Sales_data (string s = ""):bookNo(s){ }
};
```

上例：当没有给定实参或者给定了一个string实参时，类创建的对象相同。

（因为不提供实参也能调用上述构造函数，所以该构造函数实际就为类提供了默认构造函数）

1.1 构造函数初始值列表

- 默认实参和构造函数：

➢不能为构造函数的全部形参都提供默认实参

```
//接受string的构造函数
class Sales_data{
      Sales_data (string s = ""):bookNo(s){ }
};
//接受istream&参数的构造函数
class Sales_data{
      Sales_data (istream &is = cin) { is >> *this; }
};
```

错误

上例：不提供任何实参的创建类对象时，产生二义性

目录

- 构造函数初始值列表
- 委托构造函数
- 默认构造函数的作用
- 隐式的类类型转换
- 复制构造函数的调用时机

2.1 委托构造函数

- 委托构造函数使用它所属类的其他构造函数执行自己的初始化过程，或者说它把自己的一些(或全部)职责委托给了其他的构造函数
- 委托构造函数也有一个成员初始值的列表和一个函数体。在委托构造函数内，成员初始值列表只有唯一的一个入口，就是类名本身。和其他成员初始值一样，类名后面紧跟圆括号括起来的参数列表，参数列表必须与类中另外一个构造函数匹配

```
class Sales_data {
public:
        //非委托构造函数接收三个实参，使用这些实参初始化数据成员，然后结束工作
        Sales_data(string s,unsigned cnt,double price):
                bookNo(s),units_sold(cnt),revenue(cnt*price){}

        //其余构造函数全都委托给另一个构造函数
        Sales_data():Sales_data(" ", 0, 0) {}
            定义默认构造函数令其使用三参数的构造函数完成初始化过程
        Sales_data(string s):Sales_data(s,0,0){}
            定义接收一个string的构造函数，同样委托给了三参数版本
        Sales_data(istream &is):Sales_data() { read(is, *this); }
            定义接收istream &的构造函数，它委托给了默认构造函数，默认构造函数接
            着委托给三参数的构造函数。当接受委托的构造函数执行完后，接着执行
            istream &构造函数体的内容，即调用read函数读取给定的istream
private:
        string bookNo; unsigned units_sold; double revenue;
};
```

目录

- 构造函数初始值列表
- 委托构造函数
- 默认构造函数的作用
- 隐式的类类型转换
- 复制构造函数的调用时机

3.1 默认构造函数的作用

- 默认初始化发生的情况：
  - 在块作用域内不使用任何初始值定义一个非静态变量或数组
  - 类本身含有类类型的成员并且使用合成的默认构造函数
  - 当类类型的成员没有在构造函数初始值列表中显式的初始化
- 值初始化发生的情况：
  - 数组初始化的过程中如果提供的初始值少于数组的大小
  - 不使用初始值定义一个局部静态变量
  - 通过书写形如 T() 的表达式显式地请求值初始化时，T是类型名

➡ 类必须包含一个默认构造函数以便在上述情况下使用

```
//例：类的数据成员缺少默认构造函数
class NoDefault {
public:
        NoDefault(const string&){…};
};
```

注意：使用默认构造函数：NoDefault() {}; 才可以正常编译通过！！

```
struct A {
        NoDefault my_mem; //默认public
};

A a;  //错误，不能为A合成构造函数

struct B {
        B(){} //错误，b_member没有初始值
        NoDefault b_member;
};
```

合成构造函数：如果用户定义的类中没有显式的定义任何构造函数，编译器才会自动为该类型生成默认构造函数，称为合成的构造函数

```
#include <iostream>
#include <string>
using namespace std;
class NoDefault {
public:
      NoDefault(const string&) {…};
      NoDefault() {};
};
NoDefault obj1()
{     cout << "helloworld" << endl;
      NoDefault obj;
      return obj;
}
int main()
{     NoDefault obj2 = obj1();
      return 0;
}
```

```
//注意区分：
NoDefault obj1() //定义了一个obj1函数
NoDefault obj2   //定义了一个obj2对象
```

目录

- 构造函数初始值列表
- 委托构造函数
- 默认构造函数的作用
- 隐式的类类型转换
- 复制构造函数的调用时机

4.1 隐式的类类型转换

- 转换构造函数（converting constructor）：
  - 当一个构造函数只有一个参数，而且该参数又不是本类的const引用时，这种构造函数称为转换构造函数
  - 转换构造函数的作用是将一个其他类型的数据转换成一个类的对象

注意：

转换构造函数只能有一个参数。如果有多个参数，就不是转换构造函数

```
class Sales_data {
private:
    string book_no;
    unsigned units_sold = 1;
    double revenue = 1.0;

public:
    Sales_data() = default;//不接受任何实参，默认构造函数
    Sales_data(const string& s) : book_no(s) {}   //类型转换构造函数
    Sales_data(const string& s, unsigned n, double p) :
                book_no(s), units_sold(n), revenue(p* n) {}
    Sales_data(istream&) {};

    Sales_data& combine(const Sales_data&);
    //其他成员函数…
};
```

```
string null_book = "9-999-999-9";
//构造一个临时的Sales_data对象item
item.combine(null_book);
```

4.1 隐式的类类型转换

- 只允许一步类类型转换：

```
item.combine("9-999-999-9");
//错误：需要用户定义的两种转换："9-999-999-9"到string到Sales_data

item.combine(string("9-999-999-9"));
//正确：显式地转换成string，隐式地转换成Sales_data

item.combie(Sales_data("9-999-999-9"));
//正确：隐式地转换成string，显式地转换成Sales_data
```

```
Sales_data(const string& s) : book_no(s) {}   //类型转换构造函数
Sales_data& combine(const Sales_data&);
```

4.1 隐式的类类型转换

- 类类型转换不是总有效：

item.combine(cin);//隐式地将 cin 转换成 Sales_data，这个转换执行接受一个 istream 的 Sales_data 构造函数，该构造函数通过读取标准输入创建了一个临时的 Sales_data 对象，随后将得到的对象传递给 combine。该对象是一个临时量，一旦 combine 完成就不能再访问它了

```
...
Sales_data(istream&) {};
Sales_data& combine(const Sales_data&);
```

4.1 隐式的类类型转换

➢ 抑制构造函数定义的隐式类型转换：

- 将构造函数声明为 explicit 可以阻止构造函数的隐式类型转换

```
explicit Sales_data(const std::string &s):bookNo(s){ }
explicit Sales_data(std::istream&);
```

此时：

```
item.combine(null_book);    //错误
item.combine(cin);          //错误
```

4.1 隐式的类类型转换

```
explicit Sales_data(const std::string &s):bookNo(s){ }
explicit Sales_data(std::istream&);
```

- 关键字explicit只对一个实参的构造函数有效，需要多个实参的构造函数不能用于隐式转换，所以无须将这些构造函数指定为explicit
- 只能在类内声明构造函数时使用explicit，类外定义时不应重复
- 当使用explicit声明构造函数时，将只能以直接初始化的形式使用

```
Sales_data item1(null_book); //正确，直接初始化
Sales_data item2 = null_book;
        //错误，不能将explicit构造函数用于拷贝形式的初始化过程
```

4.1 隐式的类类型转换

```
explicit Sales_data(const std::string &s):bookNo(s){ }
explicit Sales_data(std::istream&);
Sales_data& combine(const Sales_data&);
```

- 为转换显式地使用构造函数：

item.combine(Sales_data(null_book));

//直接使用 Sales_data 的构造函数，该调用通过接受 string 的构造函数创建一个临时的 Sales_data 对象

item.combine(static_cast<Sales_data>(cin));

//使用_cast 执行了显式的转换：使用 istream 构造函数创建了一个临时static的 Sales_data 对象

目录

- 构造函数初始值列表
- 委托构造函数
- 默认构造函数的作用
- 隐式的类类型转换
- 复制构造函数的调用

对象复制的基本概念（复习回顾）



- 含义：建立一个新对象,其值与某个已有对象完全相同
- 使用：

  类 对象名(已有对象名)

  类 对象名=已有对象名

  Time t1(14,15,23), t2(t1), t3=t1;
- 与对象赋值的区别：

  Time t1(14,15,23),t2, t3=t1; //复制：定义语句中

  t2 = t1; //赋值：执行语句中
- 对象复制的实现：建立新对象时自动调用复制构造函数（也称为拷贝构造函数）

5.1 复制构造函数基本概念

- 形式：

  类名(const 类名 &引用名)

  - 用一个对象的值去初始化另一个对象
  - 若不定义复制构造函数，则系统自动定义一个，参数为const型引用，函数体为对应成员内存拷贝
  - 若定义了复制构造函数，则系统缺省定义的消失
  - 允许体内实现或体外实现
  - 复制构造函数和普通构造函数（可能多个）的地位平等，调用其中一个后就不再调用其它构造函数

5.2 复制构造函数调用

- 复制构造函数和普通构造函数(可能多个)的地位平等，调用其中一个后就不再调用其它构造函数：

```
class Time {
        ...
    public:
      Time(int h=0);
      Time(int h, int m, int s=0);
      Time(const Time &t);
};
```

```
int main()
{
      Time t1;
      Time t2(10);
      Time t3(1,2,3);
      Time t4(4,5);
      Time t5(t2);
      Time t6 = t4;
}
```

```
#include <iostream>
using namespace std;
int tcount = 0; //全局变量，计数器
class Time {
  private:
    int hour, minute, sec;
 public:
    Time(int h=0, int m=0, int s=0);
    Time(const Time &t);
    ~Time() {cout<<"tcount="<<--tcount<<endl;}
    void display()
    {cout<<hour<<":"<<minute<<":"<<sec<<endl;}
};
Time::Time(int h, int m, int s)
{   hour = h;
    minute = m;
    sec = s;
    ++tcount; //计数器+1
    cout << "普通构造" << endl;
}
```

```
Time::Time(const Time &t)
{  hour = t.hour - 1;
   minute = t.minute - 1;
   sec = t.sec - 1;
   ++tcount; //计数器+1
   cout << "复制构造" << endl;
}

int main()
{  //用对象初始化新对象
   Time t1(14,15,23),t2(t1);
   t2.display();
}
```

```
普通构造
复制构造
13:14:22
tcount=1
tcount=0
```

```
#include <iostream>
using namespace std;
int tcount = 0; //全局变量，计数器
class Time {
  private:
    int hour, minute, sec;
 public:
    Time(int h=0, int m=0, int s=0);
    Time(const Time &t);
    ~Time() {cout<<"tcount="<<--tcount<<endl;}
    void display()
    {cout<<hour<<":"<<minute<<":"<<sec<<endl;}
};
Time::Time(int h, int m, int s)
{   hour = h;
    minute = m;
    sec = s;
    ++tcount; //计数器+1
    cout << "普通构造" << endl;
}
```

```
Time::Time(const Time &t)
{  hour = t.hour - 1;
   minute = t.minute - 1;
   sec = t.sec - 1;
   ++tcount; //计数器+1
   cout << "复制构造" << endl;
}

void fun(Time t)
{  //函数形参为对象
   t.display();
}
int main()
{  Time t1(14,15,23);
   fun(t1);
}
```

普通构造
复制构造
13:14:22
tcount=1
tcount=0

```
#include <iostream>
```

在这个代码中，没有触发拷贝构造函数的原因是**编译器优化**，特别是**返回值优化（Return Value Optimization, RVO）** 或 **移动语义（Move Semantics）**。

具体分析如下：

1. 在函数 `fun()` 中创建了一个局部变量 `t1`，它是一个 `Time` 对象。按照传统的 C++ 构造与拷贝规则，当 `t1` 作为返回值返回时，应该调用拷贝构造函数将 `t1` 拷贝给 `t2`。
2. 然而，现代的 C++ 编译器会使用一种叫做返回值优化（RVO）的技术。编译器会直接在 `t2` 中构造 `t1` 的值，跳过临时对象的创建和拷贝构造。因此，`t1` 不会被拷贝，拷贝构造函数不会被调用。
3. 此外，C++11 及更高版本中引入了移动语义和 `std::move`，使得在某些情况下会触发移动构造函数，而不是拷贝构造函数。不过，返回值优化已经发生时，移动语义也不会生效。

优化过程：

- 当 `t1` 在 `fun()` 中返回时，编译器知道 `t1` 不会再被使用，因此直接将 `t1` 的内存返回给 `t2`。这就是返回值优化的一部分，目的是避免不必要的对象拷贝，提升性能。

因此，在你的程序中，由于 RVO 的存在，拷贝构造函数没有被调用。

```
Time::Time(int h, int m, int s)
{   hour = h;
    minute = m;
    sec = s;
    ++tcount; //计数器+1
    cout << "普通构造" << endl;
}
```

```
Time::Time(const Time &t)
{   hour = t.hour - 1;
    minute = t.minute - 1;
    sec = t.sec - 1;
    ++tcount; //计数器+1
    cout << "复制构造" << endl;
}
```

```
Time fun() //函数返回值为对象
{   Time t1(14,15,23);
    return t1;
}
int main()
{   Time t2 = fun();
    t2.display();
}
```

```
普通构造
14:15:23
tcount=0
```

➢ 变量定义时赋初值与使用赋值语句赋初值的区别：

```
Time fun()
{
    Time t1(14,15,23);
    return t1;
}
int main()
{
    Time t2 = fun();//定义时赋初值
    t2.display();
}
```

普通构造
14:15:23
tcount=0

```
Time fun()
{
    Time t1(14,15,23);
    return t1;
}
int main()
{   Time t2;
    t2 = fun();//赋值语句赋初值
    t2.display();
}
```

普通构造
普通构造
tcount=1
14:15:23
tcount=0

均针对拷贝构造函数进行过编译器优化，具体以实际编译器运行结果为准

总结

- 构造函数初始值列表（熟悉）
- 委托构造函数（了解）
- 默认构造函数的作用（熟悉）
- 隐式的类类型转换（熟悉）
- 复制构造函数的调用时机（掌握）

第六章 类和对象的使用进阶

模块6.2：数据共享与参数化类

主讲教师：同济大学计算机科学与技术学院 陈宇飞
同济大学计算机科学与技术学院 龚晓亮

目录

- 共用数据的保护
- 静态成员
- 类模板

目录

- 共用数据的保护
  - 基本概念
  - 常对象
  - 常对象成员
  - 共用数据小结

1.1 基本概念

一个数据可以通过不同的方式进行共享访问，因此可能导致数据因为误操作而改变，为了达到既能共享，又不会因误操作而改变，引入共用数据保护

- 常对象

  const 类名 对象名(初始化实参表)

- 常对象成员

  常数据成员

  常成员函数

1.2 常对象

- 常对象：

  const 类名 对象名(初始化实参表) 或 类名 const 对象名(初始化实参表)

  const Time t1(15); Time const t2(16, 30, 0);

- 在整个程序的执行过程中值不可再变化
- 必须在定义时进行初始化
- 不能调用普通成员函数(即使不改变数据成员的值)

```
//常对象
#define <iostream>
using namespace std;

class Time {
  public:
   int hour, minute, sec;
   Time(int h=0, int m=0, int s=0)
   {
     hour = h; minute = m; sec = s;
   }
   void display()
   {
     cout << hour << minute << sec;
   }
};
```

```
int main()
{
   const Time t1(15);
   Time const t2(16, 30, 0);
   t1.minute = 12;    //编译报错
   t2.sec = 27;       //编译报错
   t1.display();    //编译报错
   //display函数虽然不改变值，仍报错
}
```

```
t1始终是15:00:00
t2始终是16:30:00
```

1.3 常对象成员

- 常对象成员：

常对象中的所有数据成员在程序执行过程中值均不可变，如果只需要限制部分成员的值在执行过程中不可变，则需要引入常对象成员的概念

常数据成员：该数据成员的值在执行中不可变

常成员函数：该函数只能引用成员的值，不能修改

```
//常数据成员：
class 类名 {
      const 数据类型 数据成员名
   或 数据类型 const 数据成员名
};
class Time {
   private:
      const int hour;
      int const minute;
      int sec;
};
```

```
//常成员函数：
class 类名 {
      返回类型 成员函数名(形参表) const;
};

class Time {
   public:
      void display() const;
};
```

- 使用：常数据成员要在构造函数中初始化，使用中值不可变，在构造函数中初始化时，必须用参数初始化表形式，而不能用赋值形式



```
#define <iostream>
using namespace std;
class Time {
  public:
     const int hour;
     int minute, sec;
     Time(int h=0, int m=0, int s=0) : hour(h)
     {   minute = m;
         sec = s;
     }
};
int main()
{   Time t1;
    t1.hour = 10; //错误
}
```

```
//错误
Time(int h=0, int m=0, int s=0)
{  hour = h;
   minute = m;
   sec = s;
}
```

```
//正确
```

- 使用：常成员函数只能引用类的数据成员(无论是否常数据成员)的值，而不能修改数据成员的值



```
#define <iostream>
using namespace std;
class Time {
   public:
      int hour, minute, sec;
      void set(int h=0,int m=0,int s=0) const
      {     hour = h;
            minute = m;                    //错误
            sec = s;
      }
};
int main()
{   Time t1;
    t1.set(14,15,23);
}
```

```
void set(int h=0,int m=0,int s=0)
{   hour = h;
    minute = m;                    //正确
    sec = s;
}
```

- 使用：常成员函数写成下面形式，编译不报错但不起作用

const 返回类型 成员函数名(形参表) 或 返回类型 const 成员函数名(形参表)

```
#define <iostream>
using namespace std;
class Time {
  public:
    int hour, minute, sec;
    const void set(int h, int m,int s)
    {  hour = h;
       minute = m;
       sec = s;
    }
};
int main()
{  Time t1;
   t1.set(14,15,23);  //赋值正确，说明set不是常成员函数
}
```

```
void const set(int h, int m, int s)
{  hour = h;
   minute = m;
   sec = s;
}
```

- 使用：常成员函数可以调用本类的另一个常成员函数，但不能调用本类的非常成员函数（即使该非常成员函数不修改数据成员的值）

```
#define <iostream>
using namespace std;
class Time {
  public:
    int hour, minute, sec;
    void display()
    {  cout << hour << endl;
    }
    void fun() const
    {  display(); //错误
    }
};
int main()
{   Time t1;
    t1.fun();
}
```

```
void display() const
{  cout << hour << endl;
}
void fun() const
{  display(); //正确
}
```

- 使用：若希望常成员函数能强制修改数据成员，则要将数据成员定义为mutable

```
#include <iostream>
using namespace std;
class Time {
  public:
    int mutable hour, minute, sec;  // mutable int hour, minute, sec;
    void set(int h=0, int m=0, int s=0) const
    {    hour = h;
         minute = m;
         sec = s;
    }
};
int main()
{  Time t1;
   t1.set(14,15,23);
}
```

- 使用：若定义对象为常对象，则只能调用其中的常成员函数(不能修改数据成员的值),而不能调用其中的普通成员函数(即使该成员不修改数据成员的值)

```
class Time {
  public:
    int hour, minute, sec;
    Time(int h=0, int m=0, int s=0)
    { hour = h; minute = m; sec = s;}
    void display()
    { cout << hour << minute << sec << endl;}
};
int  main()
{  const Time t1(13, 14, 23);
   t1.minute = 12; //编译报错
   t1.display(); //编译报错
}
```

```
   …
   void display() const
   …
int main()
{  const Time t1(13, 14, 23);
   t1.display(); //正确
}
```

- 使用：若定义对象为常对象，则只能调用其中的常成员函数(不能修改数据成员的值),而不能调用其中的普通成员函数(即使该成员不修改数据成员的值)

```
class Time {
  public:
    int hour, minute, sec;
    Time(int h=0, int m=0, int s=0)
     { hour = h; minute = m; sec = s;
     }
     void display() const
     { cout << hour << minute << sec << endl;
       sec++; // 错误
     }
};
int main()
{   const Time t1(13, 14, 23);
    t1.display();
}
```

```
mutable int sec;
…
void display() const
{  cout<<hour<<minute<<sec<<endl;
   sec++; //正确
}
```

- 使用：
  - 不能定义构造/析构函数为常成员函数
  - 全局函数不能定义const

```
class Time {    //编译报错
  public:
    int hour, minute, sec;
    Time(int h=0, int m=0, int s=0) const
     { hour = h; minute = m; sec = s;
     }
    ~Time() const{}
};
int main()
{   Time t1(13, 14, 23);
    …
}
```

```
void fun() const   //编译报错
{
    …
    return;
}
int main()
{
    fun();
    …
}
```

1.4 共用数据小结

	普通 数据成员	const 数据成员	mutable 数据成员	普通 成员函数	const 成员函数
普通对象	读写	读	读写	可调用	可调用
const对象	读	读	读写	不可调用	可调用
普通成员函数	读写	读	读写	可调用	可调用
const成员函数	读	读	读写	不能调用	可调用

目录

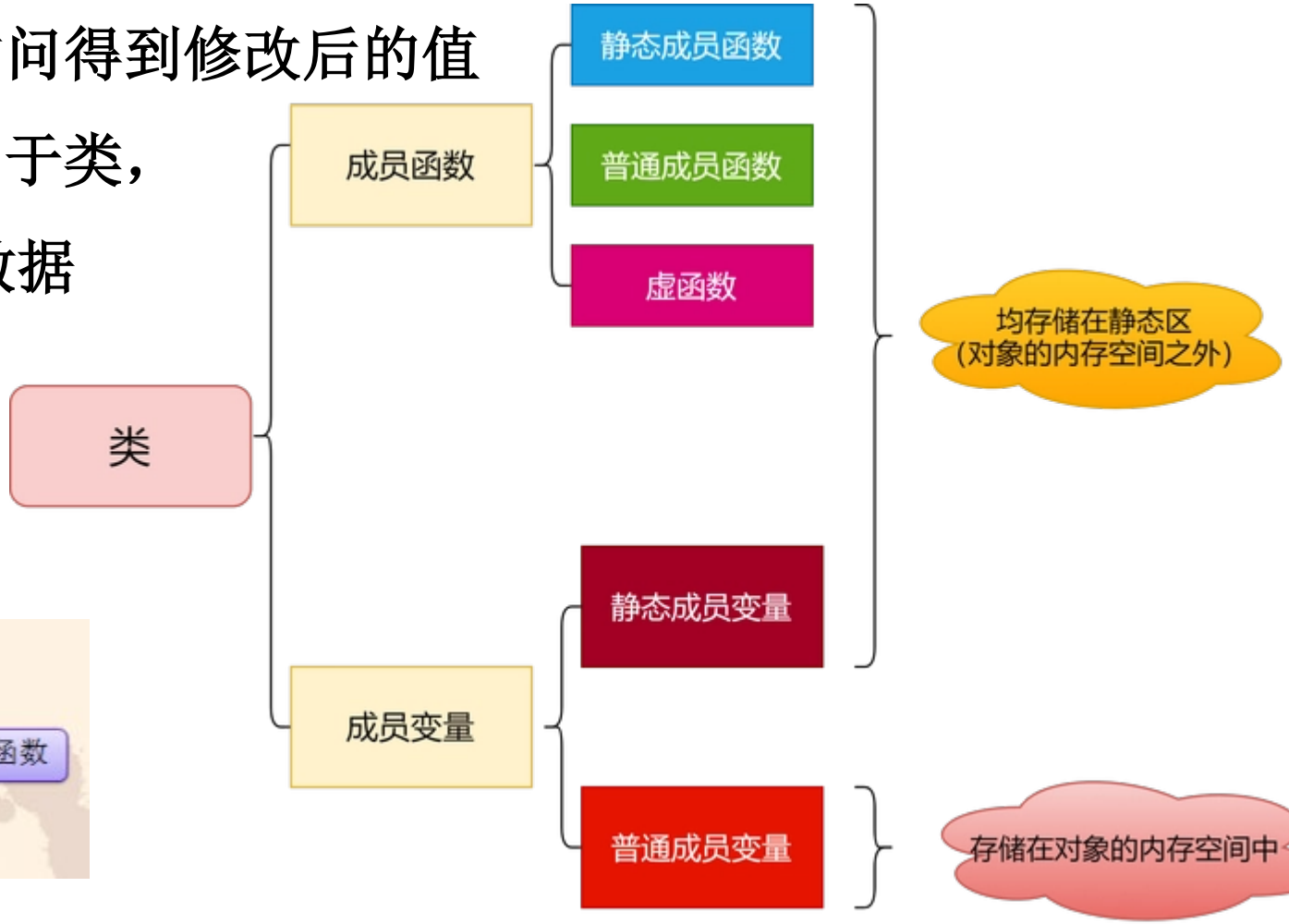
- 静态成员
  - 基本概念
  - 静态数据成员
  - 静态成员函数

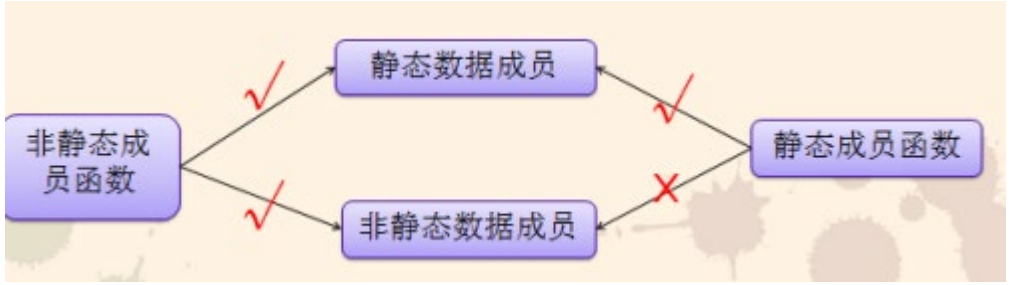
2.1 基本概念

希望在同一个类的多个对象间实现数据共享

- 一个对象修改，另一个对象访问得到修改后的值
- 类似于全局变量的概念，但属于类，

  仅供该类的不同对象间共享数据





2.2 静态数据成员

- 定义：

```
class 类名 {
    private/public:
        static 数据类型 成员名;
        …
};
```

```
class Test{
    private:
        static int a; //静态数据成员
        …
};
```

2.2 静态数据成员

- 使用：
  - 静态数据成员不属于任何一个对象，不在对象中占用空间，单独在静态数据区分配空间(初值为0，不随对象的释放而释放)，一个静态数据成员只占有一个空间，所有对象均可共享访问
  - 静态数据成员不是面向对象的概念，它破坏了数据的封装性，但方便使用，提高了运行效率

➢静态数据成员必须进行初始化，初始化位置在类定义体后，函数体外进行(此时不受类的作用域限制)　数据类型 类名::静态数据成员名=初值;



```
#include <iostream>
using namespace std;
class Test {
private:
    static int a; //静态数据成员
public:
    int GetA() const { return a; }
};
//int Test::a;  如果这样定义不赋予初值则初值为零
int Test::a = 1;
```

```
int main()
{
    Test T;
    cout << T.GetA() << endl;
    return 0;
}
```

➢既可以通过类型引用，也可以通过对象名引用

```
#include <iostream>
using namespace std;
class Test {
public:
    static int a; //静态数据成员
    int GetA() const { return a; }
};
int Test::a = 1;
```

```
int main()
{
    Test T;
    T.a++;        //对象名引用
    Test::a++;    //类型引用
    cout << T.GetA() << endl;  //3
    return 0;
}
```

➢不能通过参数初始化表进行初始化，但可以通过赋值方式初始化

```
#include <iostream>
using namespace std;
class Test {
private:
    static int a; //静态数据成员
public:
    int GetA() const { return a; }
    Test(int x) { a = x; } //正确
};
int Test::a = 1; //不可省，否则编译错
int main()
{
    Test T(5);
    cout << T.GetA() << endl;  //5
    return 0;
}
```

```
#include <iostream>
using namespace std;
class Test {
private:
    static int a; //静态数据成员
public:
    int GetA() const { return a; }
    Test(int x): a(x) {}  //错误
};
int Test::a = 1;
int main()
{
    Test T(5);
    cout << T.GetA() << endl;
    return 0;
}
```

➢静态数据成员被类的所有对象共享，包括该类的派生类对象，基类对象和派生类对象共享基类的静态数据成员（后续讲继承，此处了解）

```
#include <iostream>
using namespace std;
class Base {
public:
    static int a;//静态数据成员
};
class Derived : public Base {
};
int Base::a; //可以，初值为零
```

```
int main()
{
    Base B;
    Derived D;
    B.a++;
    cout << B.a << endl;
    D.a++;
    cout << D.a << endl;
    return 0;
}
```

1
2

- 静态数据成员可以作为成员函数的默认形参，而普通数据成员则不可以
- 静态数据成员的类型可以是所属类的类型，而普通数据成员则不可以。
  普通数据成员只能声明为所属类类型的指针或引用

```
class Test {
public:
    static int a;    //静态数据成员
    int b;
    void fun_1(int i = a) {};//正确
    void fun_2(int i = b) {};//报错
};
```

```
class Test {
public:
    static Test a;//正确
    Test b;//报错
    Test* pTest;//正确
    Test& m_Test;//正确
    static Test* pStaticObject;//正确
};
```

➢静态数据成员在const函数中可以修改，而普通的数据成员不能修改

```
class Test {
public:
    static int a;//静态数据成员
    int b;
    Test():b(0) {}
    void test() const  //不能修改当前调用该函数对象的非静态数据成员
    {   a++;
        b++;   //错误
    }
};
int Test::a;
```

const修饰的是当前this指针所指向的对象是const，但是静态数据成员不属于任何类的对象，它可被类的所有对象修改，this指针不修饰静态的数据成员，所以可以修改

2.3 静态成员函数

- 定义：

```
class 类名 {
    private/public:
        static 返回类型 函数名(形参表);
}...
```

- 调用：

类名::成员函数名(实参表);

任意对象名.成员函数名(实参表);

➢静态成员函数没有this指针，静态成员函数不能使用修饰符(函数后面的const关键字)

```
#include <iostream>
using namespace std;
class Student
{
private:
    int num; int age; float score;
    static float sum;//静态数据成员，累计学生的总分
    static int count;//静态数据成员，累计学生的人数
public:
    Student(int n, int a, float s) :num(n),age(a),score(s){ }  //构造函数
    void total();
    static float average();//声明静态成员函数，不能使用const
};
float Student::sum;
int Student::count;
```

```
//接上页
void Student::total() //定义非静态成员函数
{   sum += score;
    count++;        //有this指针，可以写this->sum，  this->score，  this->count
}
float Student::average()
//定义静态成员函数,此处不需要再次写上static，也不能用const修饰
{   return( sum/count ); //没有this指针，不可以写this->sum或this->count
}
int main()
{   Student stud[3] = { Student(1001,18,70),Student(1002,19,78),Student(1005,20,98) };
    for (int i = 0; i < 3; i++)
    {
        stud[i].total();   //调用非静态成员函数
    }
    cout<<"the average score of the students is "<<Student::average()<<endl;
    return 0;                                          //调用静态成员函数
}
```

➢静态成员函数不能调用非静态成员函数，但是反过来是可以的

```
…
void Student::total() //定义非静态成员函数
{
    sum += score;
    count++;
    average();  //可以
}
float Student::average() //定义静态成员函数
{
    total();   //不可以，报错
    return(sum / count);
}
…
```

- 在静态成员函数中不能对非静态数据成员进行直接访问，而要通过对象参数的方式(不提倡，建议静态成员函数只访问静态数据成员)

```
...
float Student::average(Student &s) //定义静态成员函数
{
    num++;      //不可以，报错
    s.num++;    //可以
    return(sum / count);
}
...
```

- 静态成员函数的地址可用普通函数指针储存，而普通成员函数地址需要用类成员函数指针来储存

…

```
float (*pf1)() = &Student::average;
```

//普通的函数指针　　静态成员函数的地址

```
void (Student:: * pf2)() = &Student::total;
```

//类成员函数指针　　普通成员函数地址

…

目录

- 类模板
  - 函数模板（复习）
  - 类模板的使用

3.1 函数模板（复习）

- 建立一个通用函数，其返回类型及参数类型不具体指定，用一个虚拟类型来代替，该通用函数称为函数模板，调用时再根据不同的实参类型来取代模板中的虚拟类型，从而实现不同的功能

一段代码,两个功能
1、两个int型求max
2、两个double型求max



```
#include <iostream>
using namespace std;
template <typename T>
T max(T x, T y)
{  cout << sizeof(x) << ' ';
   return x>y?x:y;
}
int main()
{  int a=10, b=15;
   double f1=12.34, f2=23.45;
   cout << max(a, b) << endl;
   cout << max(f1, f2) << endl;
   return 0;
}
```

3.1 函数模板（复习）

- 使用：
  - 仅适用于参数个数相同、类型不同，实现过程完全相同的情况
  - typename可用class替代
  - 类型定义允许多个

```
template <typename T1, typename T2>
template <class T1, class T2>
```

```
#include <iostream>
using namespace std;
template <typename T1, typename T2>
char max(T1 x, T2 y)
{    cout << sizeof(x) << ' ';
     cout << sizeof(y) << ' ';
     return x>y ? 'A' : 'a';
}
int main()
{   int a = 10, b = 15;
    double f1 = 12.34, f2 = 23.45;
    cout << max(a, f1) << endl;
    cout << max(f2, b) << endl;
    return 0;
}
```

3.2 类模板的使用

- 仅适用于参数个数相同、类型不同，实现过程完全相同的情况
- 类模板可以看作是类的抽象，称为参数化的类
- 模板的具体实现称为实例化（instantiation）或具体化（specialization）
- 类型定义允许多个

```
template <class T1, class T2>
```

示例：基础stack类

```
typedef unsigned long Item;     //只能处理unsigned long类型
class Stack
{
private:
    enum {MAX = 10};     // constant specific to class
    Item items[MAX];     // holds stack items
    int top;             // index for top stack item
public:
    Stack();
    bool isempty() const;
    bool isfull() const;

    // push() returns false if stack already is full, true otherwise
    bool push(const Item & item);   // add item to stack
    // pop() returns false if stack already is empty, true otherwise
    bool pop(Item & item);          // pop top into item
};
```

• 定义类模板：组织在头文件里

```
// stacktp.h -- a stack template
#ifndef STACKTP_H_
#define STACKTP_H_
template <class Type>
class Stack
{
private:
    enum {MAX = 10};
    Type items[MAX];
    int top;
public:
    Stack();
    bool isempty();
    bool isfull();
    bool push(const Type & item);
    bool pop(Type & item);
};

template <class Type>
Stack<Type>::Stack(){ top = 0; }
```

```
template <class Type>
bool Stack<Type>::isempty(){ return top == 0; }

template <class Type>
bool Stack<Type>::isfull(){return top == MAX; }

template <class Type>
bool Stack<Type>::push(const Type & item)
{   if (top < MAX) …
    else …
}

template <class Type>
bool Stack<Type>::pop(Type & item)
{   if (top > 0) …
    else …
}
#endif
```

体外实现的类限定符不要忘记泛型名！

- 使用模板类：包含头文件

```
// stacktem1.cpp
#include <iostream>
#include <string>
#include <cctype>
#include "stacktp.h"
using namespace std;

int main()
{
    Stack<string> st;
    string po;
    char ch;
    …
}
//可使用字符串作为订单ID
```

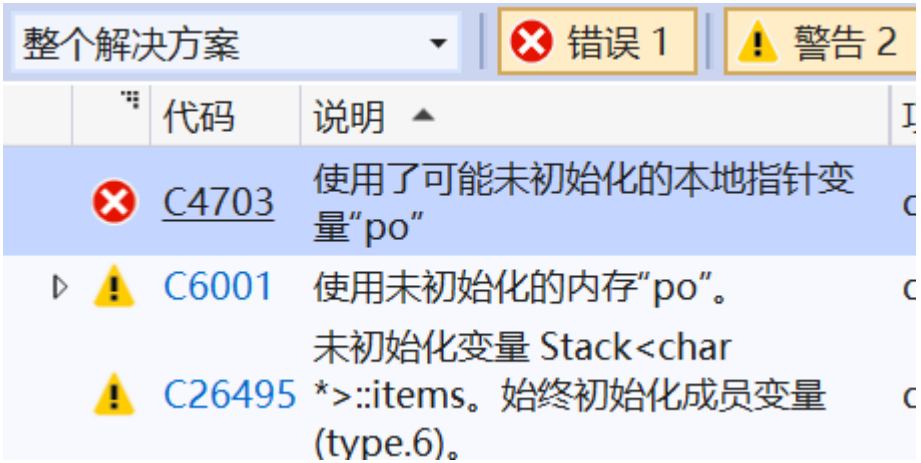
```
// stacktem2.cpp
#include <iostream>
#include <string>
#include <cctype>
#include "stacktp.h"
using namespace std;

int main()
{
    Stack<int> st;
    int po;
    char ch;
    …
}
//可使用int值作为订单ID
```

使用模板实现泛型编程：实现相同算法下不同类型参数的统一编程

- 深入探讨模板类：不正确地使用指针栈

```
//例1：
int main()
{
    Stack<char *> st;
    char * po;
    …
    cin >> po;  //错误
    …
}
//仅创建指针，没有创建用于保存输入字符串的空间
```



- 深入探讨模板类：不正确地使用指针栈

```
//例2:
int main()
{
    Stack<char *> st;
    char po[40];
    …
    st.pop(po);  //错误
    …
}
```

➡

```
template <class Type>
bool Stack<Type>::pop(Type& item)
{
    if (top > 0)
    {
        item = items[--top];
        return true;
    }
    else
        return false;
}
```



//引用变量item必须引用某类型的左值，而不是数组名；
代码不能为数组名赋值

- 深入探讨模板类：不正确地使用指针栈

```
//例3:
int main()
{
    Stack<char *> st;
    char * po = new char[40];
    …
    st.push(po);
    …
}
```



```
template <class Type>
bool Stack<Type>::push(const Type& item)
{
    if (top < MAX)
    {
        items[top++] = item;
        return true;
    }
    else
        return false;
}
```

//每次执行压入操作时，加入到栈中的地址都相同

对栈执行弹出操作时，得到的地址总是指向读入的最后一个字符串

- 数组模板：

➢ 非类型（non-type）或表达式（expression）参数

```
template <class T, int n>      //使用模板参数提供常规数组的大小
class ArrayTP          //详见primer书  arraytp.h
{
    ...
};
```

假设有：ArrayTP<double, 12> eggweights;

则：编译器定义名为ArrayTP<double, 12>的类，并创建一个类型为ArrayTP<double, 12>的eggweight对象。定义类时，编译器将使用double替换T，使用12替换n

- 模板多功能性：

➢ 递归使用模板

```
ArrayTP < ArrayTP<int,5>, 10> twodee;
```

- twodee是一个包含10个元素的数组，其中每个元素都是一个包含5个int元素的数组
- 等价于常规的数组声明：int twodee[10][5]
- 示例程序：详见primer书

➢ 使用多个类型参数

- 模板可以包含多个类型参数。假设希望类可以保存两种值，则可以创建并使用Pair模板来保存两个不同的值

```
Pair<string, int>("The Purpled Duck", 5) //调用构造函数
```

- 标准模板库提供了类似的模板，名为pair

- 模板多功能性：

➢ 默认类型模板参数

```
template <class T1, class T2 = int> class Topo {...};
```

- 类模板的新特性：为类型参数提供默认值
- 如果省略T2的值，编译器将使用int：

```
Topo<double, double> m1;  // T1 is double, T2 is double
Topo<double> m2;          // T1 is double, T2 is int
```

- 将模板用作参数：

➢ 模板可以包含类型参数（如typename T）和非类型参数（如int n）

➢ 模板还可以包含本身就是模板的参数，用于实现STL（标准模板库，不深入讨论）

```
template <template <typename T> class Thing>
class Crab {…};
```

- 模板参数是template <typename T> class Thing
- 其中template <typename T> class 是类型，Thing是参数
- 假设有：Crab<Stack> nebula; 则Stack的声明需与Thing的声明匹配：

```
template <typename T>
class Stack {…};      //stacktp.h中已定义
```

• 将模板用作参数示例：

```
// tempparm.cpp - templates as parameters
#include <iostream>
#include "stacktp.h"
using namespace std;
template <template <typename T> class Thing >
class Crab
{
  private:
     Thing<int> s1;
     Thing<double> s2;
  public:
     Crab() {};
     // assumes the thing class has push and pop() members
     bool push(int a, double x) { return s1.push(a) && s2.push(x); }
     bool pop(int& a, double& x) { return s1.pop(a) && s2.pop(x); }
};  //接下一页
```

```
// 接上一页
int main()
{
    Crab<Stack> nebula;// Stack must match template <typename T> class Thing
    int ni;
    double nb;
    cout << "Enter int double pairs, such as 4 3.5 (0 0 to end):\n";
    while (cin >> ni >> nb && ni > 0 && nb > 0)
    {
        if (!nebula.push(ni, nb)) break;
    }
    while (nebula.pop(ni, nb))
        cout << ni << ", " << nb << endl;
    cout << "Done.\n";
    return 0;
}
```

```
Enter int double pairs, such as 4 3.5 (0 0 to end):
50 22.48
25 33.87
60 19.12
0 0
60, 19.12
25, 33.87
50, 22.48
Done.
```

总结

- 共用数据的保护（熟悉）
  - 基本概念
  - 常对象
  - 常对象成员
  - 共用数据小结
- 静态成员（熟悉）
  - 基本概念
  - 静态数据成员
  - 静态成员函数
- 类模板（了解）
  - 函数模板（复习）
  - 类模板的使用

第七章 继承与派生

模块7.1：继承与派生的基本概念

主讲教师：同济大学计算机科学与技术学院 陈宇飞
同济大学计算机科学与技术学院 龚晓亮

目录

- 继承与派生基础
- 派生类的成员访问属性
- 派生类的构造函数和析构函数

目录

- 继承与派生基础
  - 基本概念
  - 派生类的声明方式
  - 派生类对象的构成

1.1 基本概念

- 继承：C++中实现重用的机制（软件代码可重复利用）
- 类的继承：在一个已有类的基础上建立一个新类
- 类的派生：对继承而言，可以理解为已有类派生出新类
  - ➢可以在已有类的基础上添加新功能。例如：数组类可以添加数学运算
  - ➢可以给类添加数据。例如：字符串类，派生出的新类添加指定字符串显式颜色的数据成员
  - ➢可以修改类方法的行为。例如：对于代表提供给飞机乘客服务的Passenger类，派生出提供更高级别服务的FirstClassPassenger类

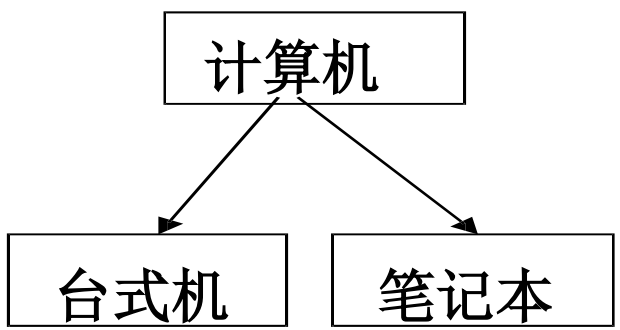
1.1 基本概念

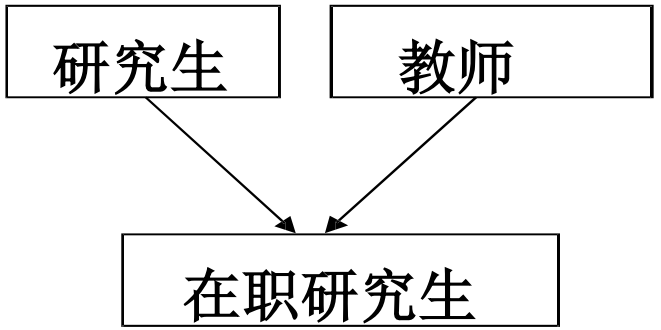
- 基类与派生类：基类(父类)，指已有的类

  派生类(子类)，新建立的类

- 派生类是基类的细化，基类是派生类的抽象

- 单继承与多继承：一个派生类从一个基类派生，则称为单继承，否则称为多继承





1.1 基本概念

- 一个简单的基类：

Webtown俱乐部决定跟踪乒乓球会会员。作为俱乐部的首席程序员，需要设计一个简单的TableTennisPlayer类：只记录会员的姓名以及是否有球桌。

数据成员：会员姓名（标准string类，比字符数组方便灵活且安全）

是否有球桌

成员函数：构造函数（建议使用成员初始化列表方式）

其他函数…

```
//tabtenn0.h -- a table-tennis base class
#ifndef TABTENN0_H_
#define TABTENN0_H_
#include <string>
class TableTennisPlayer
{
private:
    string firstname;  //比字符数组方便灵活且安全
    string lastname;
    bool hasTable;
public:
    TableTennisPlayer(const string & fn = "none",
                      const string & ln = "none", bool ht = false);
    void Name() const;
    bool HasTable() const { return hasTable; };
    void ResetTable(bool v) { hasTable = v; };
};
#endif
```

头文件
类的声明

```
//tabtenn0.cpp -- simple base-class methods
#include"tabtenn0.h"
#include <iostream>
using namespace std;
TableTennisPlayer::TableTennisPlayer(const string & fn, const string &
  ln, bool ht):firstname(fn),lastname(ln),hasTable(ht) {}
//初始化列表方式：直接使用string的复制构造函数将firstname初始化为fn…
void TableTennisPlayer::Name() const{
    cout << lastname << ", " << firstname;
}
```

源程序文件
函数的实现

- 将构造函数修改如下，注意区别：

```
TableTennisPlayer::TableTennisPlayer(const string & fn,
                                     const string & ln, bool ht){
    firstname = fn; lastname = ln; hasTable = ht;
}
//首先调用string的默认构造函数，再调用赋值运算符将firstname设置为fn…
```

```
//usett0.cpp -- using a base class
#include <iostream>
#include "tabtenn0.h"
using namespace std;
int main(){
    TableTennisPlayer player1("Chuck", "Blizzard", true);
    TableTennisPlayer player2("Tara", "Boomdea", false);
    player1.Name();
    if (player1.HasTable())
        cout << ": has a table.\n";
    else
        cout << ": hasn't a table.\n";
    player2.Name();
    if (player2.HasTable())
        cout << ": has a table.\n";
    else
        cout << ": hasn't a table.\n";
    return 0;
}
```

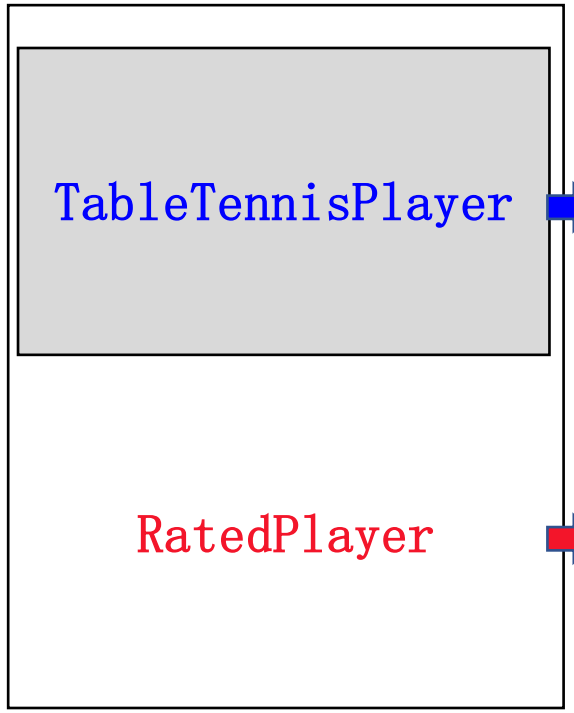
源程序文件
调用函数

构造函数的形参类型为const string &:
1)用string对象作构造函数的参数：调用接受const string&作为参数的string构造函数
2)用字符串作构造函数的参数（本例）：调用接受const char*作为参数的string构造函数

1.1 基本概念

- 派生一个类：

Webtown俱乐部的一些成员曾经参加过当地的乒乓球锦标赛，需要这样一个类，能包括成员在比赛中的比分。与其从零开始，不如从TableTennisPlayer类派生。



派生类对象存储了基类的数据成员（派生类继承了基类的实现）

派生类对象可以使用基类的方法（派生类继承了基类的接口）

派生类需要自己的构造函数

派生类可以根据需要添加额外的数据成员和成员函数

1.1 基本概念

- 派生一个类：

Webtown俱乐部的一些成员曾经参加过当地的乒乓球锦标赛，需要这样一个类，能包括成员在比赛中的比分。与其从零开始，不如从TableTennisPlayer类派生。

```
// simple derived class
class RatedPlayer : public TableTennisPlayer{
private:
   unsigned int rating;  //add a data member
public:
   RatedPlayer(unsigned int r = 0, const string & fn = "none",
               const string & ln = "none", bool ht = false);
   RatedPlayer(unsigned int r, const TableTennisPlayer & tp);
   unsigned int Rating() const { return rating; }     //add a method
   void ResetRating(unsigned int r) { rating = r; } //add a method
};
```

- 构造函数必须给新成员和继承的成员提供数据：//实现方法1

```
RatedPlayer::RatedPlayer(unsigned int r, const string & fn,
          const string & ln, bool ht) : TableTennisPlayer(fn, ln, ht)
{
   rating = r;
}
```

若有：RatedPlayer rplayer1(1140, "Mallory", "Duck", true);

- RatedPlayer构造函数将实参"Mallory", "Duck", true 赋值给形参fn, ln, ht；
- 参数fn, ln, ht作为实参传递给TableTennisPlayer构造函数，创建一个嵌套的TableTennisPlayer对象，将数据"Mallory", "Duck", true 存储在该对象中；
- 进入RatedPlayer构造函数体，完成RatedPlayer对象的创建，并将r的值（1140）赋值给rating成员。

- 构造函数必须给新成员和继承的成员提供数据： //实现方法1-引申

```
RatedPlayer::RatedPlayer(unsigned int r, const string & fn,
                         const string & ln, bool ht) //省略成员初始化列表，等效于
                                                     :TableTennisPlayer()
{
    rating = r;
}
```

若有：RatedPlayer rplayer1(1140, "Mallory", "Duck", true);

- RatedPlayer构造函数将调用默认的基类构造函数；
- 除非要使用默认构造函数，否则应显式调用正确的基类构造函数。

- 构造函数必须给新成员和继承的成员提供数据：//实现方法2

```
RatedPlayer::RatedPlayer(unsigned int r, const TableTennisPlayer & tp)
        : TableTennisPlayer(tp)
{
    rating = r;
}
```

```
RatedPlayer::RatedPlayer(unsigned int r, const TableTennisPlayer & tp)
        : TableTennisPlayer(tp), rating(r){}
```

若有：RatedPlayer rplayer1(1140, tp);

- 派生类RatedPlayer的构造函数将基类信息tp传递给基类TableTennisPlayer构造函数，即通过调用基类的复制构造函数完成；
- 使用动态内存分配时，注意隐式复制构造函数的合理性；
- 构造和析构的顺序（8.4讨论）。

```
//tabtenn1.h -- a table-tennis base class
class TableTennisPlayer{…}
class RatedPlayer : public TableTennisPlayer{…}
```

头文件
类的声明

```
//tabtenn1.cpp -- simple base-class methods
#include"tabtenn1.h"
TableTennisPlayer::TableTennisPlayer(const string & fn, const string &
   ln, bool ht):firstname(fn),lastname(ln),hasTable(ht){}
RatedPlayer::RatedPlayer(unsigned int r, const string & fn,
          const string & ln, bool ht) : TableTennisPlayer(fn, ln, ht)
{ rating = r; }
…
```

源程序文件
函数的实现

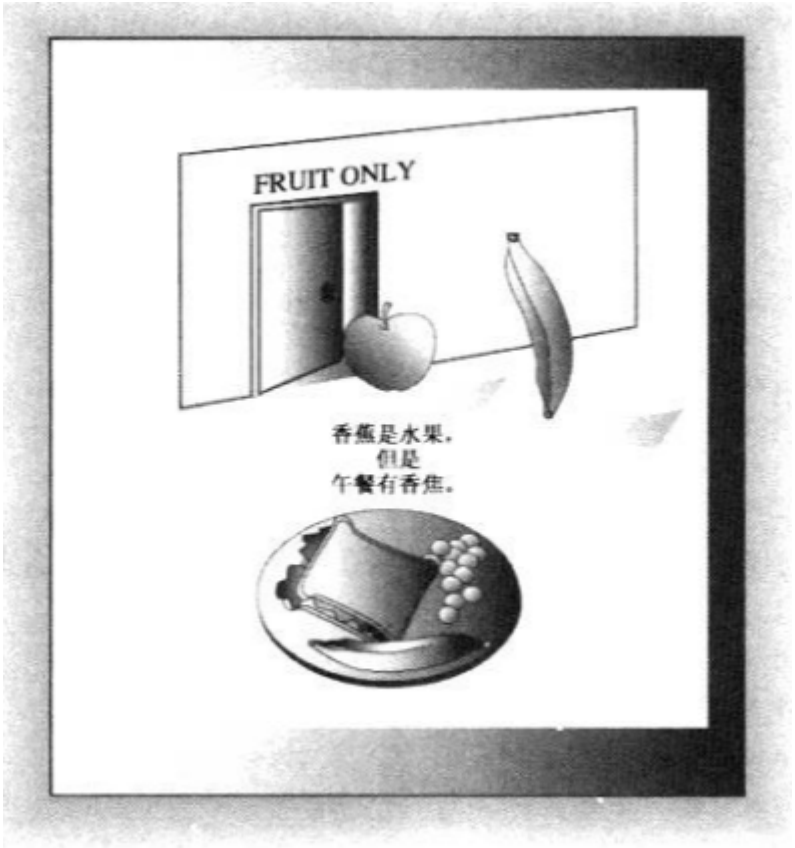
```
//usett1.cpp -- using base class and derived class
#include"tabtenn1.h"
int main()
{  TableTennisPlayer player1("Tara", "Boomdea", false);
   RatedPlayer rplayer1(1140, "Mallory", "Duck", true);
   …
}
```

源程序文件
调用函数

完整程序见primer书

1.1 基本概念

- 继承关系的本质：is-a关系



建议不建立下述关系，否则导致编程问题：

➢has-a关系：午餐有水果，但通常午餐并不是水果；

➢is-like-a关系：律师像鲨鱼，但律师不是鲨鱼（不可以在水下生活）；

➢is-implemented-as-a关系：使用数组来实现栈，但栈不是数组；

➢uses-a关系：计算机可以使用激光打印机，但从计算机派生出打印机类没有意义。

1.2 派生类的声明方式

形式：

```
class 派生类名:private/public 基类名1,
               private/public 基类名2,
                              ...
               private/public 基类名n {
    private:
        私有成员;
    public:
        公有成员;
}
```

当只有一个基类名时，单继承

当多于一个基类名时，多继承

- 根据需要加入派生类自己特有的成员

1.2 派生类的声明方式

形式：

```
class 派生类名:private/public 基类名1,
              private/public 基类名2,
                          ...
               private/public 基类名n {
    private:
        私有成员;
    public:
        公有成员;
}
```

基类存取限定符

- 根据限定符分为私有继承和公有继承，缺省是private

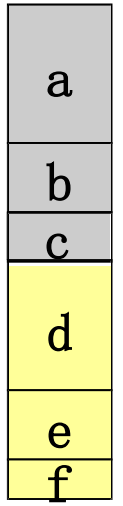
1.3 派生类对象的构成

```
class A {
   private:
      int a;
      short b;
      char c;
   public:
      void f1();
      int f2();
}
```

```
class B:public A {
   private:
      int d;
      short e;
      char f;
   public:
      void f3();
      int f4();
}
```

```
B b1;
b1.f1();
b1.f2();
b1.f3();
b1.f4();
```

内存：



存在≠可访问

- 继承基类的全部数据成员(不一定都可以访问)
- 继承基类除构造函数和析构函数外的全部成员函数(不一定都可以访问)

1.3 派生类对象的构成

- 派生类对象所占的空间：

  基类数据成员所占空间总和 + 派生类数据成员所占空间的总和

- 派生类可访问的成员函数：

  基类成员函数 + 派生类成员函数

- 友元不能继承

- 派生类的数据成员/成员函数允许和基类的同名，不同的继承方式访问方法不同（下一节讨论）

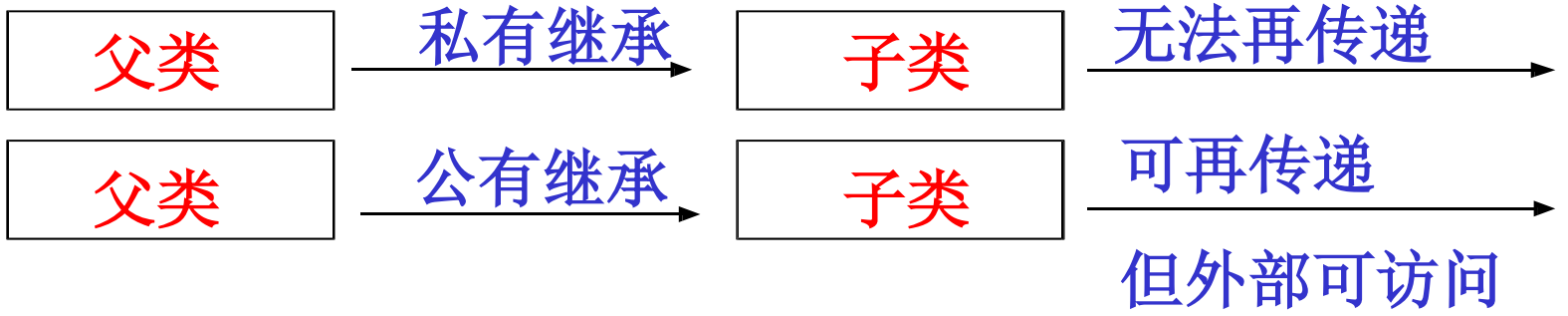
目录

- 派生类的成员访问属性
  - 继承访问控制权限
  - 继承方式与访问属性
  - 派生类的成员访问属性

2.1 继承访问控制权限

- 公有继承与私有继承

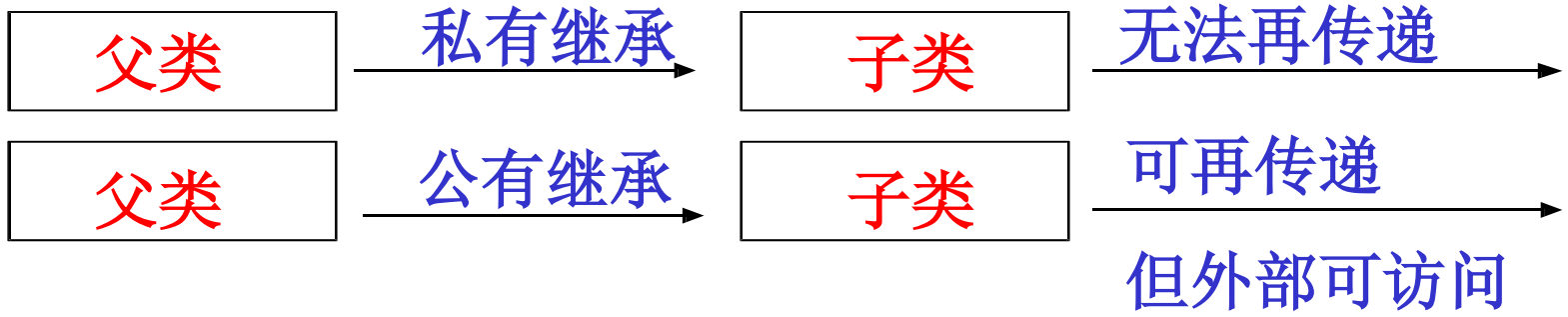
基类成员访问控制	继承访问控制	在派生类中的访问控制
public	public	public
private		不可访问
public	private	private
private		不可访问



2.1 继承访问控制权限

- 保护段与保护继承

在多级继承中，基类的private不可访问，public部分被私有继承则不可再传递，若公有继承则对于整个继承序列的外部均可访问：



为了加强灵活性，引入保护段及保护继承：

(1) 父类的成员在派生类的继承序列中内部能够访问

(2) 父类的成员在派生类的继承序列中外部不能访问

2.1 继承访问控制权限

- 保护段的定义：

```
class 类名 {
    private:
        ...
    protected:
        ...
    public:
        ...
}
```

➢protected被private继承后当做派生类的private

➢protected被public继承后当做派生类的protected

➢若该类不被其它类所继承，则声明保护段无意义（不被继承的情况下与声明为private等价）

2.1 继承访问控制权限

- 派生类的最终定义形式：

```
class 派生类名: private/protected/public 基类名{
    private:
        ...
    protected:
        ...
    public:
        ...
};
```

不同继承方式/不同限定成员的访问属性？

2.2 继承方式与访问属性

- 小结：

基类成员访问控制	继承访问控制	在派生类中的访问控制	
public	基类访问限定	public	(1)公有继承
protected	public	protected	
private		不可访问	
public	继承访问控制	protected	(2)保护继承
protected	protected	protected	
private		不可访问	
public	继承访问控制	private	(3)私有继承
protected	private	private	
private		不可访问	

```
class Base
{public:
      int x;
  protected:
      int y;
  private:
      int z;
}
```

Base B;

x
y
z

Base B;

int k=B.x; (√)

int k=B.y; (×)

int k=B.z; (×)

基类成员访问控制	继承访问控制	在派生类中的访问控制
public	基类访问限定	public
protected	public	protected
private		不可访问

(1) 公有继承

```
class C1:public Base
{public:
      int p;
      void fun()
      {只有z不可访问}
}
```

C1 C;

x, p, fun()
y
z

C1 C;

int k=C.p; (√)

int k=C.x; (√)

int k=C.y; (×)

int k=C.z; (×)

- 例1：B公有继承A

```
class A {
    private:
        int a;
        void f1();
    protected:
        int b;
        void f2();
    public:
        int c;
        void f3();
};
```

```
class B:public A{
    private:
        int d;
        void f4();
    protected:
        int e;
        void f5();
    public:
        int f;
        void f6();
        void fun();
};
```



c, f3(), f, f6(), fun()
b, f2(), e, f5()
d, f4()
a, f1()

```
void fun() //成员
{
    a=10; ×
    f1(); ×
    b=10;
    f2();
    c=10;
    f3();
    d=10;
    f4();
    e=10;
    f5();
    f=10;
    f6();
}
```

```
int main()  //域外
{  C c1;
   c1.a=10;×
   c1.f1();×
   c1.b=10;×
   c1.f2();×
   c1.c=10;
   c1.f3();
   c1.d=10;×
   c1.f4();×
   c1.e=10;×
   c1.f5();×
   c1.f=10;
   c1.f6();
}
```

```
class Base
{public:
      int x;
  protected:
      int y;
  private:
      int z;
}
```

```
Base B;
```

x
y
z

```
Base B;
int k=B.x;  (√)
int k=B.y;  (×)
int k=B.z;  (×)
```

继承访问控制		
public	protected	protected
protected	protected	protected
private		不可访问

(2) 保护继承

```
class C1:protected Base
{public:
      int p;
      void fun()
      {只有z不可访问}
}
```

```
C1 C;
```

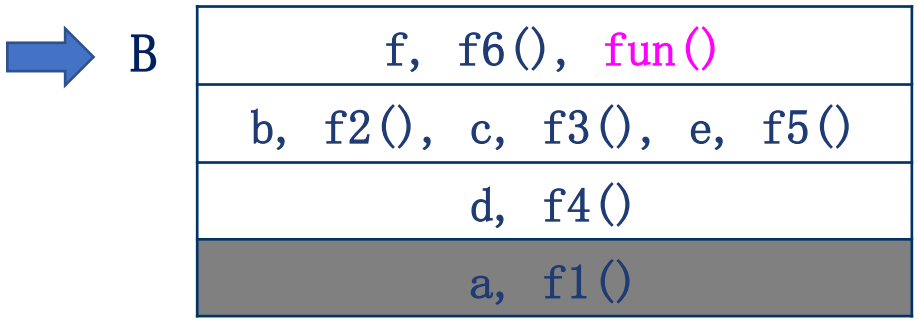
p,fun()
x, y
z

```
C1 C;
int k=C.p; (√)
int k=C.x; (×)
int k=C.y; (×)
int k=C.z; (×)
```

- 例2：B保护继承A

```
class A {
    private:
        int a;
        void f1();
    protected:
        int b;
        void f2();
    public:
        int c;
        void f3();
};
```

```
class B:protected A{
    private:
        int d;
        void f4();
    protected:
        int e;
        void f5();
    public:
        int f;
        void f6();
        void fun();
};
```




```
void fun() //成员
{
    a=10; ×
    f1(); ×
    b=10;
    f2();
    c=10;
    f3();
    d=10;
    f4();
    e=10;
    f5();
    f=10;
    f6();
}
```

```
int main() //域外
{  C c1;
   c1.a=10;×
   c1.f1();×
   c1.b=10;×
   c1.f2();×
   c1.c=10;×
   c1.f3();×
   c1.d=10;×
   c1.f4();×
   c1.e=10;×
   c1.f5();×
   c1.f=10;
   c1.f6();
}
```

```
class Base
{public:
      int x;
  protected:
      int y;
  private:
      int z;
}
```

Base B;

x
y
z

Base B;

int k=B.x; (√)

int k=B.y; (×)

int k=B.z; (×)

继承访问控制		
public		private
protected	private	private
private		不可访问

(3)私有继承

```
class C1:private Base
{public:
      int p;
      void fun()
      {只有z不可访问}
}
```

C1 C;

p, fun()
x, y
z

C1 C;

int k=C.p; (√)

int k=C.x; (×)

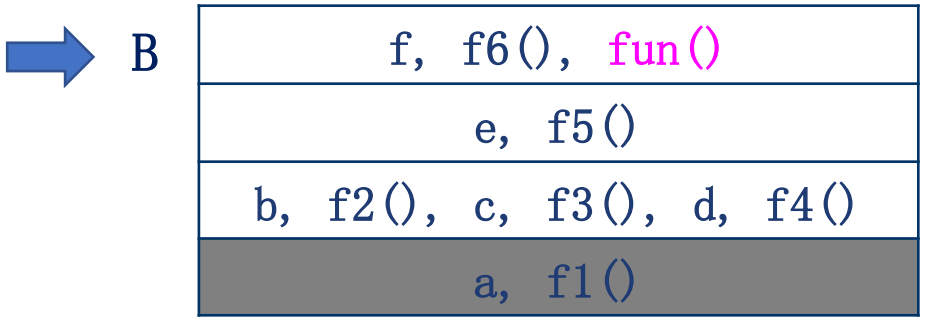
int k=C.y; (×)

int k=C.z; (×)

- 例3：B私有继承A

```
class A {
   private:
      int a;
      void f1();
   protected:
      int b;
      void f2();
   public:
      int c;
      void f3();
};
```

```
class B:private A{
   private:
      int d;
      void f4();
   protected:
      int e;
      void f5();
   public:
      int f;
      void f6();
      void fun();
};
```




```
void fun() //成员
{
    a=10; ×
    f1(); ×
    b=10;
    f2();
    c=10;
    f3();
    d=10;
    f4();
    e=10;
    f5();
    f=10;
    f6();
}
```

```
int main() //域外
{  C c1;
   c1.a=10;×
   c1.f1();×
   c1.b=10;×
   c1.f2();×
   c1.c=10;×
   c1.f3();×
   c1.d=10;×
   c1.f4();×
   c1.e=10;×
   c1.f5();×
   c1.f=10;
   c1.f6();
}
```

2.3 派生类的成员访问属性

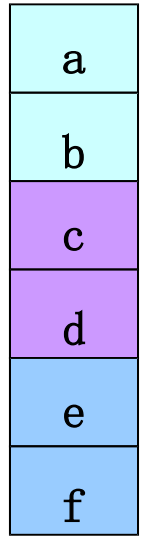
- 派生类的多级派生

```
class A {
  private:
    int a;
    void f1();
  public:
    int b;
    void f2();
}
```

```
class B: public A {
   private:
     int c;
     void f3();
  public:
     int d;
     void f4();
}
```

```
class C: public B {
  private:
    int e;
    void f5();
  public:
    int f;
    void f6();
}
```

C c1;



存在≠可访问

- 基类分为直接基类和间接基类：C的直接基类是B，间接基类是A，B的直接基类是A
- 派生类包含所有基类的数据成员(不一定可访问)
- 基类的成员的被访问关系逐级确定

2.3 派生类的成员访问属性

- 多级派生例1：B私有继承A，C私有继承B
- 多级派生例2：B公有继承A，C公有继承B
- 多级派生例3：B保护继承A，C保护继承B
- 多级派生例4：B私有继承A，C公有继承B
- 多级派生例5：B私有继承A，C保护继承B
- 多级派生例6：B公有继承A，C私有继承B
- 多级派生例7：B公有继承A，C保护继承B
- 多级派生例8：B保护继承A，C私有继承B
- 多级派生例9：B保护继承A，C公有继承B

自行补充

//基类的成员的被访问关系逐级确定，注意区分成员函数内访问和域外访问

- 多级派生例1：B私有继承A，C私有继承B

```
class A {
  private:
    int a1;
    void fa1();
  protected:
    int a2;
    void fa2();
  public:
    int a3;
    void fa3();
};
```

```
class B: private A{
  private:
    int b1;
    void fb1();
  protected:
    int b2;
    void fb2();
  public:
    int b3;
    void fb3();
};
```

```
class C: private B{
  private:
    int c1;
    void fc1();
  protected:
    int c2;
    void fc2();
  public:
    int c3;
    void fc3();
    void fun();
};
```

逐级确定

B

b3, fb3()
b2, fb2()
a2, fa2(), a3, fa3(), b1, fb1()
a1, fa1()

C

c3, fc3(), fun()
c2, fc2()
b2, fb2(), b3, fb3(), c1, fc1()
a1, fa1(), a2, fa2(), a3, fa3(), b1, fb1()

- 多级派生例1：B私有继承A，C私有继承B

```
void C::fun() //成员
{
    a1=10;×  fa1();×
    a2=10;×  fa2();×
    a3=10;×  fa3();×
    b1=10;×  fb1();×
    b2=10;   fb2();
    b3=10;   fb3();
    c1=10;   fc1();
    c2=10;   fc2();
    c3=10;   fc3();
}
```

```
int main()  //域外
{   C c;
    c.a1=10;×  c.fa1();×
    c.a2=10;×  c.fa2();×
    c.a3=10;×  c.fa3();×
    c.b1=10;×  c.fb1();×
    c.b2=10;×  c.fb2();×
    c.b3=10;×  c.fb3();×
    c.c1=10;×  c.fc1();×
    c.c2=10;×  c.fc2();×
    c.c3=10;   c.fc3();
}
```

逐级确定

B

b3, fb3()
b2, fb2()
a2, fa2(), a3, fa3(), b1, fb1()
a1, fa1()

C

c3, fc3(), fun()
c2, fc2()
b2, fb2(), b3, fb3(), c1, fc1()
a1, fa1(), a2, fa2(), a3, fa3(), b1, fb1()

- 多级派生例2：B公有继承A，C公有继承B

```
class A {
  private:
    int a1;
    void fa1();
  protected:
    int a2;
    void fa2();
  public:
    int a3;
    void fa3();
};
```

```
class B: public A{
  private:
    int b1;
    void fb1();
  protected:
    int b2;
    void fb2();
  public:
    int b3;
    void fb3();
};
```

```
class C: public B{
  private:
    int c1;
    void fc1();
  protected:
    int c2;
    void fc2();
  public:
    int c3;
    void fc3();
    void fun();
};
```

逐级确定

B

a3, fa3(), b3, fb3()
a2, fa2(), b2, fb2()
b1, fb1()
a1, fa1()

C

a3, fa3(), b3, fb3(), c3, fc3(), fun()
a2, fa2(), b2, fb2(), c2, fc2()
c1, fc1()
a1, fa1(), b1, fb1()

- 多级派生例2：B公有继承A，C公有继承B

```
void C::fun() //成员
{
    a1=10;×  fa1();×
    a2=10;   fa2();
    a3=10;   fa3();
    b1=10;×  fb1();×
    b2=10;   fb2();
    b3=10;   fb3();
    c1=10;   fc1();
    c2=10;   fc2();
    c3=10;   fc3();
}
```

```
int main()  //域外
{   C c;
    c.a1=10;×  c.fa1();×
    c.a2=10;×  c.fa2();×
    c.a3=10;   c.fa3();
    c.b1=10;×  c.fb1();×
    c.b2=10;×  c.fb2();×
    c.b3=10;   c.fb3();
    c.c1=10;×  c.fc1();×
    c.c2=10;×  c.fc2();×
    c.c3=10;   c.fc3();
}
```

逐级确定

B

a3, fa3(), b3, fb3()
a2, fa2(), b2, fb2()
b1, fb1()
a1, fa1()

C

a3, fa3(), b3, fb3(), c3, fc3(), fun()
a2, fa2(), b2, fb2(), c2, fc2()
c1, fc1()
a1, fa1(), b1, fb1()

2.3 派生类的成员访问属性

- 派生类与基类的成员同名

a）数据成员同名：

➢派生类屏蔽基类，通过直接访问方式可访问派生类的成员，通过加类的作用域符的方式可访问基类成员

b）成员函数同名：

➢参数的个数、类型完全相同：与数据成员同名的处理方式相同

➢参数的个数、类型不同：与数据成员同名的处理方式相同

上述规则一致：派生类优先于基类，称为支配规则

a）数据成员同名：

➢派生类屏蔽基类，通过直接访问方式可访问派生类的成员，通过加类的作用域符的方式可访问基类成员

```
class A{
public:
   int a;
   …
};
class B:public A{
public:
   short a;
   void fun();
   …
};
```

```
void B::fun()
{
   a=10;
   A::a=15;
}
int main()
{
   B b1;
   b1.a=10;
   b1.A::a=15;
}
```

```
int a;
class A{
public:
   int a;
   …
};
class B:public A{
public:
   short a;
   void fun();
   …
};
```

```
void B::fun()
{
   int a;
   a=10;
   A::a=15;
   B::a=20;
   ::a=30;
}
```

b）成员函数同名：

➢参数的个数、类型完全相同：与数据成员同名的处理方式相同

```
class A {
public:
    void f1();
    …
};
class B:public A {
public:
    void f1();
    void fun();
    …
};
```

```
void B::fun()
{
    f1();
    A::f1();
}
int main(){
    B b1;
    b1.f1();
    b1.A::f1();
}
```

b）成员函数同名：

➢参数的个数、类型不同：与数据成员同名的处理方式相同

```
class A {
public:
    void f1(int x);
    …
};
class B:public A {
public:
    void f1();
    void fun();
    …
};
```

```
void B::fun()
{
    f1();
    f1(10); //编译错误
}
int main()
{
    B b1;
    b1.f1();
    b1.f1(10); //编译错误
}
```

b）成员函数同名：

➢参数的个数、类型不同：与数据成员同名的处理方式相同

```
class A {
public:
    void f1(int x);
    …
};
class B:public A {
public:
    void f1();
    void fun();
    …
};
```

```
void B::fun()
{
    f1();
    A::f1(10);
}
int main()
{
    B b1;
    b1.f1();
    b1.A::f1(10);
}
```

2.3 派生类的成员访问属性

- 基类对象不可访问部分的强制访问

➢问题引入：

任何继承方式派生类均不能访问基类的private

派生类的对象中包含了基类对象(包括private部分)

=> 不可访问，又占空间？

➢解决方法：

继承后，基类的私有部分不可被派生类直接访问，但是可以通过基类的可访问的成员函数进行间接访问

- 基类对象不可访问部分的强制访问

```
#include <iostream>
using namespace std;
class A {
  private:
     int a;
  public:
     void set(int x)
     {
        a=x;
     }
     void show()
     {
        cout << a << endl;
     }
};
```

```
class B:public A {
  public:
     void fun()
     {
        a=10;   //编译错
        set(10);  //但可间接访问
     }
};
int main(){
     B b1;
     b1.set(15);
     b1.show();
     b1.fun();
     b1.show();
}
```

目录

- 派生类的构造函数和析构函数
  - 简单的派生类构造函数
  - 含有子对象的派生类构造函数
  - 多层派生时的构造函数
  - 派生类的析构函数

3.1 简单的派生类构造函数

- 含义：在派生类的数据成员中不包含基类的对象
- 形式：派生类名(参数)：基类名(参数) 初始化表方式

```
{
    函数体(一般是对派生类新增成员的初始化)
}
```

- 构造函数的调用顺序：
  - 先基类，再派生类 (在派生类的构造函数中先自动激活基类的构造函数)

3.1 简单的派生类构造函数



代码实现形式：

```
class RatedPlayer: public TableTennisPlayer {                    //体内实现
    …
    RatedPlayer(unsigned int r, const string & fn,
          const string & ln, bool ht) : TableTennisPlayer(fn, ln, ht)
    { rating = r; }
}
```

```
class RatedPlayer: public TableTennisPlayer {                    //体外实现
    …
    RatedPlayer(unsigned int r, const string & fn, const string & ln, bool ht);
}
RatedPlayer::RatedPlayer(unsigned int r, const string & fn,
          const string & ln, bool ht) : TableTennisPlayer(fn, ln, ht)
{ rating = r; }
```

3.1 简单的派生类构造函数

- 若派生类的成员初始化表中不出现基类，则系统自动调用基类的无参构造函数

  (基类不定义构造函数时调用系统缺省的无参空体构造函数)

```
RatedPlayer::RatedPlayer(unsigned int r, const string & fn,
                         const string & ln, bool ht) //省略成员初始化列表，等效于
                                                     :TableTennisPlayer()
{
   rating = r;
}
```

- 派生类的成员初始化列表中出现基类

```
#include <iostream>
using namespace std;
class A {
public:
    A() { cout << "A()" << endl; }
    A(int x) { cout << "A(x)" << endl; }
};
class B:public A {
public:
    B(int x):A(x){ cout << "B(x)" << endl; }
    B(int x, int y):A(x){ cout << "B(x,y)" << endl; }
};
int main(){
    B b1(10);  cout << endl;
    B b2(100,200);
}
```

```
A(x)
B(x)

A(x)
B(x,y)
```

- 派生类的初始化列表中不出现基类（省略成员初始化列表）

```
#include <iostream>
using namespace std;
class A {
public:
    A() { cout << "A()" << endl; }
    A(int x) { cout << "A(x)" << endl; }
};
class B:public A {
public:
    B() { cout << "B()" << endl; }
    B(int x) { cout << "B(x)" << endl; }
};
int main(){
    B b1;  cout << endl;
    B b2(100);
}
```

```
A()
B()

A()
B(x)
```

- 不能以初始化列表的方式实现重载

```
#include <iostream>
using namespace std;
class A {
  public:
    A() { cout << "A()" << endl; }
    A(int x){ cout << "A(x)" << endl; }
  };
class B:public A {
   public:
    B() { cout << "B()" << endl; }
    B(int x) { cout << "B(x1)" << endl; }//重载错误
    B(int x):A(x) { cout << "B(x2)" << endl; }
    B(int x, int y):A(x){ cout << "B(x,y)" << endl; }
  };
```

```
int main()
{
   B b1; cout << endl;
   B b2(100); cout << endl;
   B b3(4, 5);
}
```

保留第一个：

```
A()
B()

A()
B(x1)

A(x)
B(x,y)
```

保留第二个：

```
A()
B()

A(x)
B(x2)

A(x)
B(x,y)
```

3.2 含有子对象的派生类构造函数

- 含义：在派生类定义中包含了基类的对象
- 形式：

  派生类名(参数):基类名(参数),子对象名(参数)

  {

  函数体(一般是对派生类新增成员的初始化)

  }

- 构造函数的调用顺序：
  - 先基类，次子对象，再派生类

- 含有子对象的派生类构造函数

```
class student {
   protected:
      int num;
      char name[10];
   public:
      student(int n,char *nam){
         num = n; strcpy(name, nam);
      }
   …
};
class stu:public student {
   protected:
      int age;
      char addr[30];
      student monitor;
   public:
      stu(int n, char *nam, int n1, char *nam1,int a, char *ad)
                  : student(n,nam), monitor(n1,nam1){ age = a; strcpy(addr, ad);}
};
```

```
int main()
{
   stu s1(10001,"张三",10009,"李四",20,"上海");
   …
}
```

班长

s1所占空间为14*2+34=62字节，即双份student，其中1份继承，1份是monitor

3.3 多层派生时的构造函数

- 形式：

  派生类名(参数):直接基类名(参数)

  {

  函数体(一般是对派生类新增成员的初始化)

  }

  - 间接基类不需要出现在初始化表中

- 构造函数的调用顺序：
  - 按继承顺序依次调用，派生类放在最后

(派生类中先激活直接基类，直接基类再先激活其直接基类)

- 多层派生时的构造函数

```
class A {
public:
   A(int x) {
     cout << "A" << x << endl;}
};
class B: public A{
public:
   B(int x, int y):A(x) {
     cout << "B" << y << endl;}
};
class C: public B {
public:
   C(int x, int y, int z):B(x, y){
       cout << "C" << z << endl;}
};
```

```
int main()
{
   B b1(4,5);
   cout << endl;
   C c1(1,2,3);
}
```

```
A4
B5

A1
B2
C3
```

3.4 派生类的析构函数

- 析构函数的调用：
  - ➢在派生类对象出作用域时，自动调用派生类的析构函数，在其中再自动调用基类的析构函数
- 析构函数的调用顺序：
  - ➢先派生类，再基类(与构造函数的顺序相反)
- 析构函数的使用：
  - ➢派生类的数据成员无动态内存申请的情况下一般不需要定义派生类的析构函数
  - ➢若基类有动态内存申请，派生类无，则基类需要定义，派生类不需要（见8.8）

总结

- 继承与派生基础（掌握）
  - 基本概念
  - 派生类的声明方式
  - 派生类对象的构成
- 派生类的成员访问属性（掌握）
  - 继承访问控制权限
  - 继承方式与访问属性
  - 派生类的成员访问属性
- 派生类的构造函数和析构函数（熟悉）
  - 简单的派生类构造函数
  - 含有子对象的派生类构造函数
  - 多层派生时的构造函数
  - 派生类的析构函数

第七章 继承与派生

模块7.2：多重继承的概念与实现

主讲教师：同济大学计算机科学与技术学院 陈宇飞
同济大学计算机科学与技术学院 龚晓亮

目录

- 多重继承
- 虚基类
- 赋值兼容规则
- 继承和动态内存分配

目录

- 多重继承
  - 多重继承的声明及使用
  - 多重继承的派生类
  - 多重继承引起的二义性问题

1.1 多重继承的声明及使用

- 声明形式

```
class 派生类名: private/protected/public 基类名1, ... ,
                                private/protected/public 基类名n {
        private:
            私有成员;
        protected:
            保护成员;
        public:
            公有成员;
};
```

1.1 多重继承的声明及使用

多重继承的使用

- 每个基类的基类存取限定符允许不同
- 每个基类被继承后的可访问性由其基类存取限定符确定
- 派生类继承所有基类的数据成员
- 派生类继承所有基类除构造函数和析构函数外的所有成员函数
- 派生类对象所占的空间 = 所有基类的数据成员之和 + 派生类的数据成员之和

1.1 多重继承的声明及使用

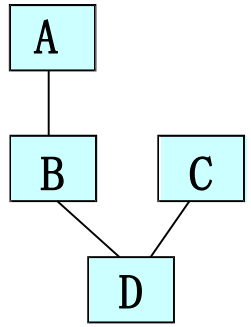
- 多重继承允许多层派生

```
class A {
  private:
    int a1;
  protected:
    int a2;
  public:
    int a3;
};
```

```
class B:public A {
  private:
    int b1;
  protected:
    int b2;
  public:
    int b3;
};
```

```
class C {
  private:
    int c1;
  protected:
    int c2;
  public:
    int c3;
};
```

```
class D:private B, public C {
 ...
};
```



1.2 多重继承的派生类

- 构造函数的形式

  派生类名(参数)：基类名1(参数),...， 基类名n(参数)

  {

  函数体(一般是对派生类新增成员的初始化)

  }

- 构造函数的使用：
  - 基类名出现的顺序无限制
  - 若调用基类的无参构造函数，则该基类可不出现在初始化参数表中
  - 若是多层派生，只出现直接基类
  - 若派生中含有基类的实例对象(子对象)，则子对象也可以出现在初始化参数表中

- 构造函数的使用：
  - 基类名出现的顺序无限制

```
class A {
  public:
    A(int x) { … }
}
class B {
  public:
    B(int y) { … }
}
```

```
class C:public A, private B{
  public:
    C(int a, int b, int c):A(a), B(b) {
                           B(b), A(a)
        …
    }
};
```

- 构造函数的使用：

  ➢ 若调用基类的无参构造函数，则该基类可不出现在初始化参数表中

```
class A {
  public:
    A() { … }
}
class B {
  public:
    B(int y) { … }
}
```

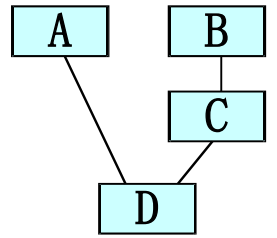
```
class C:public A, private B {
  public:
    C(int a, int b, int c):B(b) { … }
};
```

激活A的无参构造

- 构造函数的使用：
  - 若是多层派生，只出现直接基类

```
class A {
  public:
    A(int x) { … }
}
class B {
  public:
    B(int y) { … }
}
```

```
class C:public B {
  public:
      C(int z):B(z) { … };
}
class D:public A, private C {
  public:
      D(int a, int c, int d):A(a),C(c) { … }
};
```



B不需要出现

- 构造函数的使用：
  - 若派生中含有基类的实例对象(子对象)，则子对象也可以出现在初始化参数表中

```
class A {
  public:
    A(int x) { … }
}
class B {
  public:
    B(int y) { … }
}
```

```
class C:public A, private B {
  public:
    B bb;
    C(int a, int b, int b1):A(a), B(b), bb(b1){
    …
  }
};
```

1.2 多重继承的派生类

- 构造函数的调用顺序：
  - 按声明顺序依次调用基类的构造函数，最后调用派生类的构造函数
- 析构函数的调用顺序：
  - 先派生类的析构函数，再按声明顺序的反序依次调用基类的析构函数

//例1：

```
#include <iostream>
using namespace std;
class A {
   public:
      A() {cout << "A()" << endl;}
};
class B {
   public:
      B() {cout << "B()" << endl;}
};
class C {
   public:
      C() {cout << "C()" << endl;}
};
```

```
class D:public A, protected B, private C{
    public:
       D() {cout << "D()" << endl;}
};    //D():A(),B(),C() {…}
      //D():B(),A(),C() {…}
      //D():C(),B(),A() {…}
      …
int main(){
    D d1;
}
```



D对象的内存映像：

A的数据成员
B的数据成员
C的数据成员
D的数据成员

//例2：

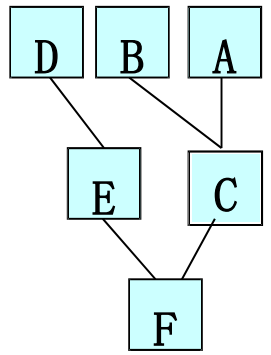
```
#include <iostream>
using namespace std;
class A {
  public:
   int a;
   A(){ a=1; }
};
class B {
  public:
   int b;
   B(){ b=2;}
};
class C {
  public:
   int c;
   C(){ c=3; }
};
class D:public A,
          protected B,
            private C {
 public:
    int d;
    D(){ d=4; }
};
int main()
{
   D d1;
   cout << sizeof(d1) << endl;
   int *p = (int *)&d1;
   cout << *p << endl;
   cout << *(p+1) << endl;
   cout << *(p+2) << endl;
   cout << *(p+3) << endl;
   return 0;
}
```

```
16
1
2
3
4
```

//例3：

```
#include <iostream>
using namespace std;
class A {
  public:
    A() {cout << "A()" << endl;}
};
class B {
  public:
    B() {cout << "B()" << endl;}
};
class C:public B, public A {
  public:
    C() {cout << "C()" << endl;}
};
class D {
  public:
    D() {cout << "D()" << endl;}
};
class E:public D {
  public:
    E() {cout << "E()" << endl;}
};
class F:public E, public C {
  public:
    F() {cout << "F()" << endl;}
};
int main(){
    F f1;
}
```

```
D()
E()
B()
A()
C()
F()
```



F对象的内存映像：



1.3 多重继承引起的二义性问题

- 成员同名
  - 以不产生二义性为基本准则

某一个基类与派生类的成员同名：

按单继承的方式进行处理(支配规则：派生类直接访问，基类加作用域符)

两个以上的基类中的成员同名：

分别加不同的基类作用域符区分

两个以上的基类与派生类的成员同名：

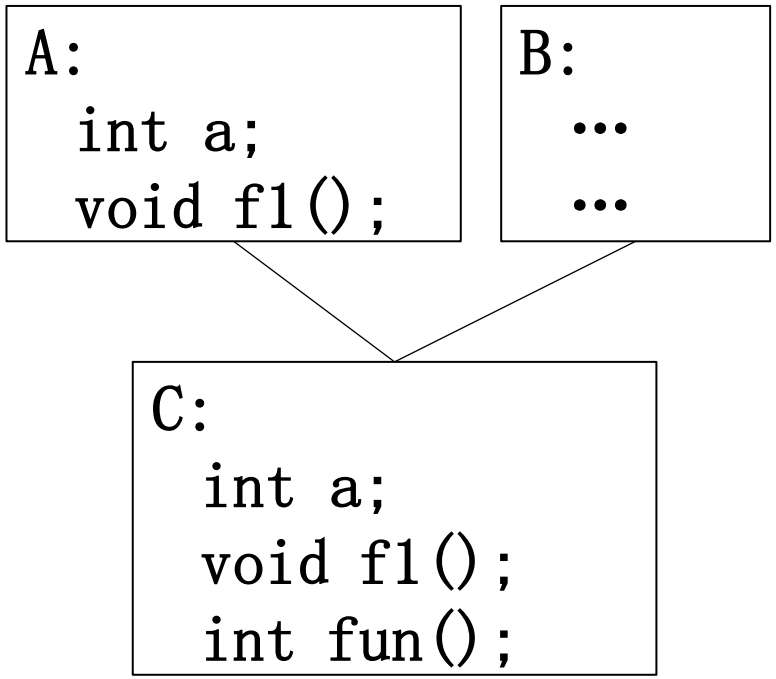
派生类直接访问，不同基类加作用域符区分

通过直接/间接方式继承同一个基类两个导致的同名：

通过可区分的类的作用域符来进行区分

- 某一个基类与派生类的成员同名：按单继承的方式进行处理

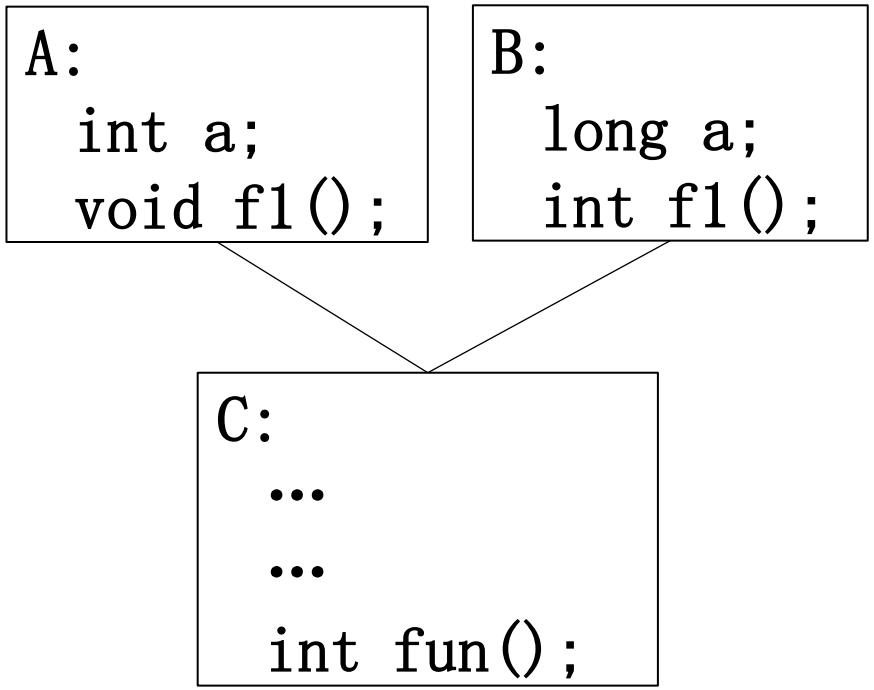
  （支配规则:派生类直接访问，基类加作用域符）




```
int C::fun()
{
    a=10;
    f1();
    A::a=15;
    A::f1();
}
```

```
int main()
{
    C c1;
    c1.a=10;
    c1.f1();
    c1.A::a=15;
    c1.A::f1();
}
```

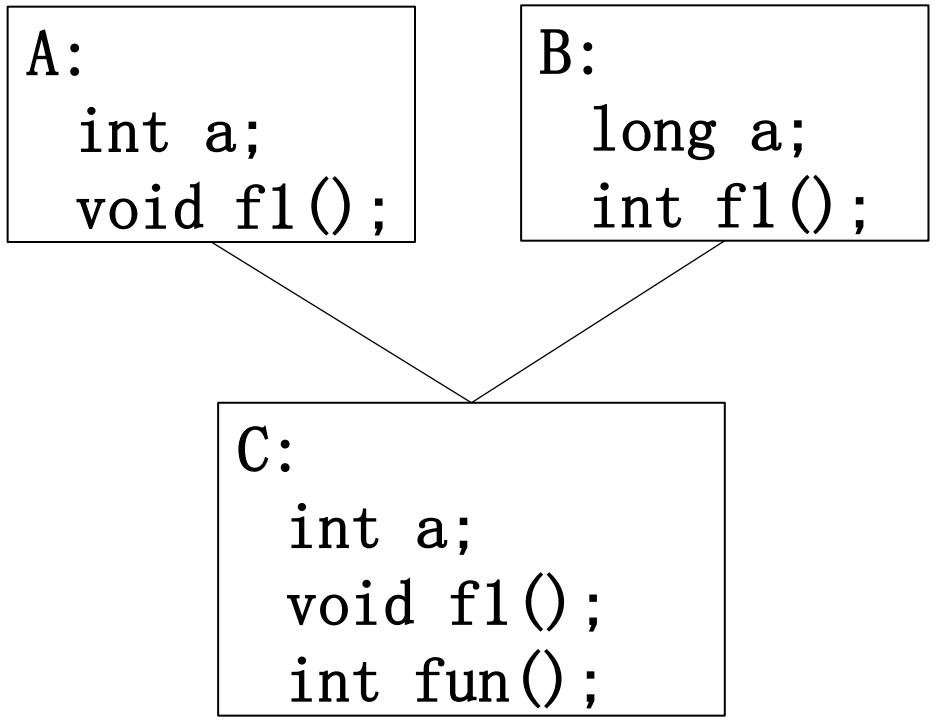
- 两个以上的基类中的成员同名：分别加不同的基类作用域符区分




```
int C::fun()
{
    A::a=10;
    A::f1();
    B::a=15;
    B::f1();
}
```

```
int main()
{
    C c1;
    c1.A::a=10;
    c1.A::f1();
    c1.B::a=15;
    c1.B::f1();
}
```

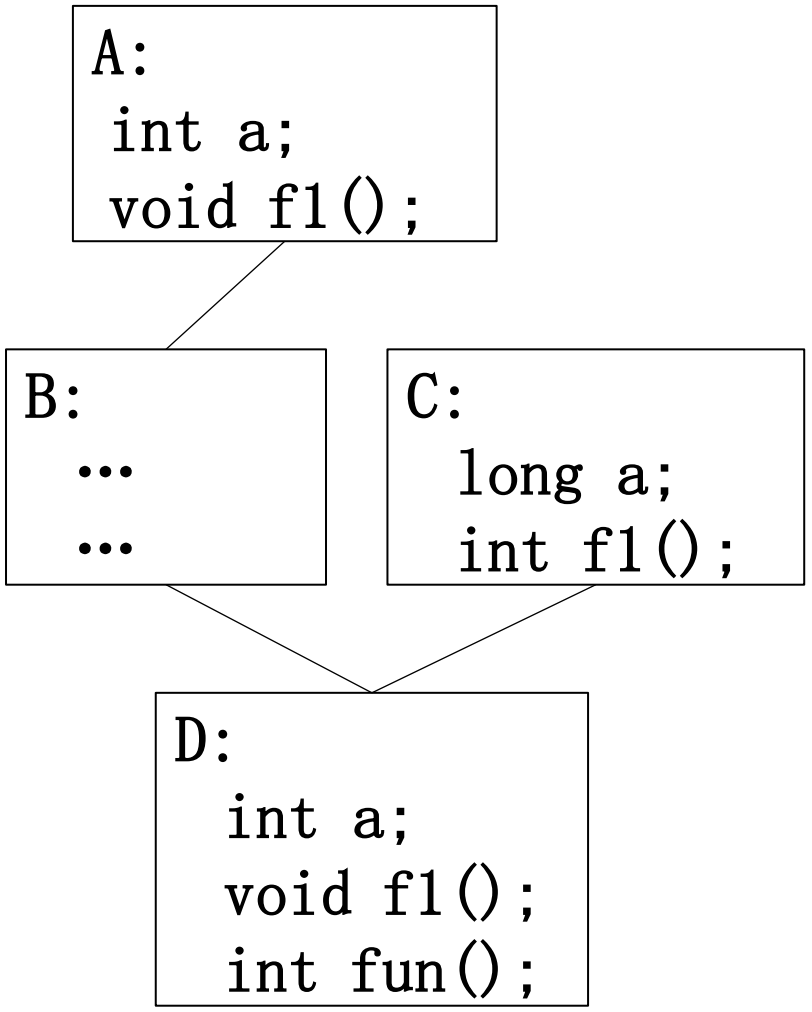
- 两个以上的基类与派生类的成员同名：派生类直接访问，不同基类加作用域符区分




```
int C::fun()
{
    a=10;
    f1();
    A::a=15;
    A::f1();
    B::a=20;
    B::f1();
}
```

```
int main()
{   C c1;
    c1.a=10;
    c1.f1();
    c1.A::a=15;
    c1.A::f1();
    c1.B::a=20;
    c1.B::f1();
}
```

- 通过直接/间接方式继承同一个基类两个导致的同名：通过可区分的类的作用域符来进行区分

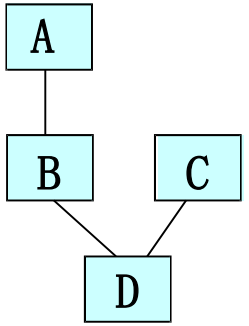



```
int D::fun()
{
   a=10;
   f1();
   A::a=15;可以B
   A::f1();可以B
   C::a=20;
   C::f1();
}
```

```
int main()
{  D d1;
   d1.a=10;
   d1.f1();
   d1.A::a=15;可以B
   d1.A::f1();可以B
   d1.C::a=20;
   d1.C::f1();
}
```

```
#include <iostream>
using namespace std;
class A {
  public:
    int a;
    void f1() { return; }
};
class B:public A {
  public:
    int b;
};
class C {
  public:
    long a;
    int f1() { return 0; }
};
class D:public B, public C {
  public:
    int a;
    char f1() { return 'A'; }
    void fun();
};
```

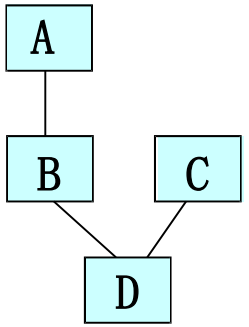
```
void D::fun(){
      a = 10;
      f1();
      A::a = 15; // B::a = 15;
      A::f1();   // B::f1();
      C::a = 20;
      C::f1();
      int *p = (int *)this;
      cout << sizeof(*this) <<endl;
      cout << *p << endl;
      cout << *(p+1) << endl;
      cout << *(p+2) << endl;
      cout << *(p+3) << endl;
}
int main(){
      D d1;
      d1.fun();
      return 0;
}
```




```
16
15
-858993460
20
10
```

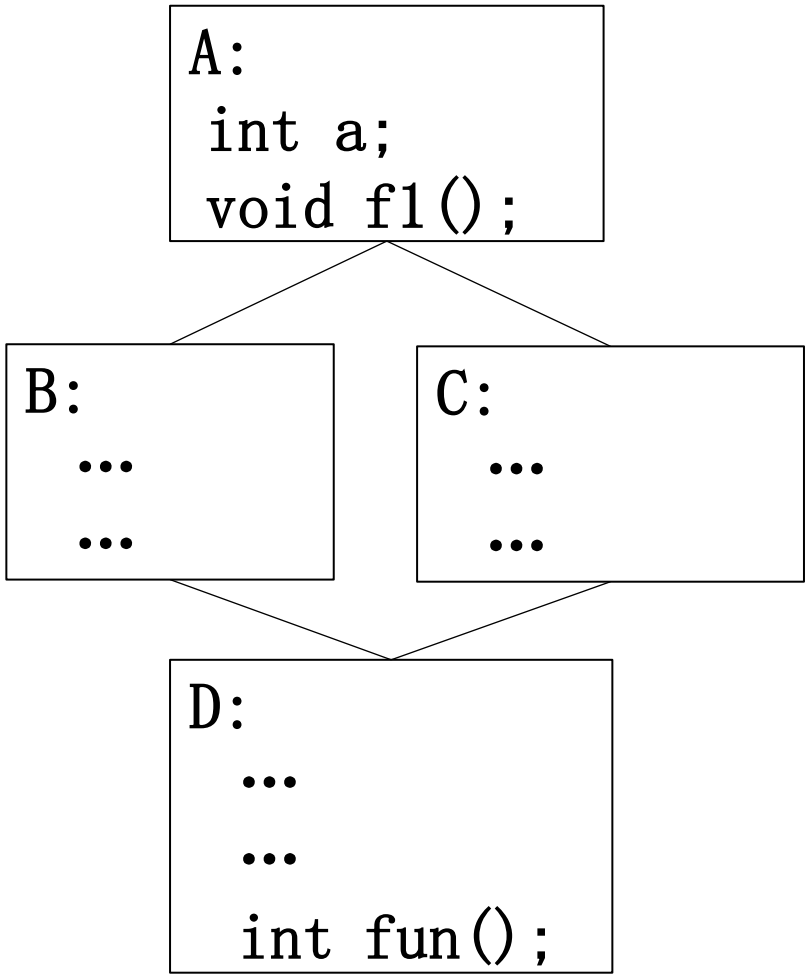
```
#include <iostream>
using namespace std;
class A {
  public:
    int a;
    void f1() { return; }
};
class B:public A {
  public:
    int b;
};
class C {
  public:
    long a;
    int f1() { return 0; }
};
class D:public B, public C {
  public:
    int a;
    char f1() { return 'A'; }
    void fun();
};
```

```
void D::fun(){
    return;
}
int main(){
    D d1;
    d1.a = 10;
    d1.f1();
    d1.A::a = 15;    // B::a = 15;
    d1.A::f1();      // B::f1();
    d1.C::a = 20;
    d1.C::f1();
    int *p = (int *)&d1;
    cout << sizeof(d1) <<endl;
    cout << *p << endl;
    cout << *(p+1) << endl;
    cout << *(p+2) << endl;
    cout << *(p+3) << endl;
    return 0;
}
```




```
16
15
-858993460
20
10
```

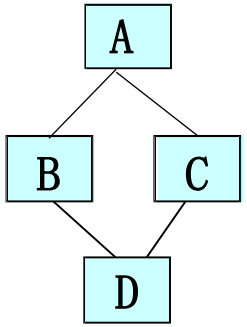
- 通过直接/间接方式继承同一个基类两个导致的同名：
  - 问题1：有几份A?



D对象的内存映象(两份A)

A的数据成员	B
B的数据成员	
A的数据成员	C
C的数据成员	
D的数据成员	D

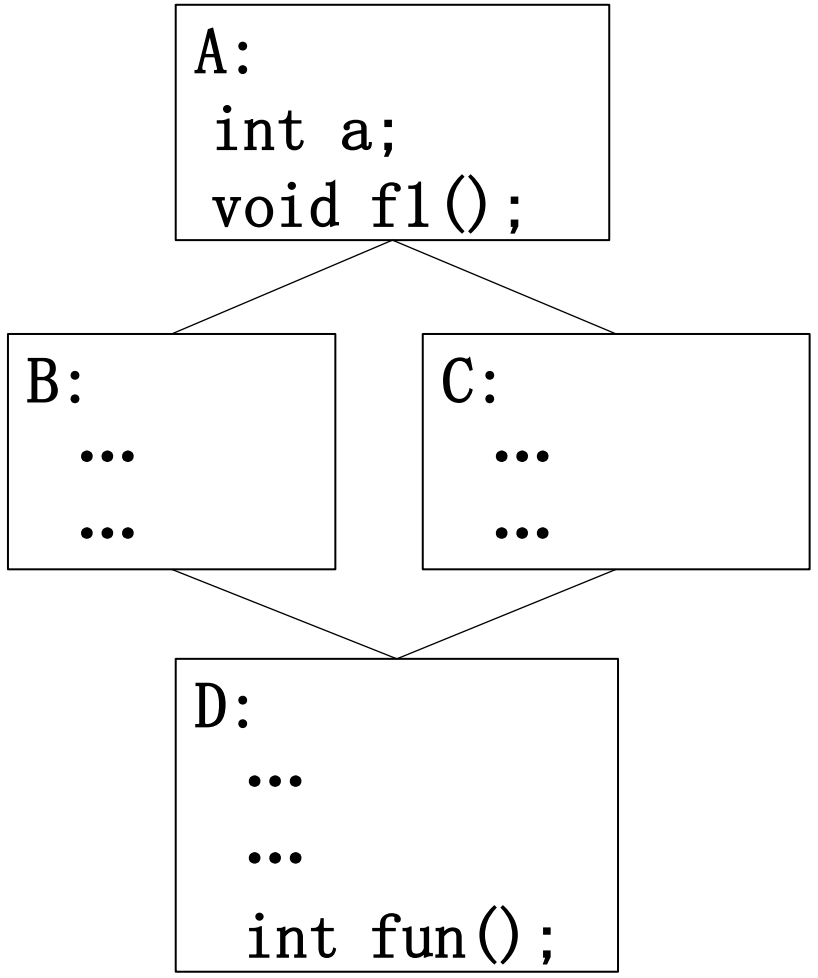
```
#include <iostream>
using namespace std;
static int x=0;
class A {
  public:
     int a;
     A() {
       a = ++x;
       cout << "A()" << endl; }
};
class B:public A {
  public:
     int b;
     B() {
       b = 20;
       cout << "B()" << endl; }
};
class C:public A {
  public:
     int c;
     C() {
       c = 30;
       cout << "C()" << endl; }
};
class D:public B, public C {
  public:
     int d;
     D(){
       d = 40;
       cout << "D()" <<endl; }
};
int main(){
  D d1;
  cout << sizeof(d1) << endl;
  int *p = (int *)&d1;
  cout << *p << endl;
  cout << *(p+1) << endl;
  cout << *(p+2) << endl;
  cout << *(p+3) << endl;
  cout << *(p+4) << endl;
  return 0;
}
```




```
A()
B()
A()
C()
D()
20
1
20
2
30
40
```

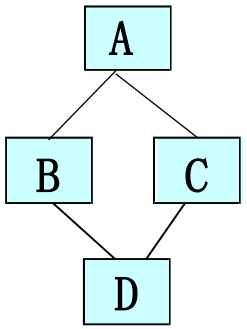
- 通过直接/间接方式继承同一个基类两个导致的同名：
  - 问题2：如何区分？

    通过可区分的类的作用域符来进行区分

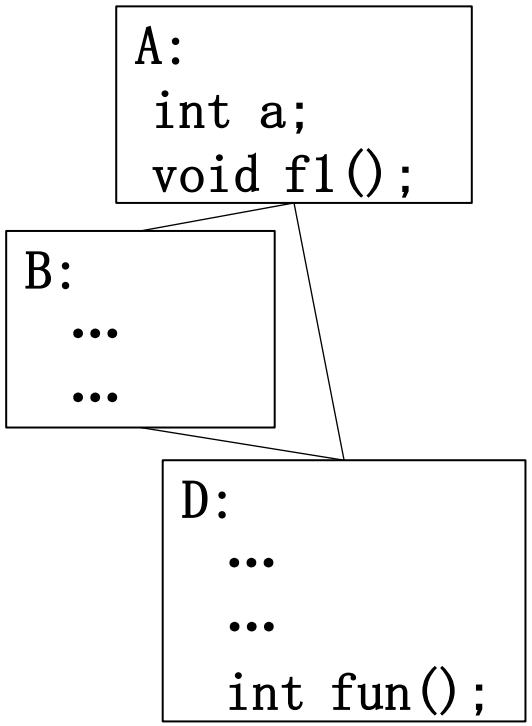



```
int D::fun()
{
    A::a = 5; //建议不用A::
    A::f1();  //建议不用A::
    B::a = 10;
    B::f1();
    C::a = 15;
    C::f1();
    return 0;
}
```

```
#include <iostream>
using namespace std;
static int x = 0;
class A {
public:
    int a;
    A() { a = ++x; }
    void f1() { cout << a << endl; }
};
class B :public A {
public:
    int b;
};
class C :public A {
public:
    int c;
};
class D :public B, public C {
public:
    int d;
    int fun();
};
int D::fun(){
    A::a = 5; //不建议
    A::f1();  //不建议
    B::a = 10;
    B::f1();
    C::a = 15;
    C::f1();
    return 0;
}
int main() {
    D d1;
    cout << sizeof(d1) << endl;
    d1.fun();
    d1.f1();   //报错：对f1的访问不明确
    d1.A::f1();//报错：对A的访问不明确
    d1.B::a=10;  但此句不同编译器会有区别
    d1.B::f1();
    d1.C::a=15;
    d1.C::f1();
    return 0;
}                    //删除报错语句后的运行结果：
```




```
20
5
10
15
10
15
```



D对象的内存映象(两份A)

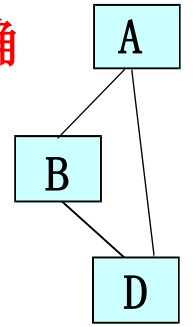
A的数据成员	B
B的数据成员	
A的数据成员	A
D的数据成员	D

```
#include <iostream>
using namespace std;
static int x=0;
class A {
  public:
    int a;
    A() {
      a = ++x;
      cout << "A()" << endl; }
};
class B :public A {
  public:
    int b;
    B() {
      b = 20;
      cout << "B()" << endl; }
};
```

```
class D :public B, public C {
  public:
    int d;
    D(){
      d = 40;
      cout << "D()" <<endl;}
};
int main(){
    D d1;
    cout << sizeof(d1) << endl;
    int *p = (int *)&d1;
    cout << *p << endl;
    cout << *(p+1) << endl;
    cout << *(p+2) << endl;
    cout << *(p+3) << endl;
    return 0;
}
```

```
A()
B()
A()
D()
16
1
20
2
40
```

```
#include <iostream>
using namespace std;
static int x = 0;
class A {
public:
    int a;
    A() { a = ++x; }
    void f1() { cout << a << endl; }
};
class B :public A {
public:
    int b;
};
class D :public B, public A {
public:
    int d;
    int fun();
};
int D::fun()
{
    A::a = 10; //不建议
    A::f1();   //不建议
    B::a = 15;
    B::f1();
    a = 15;    //报错：对a的访问不明确
    f1();      //报错：对f1的访问不明确
    return 0;
}
int main() {
    D d1;
    cout << sizeof(d1) << endl;
    d1.fun();
    d1.f1();   //报错：对f1的访问不明确
    d1.a = 15;  //报错：对a的访问不明确
    d1.A::a = 10; //不建议
    d1.A::f1();   //不建议
    d1.B::a = 15;
    d1.B::f1();
    return 0;
}
```



//删除报错语句后的运行结果：

16
10
15
10
15

目录

- 虚基类
  - 虚基类的形式
  - 虚基类的实例
  - 虚基类的构造函数

2.1 虚基类的形式

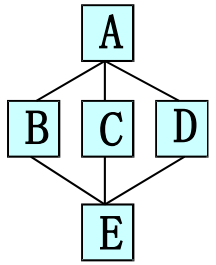
- 多重继承的问题：
  - 派生类中有多份相同的数据成员拷贝，占用较多的存储空间
  - 多个基类的相互交织可能会带来错综复杂的设计问题，命名冲突不可回避

➡引入虚基类，使相同基类只保留一份数据成员

- 虚基类声明的形式：

```
class 派生类名: virtual 存取限定符 基类名 {
    …
};
```

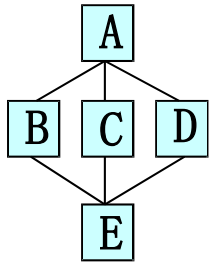
- 多重继承的构造函数调用




```
#include <iostream>
using namespace std;
class A {
  public:
    A() {cout << "A()" << endl;}
};
class B:public A {
  public:
    B() {cout << "B()" << endl;}
};
class C:public A {
  public:
    C() {cout << "C()" << endl;}
};
class D:public A {
  public:
    D() {cout << "D()" << endl;}
};
class E:public B,public C,public D{
  public:
    E() {cout << "E()" << endl;}
};
int main(){
    E e1;  //e1中有三份A的拷贝
}
```

```
A()
B()
A()
C()
A()
D()
E()
```

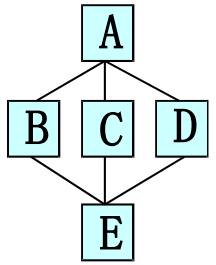
- 多重继承的构造函数调用：含虚基类




```
#include <iostream>
using namespace std;
class A {
  public:
    A() {cout << "A()" << endl;}
};
class B: virtual public A {
  public:
    B() {cout << "B()" << endl;}
};
class C: virtual public A {
  public:
    C() {cout << "C()" << endl;}
};
class D: virtual public A {
  public:
    D() {cout << "D()" << endl;}
};
class E:public B,public C,public D{
  public:
    E() {cout << "E()" << endl;}
};
int main(){
    E e1;  //e1中有一份A的拷贝
}
```

```
A()
B()
C()
D()
E()
```

- 多重继承的构造函数调用：含虚基类




```
#include <iostream>
using namespace std;
class A {
  public:
    A() {cout << "A()" << endl;}
};
class B: virtual public A {
  public:
    B() {cout << "B()" << endl;}
};
class C: virtual public A {
  public:
    C() {cout << "C()" << endl;}
};
```

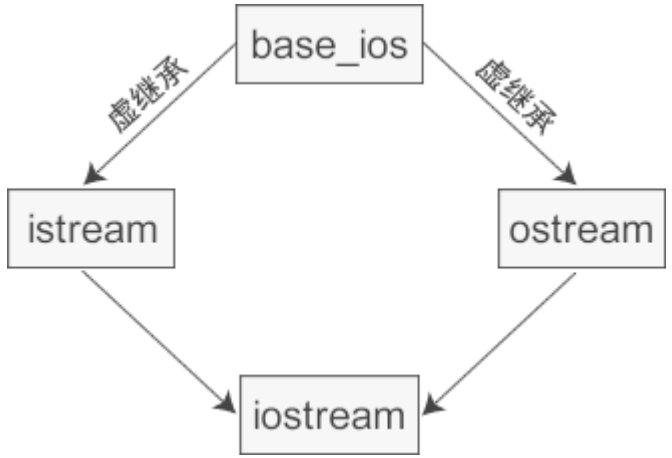
```
class D: public A {
  public:
    D() {cout << "D()" << endl;}
};
class E:public B,public C,public D{
  public:
    E() {cout << "E()" << endl;}
};
int main(){
    E e1;  //e1中有两份A的拷贝
}
```

所有继承该基类的直接派生类都应声明为虚基类才能保证只有一份数据拷贝

```
A()
B()
C()
A()
D()
E()
```

2.2 虚基类的实例

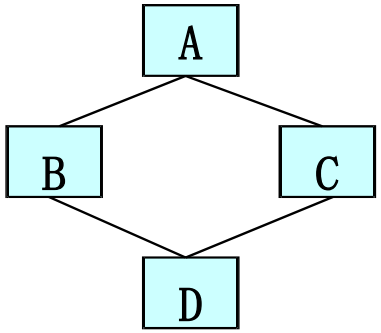
- C++标准库中的 iostream 类就是一个虚继承的实际应用案例



- iostream从istream和ostream直接继承而来，而istream和ostream又都继承自一个共同的名为base_ios的类，是典型的菱形继承
- istream和ostream必须采用虚继承，否则将导致iostream 类中保留两份 base_ios类的成员

2.3 虚基类的构造函数

- 构造函数的冲突问题：
  - 给基类自动传递信息时，存在多条不同的途径



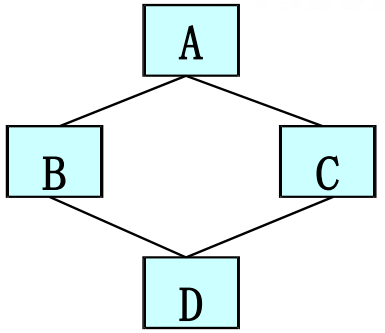
- 虚基类的构造函数：

  禁止信息通过中间类自动传递给基类

  - 使用虚基类时，必须在初始化列表最前方，显式的调用虚基类的构造函数；否则虚基类将调用默认构造函数初始化变量
  - 调用时，由派生类直接激活间接虚基类的构造函数，其直接基类不再自动激活虚基类的构造

- 禁止信息通过中间类自动传递给基类

```
class A {
   public:
      A() { cout << "A()" << endl; } // 无参构造
      A(int x) { cout << "A(" << x << ")" << endl; } //一参构造
};
class B:virtual public A {
   public:
     B(int y) { cout << "B()" << endl; }
};
class C:virtual public A {
   public:
     C(int z):A(z) { cout << "C()" << endl; }
};
class D:public B, public C {
   public:
      D(int x, int y, int z):A(x),B(y),C(z) {
               cout << "D()" << endl;}
};
```




```
int main(){
  D d1(1,2,3);
  cout << endl;
  C c1(4);
  cout << endl;
  B b1(5);
}
```

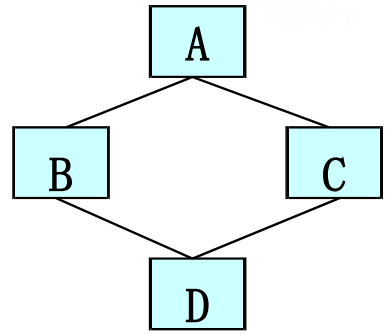
```
A(1)
B()
C()
D()

A(4)
C()

A()
B()
```

- 禁止信息通过中间类自动传递给基类

```
class A {
   public:
      A() { cout << "A()" << endl; } // 无参构造
      A(int x) { cout << "A(" << x << ")" << endl; } //一参构造
};
class B:virtual public A {
   public:
     B(int y) { cout << "B()" << endl; }
};
class C:virtual public A {
   public:
     C(int z):A(z) { cout << "C()" << endl; }
};
class D:public B, public C {
   public:
      D(int x, int y, int z): B(y),C(z) {
               cout << "D()" << endl;}
};
```




```
int main(){
  D d1(1,2,3);
  cout << endl;
  C c1(4);
  cout << endl;
  B b1(5);
}
```

```
A()
B()
C()
D()

A(4)
C()

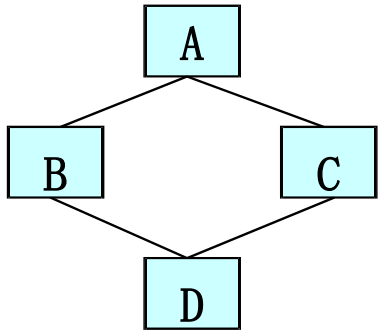
A()
B()
```

- 禁止信息通过中间类自动传递给基类

```
int main(){
  D d1(1,2,3);
  cout << endl;
  C c1(4);
  cout << endl;
  B b1(5);
}
```

➡ //d1对象生成时会自动激活A、B、C的构造函数，再调用D的构造函数。B/C的构造函数被调用时，不再自动激活A的构造函数

➡ //直接基类是虚基类时：与非虚基类一样，b1/c1生成时会自动激活A的无参/有参构造函数



目录

- 基本概念
- 派生类的声明方式及派生类对象的构成
- 派生类的成员访问属性
- 派生类的构造函数和析构函数
- 多重继承
- 虚基类
- 赋值兼容规则
- 继承和动态内存分配

目录

- 赋值兼容规则
  - 赋值兼容规则的含义
  - 赋值兼容规则的使用未知

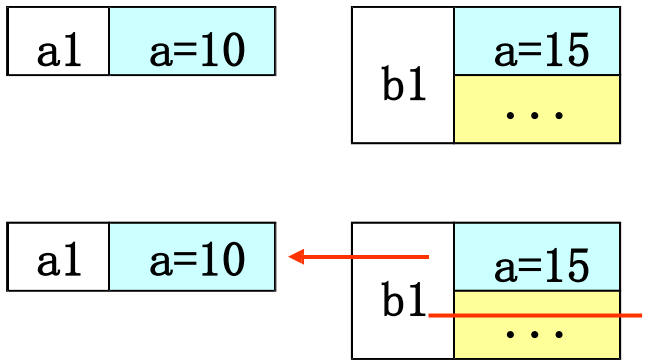
- 引例：基类与派生类的转换

```
#include <iostream>
using namespace std;
class A {
    public:
        int a;
};
class B:public A {
    public:
        int b;
};
```

```
int main()
{   A a1;
    B b1;
    a1.a = 10;
    b1.a = 15;
    cout << a1.a << endl;
    b1 = a1;  //编译错!!!
    cout << a1.a << endl;
}
```

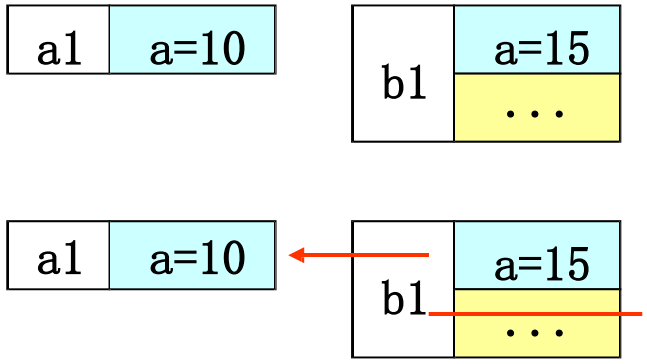
```
int main()
{   A a1;
    B b1;
    a1.a = 10;
    b1.a = 15;
    cout << a1.a << endl; 10
    a1 = b1;
    cout << a1.a << endl; 15
}
```

赋值兼容规则:



3.1 赋值兼容规则的含义

- 赋值兼容规则：在需要基类对象的任何位置，均可以使用公有继承的派生类对象
  - ➢ 只有公有继承适用，私有及保护继承会导致基类的公有部分对外不可访问
  - ➢ 将派生类中的基类无名实例对象对应空间拷贝给基类，其余部分丢弃
    - ✓ 如果不希望直接对应拷贝，则可根据需要自行定义=重载或复制构造函数
    - ✓ 当基类中包含动态申请内存时，赋值兼容规则可能出错（参考=重载及复制构造函数）



- 派生类->基类：直接对应拷贝

```
class A {
    public:
        int a;
};
class B:public A {
    public:
      int b;
};
```

```
int main()
{
    B b1;
    b1.a = 15;
    A a1(b1); //复制构造
    A a2;
    a2 = b1;
    cout << a1.a << endl; //15
    cout << a2.a << endl; //15
}
```

- 派生类->基类：=重载

```
class B; //提前声明
class A {
  public:
      int a;
      A() {} //无参空体
      A(B &b);//不能体内实现
      A& operator=(B &b);//不能体内实现
};
class B:public A {
   public:
    int b;
};
A::A(B &b) //一参构造
{  a = b.a*2; }
```

```
A& A::operator=(B &b) //=重载
{   a = b.a*2;
    return (*this);
}
int main()
{
    B b1;
    b1.a = 15;
    A a1(b1), a2;
    //一参，无参构造
    a2 = b1; //= 重载
    cout << a1.a << endl; //30
    cout << a2.a << endl; //30
}
```

- 派生类->基类：动态内存申请 运行出错

```
class A {
   private:
     char *s;
   public:
     int a;
     A()
     {
        s = new char[20];
     }
     ~A()
     {
        cout << "A析构" << endl;
        delete s;
     }
};
```

```
class B:public A {
   public:
     int b;
     ~B() { cout << "B析构" << endl;}
};
int main()
{
   B b1;
   b1.a = 15;
   A a1(b1); //复制构造
}
```

3.2 赋值兼容规则的使用位置

1. 派生类对象可初始化基类的对象或引用

```
//示例：
class A {…};
class B:public A {…};

B b1;
A a1(b1);    //需要A对象，用B对象替代
A &a2 = b1; //需要A对象，用B对象替代
```

3.2 赋值兼容规则的使用位置

2. 派生类对象可出现在函数参数/返回值为基类的地方

```
//示例：
void f1(A  a1) { ... }  //形参为A对象
void f2(A &a1) { ... }  //形参为A对象的引用
void f3(A *pa) { ... }  //形参为A对象的指针
A f4(){ static B b1; return b1; //返回值为A对象 }
int main()
{    B b1;
     f1(b1);  //B对象出现在要求A对象的位置
     f2(b1);  //B对象出现在要求A对象引用的位置
     f3(&b1); //B对象出现在要求A对象指针的位置
}
```

3.2 赋值兼容规则的使用位置

3. 派生类对象可赋值给基类

```
//示例：
A a1;
B b1;
a1 = b1;  //需要A对象，用B对象替代
```

4. 派生类对象的指针可出现在基类指针出现的位置

```
//示例：
A *pa;
B b1, *pb = &b1;
pa = pb;  //需要A对象的地址，用B对象的地址替代
pa = &b1;  //需要A对象的地址，用B对象的地址替代
```

3.2 赋值兼容规则的使用位置

- 赋值兼容规则：
  - 赋值兼容是实现多态的一个前提（模块09）
  - 赋值兼容规则+指针或引用+虚函数==多态（模块09）

目录

- 继承和动态内存分配

4.1 继承和动态内存分配

- 引例

```
class baseDMA{    //基类使用了动态内存
private:
    char *label;
    int rating;
public:
    baseDMA(const char *lb = "null", int r = 0);
    baseDMA(const baseDMA &rs);                    //复制构造函数
    virtual ~baseDMA();                            //析构函数
    baseDMA & operator=(const baseDMA &rs);        //重载赋值运算符
…
};
```

4.1 继承和动态内存分配

- 情况1：派生类不使用new

```
class lacksDMA:public baseDMA{    //派生类
private:
    char color[40];
public:
…
};
```

不需要为lacksDMA类定义显示析构函数、复制构造函数和赋值运算符：

- lacksDMA成员不需执行任何特殊操作，所以默认析构函数是合适的
- lacksDMA类的默认复制构造函数使用显示baseDMA复制构造函数来复制lacksDMA对象的baseDMA部分，因此，默认复制构造函数也是合适的
- 默认的赋值构造函数将自动使用基类的赋值构造函数完成基类组件的赋值

4.1 继承和动态内存分配

- 情况2：派生类使用new

```
class hasDMA:public baseDMA{    //派生类
private:
    char *style;  //use new in constructors
public:
    ...
};
```

必须为hasDMA类定义显示析构函数、复制构造函数和赋值运算符！

4.1 继承和动态内存分配

- 情况2：派生类使用new
  - 显示析构函数

```
baseDMA::~baseDMA() //依赖于基类的析构函数来释放指针label管理的内存
{
   delete [] label;
}
hasDMA::~hasDMA()
{
   delete [] style;  //释放指针style管理的内存
}
```

4.1 继承和动态内存分配

- 情况2：派生类使用new
  - 复制构造函数

```
baseDMA::baseDMA(const baseDMA &rs){
    label = new char[strlen(rs.label) + 1];
    strcpy(label,rs.label);
    rating = rs.rating;
}
hasDMA::hasDMA(const hasDMA &hs):baseDMA(hs){
    style = new char[strlen(hs.style) + 1];
    strcpy(style,hs.style);
}
```

基类的指针或引用可以指向派生类的指针或引用

hasDMA复制构造函数只能访问hasDMA的数据，因此它必须调用baseDMA复制构造函数来处理baseDMA共享的数据

4.1 继承和动态内存分配

- 情况2：派生类使用new
  - 赋值运算符

```
baseDMA & baseDMA::operator=(const baseDMA &rs)  //基类
{
    if(this == &rs)
        return *this;
    delete [] label;
    label = new char[strlen(rs.label)+1];
    strcpy(label,rs.label);
    rating = rs.rating;
    return *this;
}
```

4.1 继承和动态内存分配

- 情况2：派生类使用new
  - 赋值运算符

```
hasDMA & hasDMA::operator=(const hasDMA &hs)    //派生类
{
        if(this == &hs)
            return *this;
        baseDMA::operator=(hs);  //copy base portion 函数表示法
        delete [] style;   //prepare for new style
        style = new char[strlen(hs.style)+1];
        strcpy(style,hs.style);
        return *this;
}
```

//含义为*this = hs; 运算符表示法
如按此写法，编译器将使用
hasDMA::operator=()，从而形成递归调用

4.1 继承和动态内存分配

- 情况2：派生类使用new（总结）
  - 显示析构函数：自动完成
  - 复制构造函数：在初始化成员列表中调用基类的复制构造函数完成；如果不这样做，将自动调用基类的默认构造函数
  - 赋值运算符：使用作用域解析运算符显示的调用基类的赋值运算符来完成
- 使用动态内存分配和友元的继承示例
  - primer书：集成baseDMA、lacksDMA、hasDMA类

    //dma.h

    //dma.cpp

    //usedma.cpp

总结

- 多重继承（熟悉）
  - 多重继承的声明及使用
  - 多重继承的派生类
  - 多重继承引起的二义性问题
- 虚基类（熟悉）
  - 虚基类的形式
  - 虚基类的实例
  - 虚基类的构造函数
- 赋值兼容规则（熟悉）
  - 赋值兼容规则的含义
  - 赋值兼容规则的使用未知
- 继承和动态内存分配（了解）

第八章 多态性与虚函数

主讲教师：同济大学计算机科学与技术学院 陈宇飞
同济大学计算机科学与技术学院 龚晓亮

目录

- 多态的基本概念
- 虚函数的定义和使用
- 虚析构函数
- 纯虚函数与抽象类

目录

- 多态的基本概念
  - 多态的定义
  - 静态联编和动态联编
  - 多态的应用

1.1 多态的基本概念

赋值兼容规则+指针或引用+虚函数==多态

- C++程序设计中多态的含义：
  - 同一作用域内有多个不同功能的函数可以具有相同的函数名
- 多态公有继承：
  - 同一个方法在派生类和基类中的行为不同，具体取决于调用该方法的对象

1.2 静态联编和动态联编

- 静态联编和动态联编
  - 将源代码中的函数调用解释为执行特定函数代码块被称为函数名联编
  - 编译器在编译过程中进行的联编称为静态联编
  - 编译器在程序运行时选择正确的虚方法代码，称为动态联编
- 多态的分类：
  - 静态多态：在程序编译时已确定调用函数(函数重载)
  - 动态多态：在程序运行时才确定操作的对象(虚函数)

1.3 多态的应用

- 引例：

由于之前在俱乐部的工作经历，您成为了某银行的首席程序员。银行要求您完成的第一项工作是开发两个类：

① Brass类：基本账户

② BrassPlus类：添加透支保护特性的账户：如果用户签出超出其存款余额的支票且超出额不大，银行将支付支票且对超出部分收取额外费用并追加罚款

➢ 问题1：应从Brass公有派生出BrassPlus吗？

✓ 正确：BrassPlus类满足is-a条件。都将保存客户姓名、账号以及结余，都可以存款、取款和显示信息

1.3 多态的应用

- 引例：

由于之前在俱乐部的工作经历，您成为了某银行的首席程序员。银行要求您完成的第一项工作是开发两个类：

① Brass类：基本账户

② BrassPlus类：添加透支保护特性的账户：如果用户签出超出其存款余额的支票且超出额不大，银行将支付支票且对超出部分收取额外费用并追加罚款

➢ 问题2：BrassPlus是否需要添加新数据成员？

✓ 正确：BrassPlus类包含Brass所有信息及如下信息：透支上限、透支贷款利率，当前的透支总额

1.3 多态的应用

- 引例：

由于之前在俱乐部的工作经历，您成为了某银行的首席程序员。银行要求您完成的第一项工作是开发两个类：

① Brass类：基本账户

② BrassPlus类：添加透支保护特性的账户：如果用户签出超出其存款余额的支票且超出额不大，银行将支付支票且对超出部分收取额外费用并追加罚款

➢ 问题3：BrassPlus是否需要新增操作？

✓ 不需要，但有两种操作的实现不同！！！ ➡ 虚函数

a）对于取款操作，必须考虑透支保护

b）显示操作必须显示BrassPlus账户的其他信息

```
// brass.h -- bank account classes
class Brass{
private: …
public: …
    virtual void Withdraw(double amt);
    virtual void ViewAcct() const;
    virtual ~Brass() {}   //虚析构函数（模块9.3）
};
class BrassPlus : public Brass{
private:  //新增数据成员
    double maxloan;
    double rate;
    double owesBank;
public: …
    virtual void ViewAcct() const;
    virtual void Withdraw(double amt);
};
```

头文件
类的声明

完整程序见primer书

```
//brass.cpp -- bank account methods
#include"brass.h"
#include <iostream>
using namespace std;
…
void Brass::Withdraw(double amt){…}
void Brass::ViewAcct()const{…}

//redefine how Withdraw() and ViewAcct() work
void BrassPlus::Withdraw(double amt){
    …//取款操作，考虑透支保护
}
void BrassPlus::ViewAcct()const{
    …//显示操作，显示BrassPlus账户的其他信息
}
```

源程序文件
函数的实现

完整程序见primer书

```
//usebrass1.cpp -- testing bank account classes
#include <iostream>
#include "brass.h"
using namespace std;
int main()
{
    Brass Piggy("Porcelot Pigg", 381299, 4000.00);
    BrassPlus Hoggy("Horatio Hogg", 382288, 3000.00);

    Piggy.ViewAcct();
    Hoggy.ViewAcct();
    …
    Piggy.Withdraw(4200.00);
    Hoggy.Withdraw(4200.00);
    Hoggy.ViewAcct();
    …
    return 0;
}
```

源程序文件
调用函数

//方法是通过对象调用的，
并没有使用虚方法特性！

完整程序见primer书

```
//usebrass2.cpp -- polymorphic example
#include <iostream>
#include "brass.h"
const int CLIENT = 4;
using namespace std;
int main(){
    Brass * p_client[CLIENT]; //定义指向基类Brass的指针数组
    …
    for (i = 0; i < CLIENTS; i++){
        …
        if (kind == '1')
            p_clients[i] = new Brass(temp, tempnum, tempbal);
        else{  …
            p_clients[i] = new BrassPlus(temp, tempnum, tempbal, tmax, trate);
        }
    }
    for (i = 0; i < CLIENTS; i++){
        p_clients[i]->ViewAcct();  //多态
        cout << endl;
    }
    …
}
```

源程序文件
调用函数

完整程序见primer书

//usebrass2.cpp　程序分析

回顾：
赋值兼容规则+指针或引用+虚函数==多态

- Brass * p_client[CLIENT];

同时管理Brass和BrassPlus账户：定义指向基类Brass的指针数组，Brass指针既可以指向Brass对象，也可以指向BrassPlus对象。因此，可以使用一个数组来表示多种类型的对象（多态性）

- p_clients[i]->ViewAcct();

如果数组成员指向的是Brass对象，则调用Brass::ViewAcct();如果指向的是BrassPlus对象，则调用BrassPlus::ViewAcct()

如果Brass::ViewAcct()未被声明为虚的，
则在任何情况下都将调用Brass::ViewAcct()

目录

- 虚函数的定义和使用
  - 虚函数的定义
  - 虚函数的使用
  - 支配规则、赋值兼容规则、虚函数

2.1 虚函数的定义

- 在基类的函数定义前加virtual声明
  - 未加virtual前，基类指针变量 = &派生类对象形式。适用赋值兼容规则，访问的是派生类中的基类部分
  - 加virtual后，突破此限制，访问派生类的同名函数

```
#include <iostream>
using namespace std;
class A {
   public:
     virtual void display() {
       cout << "A::display()" << endl;
     }
};
class B:public A {
   public:
     void display() {
       cout << "B::display()" << endl;
     }
};
class C:public B {
   public:
     void display() {
       cout << "C::display()" << endl;
     }
};
```

```
//A类不加 virtual 的执行结果
int main(){
    A a, &ra=a, *pa=&a;
    B b, &rb=b, *pb=&b;
    C c, &rc=c, *pc=&c;
    A *p;
    a.display();   A::
    b.display();   B::
    c.display();   C::
    ra.display();  A::
    rb.display();  B::
    rc.display();  C::
    pa->display(); A::
    pb->display(); B::
    pc->display(); C::
    p=&a;
    p->display();  A::
    p=&b;
    p->display();  A::
    p=&c;
    p->display();  A::
}
```

```
//A类加 virtual 的执行结果
int main(){
    A a, &ra=a, *pa=&a;
    B b, &rb=b, *pb=&b;
    C c, &rc=c, *pc=&c;
    A *p;
    a.display();   A::
    b.display();   B::
    c.display();   C::
    ra.display();  A::
    rb.display();  B::
    rc.display();  C::
    pa->display(); A::
    pb->display(); B::
    pc->display(); C::
    p=&a;
    p->display();  A::
    p=&b;
    p->display();  B::
    p=&c;
    p->display();  C::
}
```

2.2 虚函数的使用

非类的成员函数不能声明为多态

- 类的静态成员函数不能声明为多态
- virtual在类定义时出现，函数体外实现部分不能加

```
class A {
public:
   virtual void display();
};
void A::display()
{
   cout << "A::display()" << endl;
}
```

2.2 虚函数的使用

- 只需要在最开始的基类中加virtual声明，后续派生类可以不加

```
class A {
public:
    virtual void display(); //必须加
};
class B:public A {
public:
    virtual void display(); //建议加，但可以不加
}
```

2.2 虚函数的使用

- 类的继承序列中该同名函数的参数个数、参数类型必须完全相同。若同名虚函数仅返回类型不同，则编译报错

```
class A {     //编译报错
public:
     virtual void display();
};
class B:public A {
public:
     virtual int display();
};
```

- 若派生类中无同名函数，则自动继承基类

```
class A {
public:
  virtual void display();
};
class B:public A {
  //无display函数
};
class C:public B {
public:
  virtual void display();
};
```

```
int main()
{
   A a;
   B b;
   C c;
   A *p;
   p=&a;
   p->display(); A::
   p=&b;
   p->display(); A::
   p=&c;
   p->display(); C::
}
```

- 若派生类中有同名函数，其参数个数、参数类型与基类的虚函数不同，则失去多态性，按支配规则及赋值兼容规则处理

```
class A {
public:
   virtual void display() {
      cout << "A::" << endl;
   }
};
class B:public A {
public:
   void display(int x) {
      cout << "B::x" << endl;
   }
};
class C:public B {
public:
   void display() {
     cout << "C::" << endl;
   }
};
```

```
int main()
{   A a, *p;
    B b;
    C c;
    b.display();      错
    b.display(1);     B::x
    p=&a;
    p->display();     A::
    p=&b;
    p->display();     A::
    p->display(1);    错
    p=&c;
    p->display();     C::
    p->display(1);    错
}
```

- 对于派生类中的其它非virtual仍适用赋值兼容规则

```
#include<iostream>
using namespace std;
class A {
public:
    virtual void display();
    void show();
};
class B:public A{
    void display();
    void show();
};
class C:public B{
public:
    void display();
    void show();
};
```

```
int main(){
    A a, *p;
    B b;
    C c;
    p=&a;
    p->display();   A::
    p->show();      A::
    p=&b;
    p->display();   B::
    p->show();      A::
    p=&c;
    p->display();   C::
    p->show();      A::
}
```

- 只有通过基类指针/引用方式访问时才适用虚函数规则,其它形式(对象/自身指针/引用)仍用原来的规则

```
#include <iostream>
using namespace std;
class A {
public:
   virtual void display() {
      cout << "A::display()" << endl;
   }
};
class B:public A {
public:
   void display() {
      cout << "B::display()" << endl;
   }
};
class C:public B {
public:
   void display() {
      cout << "C::display()" << endl;
   }
};
```

```
int main(){
    A a, &ra=a, *pa=&a;
    B b, &rb=b, *pb=&b;
    C c, &rc=c, *pc=&c;
    A *p;
    a.display();       A::display()
    b.display();       B::display()
    c.display();       C::display()
    ra.display();      A::display()
    rb.display();      B::display()
    rc.display();      C::display()
    pa->display();     A::display()
    pb->display();     B::display()
    pc->display();     C::display()
    p=&a;
    p->display();      A::display()
    p=&b;
    p->display();      B::display()
    p=&c;
    p->display();      C::display()
}
```

2.3 支配规则、赋值兼容规则、虚函数

区别：

- 支配规则：通过自身对象、指针、引用访问(自身的)虚函数、普通函数
- 赋值兼容规则：通过基类指针、对象、引用访问(派生类中基类部分的)普通函数
- 虚函数：通过基类指针、引用访问(基类和派生类的同名)虚函数

```
class A {
public:
  virtual f1(int x) {...}
  f2(int x) {...}
};
class B:public A {
public:
  virtual f1(int x) {...}
  f2(int x) {...}
};
```

```
void fun1(A *pa){
    pa->f1(10); //虚函数
    pa->f2(15); //赋值兼容
}
void fun2(A &ra){
    ra.f1(10); //虚函数
    ra.f2(15); //赋值兼容
}
int main(){
    A a;  B b;
    fun1(&a); fun1(&b);
    fun2(a); fun2(b);
    a.f1(10); //支配
    a.f1(10); //支配
    b.f1(10); //支配
    b.f2(15); //支配
}
```

```
class A {
public:
  virtual f1(int x) {...}
  f2(int x) {...}
};
class B:public A {
public:
  virtual f1(int x) {...}
  f2(int x) {...}
};
```

```
int main(){
    A a, *pa;
    B b;
    pa = &a;
    pa->f1(10); //支配
    pa->f2(15); //支配
    pa = &b;
    pa->f1(10); //虚函数
    pa->f2(10); //赋值兼容
    a.f1(10); //支配
    a.f2(15); //支配
    b.f1(10); //支配
    b.f2(15); //支配
}
```

```
class A {
public:
  virtual f1(int x) {...}
  f2(int x) {...}
};
class B:public A {
public:
  virtual f1(int x) {...}
  f2(int x) {...}
};
```

```
int main(){
    A a, *pa = &a;
    B b, *pb = &b;
    pa->f1(10); //支配
    pa->f2(15); //支配
    pb->f1(10); //支配
    pb->f2(10); //支配
    a.f1(10); //支配
    a.f2(15); //支配
    b.f1(10); //支配
    b.f2(15); //支配
}
```

```
class A {
public:
  virtual f1(int x) {...}
  f2(int x) {...}
};
class B:public A {
public:
  virtual f1(int x) {...}
  f2(int x) {...}
};
```

```
int main(){
    B  b;
    A &ra = b;
    ra.f1(10);  //虚函数
    ra.f2(10);  //赋值兼容
    b.f1(10);  //支配
    b.f2(15);  //支配
}
```

```
class A {
public:
  virtual f1(int x) {...}
  f2(int x) {...}
};
class B:public A {
public:
  virtual f1(int x) {...}
  f2(int x) {...}
};
```

```
int main(){
    A a, &ra = a;
    B b, &rb = b;
    ra.f1(10); //支配
    ra.f2(15); //支配
    rb.f1(10); //支配
    rb.f2(10); //支配
    a.f1(10); //支配
    a.f2(15); //支配
    b.f1(10); //支配
    b.f2(15); //支配
}
```

目录

- 虚析构函数

虚析构函数引例：

```
// brass.h -- bank account classes
class Brass{
    …
    virtual ~Brass() {}    //虚析构函数
};
class BrassPlus : public Brass{
    …
};
```

- 如果析构函数不是虚的，则将只调用对应于指针类型的析构函数。即便指针指向BrassPlus对象，也只有Brass的析构函数被调用
- 如果析构函数是虚的，将调用相应对象类型的析构函数。若指针指向BrassPlus对象，将调用BrassPlus的析构函数（上例系统默认），再自动调用基类Brass的析构函数

3.1 虚析构函数

- 虚析构函数的作用：
  - 确保正确的析构函数调用顺序。解决在通过基类指针动态申请派生对象时，会出现对象撤销时无法调用派生类析构函数的问题
- 虚析构函数的使用：
  - 适用于派生类中有动态申请空间的情况，虽然类的继承序列中析构函数名不同，系统会自动当作虚函数处理
  - 析构函数声明为虚函数后，通过基类、派生类自己生成的对象在释放时也不会出错，因此一般在类的继承序列中，无论是否包含动态申请空间的操作，都建议将析构函数声明为虚析构函数
  - 虚函数只有和对象结合才能呈现多态，构造函数时对象正在生成，因此构造函数不能声明为虚函数

```
class Base {
public:
    Base(){}
    ~Base(){cout << "Base destructor is called!" << endl;} //普通析构函数
};
class Subclass:public Base{
private: int *ptr;
public:
    Subclass(){ptr = new int;}
    ~Subclass(){
        cout << "Subclass destructor is called!" <<endl; delete ptr;
    }
};
int main(){
    Base * a = new Subclass;  //基类指针方式
    delete a;          //Base析构
    return 0;          //ptr分配的内存没有得到释放
}
```

```
Base destructor is called!
```

```
class Base {
public:
    Base(){}
    ~Base(){cout << "Base destructor is called!" << endl;} //普通析构函数
};
class Subclass:public Base{
private: int *ptr;
public:
    Subclass(){ptr = new int;}
    ~Subclass(){
        cout << "Subclass destructor is called!" <<endl; delete ptr;
    }
};
int main(){
    Subclass * a = new Subclass;  //派生类对象/指针方式
    delete a;       //Subclass析构
    return 0;       //Base析构
}
```

```
Subclass destructor is called!
Base destructor is called!
```

```
class Base {
public:
    Base(){}
    virtual ~Base(){cout << "Base destructor is called!" << endl;} //虚析
};                                                                  构函数
class Subclass:public Base{
private: int *ptr;
public:
    Subclass(){ptr = new int;}
    ~Subclass(){
        cout << "Subclass destructor is called!" <<endl; delete ptr;
    }
};
int main(){
    Base * a = new Subclass;  //基类指针方式
    delete a;          //Subclass析构
    return 0;          //Base析构
}
```

```
Subclass destructor is called!
Base destructor is called!
```

```
class Base {
public:
    Base(){}
    virtual ~Base(){cout << "Base destructor is called!" << endl;} //虚析
};                                                                  构函数
class Subclass:public Base{
private: int *ptr;
public:
    Subclass(){ptr = new int;}
    ~Subclass(){
        cout << "Subclass destructor is called!" <<endl; delete ptr;
    }
};
int main(){
    Subclass * a = new Subclass;  //派生类对象/指针方式
    delete a;          //Subclass析构
    return 0;          //Base析构
}
```

```
Subclass destructor is called!
Base destructor is called!
```

目录

- 纯虚函数与抽象类
  - 基本概念
  - 抽象类的定义
  - 抽象类的使用
  - 空虚函数
  - 应用实例

引例思考：

假设您正在开发一个图形程序，该程序会显示圆和椭圆等

问题：应从Ellipse派生出Circle吗？

✓满足is-a条件：圆是椭圆的特殊情况

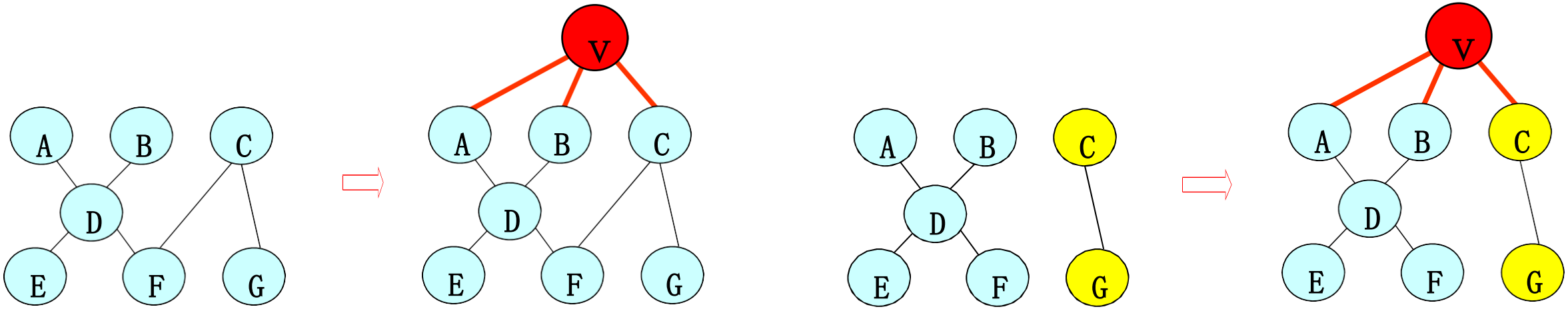
✓派生是笨拙的：圆只需要一个半径就可以描述大小和形状，不需要长轴和短轴，因此有信息冗余

✓更好的解决办法：

- 从Ellipse和Circle类抽象出共性，放到抽象类（abstract base class, ABC）中，然后从ABC派生出Ellipse和Circle类
- 方便使用基类指针同时管理Ellipse和Circle对象

4.1 基本概念

- 在类的多继承层次中，可能会出现多个基类并存的现象，若各基类有同名函数并希望使用虚函数机制，则需要引入一个更高层次的类，该类无实际意义，不进行具体操作，称为抽象类

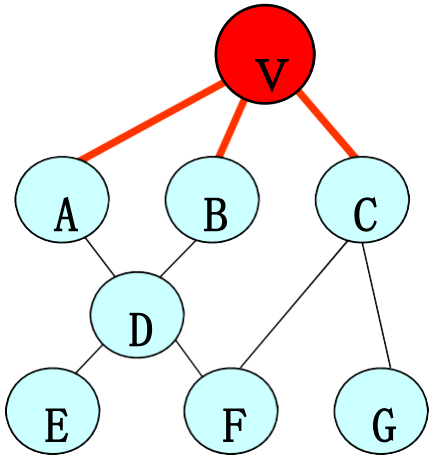


4.1 基本概念

假设A-G每个类都有display

```
class V {
  public:
    不需要其它成员及函数
    virtual void display()
    { }
};
class A:public V { //继承V
   其它成员及函数
  void display(){ 具体实现 }
};
class E:public D {
  其它成员及函数
  void display(){ 具体实现 }
};
```

```
A a1;
G g1;
V *p;
p = &a1;
p -> display();
p = &g1;
p -> display();
```



- 思考：

1）V是否还能定义对象？

可以： V v1;

2）是否有意义？

V的对象实际无意义

3）如何解决？

将V定义为抽象类

4.2 抽象类的定义

- 抽象类的定义：
  - C++中无明确的关键字定义，只要声明某一成员函数为纯虚函数即可，表示该函数没有实际意义，也不被调用
- 纯虚函数的声明：
  - virtual 返回类型 函数名(参数表) = 0;

```
class V {
 public:
   不需要其它成员及函数
   virtual void display(){ }
 };
```



```
class V {
 public:
   virtual void display()=0;
 };
```

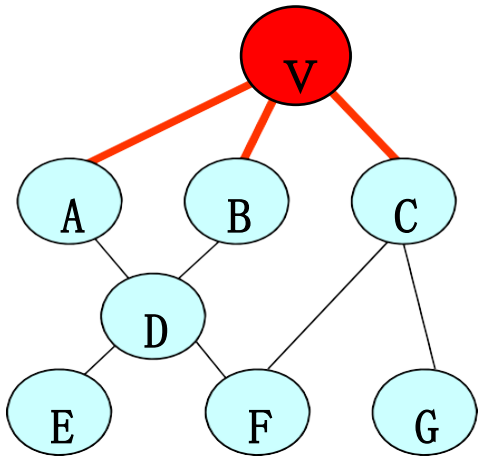
4.3 抽象类的使用

➢抽象类不能有实例对象，但可用于声明指针或引用

```
class V {
  public:
    virtual void display()=0;
  };
V v1; //错误
V *p; //正确
V &p = 派生类对象; //正确(一般用于函数形参)
```

4.3 抽象类的使用

➢抽象类中定义数据成员及有实际意义的成员函数都是无意义的，但为了简化继承序列，可以进行定义，供派生类使用(会导致理解混乱，不推荐)

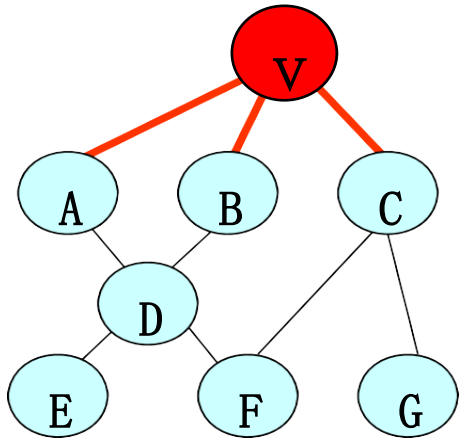



```
//假设A,B,C中都有int a,b成员
class A {
  protected: int a,b;
};
class B {
  protected: int a,b;
};
class C {
  protected: int a,b;
};
```

```
class V {
  protected: int a, b;
};
class A:public V {
  protected: …
};
class B:public V {
  protected: …
};
class C:public V {
  protected:…
};
```

4.3 抽象类的使用

➢抽象类的直接派生类的同名虚函数必须定义，否则继承抽象类的纯虚函数，也成为抽象类(若不需要，可定义为空虚函数)



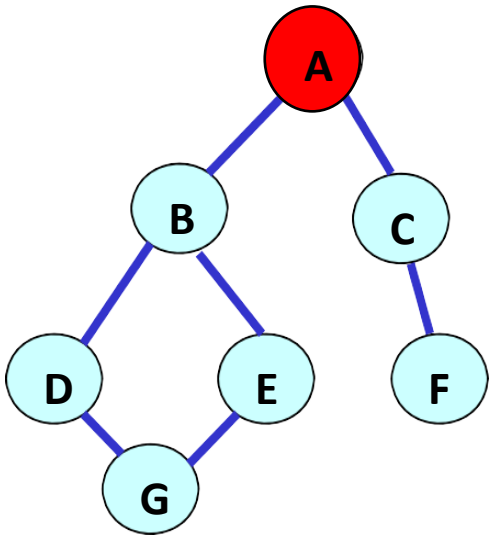
若：
```
class V {
public:
    virtual void display()=0;
};
```
则：ABC中的display()必须定义，即使B中不需要display
```
class B:public V {
public:
    void display() { }
};
```

4.4 空虚函数

- 空虚函数的概念：
  - 在类的继承层次中，派生类都有同名函数，而基类没有，为使用虚函数机制，需要建立一条从基类到派生类的虚函数路径，因此在基类中定义一个同名空虚函数
- 空虚函数的形式：
  - 函数名、参数个数、参数类型、返回值与派生类相同，函数体为空

4.4 空虚函数

- 示例：




```
class A {
 有实际意义的数据成员及函数
 virtual void display() { }
 virtual int show(){ return 0; }
};
class B:public A {
 有实际意义的数据成员及函数
 void display(){ 具体实现 }
 int show() { 具体实现 }
};
//CDEFG等相同
1.有各自有意义的数据成员及函数
2.各自独立的display及show函数,参
 数个数、参数类型、返回值同但实
 现过程各不相同
```

A中定义display及show后，可用统一方法调用，例：

```
F f;
B b;
A *pa = &f;
pa->display();
pa->show();
pa = &b;
pa->display();
pa->show();
```

4.4 空虚函数

➢基类的该函数虽然无意义，但基类的其它部分仍有意义，可定义对象、引用、指针等并进行正常操作

```
class A {
 有实际意义的数据成员及函数
 virtual void display() { }
 virtual int show(){ return 0; }
};
//BCDEFG的定义
```

```
void main(){
  C c1;
  A a1, *pa=&c1;
  a1.****; //正常操作
  pa->show(); //虚函数形式
}
```

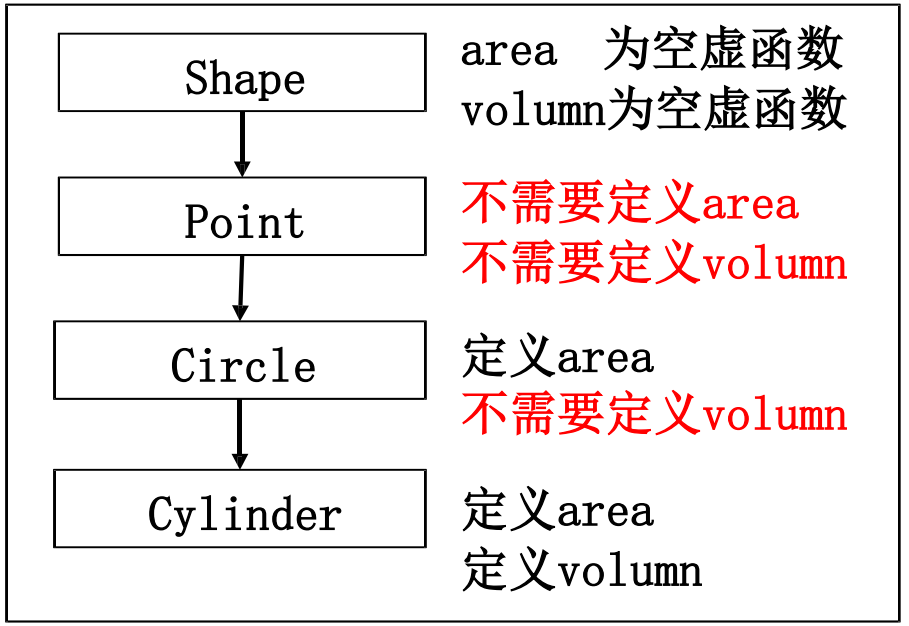
4.4 空虚函数

- 空虚函数与纯虚函数的区别：
  - 空虚函数：类的其它成员有实际含义，可生成对象
  - 纯虚函数：无实例对象，无实际含义，仅为了在更高的层次上统一类

4.5 应用实例

```
class Shape {
   public:
          virtual float area() const { return 0.0; }
          virtual float volumn() const { return 0.0; }
          virtual void shapeName() const = 0;
   };
```

- area()为空虚函数，在Point中可不再定义
- volumn()为空虚函数，在Point、Circle中可不再定义
- shapeName()为纯虚函数，为了声明抽象类，且shapeName()每个类中必须再次定义



总结

- 多态的基本概念（掌握）
  - 多态的定义
  - 静态联编和动态联编
  - 多态的应用
- 虚函数的定义和使用（熟悉）
  - 虚函数的定义
  - 虚函数的使用
  - 支配规则、赋值兼容规则、虚函数
- 虚析构函数（熟悉）
- 纯虚函数与抽象类（了解）
  - 基本概念
  - 抽象类的定义
  - 抽象类的使用
  - 空虚函数
  - 应用实例

第九章 运算符重载

模块9.1：基本概念

主讲教师：同济大学计算机科学与技术学院 陈宇飞
同济大学计算机科学与技术学院 龚晓亮

目录

- 运算符重载的方法
- 运算符重载的规则
- 成员函数和友元函数

1.1 运算符重载的方法

- 引例：今天早上在某账户上花费了2小时35分钟，下午又花费了2小时40分钟，则总共花了多少时间呢？

➢与加法概念吻合，但要相加的单位（小时与分钟的混合）与内置类型不匹配

解决方法0：在Time类中使用方法Sum()来处理加法

```
//mytime0.h -- Time class before
               operator overloading
#ifndef MYTIME0_H_
#define MYTIME0_H_
class Time {
private:
    int hours;
    int minutes;
public:
    Time();
    Time(int h, int m = 0);
    void AddMin(int m);
    void AddHr(int h);
    void Reset(int h = 0, int m = 0);
    Time Sum(const Time& t)const;
    void Show()const;
};
#endif
```

```
//mytime0.cpp -- implementing Time methods
#include<iostream>
#include"mytime0.h"
Time::Time() {…}
Time::Time(int h, int m) {…}
void Time::AddMin(int m) {…}
void Time::AddHr(int h) {…}
void Time::Reset(int h, int m) {…}
Time Time::Sum(const Time& t)const { //具体实现
   Time sum;
   sum.minutes = minutes + t.minutes;
   sum.hours = hours + t.hours + sum.minutes / 60;
   sum.minutes %= 60;
   return sum;
}
void Time::Show()const{…}
```

1.1 运算符重载的方法

- 解决方法0：在Time类中使用方法Sum()来处理加法

```
Time Time::Sum(const Time& t)const { //具体实现
  Time sum;
  sum.minutes = minutes + t.minutes;
  sum.hours = hours + t.hours + sum.minutes / 60;
  sum.minutes %= 60;
  return sum;
}
```

➢参数是引用：可以按值传递Time对象，但传递引用速度更快，占用内存更少

➢返回值不能是引用：返回对象sum将创建对象的副本，函数结束删除sum之前构造它的拷贝，供调用函数使用此拷贝。如果返回Time &，引用的是局部变量sum对象，函数结束时将被删除，引用将指向不存在的对象

```
//mytime0.h -- Time class before operator overloading
class Time {
    …
    Time Sum(const Time& t)const;
};
```

头文件
类的声明

```
//mytime0.cpp -- implementing Time methods
#include"mytime0.h"
Time Time::Sum(const Time& t)const {…}
…
```

源程序文件
函数的实现

```
//usetime0.cpp -- using the first draft of the Time class
//compile usetime0.cpp and mytime0.cpp together
#include"mytime0.h"
int main()
{
    Time coding(2, 40);
    Time fixing(5, 55);
    Time total;
    total = coding.Sum(fixing);  //想用更直观的+形式怎么办?
    ...
}
```

源程序文件
调用函数

1.1 运算符重载的方法

- 引例：今天早上在某账户上花费了2小时35分钟，下午又花费了2小时40分钟，则总共花了多少时间呢？

➢与加法概念吻合，但要相加的单位（小时与分钟的混合）与内置类型不匹配

解决方法0：在Time类中使用方法Sum()来处理加法

解决方法1：添加加法运算符来处理加法

```
//mytime1.h -- Time class before
               operator overloading
#ifndef MYTIME1_H_
#define MYTIME1_H_
class Time {
private:
    int hours;
    int minutes;
public:
    Time();
    Time(int h, int m = 0);
    void AddMin(int m);
    void AddHr(int h);
    void Reset(int h = 0, int m = 0);
    Time operator+(const Time& t)const;
    void Show()const;
};
#endif
```

```
//mytime1.cpp -- implementing Time methods
#include<iostream>
#include"mytime1.h"
Time::Time() {…}
Time::Time(int h, int m) {…}
void Time::AddMin(int m) {…}
void Time::AddHr(int h) {…}
void Time::Reset(int h, int m) {…}
Time Time::operator+ (const Time& t)const {
   Time sum;
   sum.minutes = minutes + t.minutes;
   sum.hours = hours + t.hours + sum.minutes / 60;
   sum.minutes %= 60;
   return sum;
}
void Time::Show()const{…}
```

1.1 运算符重载的方法

- 解决方法1：添加加法运算符来处理加法

```
Time Time::operator+ (const Time& t)const { //具体实现
  Time sum;
  sum.minutes = minutes + t.minutes;
  sum.hours = hours + t.hours + sum.minutes / 60;
  sum.minutes %= 60;
  return sum;
}
```

➢与sum()一样：参数是引用，返回值是对象

➢operator+()由Time对象调用，第二个Time对象作为参数，并返回一个Time对象

```
total = coding.operator+(fixing);  //function notation
total = coding + fixing;           //operator notation
```

```
//mytime1.h -- Time class before operator overloading
class Time {
    …
    Time operator+(const Time& t)const;
};
```

头文件
类的声明

```
//mytime1.cpp -- implementing Time methods
#include"mytime1.h"
Time Time::operator+ (const Time& t)const {…}
…
```

源程序文件
函数的实现

```
//usetime1.cpp -- using the second draft of the Time class
//compile usetime1.cpp and mytime1.cpp together
#include"mytime1.h"
int main()
{   Time coding(2, 40);
    Time fixing(5, 55);
    Time morefixing(3, 28);
    Time total;
    total = coding + fixing;  //operator notation
    total = morefixing.operator+(total);  //function notation
    ...
}
```

源程序文件
调用函数

1.1 运算符重载的方法

- 运算符重载的形式：

```
返回类型 operator 运算符(形参表)
{
    重载函数实现
}
```

- 用operator 运算符来表示对应运算符的函数

operator + ⇔ +

operator * ⇔ *

1.1 运算符重载的方法

- 对象 运算符 另一个值(可以不是对象、可以无)转换为函数调用：

  对象.operator运算符(另一个值)

```
Time t1, t2, t3, t4;
t4 = t1 + t2 + t3;
```

➡ `t4 = t1.operator+(t2 + t3);`

➡ `t4 = t1.operator+(t2.operator+(t3));`

返回Time对象，值t2+t3

返回Time对象，值t1+t2+t3

1.1 运算符重载的方法

- 运算符被重载后，原来用于其它数据类型上的功能仍然被保留(重载)，系统根据重载函数的规则匹配

```
int a, b, c;
Time A, B, C;
c = a + b;          //use int addition
C = A + B;          //use addition defined for Time objects
```

目录

- 运算符重载的方法
- 运算符重载的规则
- 成员函数和友元函数

2.1 运算符重载的规则

1. 重载运算符的两侧至少有一个是类对象（防止用户为标准类型重载运算符）
2. 不能违反运算符原来的句法规则
   - 不能改变操作对象的个数（例：不能将求模%重载成使用一个操作数）
   - 不能改变优先级
   - 不能改变结合性
3. 不能创建新运算符（例：不能定义operator **()函数来表示求幂）
4. 不能重载的运算符（primer书：sizeof　::　?:　.　.*　等）
5. 可重载的运算符（primer书 表11.1）
   - 只能通过成员函数进行重载：

     =：赋值运算符　　()：函数调用运算符

     []：下标运算符　　->：通过指针访问类成员的运算符

2.1 运算符重载的规则

6. 不允许带默认参数（操作数是不可缺少的）
7. 应当使重载运算符的功能与标准相同/相似(建议)
8. =和&系统缺省做了重载，=是对应内存拷贝，&取地址

例：关于=赋值（回顾）

```
test t1("hello"), t2;
t2=t1;    //整体内存拷贝
```

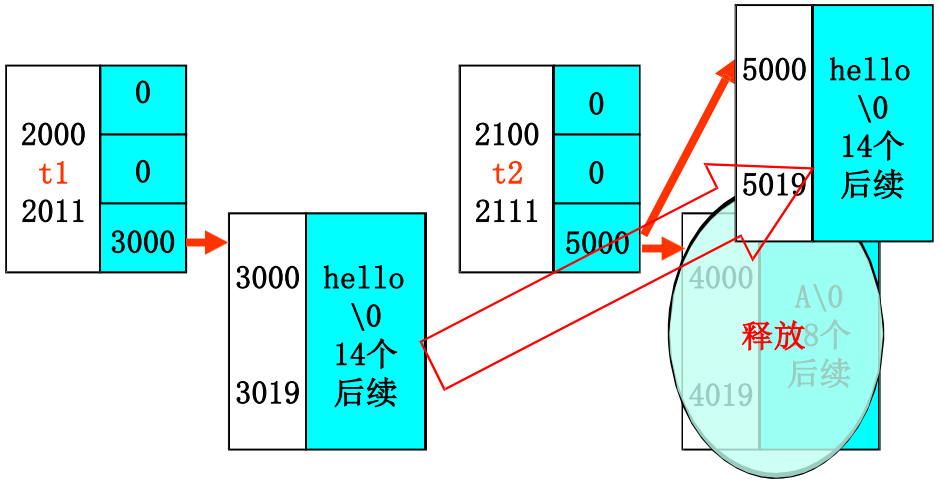
- 若对象数据成员是指针及动态分配的数据，则可能导致不可预料的后果甚至报错
- 解决方法：重载运算符=

对象的赋值与复制 （回顾）

- 对象的赋值
  - 有动态内存申请

```
int main()
{   test t1("hello"), t2;
    t1.display();   hello
    t2.display();   A
    t2=t1;
    t2.display();   hello
    t1.set("china");
    t1.display();   china
    t2.display();   hello
}
```

```
//解决方法：运算符重载！！
test &test::operator=(const test &t)
{   a = t.a;   b = t.b;
    delete c;                  //释放原空间
    c=new char[20];            //申请新空间
    strcpy(c, t.c);
    return *this;              //返回对象自身
}
```



2.1 运算符重载的规则

思考右侧运算符重载程序：

1）返回类型能否为test，而不是引用?

- 分析：返回值应符合运算符的语义

  t2 = t1理解为t2.operator=(t1)

  = 的语义希望执行后t2被改变

```
……//略
test &test::operator=(const test &t)
{  a = t.a;    b = t.b;
   delete c; //释放原空间
   c=new char[20]; //申请新空间
   strcpy(c, t.c);
   return *this; //返回对象自身
}
```

若返回test，则返回时会调用复制构造函数，返回的就是临时对象而不是t2自身

- 对比 Time类+: Time operator+(const Time& t)const;

  coding + fixing理解为coding.operator+(fixing)

  + 的语义不能改变coding，应该返回临时对象，所以返回值是Time而不是Time&

2.1 运算符重载的规则

思考右侧运算符重载程序：

```
……//略
test &test::operator=(const test &t)
{  a = t.a;    b = t.b;
   delete c; //释放原空间
   c=new char[20]; //申请新空间
   strcpy(c, t.c);
   return *this; //返回对象自身
}
```

2）返回类型能否为void?

- 分析：返回值应符合运算符的语义

  t2 = t1理解为t2.operator=(t1)

  = 的语义希望执行后t2被改变

  若返回void，this指针指向t2，对本题而言正确

- 进一步思考：t3 = (t2 = t1);

  t2 = t1：赋值表达式的值等于左值

  t3 = (t2 = t1)：没有找到接受void类型的右操作数的运算符（或没有可接受的转换），连续赋值时程序错误

2.1 运算符重载的规则

思考右侧运算符重载程序：

```
……//略
test &test::operator=(const test &t)
{  a = t.a;    b = t.b;
   delete c; //释放原空间
   c=new char[20]; //申请新空间
   strcpy(c, t.c);
   return *this; //返回对象自身
}
```

3）为什么要先释放再申请新空间？

- 分析：

  申请/释放都是20字节，内存不保证相同

  对本题而言可以仍旧使用原来已申请的空间，可删除这两条语句

- 进一步思考：若要求test类按需申请，不浪费空间

  必须释放原空间再申请新空间

```
delete c;
c=new char[strlen(t.c)+1];
```

2.1 运算符重载的规则

- 复制构造函数和重载=的区别：

	复制构造函数	重载赋值运算符
系统缺省	有，对应内存拷贝	有，对应内存拷贝
必须定义的时机	含动态内存申请时	含动态内存申请时
调用时机	定义时用对象初始化 函数形参为对象，返回值为对象	执行语句中的=操作
调用时处理	对象生成时调用，此时不可能调用其它形式的构造函数	=操作时调用，在=前对象已生成，即已调用过某种形式的构造函数(包括复制构造函数)

//复制构造函数和重载=的区别

```
class test {
 ...
 public:
  test(const test &t);
  test &operator=(const test &t);
 };
test::test(const test &s)
{    a=s.a;
     b=s.b;
     c=new char[20];  //申请新空间
     strcpy(c, s.c);
}
```

```
test &test::operator=(const test &t)
{   a = t.a;
    b = t.b;
    delete c; //释放原空间（复制构造函数不需要）
    c=new char[20]; //申请新空间
    strcpy(c, t.c);
    return *this;//返回对象自身（复制构造函数不需要）
}
int main()
{    test t1("hello"),t2(t1); //复制构造函数
     t2 = t1; //=运算
}
```

目录

- 运算符重载的方法
- 运算符重载的规则
- 成员函数和友元函数

3.1 成员函数和友元函数

- 回顾引例：今天早上在某账户上花费了2小时35分钟，下午又花费了2小时40分钟，则总共花了多少时间呢？

➢与加法概念吻合，但要相加的单位（小时与分钟的混合）与内置类型不匹配

解决方法0：在Time类中使用方法Sum()来处理加法

解决方法1：添加加法运算符来处理加法（成员函数）

➢其它重载运算符（成员函数）：

```
Time operator-(const Time& t)const;
Time operator*(double n)const;
```

//具体实现详见primer书：mytime2.h　mytime2.cpp　usetime2.cpp

➢进一步思考：乘法运算符重载的合理性

3.1 成员函数和友元函数

➢进一步思考：乘法运算符重载的合理性

```
Time operator*(double n)const;
A = B * 2.75;  //合理，左操作数是对象
A = 2.75 * B;  //不合理
```

➢上述二者从概念上应该相同

解决方法0：非成员函数

```
Time operator*(double m, const Time & t);  //无法访问类的私有数据
```

解决方法1：友元函数（非成员函数）

```
friend Time operator*(double m, const Time & t);
```

3.1 成员函数和友元函数

- 成员函数与友元函数的区别（结合后续“案例应用”理解）

	成员函数	友元函数
单目运算符	空参数	一个参数(必须是对象)
双目运算符	一个参数 (可不是对象)	两个参数 (一个必须是对象,一个可不是)

- 两个操作数都是对象：没区别
- 一个操作数是对象：若希望 2.75 * B正确，则需要重载实现 double * Time，且该方式只能通过友元函数实现
- 建议对单目运算符采用成员函数方式，双目运算符采用友元函数方式
- C++规定，某些运算符必须是成员函数形式(赋值=，下标[]，函数())，某些运算符必须是友元函数形式(流插入<<，流提取>>，类型转换　类型(值))，可能因编译系统不同而不同

总结

- 运算符重载的方法（掌握）
- 运算符重载的规则（熟悉）
- 成员函数和友元函数（熟悉）

第九章 运算符重载

模块9.2：实例应用

主讲教师：同济大学计算机科学与技术学院 陈宇飞
同济大学计算机科学与技术学院 龚晓亮

目录

- 单目运算符的重载
- 双目运算符的重载
- 流运算符的重载
- 不同类型间数据的转换

1.1 单目运算符的重载

例1：对复数的-重载，规则为实部虚部全部换符号

- 成员函数的实现：

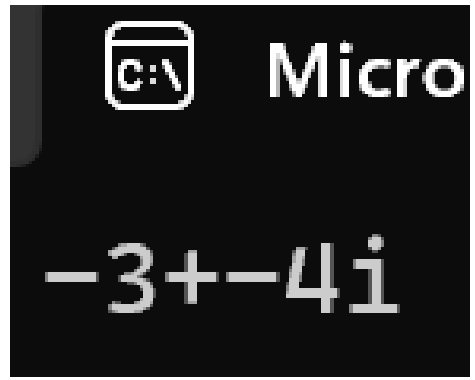
```
Complex operator-();
```

- 友元函数的实现：

```
friend Complex operator-(Complex &a);
```

```
class Complex {
 private:
  double real;
  double imag;
public:
  Complex(double r=0, double i=0) {
     real = r; imag = i;
  }
  void display(){
  cout << real <<"+" << imag <<"i" <<endl;
  }
  Complex operator-();
};
```

```
Complex Complex::operator-() //成员函数
{  Complex c1;
   c1.real = -real;  //实部取反
   c1.imag = -imag;  //虚部取反
   return c1;
}
int main()
{  Complex c1(3,4),c2;
   c2 = -c1;
   c2.display();
}    //输出为-3+-4i形式
```



1.1 单目运算符的重载

思考右侧运算符重载程序：

- 返回类型能否为引用Complex & ？

➢分析：

若只修改函数返回类型为Complex &，不可以

因为不能返回自动变量的引用

➢进一步思考：

若改为右侧程序，不可以

c2 = -c1;

```
……//略
Complex Complex::operator-() //成员函数
{
   Complex c1;
   c1.real = -real; //实部取反
   c1.imag = -imag; //虚部取反
   return c1;
}
```

```
……//略
Complex & Complex::operator-() //成员函数
{
   real = -real; //实部取反
   imag = -imag; //虚部取反
   return *this;
}
```

-的语义不希望改变对象自身，所以要返回临时对象，而不是this

```
class Complex {
 private:
  double real;
  double imag;
public:
  Complex(double r=0, double i=0) {
      real = r; imag = i;
  }
  void display(){
  cout << real <<"+" << imag <<"i" <<endl;
  }
  friend Complex operator-(Complex &a);
};
```

```
//该友元函数返回值不可以是Complex &
Complex operator-(Complex &a) //友元函数
{   Complex c1;
    c1.real = -a.real; //实部取反
    c1.imag = -a.imag; //虚部取反
    return c1;
}
int main()
{   Complex c1(3,4),c2;
    c2 = -c1;
    c2.display();
}  //输出为-3+-4i形式
```



1.1 单目运算符的重载

例2：对++/--的前后缀运算符重载，规则为只对实部++/--，虚部不动

- 思考语义：程序中前缀++/--返回引用，后缀++/--返回对象
  - 前缀是自身先++/--，再自身参与运算，因此返回引用，即对象自身，且不需调用复制构造
  - 后缀是保存旧值，自身++/--，再旧值参与运算，因此返回对象，返回时调用复制构造产生临时对象
- 实现：
  - 前缀：正常方式
  - 后缀：多一个int型参数，不访问，仅进行区别

引申思考：i++和++i哪个效率更高？

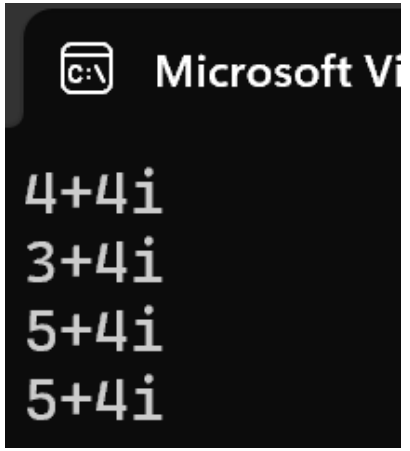
```
class Complex {    //成员函数的实现方式
  private:
    double real;  double imag;
 public:
    Complex(double r=0, double i=0) {
        real = r; imag = i;
    }
    void display(){
        cout<<real<<"+"<<imag<<"i"<<endl;
    }
    Complex& operator++();   //前缀
    Complex  operator++(int); //后缀
};
Complex& Complex::operator++() //前缀
{   real++; //前后缀无所谓
    return *this; //返回自身
}
```

```
Complex Complex::operator++(int) //后缀
{  Complex c1(*this); //用对象自身初始化c1
    real++;    //前后缀无所谓，注意不是c1.real
    return c1; //返回++前的值，符合后缀语义
}
int main()
{  Complex c1(3,4), c2;
    c2 = c1++;
    c1.display();
    c2.display();
    c2 = ++c1;
    c1.display();
    c2.display();
}
```

```
Microso
4+4i
3+4i
5+4i
5+4i
```

```
class Complex {      //友元函数的实现方式
  private:
     double real;
     double imag;
  public:
     Complex(double r=0, double i=0) {
        real = r; imag = i;
     }
     void display(){
        cout << real <<"+" << imag <<"i" <<endl;
     }
     friend Complex& operator++(Complex &a);      //前缀
     friend Complex  operator++(Complex &a, int); //后缀
 };
Complex& operator++(Complex &a) //前缀
{    a.real++; //前后缀无所谓
     return a; //全局函数没有this指针
}
```

```
Complex operator++(Complex &a, int) //后缀
{  Complex c1(a); //用对象a初始化c1
   a.real++; //前后缀无所谓，注意不是c1.real
   return c1; //返回++前的值，符合后缀语义
}
int main()
{  Complex c1(3,4), c2;
   c2 = c1++;
   c1.display();
   c2.display();
   c2 = ++c1;
   c1.display();
   c2.display();
}
```



目录

- 单目运算符的重载
- 双目运算符的重载
- 流运算符的重载
- 不同类型间数据的转换

2.1 双目运算符的重载

例3：对复数的+重载，需考虑复数+复数，复数+double，double+复数

➢两个操作数都是对象（复数+复数）：

```
friend Complex operator+(Complex &a, Complex &b);
Complex operator+(Complex &b);
```

➢一个操作数是对象（复数+double，double+复数）：

(1) 复数+double：`friend Complex operator+(Complex &a, double b);`

`Complex operator+(double b);`

(2) double+复数：`friend Complex operator+(double a, Complex &b);`

• 双目运算符--两个操作数都是对象：友元函数、成员函数均可

```
class Complex {
    …
    friend Complex operator+(Complex &a, Complex &b);
};
Complex operator+(Complex &a, Complex &b)//友元函数
{  Complex c;
    c.real=a.real+b.real;
    c.imag=a.imag+b.imag;
    return c;
}
int main()
{  Complex c1(3,4),c2(4,5),c3;
    c3 = c1+c2;
    c3.display();
}
```

//全局友元函数，没有this指针，通过对象.成员的形式调用，返回时调用复制构造函数

• 双目运算符--两个操作数都是对象：友元函数、成员函数均可

```
class Complex {
    …
    Complex operator+(Complex &b);
};
Complex Complex::operator+(Complex &b)//成员函数
{   Complex c;
    c.real=real+b.real;
    c.imag=imag+b.imag;
    return c;
}
int main()
{   Complex c1(3,4),c2(4,5),c3;
    c3 = c1+c2;
    c3.display();
}
```

//成员函数，有this指针，直接成员的形式调用，返回时调用复制构造函数

• 双目运算符--一个操作数是对象：复数+double使用友元、成员均可

```
class Complex {
    …
    friend Complex operator+(Complex &a, double b);
};
Complex operator+(Complex &a, double b)  //友元函数
{  Complex c;
    c.real=a.real+b;//实部相加
    c.imag=a.imag; //虚部不变
    return c;
}
int main()
{    Complex c1(3,4),c2;
     c2 = c1 + 4;  //正确
     c2 = 4 + c1;  //编译错
}
```

- 双目运算符--一个操作数是对象：复数+double使用友元、成员均可

```
class Complex {
    …
   Complex operator+(double b);
};
Complex Complex::operator+(double b)   //成员函数
{  Complex c;
   c.real=real+b;//实部相加
   c.imag=imag; //虚部不变
   return c;
}
int main()
{   Complex c1(3,4),c2;
    c2 = c1 + 4;  //正确
    c2 = 4 + c1;  //编译错
}
```

- 双目运算符--一个操作数是对象：两个友元函数重载

```
class Complex { ...
    friend Complex operator+(Complex &a, double b);
    friend Complex operator+(double a, Complex &b);
};
Complex operator+(Complex &a, double b) //复数+double
{   Complex c;
    c.real=a.real+b; //实部相加
    c.imag=a.imag;  //虚部不变
    return c;
}
Complex operator+(double a, Complex &b) //double+复数
{   Complex c;
    c.real=a+b.real;//实部相加
    c.imag=b.imag; //虚部不变
    return c;
}
```

```
int main()
{ Complex c1(3,4);
  Complex c2, c3;
  c2 = c1 + 4; //正确
  c2.display();
  c3 = 5 + c1; //正确
  c3.display();
} //double型常量
```

```
int main()
{ Complex c1(3,4);
  Complex c2, c3;
  double d1=4, d2=5;
  c2 = c1 + d1;//正确
  c2.display();
  c3 = d2 + c1;//正确
  c3.display();
} //double型变量
```

• 双目运算符--一个操作数是对象：两个友元函数重载

```
class Complex {  //引申修改1：将double换成double &
    ...
    friend Complex operator+(Complex &a, double &b);
    friend Complex operator+(double &a, Complex &b);
};
Complex operator+(Complex &a, double &b) //复数+double
{   Complex c;
    c.real=a.real+b; //实部相加
    c.imag=a.imag;  //虚部不变
    return c;
}
Complex operator+(double &a, Complex &b) //double+复数
{   Complex c;
    c.real=a+b.real;//实部相加
    c.imag=b.imag; //虚部不变
    return c;
}
```

原因：引用是变量的别名，因此若函数形参为引用，则实参只能是变量，不能是常量/表达式

```
int main()
{ Complex c1(3,4);
  Complex c2, c3;
  c2 = c1 + 4; //错误
  c2.display();
  c3 = 5 + c1; //错误
  c3.display();
} //double型常量
```

```
int main()
{ Complex c1(3,4);
  Complex c2, c3;
  double d1=4, d2=5;
  c2 = c1 + d1;//正确
  c2.display();
  c3 = d2 + c1;//正确
  c3.display();
} //double型变量
```

- 双目运算符--一个操作数是对象：两个友元函数重载

```
class Complex {  //引申修改2：将double换成const double &
    ...
    friend Complex operator+(Complex &a, const double &b);
    friend Complex operator+(const double &a, Complex &b);
};
Complex operator+(Complex &a, const double &b) //复数+double
{   Complex c;
    c.real=a.real+b; //实部相加
    c.imag=a.imag;  //虚部不变
    return c;
}
Complex operator+(const double &a, Complex &b) //double+复数
{   Complex c;
    c.real=a+b.real;//实部相加
    c.imag=b.imag; //虚部不变
    return c;
}
```

若函数形参为常引用，则实参可以是变量/常量/表达式

```
int main()
{ Complex c1(3,4);
  Complex c2, c3;
  c2 = c1 + 4; //正确
  c2.display();
  c3 = 5 + c1; //正确
  c3.display();
} //double型常量
```

```
int main()
{ Complex c1(3,4);
  Complex c2, c3;
  double d1=4, d2=5;
  c2 = c1 + d1;//正确
  c2.display();
  c3 = d2 + c1;//正确
  c3.display();
} //double型变量
```

- 双目运算符---一个操作数是对象：友元和成员函数重载

```
class Complex {
   ...
   Complex operator+(double b);
   friend Complex operator+(double  a, Complex &b);
};
Complex Complex::operator+(double b) //成员函数实现复数+double
{  Complex c;
   c.real=real+b; //实部相加
   c.imag=imag;  //虚部不变
   return c;
}
Complex operator+(double a, Complex &b) //友元函数实现double+复数
{  Complex c;
   c.real=a+b.real;//实部相加
   c.imag=b.imag; //虚部不变
   return c;
}
```

无法做到两个成员函数重载：
因为5+c1无法表示为成员函数形式
原因：第1个参数(左值)不是类

```
int main()
{ Complex c1(3,4);
  Complex c2, c3;
  c2 = c1 + 4; //正确
  c2.display();
  c3 = 5 + c1; //正确
  c3.display();
} //double型常量
```

```
int main()
{ Complex c1(3,4);
  Complex c2, c3;
  double d1=4, d2=5;
  c2 = c1 + d1;//正确
  c2.display();
  c3 = d2 + c1;//正确
  c3.display();
} //double型变量
```

2.1 双目运算符的重载



- 关于 + 交换律的说明 ：

➢两个对象+，即定义两个类的+重载后，无论c1+c2还是c2+c1，都调用同一重载函数但本质不同：c1.operator+(c2)/c2.operator+(c1)结果相同，可以理解为交换律存在

➢对象+其它类型，例如 Complex+double，交换律不存在，交换律的表面现象是通过多个函数的重载来实现的

➢同理适用于其它存在交换律的运算符

目录

- 单目运算符的重载
- 双目运算符的重载
- 流运算符的重载
- 不同类型间数据的转换

3.1 流运算符的重载

- 形式：

```
istream& operator >> (istream &, 自定义类 &);
ostream& operator << (ostream &, 自定义类 &);
```

  - >> 和 << 已被多次重载：

    最初：位移运算符

    istream类：重载为一个输入工具

    ostream类：重载为一个输出工具

  - >> 本身是istream类(系统定义类)的成员函数，因此希望对istream类的 >> 运算符重载，使其能输入自定义类的内容
  - << 本身是ostream类(系统定义类)的成员函数，因此希望对ostream类的 << 运算符重载，使其能输出自定义类的内容

3.1 流运算符的重载

- 重载<<运算符（>>的实现类似）：

```
Time trip;  //trip是已定义的Time类对象

cout << trip;  //需要重载<<
```

  - 必须使用友元函数的方式来实现

    假设：使用Time的成员函数来重载<<（Time对象将是第一个操作数）

    则意味着使用形式为： trip << cout

    结论：使用友元函数

3.1 流运算符的重载

- 重载<<运算符：
  - 第一种重载版本

```
void operator<<(ostream & os, const Time & t)
{
      os << t.hours << " hours, " << t.minutes << " minutes";
}
```

```
cout << trip; //可以
cout << "Trip time: " << trip << " (Tuesday)\n";   //不可以
```

ostream &

void

3.1 流运算符的重载

- 重载<<运算符：
  - 第二种重载版本

```
ostream & operator<<(ostream & os, const Time & t){
        os << t.hours << " hours, " << t.minutes << " minutes";
        return os;
}
```

```
cout << trip; //可以
cout << "Trip time: " << trip << " (Tuesday)\n";  //可以
```

ostream &

ostream &

```
//mytime3.h -- Time class with friends
class Time {
    …
    friend ostream & operator<<(ostream & os, const Time& t);
};
```

头文件
类的声明

```
//mytime3.cpp -- implementing Time methods
#include"mytime3.h"
ostream & operator<<(ostream & os, const Time& t);{…}
…
```

源程序文件
函数的实现

```
//usetime3.cpp -- using the fourth draft of the Time class
//compile usetime3.cpp and mytime3.cpp together
#include"mytime3.h"
int main()
{   Time aida(3, 35);
    Time tosca(2, 48);
    cout << aida << "; " << tosca << endl;  //operator<<
    ...
}
```

源程序文件
调用函数

完整程序见primer书

目录

- 单目运算符的重载
- 双目运算符的重载
- 流运算符的重载
- 不同类型间数据的转换

4.1 不同类型间数据的转换

- 内置类型
  - 隐式转换：

    ```
    int + double; //按照转换规则（高程基础第二章）
    int = double; //以左值为准进行转换
    ```

  - 显式转换：　（强制类型转换）

    C方式：　(int)89.5

    C++方式：　int(89.5)

- 自定义类型，能否进行类型转换？

• 例：设计一种合适的类型，以两种方式（磅和英石）来表示重量

```
// stonewt.h  -  definition for the Stonewt class
class Stonewt{
  private:
     enum {Lbs_per_stn = 14};        //类特定常量，相当于static const int
     int stone;                      Lbs_per_Stn = 14
     double pds_left;
     double pounds;
  public:                            //3种构造函数：
     Stonewt(double lbs);            Stonewt blossom(132.5);//132.5 pounds
     Stonewt(int stn, double lbs);   Stonewt buttercup(10,2);//10stone,2pounds
     Stonewt();                      Stonewt bubbles;//default value
     ~Stonewt();
     void show_lbs() const;
     void show_stn() const;
};

// stonewt.cpp  -  stonewt methods  具体程序见primer书
```

4.1 不同类型间数据的转换

1）转换构造函数（某类型->类类型）

```
Stonewt(double lbs); //template for double-to-Stonewt conversion
```

- 只有接受一个参数的构造函数才能作为转换函数：

```
Stonewt(int stn, double lbs); //not a conversion function
```

如果第二个参数提供默认值，可用于转换int

```
Stonewt(int stn, double lbs = 0); //int-to-Stonewt conversion
```

- 单一个参数的构造函数，只有完成转换功能才称为转换构造函数：

```
Stonewt::Stonewt(double lbs){
    stone = 0; pds_left = 0; pounds = 0;
}  //not a conversion function
```

4.1 不同类型间数据的转换

1）转换构造函数（某类型->类类型）

```
Stonewt(double lbs); //template for double-to-Stonewt conversion
```

- 隐式的自动类型转换：

```
Stonewt myCat; //create a Stonewt object
myCat = 19.6; //use Stonewt(double) to convert 19.6 to Stonewt
```

- 显式的类型强制转换：（使用关键字explicit关闭隐式自动转换）

```
explicit Stonewt(double lbs);//只能显式类型转换
myCat = 19.6; //not valid
myCat = Stonewt(19.6)  //ok, an explicit conversion
myCat = (Stonewt) 19.6  //ok, old form for explicit typecast
```

4.1 不同类型间数据的转换

1）转换构造函数（某类型->类类型）

```
Stonewt(double lbs); //template for double-to-Stonewt conversion
```

- 隐式转换的时机：

1. 将Stonewt对象初始化为double值时；

2. 将double值赋给Stonewt对象时；

3. 将double值传递给接受Stonewt参数的函数时；

4. 返回值被声明为Stonewt的函数试图返回double值时；

5. 在上述任意一种情况下，使用可转换为double类型的内置类型时；

4.1 不同类型间数据的转换

1）转换构造函数（某类型->类类型）

```
Stonewt(double lbs); //template for double-to-Stonewt conversion
```

➢允许使用Stonewt(double)构造函数来转换其他数值类型：

```
Stonewt Jumb(7000);//uses Stonewt(double), converting int to double
Jumb = 7300; //uses Stonewt(double), converting int to double
```

二步转换的前提：不存在二义性

如果还存在Stonewt(long)，则编译器将拒绝执行上述语句

因为int可以转为long或者double，一次调用会出现二义性

```
//stonewt.h  -  definition for the Stonewt class
class Stonewt {
    …
    Stonewt(double lbs);
};
```

头文件
类的声明

```
//stonewt.cpp -- Stonewt methods
#include"stonewt.h"
Stonewt::Stonewt(double lbs){…}
…
```

源程序文件
函数的实现

```
//stone.cpp  -  user-defined conversions
//compile with stonewt.cpp
#include"stonewt.h"
int main()
{   Stonewt incognito = 275;
    Stonewt wolfe(285.7);
    Stonewt taft(21, 8);
    incognito = 276.8;
    taft = 325;
    display(422, 2);
    …
}
```

源程序文件
调用函数

完整程序见primer书

• 程序分析：

```
//stone.cpp  -  user-defined conversions
int main()
{    Stonewt incognito = 275;
     Stonewt wolfe(285.7);
     Stonewt taft(21, 8);
     incognito = 276.8;
     taft = 325;
     display(422, 2);
     …
}
void display(const Stonewt & st, int n){
     …
}
```

```
//隐式
等价形式：
Stonewt incognito(275);//隐式
Stonewt incognito = Stonewt(275)//显式
等价形式相比左侧标红程序的好处：
可以接受多个参数的构造函数
```



• 程序分析：

```
//stone.cpp  -  user-defined conversions
int main()
{    Stonewt incognito = 275;
     Stonewt wolfe(285.7);
     Stonewt taft(21, 8);
     incognito = 276.8;          //将double转换为Stonewt
     taft = 325;                 //将int转换为double再转换为Stonewt
     display(422, 2);
     …
}
void display(const Stonewt & st, int n){
     …
}
```

• 程序分析：

```
//stone.cpp  -  user-defined conversions
int main()
{    Stonewt incognito = 275;
     Stonewt wolfe(285.7);
     Stonewt taft(21, 8);
     incognito = 276.8;
     taft = 325;
     display(422, 2);
     …
}
void display(const Stonewt & st, int n){
     …
}
```

查找匹配顺序：

(1)是否有Stonewt(int)？　　无

(2)是否有系统内置类型的转换？　有

　　int->double

(3)是否有用户定义的内部转换？　有

　　Stonewt(double)

//将int转换为double再转换为Stonewt

4.1 不同类型间数据的转换

2）类型转换函数（类类型->某类型）

```
operator double(); //template for Stonewt-to-double conversion
```

- 类型转换函数是用户定义的强制类型转换：(若已定义operator double();)
  - 显式使用：

```
Stonewt wolfe(285.7);
double host = double (wolfe);
double thinker = (double) wolfe;
```

  - 隐式使用：

```
Stonewt wells(20, 3);
double star = wells;
```

4.1 不同类型间数据的转换

2）类型转换函数（类类型->某类型）

```
operator double(); //template for Stonewt-to-double conversion
```

➢ 注意事项：

- 转换函数必须是类方法
- 转换函数不能指定返回类型
- 转换函数不能有参数
- 可以使用explicit关键词关闭隐式自动转换

```
//stonewt1.h -- revised definition for the Stonewt class
class Stonewt {
    …
    operator int() const;
    operator double() const;
};
```

头文件
类的声明

```
//stonewt1.cpp -- Stonewt class methods + conversion functions
#include"stonewt1.h"
Stonewt::operator int() const{…}
Stonewt::operator double() const{…}
…
```

源程序文件
函数的实现

```
//stone1.cpp  -  user-defined conversion functions
//compile with stonewt1.cpp
#include"stonewt1.h"
int main()
{   Stonewt poppins(9,2.8);
    double p_wt = poppins;
    …
    cout << "Poppins: " << int(poppins) << " pounds.\n";
    …
}
```

源程序文件
调用函数

完整程序见primer书

• 程序分析：

```
//已定义了：
    operator int() const;
    operator double() const;
//stone1.cpp
int main()
{   Stonewt poppins(9,2.8);
    double p_wt = poppins;  //隐式
    …
    cout << "Poppins: " << int(poppins) << " pounds.\n";   //显式
    …
}
```

```
//左侧程序将显式修改为隐式：
cout << "Poppins: " << poppins
     << " pounds.\n";  //不可以
原因：二义性转换
     （未指出转换为int还是double）
若只定义：operator double() const;
则正确，没有二义性
```

• 程序分析：

```
//已定义了：
    operator int() const;
    operator double() const;
//stone1.cpp
int main()
{   Stonewt poppins(9,2.8);
    double p_wt = poppins;  //隐式
    …
    cout << "Poppins: " << int(poppins) << " pounds.\n";   //显式
    …
}
```

```
//若有以下赋值：
long gone = poppins;  //不可以
原因：二义性
    （int和double均可被赋值给long）
若删除任一转换函数：
则正确，没有二义性
```

• 程序分析：

```
//已定义了：
    operator int() const;
    operator double() const;
//stone1.cpp
int main()
{   Stonewt poppins(9,2.8);
    double p_wt = poppins;  //隐式
    …
    cout << "Poppins: " << int(poppins) << " pounds.\n";   //显式
    …
}
```

```
//可使用显式的强制类型转换：
long gone = (double) poppins;
                //use double conversion
long gone = int (poppins);
                //use int conversion
```

• 程序分析：应谨慎的使用隐式转换函数

```
//手误程序：
int ar[20];
…
Stonewt temp(14,4);
…
int Temp = 1;
…
cout << ar[temp] << "!\n";
//used temp instead of Temp
```

```
//隐式的将temp转换为int，用作了数组索引（结果越界）：
结论：最好使用显式转换
      (使用explicit关键字)

class Stonewt{
   …
   explicit operator int() const;
   explicit operator double() const;
};
```

总结

- 单目运算符的重载（掌握）
- 双目运算符的重载（掌握）
- 流运算符的重载（熟悉）
- 不同类型间数据的转换（了解）

第十章 模板类

主讲教师：同济大学计算机科学与技术学院 陈宇飞
同济大学计算机科学与技术学院 龚晓亮

目录

- 智能指针模板类
- 标准模板库

目录

- 智能指针模板类
  - 基本概念
  - 使用智能指针
  - 有关智能指针的注意事项
  - unique_ptr为何优于auto_ptr
  - 选择智能指针

1.1 基本概念

- 请查找下面函数的缺陷：

```
void remodel(std::string& str)
{
    std::string * ps = new std::string(str);
    ...
    str = *ps;
    return;
}
```

- 每当调用函数时，该函数都分配堆中的内存，但从不收回，从而导致内存泄露

1.1 基本概念

- “别忘了delete ps” 有时可能记住，但有时可能在不经意间删除或注释掉

```
void remodel(std::string& str)
{
    std::string * ps = new std::string(str);
    ...
    str = *ps;
    delete ps;
    return;
}
```

1.1 基本概念

- 当出现异常时，delete将不被执行，因此也会出现内存泄露

```
void remodel(std::string& str)
{
      std::string * ps = new std::string(str);
      ...
      if (weird_thing())
          throw exception();
      str = *ps;
      delete ps;
      return;
}
```

- 现在的需求是希望remodel()这样的函数终止时（不论是正常终止，还是因异常终止），本地变量都将从栈内存中删除（包括指针ps占据的内存被释放）

1.2 使用智能指针

- 三个智能指针模板（auto_ptr、unique_ptr和shared_ptr）都定义了类似指针的对象，可以将new获得（直接或间接）的地址赋给这种对象。当智能指针过期时，其析构函数将使用delete来释放内存

```
void demo1()
{
    double* pd = new double;//#1
    *pd = 25.5;             //#2
    return;                 //#3
}
```

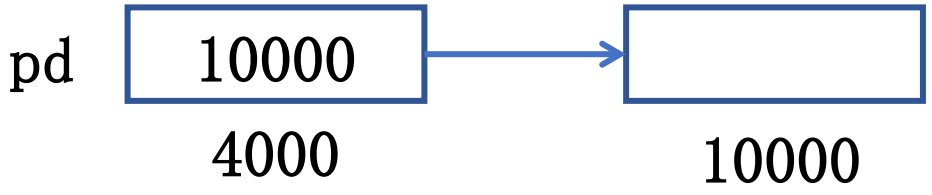
```
void demo1()
{
  auto_ptr<double> ap(new double); //#1
  *ap= 25.5;                       //#2
  return;                          //#3
}
```

- 智能指针是行为类似于指针的类对象，但这种对象还有其他功能

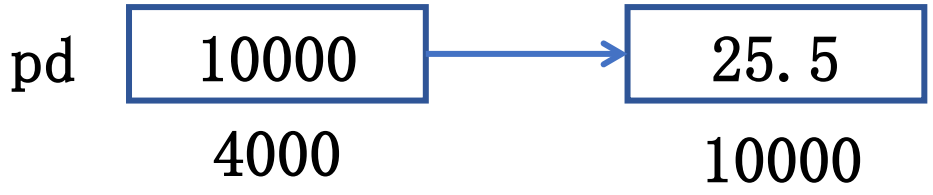
1.2 使用智能指针

```
void demo1()
{
    double* pd = new double;//#1
    *pd = 25.5;                //#2
    return;                    //#3
}
```

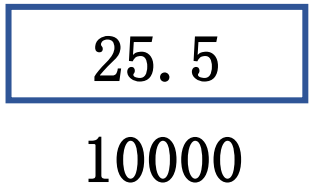
#1. 为pd和double值分配存储空间，保存地址：



#2. 将值赋值到动态内存中：



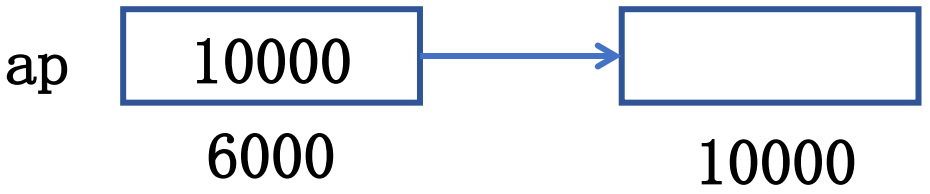
#3. 删除pd，值被保留在动态内存中：



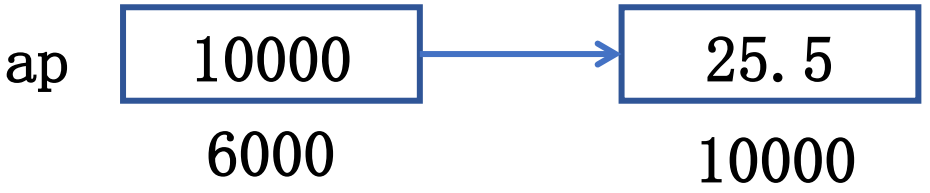
1.2 使用智能指针

```
void demo2()
{
    auto_ptr<double> ap(new double);//#1
    *ap= 25.5;                      //#2
    return;                         //#3
}
```

#1. 为ap和double值分配存储空间，保存地址：



#2. 将值赋值到动态内存中：



#3. 删除ap, ap的析构函数释放动态内存

1.2 使用智能指针

- 创建智能指针对象，必须包含头文件memory，实现该文件模板定义

```
template<class X> class auto_ptr {
public:
    explicit auto_ptr(X* p = 0) throw()   {};
   ...
};
```

- 使用通常的模板语法来实例化所需类型的指针

```
auto_ptr<double> pd(new double);
auto_ptr<string> ps(new string);
```

```
unique_ptr<double> pdu(new double);
shared_ptr<string> pss(new string);
```

1.2 使用智能指针

- 注意：

➢异常规范（包括 throw()）在 C++11 中被弃用，并在 C++17 中被完全移除。原因是它们在实际使用中增加了编译器的复杂性，并可能导致意外的程序行为（如 std::unexpected() 的调用）。C++11 引入了 noexcept 关键字作为异常规范的现代替代品

➢auto_ptr 本身由于其不安全的复制语义已经被弃用，并在 C++17 中被移除。最佳实践是使用 std::unique_ptr 或 std::shared_ptr 来管理动态分配的内存

```
auto_ptr<double> pd(new double);
auto_ptr<string> ps(new string);
```

```
unique_ptr<double> pdu(new double);
shared_ptr<string> pss(new string);
```

1.2 使用智能指针

- 要转换remodel()函数，应按3个步骤进行：

  #1.包括头文件memory；

  #2.将指向string的指针替换为指向string的智能指针对象；

  #3.删除delete语句

```
void remodel(std::string& str)
{
   std::string * ps = new std::string(str);
   ...
   if (weird_thing())
      throw exception();
   str = *ps;
   delete ps;
   return;
}
```

```
#include <memory> //# 1
void remodel(std::string& str)
{
   std::auto_ptr<std::stringbuf> ps (new
std::string(str));//# 2
   ...
   if (weird_thing())
       throw exception();
   str = *ps;
   // delete ps; NO LONGER NEEDED//# 3
   return;
}
```

```
// smrtptrs.cpp -- using three kinds of smart pointers
```

接后页

```
#include <iostream>
#include <string>
#include <memory>

class Report
{
private:
    std::string str;
public:
    Report(const std::string s):str(s) { std::cout << "Object created!\n"; }
    ~Report() { std::cout << "Object deleted!\n"; }
    void comment() const { std::cout << str << "\n"; }
};
```

续前页

```
// smrtptrs.cpp -- using three kinds of smart pointers
int main()
{
    {
        std::auto_ptr<Report> ps (new Report("using auto_ptr"));
        ps->comment();   // use -> to invoke a member function
    }
    {
        std::shared_ptr<Report> ps (new Report("using shared_ptr"));
        ps->comment();
    }
    {
        std::unique_ptr<Report> ps (new Report("using unique_ptr"));
        ps->comment();
    }

    return 0;
}
```

```
Object created!
using auto_ptr
Object deleted!
Object created!
using shared_ptr
Object deleted!
Object created!
using unique_ptr
Object deleted!
```

1.2 使用智能指针

- 所有智能指针类都有一个explicit构造函数，该构造函数将指针作为参数。因此，需要显式将指针转换为智能指针对象

```
shared_ptr<double>pd;
double* p_reg = new double;
pd = p_reg;                              // not allowed (implicit conversion)
pd =  shared_ptr<double>(p_reg>;         // allowed (explicit conversion)
shared_ptr<double> pshared = p_reg;      // not allowed (implicit conversion)
shared_ptr<double> pshared(p_reg);       // allowed (explicit conversion)
```

1.2 使用智能指针

- 由于智能指针模板类的定义方式，智能指针对象的很多方面类似于常规指针

```
shared_ptr<string> ps;
```

  - 可以对智能指针执行解引用操作（*ps）
  - 可以用它来访问结构成员（ps->pufflndex）
  - 可以将它赋给指向相同类型的常规指针
  - 还可以将智能指针对象赋给另一个同类型的智能指针对象
  - 但要注意可能引起问题（见后）

1.2 使用智能指针

- 由于智能指针模板类的定义方式，智能指针对象的很多方便类似于常规指针
  - 但要注意可能引起问题

```
string vacation("I wandered lonely as a cloud.");
shared_ptr<string> pvac(&vacation);// NO!
```

当pvac过期时，程序将把delete运算符用于非堆内存

```
string* vacation = new string("I wandered lonely as a cloud.");
shared_ptr<string> pvac(vacation);
```

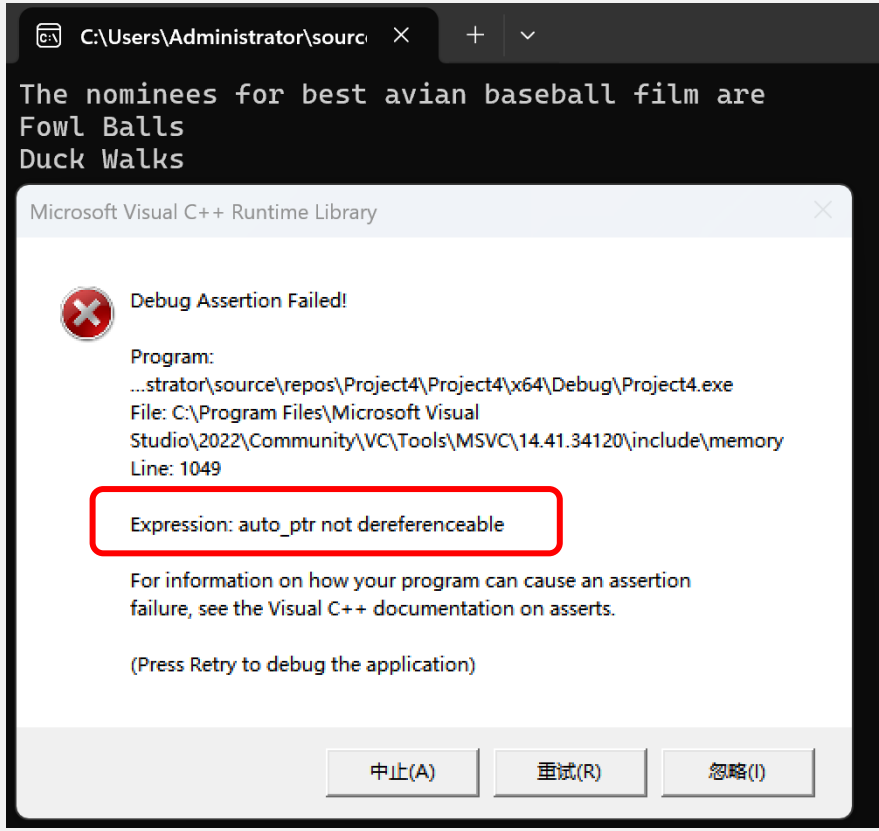
1.3 有关智能指针的注意事项

```
auto_ptr<string> ps(new string("I reigned in the year 2010"));
auto_ptr<string> vocation;
vocation = ps;//两个指针指向同一个string，程序将试图删除同一个对象两次，一次是ps过期时，一次是vocation过期时
```

避免这一问题的方法有：

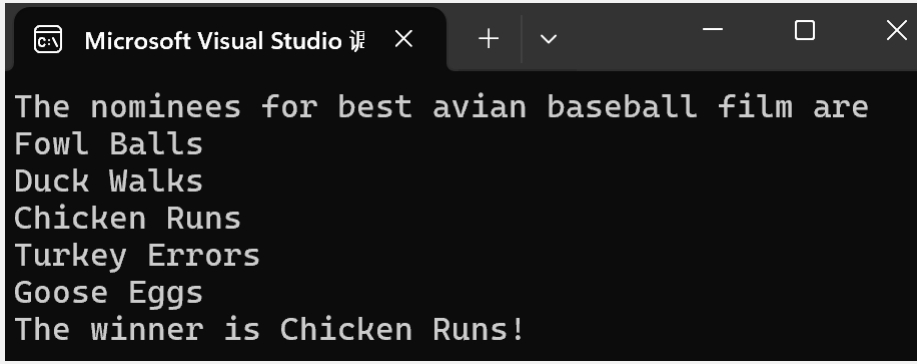
- 定义赋值运算符，使之执行深拷贝，这样两个智能指针指向不同的内存
- 建立所有权概念，对于特定的对象，只能有一个智能指针可拥用它。只有拥有对象的智能指针的构造函数会删除该对象。然后，让赋值操作转让所有权。（auto_ptr/unique_ptr的策略，unique_ptr的更严格）
- 创建智能更高的指针，跟踪引用特定对象的智能指针数（引用计数reference counting）。赋值时计数加1，指针过期计数减1，仅当最后一个指针过期时，才调用delete。（shared_ptr的策略）

```
// fowl.cpp  -- auto_ptr a poor choice
#include <iostream>
#include <string>
#include <memory>
int main()
{
    using namespace std;
    auto_ptr<string> films[5] =
    {
        auto_ptr<string>(new string("Fowl Balls")),
        auto_ptr<string>(new string("Duck Walks")),
        auto_ptr<string>(new string("Chicken Runs")),
        auto_ptr<string>(new string("Turkey Errors")),
        auto_ptr<string>(new string("Goose Eggs"))
    };
    auto_ptr<string> pwin;
    pwin = films[2];   // films[2] loses ownership
    cout << "The nominees for best avian baseball film are\n";
    for (int i = 0; i < 5; i++)
        cout << *films[i] << endl;
    cout << "The winner is " << *pwin << "!\n";
      return 0;}
```



- 该语句将所有权从films[2]转让给pwin，这导致films[2]不再指向该字符串
- 但程序打印films[2]指向的字符串时，发现其是一个空指针

```
// fowl.cpp  -- auto_ptr a poor choice
#include <iostream>
#include <string>
#include <memory>
int main()
{
    using namespace std;
    shared_ptr<string> films[5] =
    {
        shared_ptr<string>(new string("Fowl Balls")),
        shared_ptr<string>(new string("Duck Walks")),
        shared_ptr<string>(new string("Chicken Runs")),
        shared_ptr<string>(new string("Turkey Errors")),
        shared_ptr<string>(new string("Goose Eggs"))
    };
    shared_ptr<string> pwin;
    pwin = films[2];   // films[2] loses ownership
    cout << "The nominees for best avian baseball film are\n";
    for (int i = 0; i < 5; i++)
        cout << *films[i] << endl;
    cout << "The winner is " << *pwin << "!\n";
      return 0;}
```



- 该语句使得两个两个指针指向同一个对象，因此引用计数从1增加到2
- 后声明的pwin首先调用其析构函数，使得引用计数降到1
- 然后，shared_ptr数组的成员被释放，films[2]调用析构函数时，引用计数降低到0，并释放以前分配的空间

```cpp
// fowl.cpp  -- auto_ptr a poor choice
#include <iostream>
#include <string>
#include <memory>

int main()
{
    using namespace std;
    unique_ptr<string> films[5] =
    {
        unique_ptr<string>(new string("Fowl Balls")),
        unique_ptr<string>(new string("Duck Walks")),
        unique_ptr<string>(new string("Chicken Runs")),
        unique_ptr<string>(new string("Turkey Errors")),
        unique_ptr<string>(new string("Goose Eggs"))
    };
    unique_ptr<string> pwin;
    pwin = films[2];   // films[2] loses ownership

    cout << "The nominees for best avian baseball film are\n";
    for (int i = 0; i < 5; i++)
        cout << *films[i] << endl;
    cout << "The winner is " << *pwin << "!\n";
    // cin.get();
    return 0;
}
```

- 如果使用unique_ptr时，程序不会等到运行阶段崩溃，而是在编译器下就报错

整个解决方案 | 错误 2 | 警告 0 | 消息 0 | 生成 + IntelliSense

	代码	说明	项目
abc	E1776	无法引用 函数 "std::unique_ptr<_Ty, _Dx>::operator=(const std::unique_ptr<_Ty, _Dx> &) [其中 _Ty=std::string, _Dx=std::default_delete<std::string>]" (已声明 所在行数:3452，所属文件:"C:\Program Files\Microsoft Visual Studio\2022\Community\VC\Tools\MSVC\14.41.34120\include\memory") -- 它是已删除的函数	Project4
⊗	C2280	"std::unique_ptr<std::string,std::default_delete<std::string>> &std::unique_ptr<std::string,std::default_delete<std::string>>::operator =(const std::unique_ptr<std::string,std::default_delete<std::string>> &)": 尝试引用已删除的函数	Project4

1.4 unique_ptr为何优于auto_ptr

```
auto_ptr<string> p1(new string("auto")); //#1
auto_ptr<string> p2;                     //#2
p2 = p1;                                 //#3
```

- 在语句#3，p2接管string对象的所有权后，p1的所有权将被剥夺，成为悬挂指针，不再指向有效的数据

```
unique_ptr<string> p3(new string("auto")); //#4
unique_ptr<string> p4;                     //#5
p4 = p3;                                   //#6
```

- 编译器认为语句#6非法，避免了p3不再指向有效数据的问题。因此unique_ptr比auto_ptr更安全（编译阶段错误比潜在的程序崩溃更安全）

1.4 unique_ptr为何优于auto_ptr

```
unique_ptr<string> demo(const char* s)
{
      unique_ptr<string> temp(new string(s));
      return temp;
}
unique_ptr<string> ps;
ps = demo("Uniquely special");
```

- demo()返回一个临时unique_ptr，ps接管了原本归返回的unique_ptr所有的string对象，而返回的unique_ptr被销毁。这样的方式没有出现指向无效数据的悬挂指针
- 如果unique_ptr是临时右值，这种赋值是编译器允许的；但如果源unique_ptr将存在一段时间，编译器将禁止

1.4 unique_ptr为何优于auto_ptr

```
using namespace std;
unique_ptr<string> pu1(new string("Hi ho!"));
unique_ptr<sting> pu2;
pu2 = pu1;  // #1 illegal

unique_ptr<string> pu3;
pu3 = unique_ptr<string>(new string("Yo!"));// #2 legal
```

- 语句#1将留下悬挂的unique_ptr(pul)，导致危害，因此不允许
- 语句#2调用unique_ptr构造函数，创建的临时对象将所有权转让给pu3后就会被销毁，因此是允许
- 因此，建议在容器对象中禁止使用auto_ptr，但允许使用unique_ptr

1.4 unique_ptr为何优于auto_ptr

- C++中有一个标准库函数std::move(),可以将一个unique_ptr赋给另一个unique_ptr
- C++新增的移动构造函数和右值引用，来确保unique_ptr的安全使用

```
using namespace std;
unique_ptr<string> ps1, ps2;
ps1 = demo("Uniquely special");
ps2 = move(ps1); // transfer ownership
ps1 = demo(" and more");
cout << *ps2 << *ps1 << endl;
```

- unique_ptr有一个可用于数组的变体

1.4 unique_ptr为何优于auto_ptr

	指向栈内变量	指向堆内（动态内存）变量 new/delete	指向堆内（动态内存）变量数组 new[]/delete[]
auto_ptr	×	√	×
shared_ptr	×	√	×
unique_ptr	×	√	√

```
std::unique_ptr<double[]>pda(new doubel(5)); // will use delete[] to release memory
```

1.5 选择智能指针

	选择智能指针的情况	
shared_ptr	如果程序需要使用多个指向同一个对象的指针时，选择shared_ptr	共享所有权。一个shared_ptr指向的资源可以被多个shared_ptr实例共享。这是通过内部的引用计数机制来实现的。当最后一个拥有资源的 shared_ptr 被销毁时，资源才会被释放
unique_ptr	如果程序不需要多个指向同一个对象的指针时，可使用 unique_ptr	独占所有权。一个unique_ptr 指向的资源只能被一个 unique_ptr 实例拥有。当尝试将一个 unique_ptr赋值给另一个 unique_ptr 或者传递给函数时，默认的行为是转移所有权，这要求源 unique_ptr 被销毁（即不再拥有资源），并且目标 unique_ptr 开始拥有这个资源。这种转移所有权的操作不允许复制，只允许移动

1.5 选择智能指针

- 如果程序需要使用多个指向同一个对象的指针时，选择shared_ptr

具体指：

- 有一个指针数组，并使用一些辅助指针来标识特定的元素，如最大元素和最小元素
- 两个对象包含指向第三个对象的指针
- STL容器包含指针

1.5 选择智能指针

- 如果程序不需要多个指向同一个对象的指针时，可使用unique_ptr

具体指：

- 如果函数使用new分配内存，并返回指向该内存的指针，将其返回类型声明为unique_ptr是不错的选择。当所有权转让给接受返回值的unique_ptr，该智能指针将负责调用delete
- 可以将unique_ptr存储到STL容器中，只要不调用将一个unique_ptr复制或赋值给另一个的方法或算法（如sort()）

1.5 选择智能指针

```
unique_ptr<int> make_int(int n)
{
        return unique_ptr<int>(new int(n));
}
void show(unique_ptr<int> &p)        // pass by reference
{
        cout << *a << ' ';
}
int main()
{       ...
        vector<unique_ptr<int>> vp(size);
        for (int i = 0; i < vp.size(); i++)
                vp[i] = make_int(rand()%1000;      //copy temporary unique_ptr
        vp.push_back(make_int(rand() % 1000));  //ok because arg is temporary
        for_each(vp.begin(), vp.end(), show);  //use for_each() to show all elements
        ...
}
```

1.5 选择智能指针

```
unique_ptr<int> pup(make_int(rand() % 1000));//ok
```

- 创建了一个 unique_ptr<int> 实例 pup，它拥有通过 make_int返回的 unique_ptr<int> 指针

```
shared_ptr<int> spp(pup);// not allowed, pup an lvalue (left value)
shared_ptr<int> spp(move(pup)); // ok, moves ownership from pup to spp
```

- 试图将一个unique_ptr直接转换为shared_ptr，这是不允许的。因为unique_ptr不能被复制，它只能被移动。如果想要将unique_ptr 转换为shared_ptr，需要使用 move 来转移所有权

```
shared_ptr<int> spr(make_int(rand() % 1000));//ok
```

- 直接创建了一个shared_ptr，它拥有一个 int 对象，因为 make_int 返回的是一个右值unique_ptr<int> ，可以被用来初始化shared_ptr（通过shared_ptr的一个显式构造函数进行转换）

目录

- 智能指针模板类
- 标准模板库

目录

- 标准模板库
  - 基本概念
  - 模板类vector
  - 可对矢量执行的操作
  - 对矢量可执行的其他操作
  - 基于范围的for循环

2.1 基本概念

- STL提供了一组表示容器、迭代器、函数对象和算法的模板
  - 容器是一个与数组类似的单元，可以存储若干个值。STL容器是同质的，即存储的值的类型相同
  - 迭代器能够用来遍历容器的对象，与能够遍历数组的指针，是广义的指针
  - 函数对象可以是类对象或函数指针（包括函数名，函数名被用作指针）
  - 算法是完成特定任务的方法
- STL能够构造各种容器（包括数组、队列和链表）和执行各种操作（包括搜索、排序和随机排列）

2.2 模板类vector

- 模板类vector类似于string类，是一种动态数组。它是使用new创建动态数组的替代品。本质上，vector类使用new和delete来管理内存，但这些工作是自动完成的
  - 要使用vector对象，必须要声明头文件vector
  - vector包含在名称空间std中
  - 模板使用不同的语法来指出它存储的类型
  - vector类使用不同的语法来指定元素数

2.2 模板类vector

- 声明创建一个名为vt的vector对象，它可存储n_elem个类型为typeName的元素，n_elem可以是整型常量，也可以是整型变量

```
vector <typeName> vt (n_elem);
```

```
#include<vector>
...
using namespace std;
vector<int> vi;//create a zero-size array of int
int n;
cin >> n;
vector<double> vd(n);//create an array of n doubles
```

2.2 模板类vector

- vector类的功能比数组强大，但代价是效率稍低。如果需要固定长度的数组，使用数组是更佳的选择，但代价是不那么方便和安全
- C++11新增了模板类array，它也位于名称空间std中
- 和数组一样，array对象的长度也固定，也使用栈（静态内存分配），而不是堆（自由存储区），因此效率与数组相同，但更方便，更安全
- 需要包含头文件array，array对象的创建语法与vector稍有不同

2.2 模板类vector

- 与创建vector对象不同，n_elem不能是变量

```
array <typeName, n_elem> arr;
```

```
#include<array>
...
using namespace std;
array <int, 5> ai; // create array object of 5 integers
array <double, 4> ad={1.2, 2.1, 3.43, 4.3} ;
```

```
// choices.cpp -- array variations                                    //后页续
#include <iostream>
#include <vector>   // STL C++98
#include <array>    // C++0x
int main()
{
    using namespace std;
// C, original C++
    double a1[4] = {1.2, 2.4, 3.6, 4.8};
// C++98 STL
    vector<double> a2(4);   // create vector with 4 elements
// no simple way to initialize in C98
    a2[0] = 1.0/3.0;
    a2[1] = 1.0/5.0;
    a2[2] = 1.0/7.0;
    a2[3] = 1.0/9.0;
// C++0x -- create and initialize array object
    array<double, 4> a3 = {3.14, 2.72, 1.62, 1.41};
    array<double, 4> a4;
    a4 = a3;     // valid for array objects of same size
```

```
// choices.cpp -- array variations                                    //续前页
// use array notation
    cout << "a1[2]: " << a1[2] << " at " << &a1[2] << endl;
    cout << "a2[2]: " << a2[2] << " at " << &a2[2] << endl;
    cout << "a3[2]: " << a3[2] << " at " << &a3[2] << endl;
    cout << "a4[2]: " << a4[2] << " at " << &a4[2] << endl;
// misdeed
    a1[-2] = 20.2;
    cout << "a1[-2]: " << a1[-2] <<" at " << &a1[-2] << endl;
    cout << "a3[2]: " << a3[2] << " at " << &a3[2] << endl;
    cout << "a4[2]: " << a4[2] << " at " << &a4[2] << endl;

    return 0;
}
```

```
a1[2]: 3.6 at 000000192651F458
a2[2]: 0.142857 at 000001DCE6545C60
a3[2]: 1.62 at 000000192651F4D8
a4[2]: 1.62 at 000000192651F518
a1[-2]: 20.2 at 000000192651F438
a3[2]: 1.62 at 000000192651F4D8
a4[2]: 1.62 at 000000192651F518
```

- 数组、vector对象还是array对象，都可以使用标准数组表示来访问各个元素（相同点）（注意：运算符[]被重载的原因）
- 从地址可知，array对象和数组对象存储在相同的内存区域（即栈）中，而vector对象存储在另一个区域（自由存储区域或堆中）（区别）
- 可以将一个array对象赋给另一个array对象；而对于数组，必须逐个元素复制（区别）
- vector和array对象，通过成员函数at(),begin(),end()等，来确定边界，以免无意超界（区别）

```
// vect1.cpp -- introducing the vector template                    //后页续

#include <iostream>
#include <string>
#include <vector>

const int NUM = 5;
int main()
{
    using std::vector;
    using std::string;
    using std::cin;
    using std::cout;
    using std::endl;

    vector<int> ratings(NUM);
    vector<string> titles(NUM);
    cout << "You will do exactly as told. You will enter\n"
         << NUM << " book titles and your ratings (0-10).\n";
```

```
// vect1.cpp -- introducing the vector template                    //续前页

    int i;
    for (i = 0; i < NUM; i++)
    {
        cout << "Enter title #" << i + 1 << ": ";
        getline(cin, titles[i]);
        cout << "Enter your rating (0-10): ";
        cin >> ratings[i];
        cin.get();
    }
    cout << "Thank you. You entered the following:\n"
         << "Rating\tBook\n";
    for (i = 0; i < NUM; i++)
    {
        cout << ratings[i] << "\t" << titles[i] << endl;
    }
    return 0;
}
```

```
You will do exactly as told. You will enter
5 book titles and your ratings (0-10).
Enter title #1: The Cat Who Knew C++
Enter your rating (0-10): 6
Enter title #2: Felonious Felines
Enter your rating (0-10): 4
Enter title #3: Warlords of Wonk
Enter your rating (0-10): 3
Enter title #4: Don't Tounch That Metaphor
Enter your rating (0-10): 5
Enter title #5: Panic Oriented Programming
Enter your rating (0-10): 8
Thank you. You entered the following:
Rating  Book
6       The Cat Who Knew C++
4       Felonious Felines
3       Warlords of Wonk
5       Don't Tounch That Metaphor
8       Panic Oriented Programming
```

- 该程序创建两个vector对象--一个是int规范，另一个是string规范，都包含5个元素
- 该程序使用vector模板只是为方便创建动态分配的数组

2.3 可对矢量执行的操作

- 除分配存储空间外，vector模板还可以完成的操作（所有STL容器提供的一些基本方法）
  - size():返回容器中元素数目
  - swap():交换两个容器的内容
  - begin():返回一个指向容器中第一个元素的迭代器
  - end():返回一个表示超过容器尾的迭代器

迭代器，是广义的指针，它可以是指针，也可以是可对其执行类似指针的操作，例如：

解引用（operator*()）和
递增（operator++()）的对象

2.3 可对矢量执行的操作

- 每个容器类都定义了一个合适的迭代器，该迭代器的类型是一个名为iterator的typedef，其作用域为整个类

```
vector<double>::iterator pd; // pd is an iterator for type double

vector<double> scores; //scores是一个vector<double>对象
pd = scores.begin(); // set pd point to the first element of scores
*pd = 22.3; // set the value of the first element of scores to 22.3
pd++; // increment pd to point to the next element of scores
```

- 结合C++11自动类型推断

```
vector<double>::iterator pd = scores.begin();
auto pd = scores.begin(); //C++11 automatic type deduction (与前句等价)
```

2.3 可对矢量执行的操作

- 超过结尾（past-the-end）迭代器，指向容器最后一个元素后面那个元素。end()成员函数标识超过结尾的位置
- vector模板类中只有某些STL容器才有的方法：
  - push_back()：将元素添加到矢量末尾，负责内存管理，增加矢量的长度
  - erase()：删除矢量中给定区间的元素，它接受两个迭代器参数，定义要删除的区间
  - insert()：接受三个迭代器参数，第一个参数指定新元素的插入位置，第二和第三个迭代器参数定义了被插入区间，该区间通常是另一个容器对象的一部分

2.3 可对矢量执行的操作

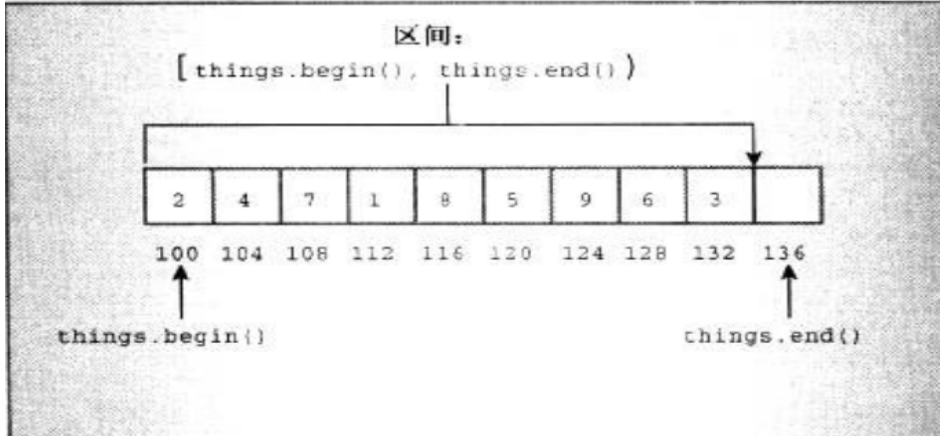
```
for(pd = scores.begin(); pd != scores.end(); pd++)//遍历整个容器内容
      cout << *pd << endl;
```

```
//每次循环都给scores对象增加一个元素，无需了解元素数目
vector<double> scores; // create an empty vector
double temp;
while (cin >> temp && temp >= 0)
      scores.push_back(temp);
cout << "You entered " << scores.size() << " scores\n";
```

```
//删除begin()和begin()+1指向的元素
scores.erase(scores.begin(), scores.begin() + 2);
```

注意：迭代器指定的begin()到begin()+2，其范围为begin()到begin()+2（但不包括begin()+2）

2.3 可对矢量执行的操作



STL的区间概念

```
//将矢量new_v中除第一个元素外的所有元素插入到old_v矢量的第一个元素前面:
vector<int> old_v;
vector<int> new_v;
...
old_v.insert(old_v.begin(), new_v.begin()+1, new_v.end());

//将矢量new_v中除第一个元素外的所有元素插入到old_v矢量的最后一个元素后面:
old_v.insert(old_v.end(), new_v.begin() + 1, new_v.end());
```

2.4 对矢量可执行的其他操作

- STL定义了非成员（non-memner）函数来执行如搜索、排序、随机排序等操作
- 有时即使有执行相同任务的非成员函数，STL也会定义一个成员函数。因为，成员函数的效率比通用非成员函数高
  - 如vector的成员函数swap()的效率比非成员函数swap()高，但非成员函数能交换两个不同类型的容器的内容
- 具有代表性的STL函数：for_each()、random_shuffle()和sort()

2.4 对矢量可执行的其他操作

- for_each()：可用于多容器类，接受3个参数。前两个定义容器中区间的迭代器，最后一个指向函数的指针（函数对象）。该函数将被指向的函数应用于容器区间中的各个元素，被指向函数不能修改容器元素值，可用来替代for循环

```
vector<Review>::iterator pr;
for (pr = books.begin(); pr != books.end(); pr++)
      ShowReview(*pr);
```



等价

```
for_each(books.begin(), books.end(), ShowReview);
```

2.4 对矢量可执行的其他操作

- random_shuffle()：接受2个指定区域的迭代器参数，并随机排列该区域中的元素。该函数要求容器类允许随机访问，vector类满足

```
random_shuffle(books.begin(), books.end());
```

- sort()：也要求容器支持随机访问
  - 接受两个定义区间迭代器参数，并使用为存储在容器中的类型元素定义的比较运算符operator<，对区间中的元素进行操作
  - 接受3个参数版本，前两参数也是指定区间的迭代器，最后一个参数是指向要使用的函数的指针（函数对象），不使用比较运算符operator<

2.4 对矢量可执行的其他操作

```
//sort()的2参数版本

vector<int>coolstuff;
...
sort(coolstuff.begin(), coolstuff.end());

bool operator<(const Review& r1, const Review& r2)
{
      if (r1.title < r2.title)
            return true;
      else if (r1.title == r2.title && r1.rating < r2.rating)
            return true;
      else
            return false;
}
```

如果title成员都相同，则再按rating排序比较

```
struct Review {
    std::string title;
    int rating;
};
```

2.4 对矢量可执行的其他操作

```
//sort()的3参数版本

bool WorseThan(const Review& r1, const Review& r2)
{
      if (r1.rating < r2.rating)
            return true;
      else
            return false;
}
sort(books.begin(), books.end(), WorseThan);
```

注意：与operator<()相比，WorseThan()函数执行的对Review对象进行排序的工作不那么完整

全排序（total ordering），是相同

完整弱排序（strict weak ordering），是等价

2.5 基于范围的for循环（C++11）

```
double prices[5] = { 4.99, 10.99, 6.87, 7.99, 8.49 };
for (double x : prices)
      cout << x << endl;
```

```
for_each(books.begin(), books.edn(), ShowReview));
```

```
for (auto x : books) ShowReview(x);
```

```
void InflateReview(Review & r) { r.rating++; }
```

```
for (auto x : books) InflateReview(x);
```

总结

- 智能指针模板类（了解）
  - 基本概念
  - 使用智能指针
  - 有关智能指针的注意事项
  - unique_ptr为何优于auto_ptr
  - 选择智能指针
- 标准模板库（掌握）
  - 基本概念
  - 模板类vector
  - 可对矢量执行的操作
  - 对矢量可执行的其他操作
  - 基于范围的for循环

第十一章 泛型编程

主讲教师：同济大学计算机科学与技术学院 陈宇飞
同济大学计算机科学与技术学院 龚晓亮

目录

- STL基本概念
- 使用迭代器
- 迭代器类型
- 迭代器结构
- 容器种类

目录

- STL基本概念

1.1 STL基本概念

- STL（Standard Template Library，标准模板库）是一种泛型编程（generic programming），它允许程序员编写与数据类型无关的代码，从而提高代码的复用性和灵活性
- STL是泛型编程在C++中的具体实现，它利用模板（包括类模板和函数模板）技术，实现了数据结构和算法的分离

	面向对象编程	泛型编程
编程关注点	数据	算法
使用方法		模板 （包括类模板和函数模板）
共同点	抽象和创建可重用代码	

1.1 STL基本概念

STL的代码从广义上讲分为三类：

- 容器（container）：是一种数据结构用于存储数据，并提供了各种方法来访问和操作这些数据，如list、vector和deque等，以模板类的方式提供
- 算法（algorithm）：对容器中的数据进行操作和处理的函数，它们被实现为模板函数，可以操作不同类型的数据，如sort()函数来对一个vector中的数据进行排序，提供了find()函数来搜索一个list中的对象
- 迭代器（iterator）：是访问容器中元素的方式。迭代器是广义的指针，但比指针更灵活，适用于不同类型的容器。通过使用迭代器，程序员可以遍历容器中的数据，并对其进行操作

目录

- 使用迭代器

2.1 使用迭代器

```
double* find_ar(double* ar, int n, const doubel& val)
{
        for (int i = 0; i < n; i++)
                if (ar[i] == val)
                        return &ar[i];
        return 0; // or , in C++11, return nullptr;
}
```

两种不同数据表示实现find函数

与特定数据结构数组关联

与特定数据结构链表关联

```
struct Node
{
        double item;
        Node* p_next;
};
Node* find_ll(Node* head, const double& val)
{
        Node* start;
        for (start=head; start!=0; start = start->p_next)
                if (start->item == val) return start;
        return 0;
}
```

2.1 使用迭代器

- 泛型编程旨在使用同一个函数来处理数组、链表或任何其他容器类型
- 函数不仅独立于容器中存储的数据类型，而且独立于容器本身的数据结构
- 模板提供了存储在容器中的数据类型的通用表示，再用通用表示（即迭代器）来遍历容器中的值
- 在STL（标准模板库）中使用迭代器提供了一种统一和灵活的方式来访问和操作容器中的元素

2.1 使用迭代器

迭代器应具备的特征：

- 应能够对迭代器执行解除引用的操作，以便能够访问它的引用的值。即如果p是一个迭代器，则应对*p进行定义
- 应能够将一个迭代器赋给另一个。即如果p和q都是迭代器，则应对表达式p=q进行定义
- 应该能够将一个迭代器与另一个进行比较，看它们是否相等。即如果p和q都是迭代器，则应对p==q和p!=q进行定义
- 应能够使用迭代器遍历容器中的所有元素，这可以通过为迭代器p定义++p和p++来实现

2.1 使用迭代器

```
//常规指针满足迭代器要求
typedef double* iterator;
iterator find_ar(iterator ar, int n, const double& val) {
        for (int i = 0; i < n; i++,ar++)
                if (*ar == val)
                        return ar;
        return 0;
}
```

使用迭代器实现find_ar函数

```
//接受两个指示区间的指针参数
typedef double* iterator;
iterator find_ar(iterator begin, iterator end, const double& val)
{
        iterator ar;
        for (ar = begin; ar!=end;ar++)
                if (*ar == val)
                        return ar;
        return end;                     // indicates val not found
}
```

```
struct Node
{
        double item;
        Node* p_next;
};
//定义一个迭代器类，并在其中定义了运算符*和++
class iterator
{
        Node* pt;
public:
        iterator() : pt(0) {};
        iterator(Node* pn) : pt(pn) {}
        double& operator*() { return pt->item; }
        iterator& operator++()                  // for ++it
        {       pt = pt->p_next;
                return *this; }
        iterator& operator++(int)               // for it++
        {       iterator tmp = *this;
                pt = pt->p_next;
                return tmp;
        }
}
```

```
//find函数修改如下:
iterator find_ll(iterator head, const double& val)
{
        iterator start;
//到达最后一个值的判据：存储在最后一个节点中的空值
        for (start = head; start != 0; ++start )
                if (*start == val)
                        return start;
        return 0;
}
```

使用迭代器实现find_ll函数

2.1 使用迭代器

```
//find函数修改如下:
iterator find_ll(iterator head, const double& val)
{
        iterator start;
//到达最后一个值的判据：存储在最后一个节点中的空值
        for (start = head; start != 0; ++start )
        if (*start == val) return start;
        return 0;
}
```

```
//接受两个指示区间的指针参数
typedef double* iterator;
iterator find_ar(iterator begin, iterator end, const double& val)
{
        iterator ar;
 //到达最后一个值的判据：超尾迭代器
        for (ar = begin; ar != end; ar++)
                if (*ar == val) return ar;
        return end;
}
```

2.1 使用迭代器

- STL通过为每个类定义适当的迭代器，并以统一的风格设计类，能够对内部表示绝然不同的容器编写相同的代码

```
vector<double>::iterator pr;
for (pr = scores.begin(); pr != scores.end(); pr++)
     cout << *pr << end;
```

```
list<double>::iterator pr;
for (pr = scores.begin(); pr != scores.end(); pr++)
     cout << *pr << end;
```

- 使用C++11新增的自动类型推断可进一步简化，对于矢量或列表，都可使用如下代码：

```
for (auto pr = scores.begin(); pr != scores.end(); pr++)
     cout << *pr << endl;
```

2.1 使用迭代器

- 实际上，作为一种编程风格，最好避免直接使用迭代器，而应尽可能使用STL函数（如for_each()）
- 也可使用C++11新增的基于范围的for循环：

```
for (auto x : scores)
      cout <<*pr << endl;
```

2.1 使用迭代器

总结STL方法：

- 处理容器的算法，应尽可能用通用的术语来表达算法，使之独立于数据类型和容器类型
- 为使通用算法能够适用于具体情况，应定义能够满足算法需求的迭代器，并把要求加到容器设计上
- 基于算法的要求，设计基本迭代器的特征和容器特征

目录

- 迭代器类型
  - 迭代器类型介绍
  - 输入迭代器
  - 输出迭代器
  - 正向迭代器
  - 双向迭代器
  - 随机访问迭代器

3.1 迭代器类型介绍

- 不同的算法对迭代器的要求也不同
  - 如，查找算法需要定义++运算法，以便迭代器能够遍历整个容器
  - 它要求能够读取数据，但不要求能够写数据（它只是查看数据，而并不修改数据）
  - 而排序算法要求能够随机访问，以便能够交换两个不相邻的元素

3.1 迭代器类型介绍

- STL定义了5种迭代器： 输入迭代器、输出迭代器、正向迭代器、双向迭代器和随机访问迭代器
  - 已经为它们定义了*运算符，可以执行解除引用操作
  - 也可以进行比较，看其是相等还是不相等

3.2 输入迭代器

- 对输入迭代器进行解引用将使程序读取容器中的值
- 需要输入迭代器的算法将不会修改容器中的值
  - ➢输入迭代器必须能够访问容器中所有的值（通过++运算符来实现）
  - ➢基于输入迭代器的任何算法都应当是单通行（single-pass），不依赖于前一次遍历时的迭代器值，也不依赖于本次遍历中前面的迭代器值
  - ➢注意：输入迭代器是单向迭代器，可以递增，但不能倒退

```
template<class InputIterator, class T>
InputIterator find(InputIterator first, InputIterator last, const T& value)
{
        while (first != last && *first != value) {
                ++first;
        }
        return first;
}
```

```
//输入迭代器只能顺序读取容器中的元素，并且只能向前移动一次（单次传递算法）
//常见的输入迭代器包括指向常量数据的指针和某些输入流迭代器
#include <iostream>
#include <vector>
#include <iterator>
int main() {
    std::istream_iterator<int> input_it(std::cin);
    std::istream_iterator<int> end;
    std::vector<int> numbers;

    std::cout << "Enter integers (Ctrl+D to end input): ";
    std::copy(input_it, end, std::back_inserter(numbers));

    std::cout << "You entered: ";
    for (int num : numbers)
        std::cout << num << " ";
    std::cout << std::endl;
    return 0;
}
```

```
1 2
Enter integers (Ctrl+D to end input): 43
334
455
^D
You entered: 1 2 43 334 455
```

3.3 输出迭代器

- 输出迭代器的解引用让程序能修改容器值，用于将信息从程序输给容器，但不能读取
- 对于单通行、只读算法，可以使用输入迭代器；而对于单通行、只写算法，则可以使用输出迭代器

```
#include <iostream>
#include <vector>
#include <iterator>
int main() {
    std::vector<int> numbers = {1, 2, 3, 4, 5};
    std::ostream_iterator<int> output_it(std::cout, " ");

    std::cout << "Numbers: ";
    std::copy(numbers.begin(), numbers.end(), output_it);
    std::cout << std::endl;

    return 0;
}
```

```
Numbers: 1 2 3 4 5
```

3.4 正向迭代器

- 正向迭代器允许读取和写入序列中的元素，且只能单向遍历，但可以多次遍历相同的序列。std::vector、std::list等容器的迭代器都是正向迭代器
- 正向迭代器与输入迭代器和输出迭代器相似，只使用++运算符来遍历容器，所以每次沿着容器向前移动一个元素；不同的是，正向迭代器总按相同的顺序遍历一系列值
- 将正向迭代器递增后，仍然可以对前面迭代器值解除引用（如果保存了它），并可以得到相同的值
- 正向迭代器既可以读取和修改数据，也可以只读数据

```
int * pirw;  // read-write iterator
const int *pir;  // read-only iterator
```

```cpp
#include <iostream>
#include <vector>
int main() {
    std::vector<int> numbers = {1, 2, 3, 4, 5};
    std::cout << "Numbers: ";
    for (std::vector<int>::iterator it = numbers.begin(); it != numbers.end(); ++it) {
        std::cout << *it << " ";
    }
    std::cout << std::endl;

    // 修改元素
    for (std::vector<int>::iterator it = numbers.begin(); it != numbers.end(); ++it) {
        *it *= 2;
    }
    std::cout << "Modified numbers: ";
    for (int num : numbers)
        std::cout << num << " ";
    std::cout << std::endl;
    return 0;
}
```

```
Numbers: 1 2 3 4 5
Modified numbers: 2 4 6 8 10
```

3.5 双向迭代器

- 双向迭代器允许读取和写入序列中的元素，且可以双向遍历。std::list、std::set等容器的迭代器都是双向迭代器
- 双向迭代器具有正向迭代器的所有特性，同时支持两种（前缀和后缀）递减运算符

```cpp
#include <iostream>
#include <list>

int main() {
    std::list<int> numbers = { 1, 2, 3, 4, 5 };
    // 正向遍历
    std::cout << "Numbers (forward): ";
    for (std::list<int>::iterator it = numbers.begin(); it != numbers.end(); ++it) {
        std::cout << *it << " ";
    }
    std::cout << std::endl;
    // 反向遍历
    std::cout << "Numbers (backward): ";
    for(std::list<int>::reverse_iterator rit = numbers.rbegin(); rit != numbers.rend(); ++rit)
        std::cout << *rit << " ";

    std::cout << std::endl;

    return 0;
}
```

```
Numbers (forward): 1 2 3 4 5
Numbers (backward): 5 4 3 2 1
```

3.6 随机访问迭代器

- 随机访问迭代器允许读取和写入序列中的元素，且支持常数时间内的任意位置访问（如通过索引访问）。std::vector、std::deque等容器的迭代器都是随机访问迭代器
- 表中a和b都是迭代器值，n为整数，r为随机迭代器变量或引用

随机访问迭代器操作

表 达 式	描 述
a + n	指向 a 所指向的元素后的第 n 个元素
n + a	与 a + n 相同
a - n	指向 a 所指向的元素前的第 n 个元素
r += n	等价于 r = r + n
r -= n	等价于 r = r - n
a[n]	等价于*(a + n)
b - a	结果为这样的 n 值，即 b = a + n
a < b	如果 b - a > 0，则为真
a > b	如果 b < a，则为真
a >= b	如果 !(a < b)，则为真
a <= b	如果 !(a > b)，则为真

```cpp
#include <iostream>
#include <vector>
int main() {
    std::vector<int> numbers = { 1, 2, 3, 4, 5 };
    // 访问第一个元素
    std::cout << "First element: " << numbers.front() << std::endl;
    // 访问最后一个元素
    std::cout << "Last element: " << numbers.back() << std::endl;
    // 通过索引访问元素
    std::cout << "Element at index 2: " << numbers[2] << std::endl;
    // 修改元素
    numbers[3] = 10;

    // 遍历并打印所有元素
    std::cout << "Modified numbers: ";
    for (int num : numbers) {
        std::cout << num << " ";
    }
    std::cout << std::endl;
    return 0;
}
```

```
First element: 1
Last element: 5
Element at index 2: 3
Modified numbers: 1 2 3 10 5
```

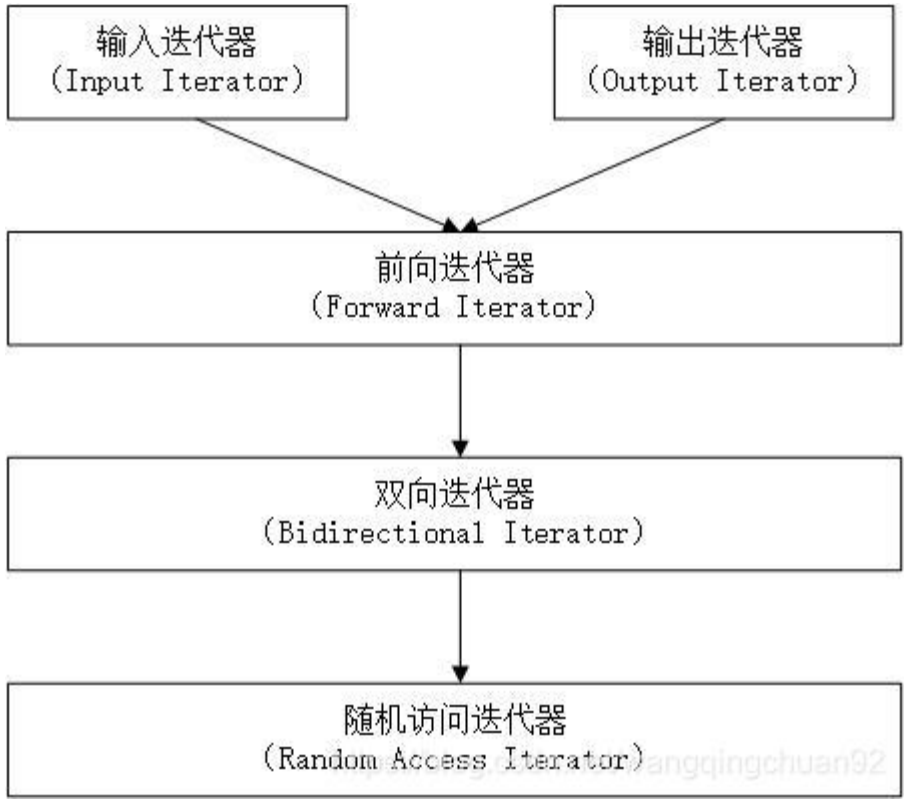
目录

- 迭代器结构
  - 迭代器基本结构
  - 概念-改进-模型
  - 将指针用作迭代器
  - 其他有用的迭代器

4.1 迭代器基本结构

- 正向迭代器具有输入迭代器和输出迭代器的全部功能，同时还有自己的功能；
- 双向迭代器具有正向迭代器的全部功能，同时还具有自己的功能
- 随机访问迭代器具有正向迭代器的全部功能，同时还具有自己的功能

见下表



4.1 迭代器基本结构

迭代器功能	输入	输出	正向	双向	随机访问
解除引用读取	√		√	√	√
解除引用写入		√	√	√	√
固定和可重复排序			√	√	√
++i，i++	√	√	√	√	√
--i，i--				√	√
i[n]					√
i+n					√
i-n					√
i+=n					√
i-=n					√

4.2 概念-改进-模型

- STL的文献中使用术语“概念（Concept）”来描述一列要求，这些要求定义了迭代器必须满足的行为和属性。例如：
  - 正向迭代器的概念包含这样的要求：正向迭代器能够被解除引用，以便读写，同时能够被递增
- 概念是对迭代器行为的一种抽象描述，它使得STL算法可以独立于具体的容器类型进行实现

4.2 概念-改进-模型

- 在STL中，有些概念是基于其他概念构建的，这种基于其他概念的概念被称为“改进（Refinement）”。例如：
  - 双向迭代器是正向迭代器概念的改进，它继承了正向迭代器的所有要求，并添加了能够递减的能力
- 这种改进关系使得STL能够以一种层次结构来组织迭代器类型，从而提高了算法和容器的灵活性

4.2 概念-改进-模型

- 概念的具体实现被称为“模型（Model）”。在STL中，模型是满足特定概念要求的迭代器或容器的具体实例。例如：
  - 指向int的常规指针是一个随机访问迭代器模型，因为它满足随机访问迭代器概念的所有要求
  - vector容器是一个序列容器模型，因为它满足序列容器概念的所有要求

4.3 将指针用作迭代器

- 迭代器是STL算法的接口，而指针是迭代器，因此STL算法可以使用指针来对基于指针的非STL容器进行操作

```
cosnt int SIZE =100;
double Receipts[SIZE];
//sort()函数接受指向容器第一个元素的迭代器和指向超尾的迭代器作为参数
sort(Receipts,Receipts + SIZE);
```

- 指针是迭代器，而算法是基于迭代器的，这使得可将STL算法用于常规数组
- 可以将STL算法用于自己设计的数组形式，只要提供适当的迭代器（可以是指针，也可以是对象）和超尾指示器即可

4.3 将指针用作迭代器

- STL提供了一些预定义迭代器，如copy()可以将数据从一个容器复制到另一个容器中

```
int casts[10] = {6, 7, 2, 9, 4, 11, 8, 7, 10, 5};
vector<int> dice(10);
copy(casts, casts + 10, dice.begin()); // copy array to vector
```

- 假设要将信息复制到显示器上，如果有一个表示输出流的迭代器，则可以使用copy()
- STL 为这种迭代器提供了ostream_iterator模板，它是输出迭代器概念的一个模型，也是一个适配器（adapter）--一个类或函数，可以将一些其他接口转换为STL使用的接口

4.3 将指针用作迭代器

- 创建迭代器

```
#include <iterator>
...
ostream_iterator<int,char> out_iter(cout, " ");
```

- 第一个模板参数(这里为int)指出了被发送给输出流的数据类型
- 第二个模板参数(这里为char)指出了输出流使用的字符类型(另一个可能的值是wchar_t)
- 构造函数的第一个参数(这里为cout)是要使用的输出流，它也可以是用于文件输出的流
- 最后一个字符串参数是在发送给输出流的每个数据项目后显示的分隔符

4.3 将指针用作迭代器

```
#include <iterator>
...
ostream_iterator<int,char> out_iter(cout, " ");
//out_iter迭代器现在是一个接口，能够使用cout来显示信息
```

```
*out_iter++ = 15; // works like cout<< 15 << " ";
copy(dice.begin(),dice.end, out_iter); //copy vector to output stream
//也可以不创建命名的迭代器，直接构建一个匿名迭代器
copy(dice.begin(),dice.end, ostream_iterator<int,char> (cout, " "));
```

4.4 其他有用的迭代器

- 除了ostream_iterator和istream_iterator之外，头文件iterator还提供了其他一些专用的预定义迭代器类型
  - reverse_iterator
  - back_insert_iterator
  - front_insert_iterator
  - insert_iterator

4.4 其他有用的迭代器

- 除了ostream_iterator和istream_iterator之外，头文件iterator还提供了其他一些专用的预定义迭代器类型
  - reverse_iterator

	rbegin()	end()	rend()	begin()
返回值	vector的超尾	vector的超尾	vector的第一个元素	vector的第一个元素
返回类型	reverse_iterator	iterator	reverse_iterator	iterator

```
ostream_iterator<int,char> out_iter(cout, " ");
copy(dice.begin(),dice.end, out_iter); // display in forward order
copy(dice.rbegin(),dice.rend(),out_iter);//display in reverse order
```

```
// copyit.cpp -- copy() and iterators
#include <iostream>
#include <iterator>
#include <vector>
int main()
{
    using namespace std;
    int casts[10] = { 6, 7, 2, 9 ,4 , 11, 8, 7, 10, 5 };
    vector<int> dice(10);
    // copy from array to vector
    copy(casts, casts + 10, dice.begin());
    cout << "Let the dice be cast!\n";
    // create an ostream iterator
    ostream_iterator<int, char> out_iter(cout, " ");
    // copy from vector to output
    copy(dice.begin(), dice.end(), out_iter);
    cout << endl;
    cout << "Implicit use of reverse iterator.\n";
    copy(dice.rbegin(), dice.rend(), out_iter);
    cout << endl;
    cout << "Explicit use of reverse iterator.\n";
```

```
Let the dice be cast!
6 7 2 9 4 11 8 7 10 5
Implicit use of reverse iterator.
5 10 7 8 11 4 9 2 7 6
Explicit use of reverse iterator.
5 10 7 8 11 4 9 2 7 6
```

- 如果可以在显示声明迭代器和使用STL函数来处理内部问题（如将rbegin()返回值传递给函数）之间选择，推荐采用后者
- 后者方法要做的工作较少，人为出错的机会较少

```
// vector<int>::reverse_iterator ri;
// use if auto doesn't work
    for (auto ri = dice.rbegin(); ri
!= dice.rend(); ++ri)
        cout << *ri << ' ';
    cout << endl;
    return 0;
}
```

4.4 其他有用的迭代器

```
copy(casts, casts + 10, dice.begin()); // copy array to vector会覆盖原内容
```

	相同点	区别
back_insert_iterator	➢ 插入将添加新的元素 ➢ 不会覆盖已有的数据 ➢ 并使用自动内存分配来确保能够容纳新的信息 ➢ 输出容器概念的模型	▪ 将元素插入到容器尾部 ▪ queue类满足
front_insert_iterator		▪ 将元素插入到容器前端 ▪ vector类满足（完成任务速度最快）
insert_iterator		▪ 将元素插入到insert_iterator构造函数的参数指定的位置前面 ▪ 没有限制 ▪ 可以用来将复制数据的算法转换为插入数据的算法

接后页




```
// inserts.cpp -- copy() and insert iterators
#include <iostream>
#include <string>
#include <iterator>
#include <vector>
#include <algorithm>

void output(const std::string& s) { std::cout << s << " "; }

int main()
{
    using namespace std;
    string s1[4] = { "fine", "fish", "fashion", "fate" };
    string s2[2] = { "busy", "bats" };
    string s3[2] = { "silly", "singers" };
    vector<string> words(4);
    copy(s1, s1 + 4, words.begin());
    for_each(words.begin(), words.end(), output);
    cout << endl;
```

```
fine fish fashion fate
```

续前页

```
// construct anonymous back_insert_iterator object
    copy(s2, s2 + 2, back_insert_iterator<vector<string> >(words));
    for_each(words.begin(), words.end(), output);
    cout << endl;

    // construct anonymous insert_iterator object
    copy(s3, s3 + 2, insert_iterator<vector<string> >(words, words.begin()));
    for_each(words.begin(), words.end(), output);
    cout << endl;
    return 0;
}
```

```
fine fish fashion fate busy bats
```

```
silly singers fine fish fashion fate busy bats
```

```
string s1[4] = { "fine", "fish", "fashion", "fate" };
string s2[2] = { "busy", "bats" };
string s3[2] = { "silly", "singers" };
vector<string> words(4);
```

```
fine fish fashion fate
fine fish fashion fate busy bats
silly singers fine fish fashion fate busy bats
```

目录

- 容器种类
  - 容器基本种类
  - 容器概念
  - 序列容器类型
  - 关联容器

5.1 容器基本种类

- STL里容器概念有容器、序列容器、关联容器等的通用类别
- STL里容器类型是可用于创建具体容器对象的模板
  - ➢原有11个容器：deque、list、queue、priority_queue、stack、vector、map、multimap、set、multiset和bitset
  - ➢C++11新增容器：forward_list、unordered_map、unordered_multimap、unordered_set和unordered_multiset，且将bitset视为独立类别而不是容器
    - ✓ bitset是在比特级别处理数据的容器

5.2 容器概念

- 容器概念指定了所有STL容器类都必须满足的一系列要求
  - ➢容器是存储其他对象的对象，被存储对象必须是同一种类型，可以是OOP意义上的对象，也可以是内置类型值
  - ➢但容器过期时，存储在容器中的数据也将过期（但如果数据是指针的话，则它指向的数据并不一定过期）
  - ➢容器中存储类型必须是可复制构造的和可赋值的
    - ✓ 基本类型满足
    - ✓ 只要类定义没有将复制构造函数和赋值运算符声明为私有或保护的，也满足
  - ➢所有容器都提供某些特征和操作（见后页）

5.2 容器概念

表 16.5　　一些基本的容器特征

表达式	返回类型	说明	复杂度
X :: iterator	指向 T 的迭代器类型	满足正向迭代器要求的任何迭代器	编译时间
X :: value_type	T	T 的类型	编译时间
X u;		创建一个名为 u 的空容器	固定
X( );		创建一个匿名的空容器	固定
X u(a);		调用复制构造函数后 u == a	线性
X u = a;		作用同 X u(a);	线性
r = a;	X&	调用赋值运算符后 r == a	线性
(&a)->~X( )	void	对容器中每个元素应用析构函数	线性
a.begin( )	迭代器	返回指向容器第一个元素的迭代器	固定
a.end( )	迭代器	返回超尾值迭代器	固定
a.size( )	无符号整型	返回元素个数，等价于 a.end( ) - a.begin( )	固定
a.swap(b)	void	交换 a 和 b 的内容	固定
a == b	可转换为 bool	如果 a 和 b 的长度相同，且 a 中每个元素都等于（== 为真）b 中相应的元素，则为真	线性
a != b	可转换为 bool	返回!(a == b)	线性

- “复杂度”描述了执行操作所需要的时间
- 从快到慢：编译时间>固定时间>线性时间

5.2 容器概念

表 16.6 C++11 新增的基本容器要求

表达式	返回类型	说明	复杂度
X u(rv);		调用移动构造函数后，u 的值与 rv 的原始值相同	线性
X u = rv;		作用同 X u(rv);	
a = rv;	X&	调用移动赋值运算符后，u 的值与 rv 的原始值相同	线性
a.cbegin( )	const_iterator	返回指向容器第一个元素的 const 迭代器	固定
a.cend( )	const_iterator	返回超尾值 const 迭代器	固定

- 复制构造和复制赋值以及移动构造和移动赋值之间的差别在于，复制操作保留源对象，而移动操作可修改源对象，还可能转让所有权，而不作任何复制
- 如果源对象是临时的，移动操作的效率将高于常规复制

5.2 容器概念

- 可以通过添加要求来改进基本的容器概念。序列（sequence）是一种重要的改进
- deque、forward_list、list、queue、priority_queue、stack和vector都是序列概念的模型
- 在运行的情况下，它们的复杂度是固定时间

表 16.7 序列的要求

表 达 式	返 回 类 型	说 明
X a(n, t);		声明一个名为 a 的由 n 个 t 值组成的序列
X(n, t)		创建一个由 n 个 t 值组成的匿名序列
X a(i, j)		声明一个名为 a 的序列，并将其初始化为区间[i，j)的内容
X(i, j)		创建一个匿名序列，并将其初始化为区间[i，j)的内容
a. insert(p, t)	迭代器	将 t 插入到 p 的前面
a.insert(p, n, t)	void	将 n 个 t 插入到 p 的前面
a.insert(p, i, j)	void	将区间[i，j)中的元素插入到 p 的前面
a.erase(p)	迭代器	删除 p 指向的元素
a.erase(p, q)	迭代器	删除区间[p，q)中的元素
a.clear( )	void	等价于 erase(begin( ), end( ))

5.2 容器概念

表 16.8 序列的可选要求

表 达 式	返 回 类 型	含 义	容 器
a.front( )	T&	*a.begin( )	vector、list、deque
a.back( )	T&	*- -a.end( )	vector、list、deque
a.push_front(t)	void	a.insert(a.begin( ), t)	list、deque
a.push_back(t)	void	a.insert(a.end( ), t)	vector、list、deque
a.pop_front(t)	void	a.erase(a.begin( ))	list、deque
a.pop_back(t)	void	a.erase(- -a.end( ))	vector、list、deque
a[n]	T&	*(a.begin( )+ n)	vector、deque
a.at(t)	T&	*(a.begin( )+ n)	vector、deque

5.3 序列容器类型

（1）vector模板类

- 是动态数组
- 提供了对元素随机访问
- 在尾部添加和删除元素的时间固定，在头部或中间插入或删除元素的复杂度为线性时间

```
char word[4] = "cow";
deque<char>dword(word, word+3);
```

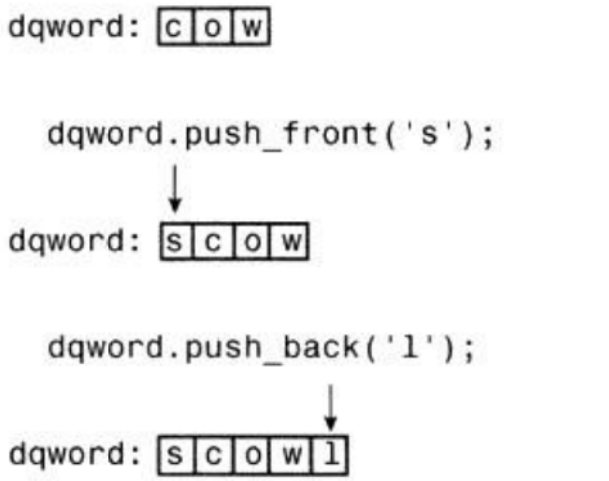


图 16.4 push_front( )和 push_back( )

- 除序列外，vector还是可反转容器（reversible container）概念的模型。增加了两个类方法：rbegin( )和rend( )
- vector 模板是最简单的序列类型，除非其他类型的特殊优点能够更好地满足程序的要求，否则应默认使用这种类型

5.3 序列容器类型

（2）deque模板类

- deque模板类（在deque头文件中），表示双端队列（double-ended queue）
- 如果多数操作发生在序列的起始和结尾处，则应考虑deque数据结构

	vector类	deque
随机访问	√	√
插入和删除元素在头部位置	线性时间	固定时间
插入和删除元素在其他位置	线性时间	线性时间
插入和删除元素在尾部位置	固定时间	线性时间
类的对象设计		更复杂

5.3 序列容器类型

（3）list模板类

- list模板类（在list头文件中），表示双向链表，可以双向遍历链表

	vector类	list
随机访问	√	×
数组[]表示法访问	√	×
序列和可反转容器的函数	√	√
插入和删除元素在头部位置	线性时间	固定时间
插入和删除元素在其他位置	线性时间	固定时间
插入和删除元素在尾部位置	固定时间	固定时间

5.3 序列容器类型

（3）list模板类

- list模板类（在list头文件中），表示双向链表，可以双向遍历链表

表 16.9　　list 成员函数

函　　数	说　　明
void merge(list<T, Alloc>& x)	将链表 x 与调用链表合并。两个链表必须已经排序。合并后的经过排序的链表保存在调用链表中，x 为空。这个函数的复杂度为线性时间
void remove(const T & val)	从链表中删除 val 的所有实例。这个函数的复杂度为线性时间
void sort( )	使用<运算符对链表进行排序；N 个元素的复杂度为 NlogN
void splice(iterator pos, list<T, Alloc>x)	将链表 x 的内容插入到 pos 的前面，x 将为空。这个函数的的复杂度为固定时间
void unique( )	将连续的相同元素压缩为单个元素。这个函数的复杂度为线性时间

5.3 序列容器类型

（4）forward_list模板类

- forward_list模板类，表示单链表
- forward_list只需要正向迭代器，不需要双向迭代器，forward_list是不可反转的容器
- 相比于list，forward_list更简单、更紧凑，功能更少

5.3 序列容器类型

（5）queue模板类

- queue模板类（在头文件queue（以前为queue.h）中声明）是一个适配器类，让底层类（默认情况下为vector）展示典型的列队接口

	vector类	deque类	queue类
随机访问	√	√	×
遍历	√	√	×

5.3 序列容器类型

（5）queue模板类

- queue模板类（在头文件queue（以前为queue.h）中声明）是一个适配器类，让底层类（默认情况下为vector）展示典型的列队接口

表 16.10　　queue 的操作

方　法	说　明
bool empty( )const	如果队列为空，则返回 true；否则返回 false
size_type size( )const	返回队列中元素的数目
T& front( )	返回指向队首元素的引用
T& back( )	返回指向队尾元素的引用
void push(const T& x)	在队尾插入 x
void pop( )	删除队首元素

5.3 序列容器类型

（6）priority_queue模板类

- priority_queue模板类（在头文件queue中声明）是另一个适配器类，它支持的操作与queue相同
- 区别在于，在priority_queue中，最大的元素被移到队首，可以修改用于确定哪个元素放到队首的比较方式，方法是提供一个可选的构造函数参数：

```
priority_queue<int> pq1;  //default version
priority_queue<int> pq2(greater<int>); // use greater<int> to order
//greater<>()函数是一个预定义的函数对象
```

5.3 序列容器类型

（7）stack模板类

- 与queue相似，stack（在头文件stack（以前是stack.h）中声明）也是一个适配器类，它给底层类（默认情况下为vector）提供典型的栈接口
- 它不允许随机访问栈元素，甚至不允许遍历栈
- 它把使用限制在定义栈的基本操作上，即可以将压入推到栈顶、从栈顶弹出元素、查看栈顶的值、检查元素数目和测试栈是否为空

表 16.11　　stack 的操作

方　　法	说　　明
bool empty( )const	如果栈为空，则返回 true；否则返回 false
size_type size( )const	返回栈中的元素数目
T& top( )	返回指向栈顶元素的引用
void push(const T& x)	在栈顶部插入 x
void pop( )	删除栈顶元素

5.3 序列容器类型

（8）array模板类

- array是在头文件array中定义的，它并非STL 容器，因为其长度固定
- 因此array没有定义调整容器大小的操作（如push_back()和insert()），但定义了有意义的成员函数，如operator[]()和at
- 可将很多标准STL算法用于array对象，如copy()和 for_each()

5.4 关联容器

- 关联容器（associative container）是对容器概念的另一个改进
- 关联容器将值和键关联起来，并使用键来查找值
  - 值可以使表示雇员信息的结构，对于容器X，X::value_type指容器中的值类型
  - 键可以是唯一的员工编号，对于容器X，X::key_type指容器中的键类型
- 关联容器的优点在于，它提供了对元素的快速访问
- 与序列相似，关联容器也允许插入新元素，但不能指定元素的插入位置
- STL提供了4种关联容器：set、multiset、map和multimap，分别在头文件set（set.h）和map（map.h）中定义

```
// setops.cpp -- some set operations                                    part1
#include <iostream>
#include <string>
#include <set>
#include <algorithm>
#include <iterator>
int main()
{
    using namespace std;
    const int N = 6;
    string s1[N] = {"buffoon", "thinkers", "for", "heavy", "can", "for"};
    string s2[N] = {"metal", "any", "food", "elegant", "deliver","for"};
    //利用array进行初始化set A,注意：区间、键是唯一的for只出现1次、且集合被排序
    set<string> A(s1, s1 + N);
    set<string> B(s2, s2 + N);

    ostream_iterator<string, char> out(cout, " ");
    cout << "Set A: ";
    copy(A.begin(), A.end(), out);
    cout << endl;
    cout << "Set B: ";
    copy(B.begin(), B.end(), out);    cout << endl;
```

```
Set A: buffoon can for heavy thinkers
Set B: any deliver elegant food for metal
```

```
// setops.cpp -- some set operations                                    part2
    cout << "Union of A and B:\n";
    set_union(A.begin(), A.end(), B.begin(), B.end(), out);
    cout << endl;

    cout << "Intersection of A and B:\n";
    set_intersection(A.begin(), A.end(), B.begin(), B.end(), out);
    cout << endl;

    cout << "Difference of A and B:\n";
    set_difference(A.begin(), A.end(), B.begin(), B.end(), out);
    cout << endl;

    set<string> C;
    cout << "Set C:\n";
    set_union(A.begin(), A.end(), B.begin(), B.end(),
        insert_iterator<set<string> >(C, C.begin()));
    copy(C.begin(), C.end(), out);
    cout << endl;
```

```
Set A: buffoon can for heavy thinkers
Set B: any deliver elegant food for metal
```

```
Union of A and B:
any buffoon can deliver elegant food for heavy metal thinkers
```

```
Intersection of A and B:
for
```

```
Difference of A and B:
buffoon can heavy thinkers
```

```
Set C:
any buffoon can deliver elegant food for heavy metal thinkers
```

```
// setops.cpp -- some set operations                                    part3

    string s3("grungy");
    C.insert(s3);
    cout << "Set C after insertion:\n";
    copy(C.begin(), C.end(),out);
    cout << endl;

    cout << "Showing a range:\n";
    copy(C.lower_bound("ghost"),C.upper_bound("spook"), out);
    cout << endl;

    return 0;
}
```

```
Union of A and B:
any buffoon can deliver elegant food for heavy metal thinkers
```

```
Set C after insertion:
any buffoon can deliver elegant food for grungy heavy metal thinkers
```

```
Showing a range:
grungy heavy metal
```

```
Set A: buffoon can for heavy thinkers
Set B: any deliver elegant food for metal
Union of A and B:
any buffoon can deliver elegant food for heavy metal thinkers
Intersection of A and B:
for
Difference of A and B:
buffoon can heavy thinkers
Set C:
any buffoon can deliver elegant food for heavy metal thinkers
Set C after insertion:
any buffoon can deliver elegant food for grungy heavy metal thinkers
Showing a range:
grungy heavy metal
```

```cpp
// multmap.cpp -- use a multimap                                    part1
#include <iostream>
#include <string>
#include <map>
#include <algorithm>

typedef int KeyType;
typedef std::pair<const KeyType, std::string> Pair;
typedef std::multimap<KeyType, std::string> MapCode;

int main()
{
    using namespace std;
    MapCode codes;

    codes.insert(Pair(415, "San Francisco"));
    codes.insert(Pair(510, "Oakland"));
    codes.insert(Pair(718, "Brooklyn"));
    codes.insert(Pair(718, "Staten Island"));
    codes.insert(Pair(415, "San Rafael"));
    codes.insert(Pair(510, "Berkeley"));
```

```
// multmap.cpp -- use a multimap                                    part2

    cout << "Number of cities with area code 415: "
         << codes.count(415) << endl;
    cout << "Number of cities with area code 718: "
         << codes.count(718) << endl;
    cout << "Number of cities with area code 510: "
         << codes.count(510) << endl;
    cout << "Area Code   City\n";
    MapCode::iterator it;
    for (it = codes.begin(); it != codes.end(); ++it)
        cout << "    " << (*it).first << "     "
         << (*it).second << endl;

    pair<MapCode::iterator, MapCode::iterator> range
        = codes.equal_range(718);
    cout << "Cities with area code 718:\n";
    for (it = range.first; it != range.second; ++it)
        cout << (*it).second << endl;
    return 0;
}
```

```
Number of cities with area code 415: 2
Number of cities with area code 718: 2
Number of cities with area code 510: 2
```

```
Area Code   City
    415     San Francisco
    415     San Rafael
    510     Oakland
    510     Berkeley
    718     Brooklyn
    718     Staten Island
```

```
Cities with area code 718:
Brooklyn
Staten Island
```

总结

- STL基本概念（熟悉）
- 使用迭代器（熟悉）
- 迭代器类型（了解）
- 迭代器结构（了解）
- 容器种类（了解）

第十二章 跨平台编译

主讲教师：同济大学计算机科学与技术学院 陈宇飞
同济大学计算机科学与技术学院 龚晓亮

目录

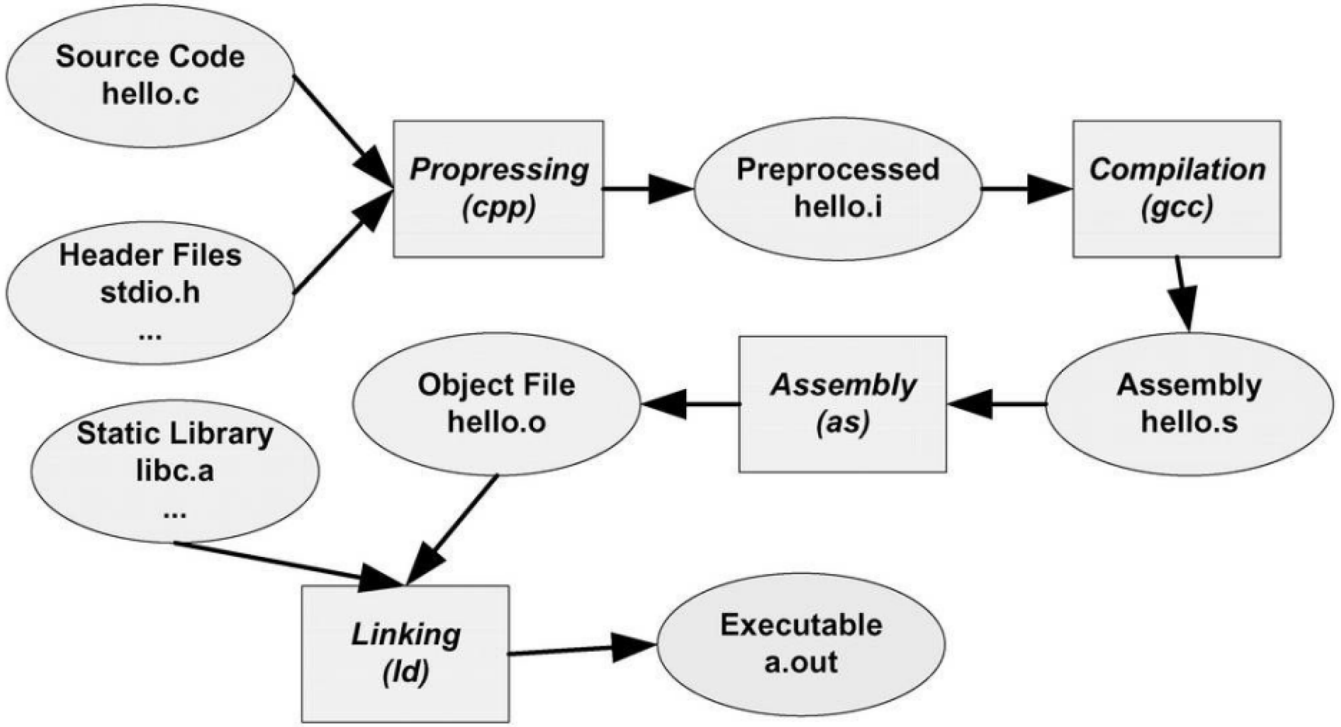
- 跨平台编译概述
- Cmake编译原理
- Makefile的使用

目录

- 跨平台编译概述
  - 需求背景
  - 基本概念

1.1 需求背景

- 编译器会将源代码翻译为目标机器的机器语言指令，链接器则把多个目标文件以及库文件组合起来形成完整的可执行文件。
- 不同操作系统底层架构和运行环境有很大差异，如二进制文件格式、系统调用接口、库文件实现等。因此，平台A（如Windows）编译出的可执行文件很可能无法直接在平台B（如Linux）运行。



1.1 需求背景

- 编程角度：
  - ① 避免使用平台特定的库函数和系统调用；
  - ② 对数据类型进行严格的定义和处理；
  - ③ 合理组织代码结构，隔离平台相关代码等等
- 编译角度：针对不同平台的特性，在编译过程中进行适当的配置和调整，例如使用不同的编译器选项、链接不同的库文件，从而让同一份源代码生成符合不同平台要求的可执行文件。（跨平台编译）

1.2 基本概念

- gcc：GNU编译器套件，相当于编译器，一般用于单文件的项目编译。
- makefile：一种自动化编译的脚本文件，定义了不同文件（如源文件、目标文件）之间的依赖关系，以及将源文件构建为目标文件的具体命令。
- make：一个命令行工具，读取 makefile 文件中的规则和命令，调用相应的编译器和链接器按照其中指定的命令进行编译和链接操作。
- Cmake：一个跨平台的构建系统生成工具。根据项目配置文件生成适合特定平台的构建文件，比如生成 make 工具使用的 makefile 文件。
- 简单来说，gcc 是编译的执行者， makefile 是编译规则的描述者， make 是根据 makefile 进行编译的调度者， Cmake 是更高级的构建文件生成者。

目录

- Cmake编译原理
  - 基本概念
  - 工作流程
  - 指令与变量
  - 控制结构
  - 综合案例

2.1 基本概念

- CMake本身是一个工具集，由五个可执行程序组成：cmake、ctest、cpack、cmake-gui和ccmake
  - cmake：核心工具，负责项目配置、生成构建文件并构建项目，产出可执行或库文件。
  - ctest：测试驱动程序，运行测试用例并报告结果。
  - cpack：打包程序，依据项目构建结果生成安装和源包。
  - cmake-gui：图形界面工具，方便用户设置。
  - ccmake：基于终端的图形界面，在终端以图形化交互修改构建配置。

- 官方网站：https://cmake.org/ （学会查文档）



2.2 工作流程

（1）配置阶段：编写配置文件

- 定义项目的基本信息，如项目名称、项目版本号等。
- 指定源文件和头文件的位置，可用add_executable()定义可执行文件。
- 定义目标文件之间的依赖关系和链接选项。如果项目使用了外部库，需要显式指定链接哪些库。

```
Msg.hpp + Msg.cpp
main.cpp
  → #include "Msg.hpp"
```

```
# CMakeLists.txt （CMake语法中，注释使用 # ）
# 设置最低 CMake 版本，低于 3.5 则报错
cmake_minimum_required(VERSION 3.5 FATAL_ERROR)
# 定义项目名为my_project，定义语言为 CXX (C++)
project(my_project LANGUAGES CXX)
# 从源文件生成静态库，名为 msg
add_library(msg STATIC Msg.hpp Msg.cpp)
# 创建可执行文件 hello-world
add_executable(hello-world main.cpp)
# 把 msg 库链接到 hello-world 可执行文件
target_link_libraries(hello-world message)
```

2.2 工作流程

（2）生成阶段：运行CMake命令

- CMake会读取CMakeLists.txt文件，并且检查系统环境，包括操作系统类型、编译器类型和版本等信息。（如，Linux可能检测到gcc编译器，Windows则可能检测到Visual C++编译器）
- 根据上述条件，CMake会生成适合该平台的构建文件。在Unix-like系统上，通常会生成Makefile文件；在Windows系统上，可能会生成Visual Studio项目文件（.vcxproj等）。

（3）构建阶段：使用生成的构建文件进行编译

- 在Unix-like系统上，可以使用make命令来进行实际的编译和链接操作。
- 在Windows系统上，生成的Visual Studio项目文件可以直接在Visual Studio集成开发环境中打开，然后使用Visual Studio的编译功能来构建项目。

2.3 指令与变量

- cmake_minimum_required指令 - 规定所需Cmake的最低版本。
  - 基本语法：cmake_minimum_required(VERSION <min_version>[...<policy_max>] [FATAL_ERROR])
  - ① <min_version>：项目所需的最低 CMake 版本号。版本号的格式通常为 major.minor.patch，例如3.10.0。
  - ② [...<policy_max>]：与<min_version>结合作为版本号范围。
  - ③ [FATAL_ERROR]：当添加了这个参数后，如果当前系统中的 CMake 版本低于指定的最低版本，CMake 会立即停止配置过程，并输出错误信息。
  - 例：cmake_minimum_required (VERSION 3.10.0...3.12.0 FATAL_ERROR)
- 注意，方括号代表可选参数，尖括号代表必填参数；小写变量名可以替换。

2.3 指令与变量

- project指令 - 提供项目名称，指定项目相关信息。
  - 基本语法：project(<PROJECT - NAME> [LANGUAGES <language - names>...]
    [VERSION <major>[.<minor>[.<patch>[.<tweak>]]]] [HOMEPAGE_URL <url>]
    [DESCRIPTION <project - description>])
  - ① <PROJECT - NAME>：指定项目的名称。
  - ② [LANGUAGES <language - names>...]：用于指定项目所使用的编程语言。如果不指定，CMake 会根据源文件的扩展名自动推断编程语言。
  - ③ [VERSION <major>[.<minor>[.<patch>[.<tweak>]]]]：用于指定项目的版本号。
  - ④ [HOMEPAGE_URL <url>]：可以用来指定项目的主页网址。
  - ⑤ [DESCRIPTION <project - description>]：用于提供项目的简单描述信息。
  - 例：project(my_project LANGUAGES C CXX VERSION 3.10.2)

2.3 指令与变量

- add_executable指令 - 定义如何从给定的源文件构建出一个可执行程序。
  - 基本语法：add_executable(<target_name> [WIN32] [MACOSX_BUNDLE] [EXCLUDE_FROM_ALL] [<source>...])

① <target_name>：指定生成的可执行文件目标的名称。

② [WIN32]：在 Windows 平台上构建项目并且希望生成一个带有 Windows 特定属性（如控制台应用程序或窗口应用程序）的可执行文件时添加。

③ [MACOSX_BUNDLE]：在Mac OS 平台上构建项目时，希望将可执行文件打包成 Mac 应用程序包（Bundle）的形式时添加。

④ [EXCLUDE_FROM_ALL]：当添加这个参数时，生成的可执行文件目标将不会被默认构建。

⑤ [<source>...]：可变长度，用于指定构建可执行文件所需的源文件。

  - 例：add_executable(MyApp main.cpp helper.cpp)

2.3 指令与变量

- set指令 - 创建变量
  - 基本语法：set(<variable> <value>... [PARENT_SCOPE])。
  - ① <variable> <value>：变量名和变量值。如果不给变量值，则会销毁变量。如果给多个值，相当于创建了一个列表变量。
  - [PARENT_SCOPE]：如果指定，则变量会在上一级作用域创建。（CMake的作用域概念与C++中变量的作用域相似）
- 使用变量
  - 可以通过 ${<variable_name>} 的语法形式进行引用。
  - 还可以在源文件中通过配置头文件的方式进行使用。

2.3 指令与变量

- 使用变量
  - 还可以在源文件中通过配置头文件的方式进行使用。
  - ① 在CMakeLists.txt中使用configure_file指令创建一个配置头文件。
  - ② 在MyProjectConfig.h.in文件中使用 @<variable_name>@ 的语法形式引用变量。
  - ③ 在源文件中包含这个配置头文件，即可以使用其中定义的变量值。

- 变量名区分大小写并可以包含字符。虽然使用括号和引号参数允许在变量名中包含空格，但以后引用时，必须使用反斜杠来转义空格；因此，建议在变量名中只使用字母、数字字符、减号和下划线。

```
set(MyString1 "Text1")
set([My String2] "Text2")
set("My String 3" "Text3")

message(${MyString1})
message(${My\ String2})
message(${My\ String\ 3})
```

2.3 指令与变量

- message指令 - 打印信息。
  - 基本语法：message([<mode>] "message text" ...)。
  - ① <mode>：用于指定消息的输出模式，可能的取值有STATUS、WARNING、AUTHOR_WARNING、SEND_ERROR、FATAL_ERROR等。
  - ② “message text”：用于指定要输出的消息内容。消息内容可以是一个简单的字符串，也可以包含 CMake 变量，通过${}语法来引用变量的值。
  - 例：message(STATUS "Project source directory: ${PROJECT_SOURCE_DIR}")

```
# CMakeLists.txt
set(MyString "What a nice day!")
message(${MyString})
```

2.3 指令与变量

- configure_file指令 - 将一个输入文件复制到一个输出文件，并进行变量替换。
  - 通常用于将 CMake 中的变量值传递到源文件中，从而让源文件获取构建时的相关信息，如项目版本号、编译选项等。
  - 基本语法：configure_file(<input> <output> [COPYONLY] [ESCAPE_QUOTES] [@ONLY])

① <input>：输入文件的路径。这个文件通常是一个模板文件，其中包含了一些特殊的变量占位符，如 @<variable_name>@。

② <output>：输出文件的路径，即经过变量替换后的最终文件路径。

③ [COPYONLY]：如果指定了这个参数，那么 CMake 只会简单地复制文件，而不会进行任何变量替换操作。

④ [ESCAPE_QUOTES]：用于指定在变量替换过程中是否对引号进行转义。

⑤ [@ONLY]：用于指定只替换格式为@<variable_name>@的变量，忽略其他可能的变量语法。

2.3 指令与变量

```
# ① CMakeLists.txt
cmake_minimum_required(VERSION 3.5 FATAL_ERROR)
set(MY_PROJECT_VERSION "1.0.0")
project(MyProject VERSION ${MY_PROJECT_VERSION})
configure_file(MyProjectConfig.h.in MyProjectConfig.h)
```

```
// ② MyProjectConfig.h.in
#define MY_PROJECT_VERSION_STRING "@MY_PROJECT_VERSION@"
```

```
// ③ main.cpp
#include "MyProjectConfig.h"
#include <iostream>
int main()
{
    std::cout << "Version: " << MY_PROJECT_VERSION_STRING << std::endl;
    return 0;
}
```

2.3 指令与变量

- add_library指令 - 使用指定的一个或多个文件为项目添加一个库。
  - 基本语法：add_library(<name> [<type>] [EXCLUDE_FROM_ALL] <sources>...)
  - ① <name>：指定库的名字。
  - ② [<type>]：指定库的类型，包括STATIC（静态库），SHARED（动态库）和MODULE（模块库）。如果不指定该参数，默认库的类型为STATIC；如果设置了BUILD_SHARED_LIBS变量，则默认为SHARED。
  - ③ [EXCLUDE_FROM_ALL]：当添加这个参数时，生成的库目标将不会被默认构建。
  - ④ <sources>...：可变长度，用于指定构建库所需的源文件。
  - 生成的library名会根据TYPE属性（STATIC或SHARED）转换为name.a或name.lib
  - 例：add_library(MyLibrary STATIC source1.cpp source2.cpp)

2.3 指令与变量

- target_link_libraries指令 - 将库文件与目标文件进行链接。
  - 基本语法：target_link_libraries(<target> <item>...)。（还有其他很多签名）
  - ① <target>：用于指定要链接库的目标。这个目标可以是通过add_executable创建的可执行文件目标，也可以是通过add_library创建的库目标。
  - ② <item>...：可变长度，用于指定要链接的库。可以是库名称（包括自定义的库和系统提供的库）或全路径库文件，后者不建议使用。
  - 例：target_link_libraries(MyExecutable MyLib)

2.3 指令与变量

- target_link_libraries指令 - 将库文件与目标文件进行链接。
  - 例1：一个图形用户界面（GUI）程序可能依赖于图形库（如 Qt 库）和操作系统的窗口管理库（如在 Windows 上的user32.dll）。

```
# 将GUIApp可执行文件与Qt5::Widgets库（假设已经正确配置了 Qt 库）和user32系统库进行链接
add_executable(GUIApp main.cpp)
target_link_libraries(GUIApp Qt5::Widgets user32)
```

  - 例2：高级库HighLevelLib依赖于一个基础库BaseLib.

```
add_library(BaseLib STATIC base_source.cpp)
add_library(HighLevelLib STATIC highlevel_source.cpp)
target_link_libraries(HighLevelLib BaseLib)
```

2.3 指令与变量

- set_target_properties指令 - 更改target属性。
  - 基本语法：set_target_properties(<target> PROPERTIES <property> <value> [<property> <value>]...)
  - ① <target>：用于指定要设置属性的target名称。target必须是在 CMakeLists.txt 文件中通过add_executable或add_library指令定义的目标。
  - ② <property> <value>：一组属性 - 值对。
  - 例：set_target_properties(MyExecutable PROPERTIES RUNTIME_OUTPUT_DIRECTORY ${PROJECT_BINARY_DIR}/bin)将可执行文件MyExecutable的运行时输出目录设置为${PROJECT_BINARY_DIR}/bin。

2.3 指令与变量

- target_include_directories指令 - 指定target的头文件搜索路径。
  - 基本语法：target_include_directories(<target> [SYSTEM] [BEFORE] <INTERFACE|PUBLIC|PRIVATE> [items1...] [<INTERFACE|PUBLIC|PRIVATE> [items2...]...])

① <target>：用于指定要设置头文件搜索路径的target。

② [SYSTEM]：用于指定头文件路径为系统头文件路径。

③ [BEFORE]：用于控制添加的头文件搜索路径的优先级。如果使用了BEFORE，添加的路径会被放在头文件搜索路径列表的前面。

④ <INTERFACE|PUBLIC|PRIVATE>：用于指定头文件路径的作用范围。包括

- PRIVATE 仅用于目标本身的编译，不会传递给链接这个目标的其他目标。
- INTERFACE 作为接口，当其他目标链接这个库时，可以使用这些路径来搜索头文件。
- PUBLIC 包含了PRIVATE和INTERFACE的功能。既可用于本身，也可被其他库使用。

2.3 指令与变量

- target_include_directories指令 - 指定target的头文件搜索路径。
  - 基本语法：target_include_directories(<target> [SYSTEM] [BEFORE] <INTERFACE|PUBLIC|PRIVATE> [items1...] [<INTERFACE|PUBLIC|PRIVATE> [items2...]...])

⑤ <items1...>：可变长度，用于指定头文件搜索路径。

- 例如，有一个数学库MathLib，头文件存放在${PROJECT_SOURCE_DIR}/math_include目录下，可以这样设置：

```
add_library(MathLib STATIC math_sources.cpp)
target_include_directories(MathLib PUBLIC ${PROJECT_SOURCE_DIR}/math_include)
```

当其他目标（如可执行文件或者其他库）链接MathLib时，就能正确找到数学库的头文件来进行编译。

2.3 指令与变量

- file指令 - 对文件和目录进行各种操作。
  - 读取文件内容到变量：file(READ <filename> <variable> [LIMIT <maxbytes>] [OFFSET <startbytes>] [HEX])。

① <filename>：要读取的文件的名称或路径。

② <variable>：用于存储读取内容的变量。

③ [LIMIT <maxbytes>]：用于限制读取的字节数。

④ [OFFSET <startbytes>]：用于指定从文件的哪个字节位置开始读取。

⑤ [HEX]：如果指定这个参数，文件内容将以十六进制形式读取。

  - 例如，file(READ ${PROJECT_SOURCE_DIR}/config.ini CONFIG_CONTENT)，尝试读取${PROJECT_SOURCE_DIR}/config.ini文件的内容到CONFIG_CONTENT变量中。

2.3 指令与变量

- file指令 – 对文件和目录进行各种操作。
  - 复制文件：file(COPY <files>... DESTINATION <dir> [FILE_PERMISSIONS <permissions>...] [DIRECTORY_PERMISSIONS <permissions>...] [NO_SOURCE_PERMISSIONS] [USE_SOURCE_PERMISSIONS])。

① <files>...：可变长度，用于指定要复制的文件。

② <dir>：目标目录的路径，即文件要复制到的位置。

③ [FILE_PERMISSIONS <permissions>...]：用于指定复制后的文件权限。

④ [DIRECTORY_PERMISSIONS <permissions>...]：用于指定复制后的目录权限。

⑤ [NO_SOURCE_PERMISSIONS]和[USE_SOURCE_PERMISSIONS]：控制是否使用源文件权限。

  - 例：file(COPY ${PROJECT_SOURCE_DIR}/src/file1.cpp ${PROJECT_SOURCE_DIR}/src/file2.cpp DESTINATION ${PROJECT_BINARY_DIR}/src)

2.3 指令与变量

- PROJECT_SOURCE_DIR 与 PROJECT_BINARY_DIR
  - 分别用于指定项目源代码根目录和CMake构建时的输出目录。
  - CMake自动生成，可更改。
- CMAKE_CXX_COMPILER 与 CMAKE_C_COMPILER
  - 分别用于指定 C++ 和 C 编译器的路径。
  - 在不同的操作系统环境下，CMake 会依据系统配置自动检测并填充相应的编译器路径。例如，在常见的 Linux 系统中，CMAKE_CXX_COMPILER 通常指向 g++ 编译器的路径，而在 Windows 系统中，若安装了 Visual C++ 编译器，则会指向其对应的可执行文件路径。
  - 在项目构建过程中，CMake 会依据这些变量所指定的编译器来执行源文件的编译操作，确保代码能够正确转换为目标平台可执行的机器码。

2.3 指令与变量

- CMAKE_CXX_FLAGS 与 CMAKE_C_FLAGS
  - 主要用于存储传递给 C++ 和 C 编译器的各类编译选项，例如优化级别、警告处理以及调试信息生成等。
  - 常见的编译选项包括
    ① -O2 或 -O3 ：用于指定优化级别，以提升生成代码的执行效率；
    ② -Wall 用于开启所有警告信息，帮助开发者发现潜在的代码问题；
    ③ -g 用于生成调试信息，以便在调试阶段能够更方便地进行代码调试和追踪。
- CMAKE_SYSTEM_NAME
  - 用于存储当前构建平台的名称，可能的取值包括Linux、Windows、Darwin（Mac OS）等。
  - 可以依据该变量的值，编写条件判断语句，针对不同的平台执行特定的构建配置操作。

2.4 控制结构

- 条件块 - 语法格式
  - 必须以if()开头并以endif()结尾
  - 在开头和结尾之间可以添加任意数量的elseif()，但只能有单独一个的、可选的else()
- 条件表达式的类型
  - 可以使用STREQUAL（字符串相等比较）、LESS（小于）、GREATER（大于）等操作符来比较变量的值。例如，if(${CMAKE_VERSION} GREATER 3.10)用于判断 CMake 版本是否大于 3.10。 也可以使用逻辑操作符组合多个比较条件，如AND、OR、NOT。
  - 可以使用DEFINED关键字来检查变量是否已经定义。例如，if(DEFINED MY_VARIABLE)用于判断MY_VARIABLE是否已经在之前的 CMake 脚本中定义；也可以使用EXISTS关键字来检查文件或目录是否存在。例如，if(EXISTS ${PROJECT_SOURCE_DIR}/config.ini)用于判断config.ini文件是否存在于项目源目录中。

2.4 控制结构

- while循环
  - 必须以while()开头创建并以endwhile()结尾。

```
while(<condition>)
<commands>
endwhile()
```

```
set(COUNT 0)
while(${COUNT} LESS 10)
    message(STATUS "Count: ${COUNT}")
    math(EXPR COUNT "${COUNT}+1")
endwhile()
```

- foreach循环
  - 类似C/C++的for循环风格控制，首尾分别是foreach()和endforeach()。

```
foreach(<loop_var> RANGE <min> <max> [<step>])
<commands>
endforeach()
```

  - <min>和<step>参数变量可选择配置，默认从0开始。min和max都必须是非负整数。
  - foreach()处理列表变量时十分便捷，在CMake中是更加常用和简洁的循环结构块。

2.4 控制结构

- foreach循环
  - foreach() 处理列表变量时，可以在LISTS后添加若干ITEMS一起遍历，也可以不写LISTS，直接写若干ITEMS用于遍历。

```
foreach(<loop_variable> IN [LISTS <lists>] [ITEMS <items>])
<commands>
endforeach()
```

```
set(SOURCE_FILES main.cpp helper.cpp utility.cpp)
foreach(file IN ${SOURCE_FILES})
    message(STATUS "Processing file: ${file}")
endforeach()
```

- 定义指令：包括宏定义指令 macro() 和函数定义指令 function() 。（结合CMake变量作用域相关知识自行学习）

2.5 综合案例

- 模拟一个简单的图形绘制项目，在不同操作系统下有不同的依赖库，并且会根据源文件目录下的多个源文件来构建可执行文件和库文件，同时通过配置文件传递项目版本信息到代码中

```
GraphicsProject/
├── CMakeLists.txt
├── src
│   ├── main.cpp
│   ├── draw_functions.cpp
│   ├── draw_functions.h
│   └── utils.cpp
├── include
│   └── common_utils.h
└── config
    └── project_config.h.in
```

2.5 综合案例

```
# 指定项目所需的最低CMake版本，若版本过低则报错并停止配置
cmake_minimum_required(VERSION 3.15 FATAL_ERROR)

# 定义项目名称、版本号，并指定使用的编程语言为C++
project(GraphicsProject VERSION 1.0 LANGUAGES CXX)

# 定义源文件列表变量，先初始化为空
set(ALL_SOURCE_FILES "")
set(SOURCE_FILES_FOR_EXECUTABLE "")
set(SOURCE_FILES_FOR_LIBRARY "")

# 使用file命令获取src目录下所有的.cpp源文件，将其存入ALL_SOURCE_FILES变量
file(GLOB_RECURSE ALL_SOURCE_FILES ${PROJECT_SOURCE_DIR}/src/*.cpp)
```

2.5 综合案例

```
# 循环遍历所有源文件，区分出用于构建可执行文件和库文件的源文件
foreach(file ${ALL_SOURCE_FILES})
    if(${file} MATCHES "main.cpp")
        list(APPEND SOURCE_FILES_FOR_EXECUTABLE ${file})
    else()
        list(APPEND SOURCE_FILES_FOR_LIBRARY ${file})
    endif()
endforeach()
```

2.5 综合案例

```
# 根据不同操作系统进行条件判断，添加特定的配置（这里以链接不同系统库为例）
if(${CMAKE_SYSTEM_NAME} STREQUAL "Linux")
    message(STATUS "Building on Linux system")
    set(SYSTEM_LIBRARY_FOR_DRAWING "X11")
    set(SYSTEM_LIBRARY_FOR_UTILS "pthread")
elseif(${CMAKE_SYSTEM_NAME} STREQUAL "Windows")
    message(STATUS "Building on Windows system")
    set(SYSTEM_LIBRARY_FOR_DRAWING "gdi32")
    set(SYSTEM_LIBRARY_FOR_UTILS "ws2_32")
else()
    message(STATUS "Building on other system")
    set(SYSTEM_LIBRARY_FOR_DRAWING "")
    set(SYSTEM_LIBRARY_FOR_UTILS "")
endif()
```

2.5 综合案例

```
# 创建一个名为DrawLib的静态库，使用相应的源文件构建
add_library(DrawLib STATIC ${SOURCE_FILES_FOR_LIBRARY})

# 将include目录添加到DrawLib库的头文件搜索路径，便于找到头文件
target_include_directories(DrawLib PUBLIC ${PROJECT_SOURCE_DIR}/include)

# 创建可执行文件GraphicsProject，使用之前定义好的源文件列表构建
add_executable(GraphicsProject ${SOURCE_FILES_FOR_EXECUTABLE})

# 将DrawLib库与GraphicsProject可执行文件进行链接
target_link_libraries(GraphicsProject DrawLib)
```

2.5 综合案例

```
# 根据操作系统链接对应的特定系统库（如果有）到可执行文件
if(SYSTEM_LIBRARY_FOR_DRAWING)
    target_link_libraries(GraphicsProject ${SYSTEM_LIBRARY_FOR_DRAWING})
endif()
if(SYSTEM_LIBRARY_FOR_UTILS)
    target_link_libraries(GraphicsProject ${SYSTEM_LIBRARY_FOR_UTILS})
endif()

# 设置可执行文件的输出目录属性
set_target_properties(GraphicsProject PROPERTIES RUNTIME_OUTPUT_DIRECTORY
${PROJECT_BINARY_DIR}/bin)
```

2.5 综合案例

```
# 使用configure_file指令生成配置文件，将项目版本号传递到代码中
configure_file(${PROJECT_SOURCE_DIR}/config/project_config.h.in
${PROJECT_BINARY_DIR}/config/project_config.h)

# 输出项目源文件目录和构建目录的路径信息，方便查看构建相关情况
message(STATUS "Project source directory: ${PROJECT_SOURCE_DIR}")
message(STATUS "Project binary directory: ${PROJECT_BINARY_DIR}")
```

目录

- Makefile的使用
  - 基本概念
  - 常用命令与选项
  - 变量
  - 通配符
  - 伪目标
  - 基本结构
  - 综合案例

3.1 基本概念

- 基本结构：主要由一系列规则构成，格式见右侧

```
target: dependencies
	command
```

① target：目标，可以是需要生成的文件名或自定义的操作名。

② dependencies：生成目标所依赖的文件或其他目标，可多个，也可不填。

③ command：构建目标的命令，编写当前操作系统支持的命令。必须以 Tab 键开头，可多个，也可不填（.all，见后续）。

➢ 如果存在相同的 target ，会合并依赖或报出警告：

```
target: depend1
target: depend2
# 两行会被合并为target1: depend1,depend2
	command2
```

```
target: depend1
	command1
target: depend2 # 多个目标无法合并
	command2 # 仅保留最后的目标
```

- 例：

```
my_target: # 下面的指令没有用到外部文件，因此此处不需要写依赖
	echo "This is my target" # Linux命令，用于输出
```

3.1 基本概念

- 工作原理（如何执行一个规则）：

① 检查目录下是否有和目标重名的文件，如果有，退出执行。

② 检查否有依赖项。如果有，先确保对应的依赖文件是否存在。

- 如果文件不存在，先在 Makefile 文件中寻找生成该文件的规则。
  - 如果能找到，先执行这些规则（递归），生成依赖文件；
  - 如果不能找到，退出执行。

③ 检查其时间戳是否比目标新，若存在时间戳更新的依赖文件，则执行指令。

3.2 常用命令与选项

- make：选择 Makefile中第一个目标执行。
- make [target]：选择 Makefile 中名为target的目标执行。
  - 命令行输入 make
    - 输出 This is the first target
  - 命令行输入 make second_target
    - 输出 This is the second target

```
# Makefile

first_target:
    echo "This is the first target"
second_target:
    echo "This is the second target"
```

- 注意，本节所有命令均在命令行输入，而非编写在 Makefile 中。
  - 拓展：在文件压缩大作业中我们尝试过在命令行运行exe并传参。然而为什么直接在命令行输入 make 系列命令，计算机也能找到并调用 exe 程序？（考虑环境变量）

3.2 常用命令与选项

- -n：只打印出执行构建任务时将要执行的命令，而不会真正执行。
  - 也被称为 “dry - run”（空运行）。
  - 一般用于检查 Makefile 中的规则是否正确编写。比如，在修改了 Makefile 中的编译规则后，可以先用 make -n 查看即将执行的命令是否符合预期，再实际执行构建。
- -p：会输出 Makefile 中的所有规则，包括默认规则和变量定义等信息。
- -s：在执行构建任务时，它会抑制命令的输出。
  - 指令被执行会默认输出指令内容（类似于回显），此时可以使用 make -s 抑制所有的输出，也可以针对特定不想要输出的指令，在行首添加@字符。

3.2 常用命令与选项

- -f [filename]：指定一个非标准名称的 Makefile 文件。
  - 在一些复杂的项目中，可能会有多个不同名称的构建脚本，使用这个命令就可以指定特定的构建脚本。
  - 例如，对于名为 build.mk 的构建文件，可以使用 make -f build.mk 执行构建规则。
- -C [directory]：用于改变工作目录到指定的目录后再读取 Makefile。
  - 例如，项目的源代码在 src 目录，而 Makefile 在项目根目录，通过 make -C src 可以在 src 目录下执行构建操作。
- -j [n]：用于开启并行构建。
  - n 表示并行任务的数量。例如，make -j4 会尝试同时执行 4 个构建任务，前提是这些任务之间没有依赖关系或者依赖关系已经满足。

3.3 变量

- 定义变量的格式通常是 变量名 =变量值。例如，CC = gcc 定义了一个名为 CC 的变量，其值为 gcc。
- 引用变量时，需要使用 $(变量名) 的格式。（注意，CMake是 ${} 格式）
- 变量赋值方式分为简单赋值和延迟赋值。
  - 简单赋值（:=）：当定义变量时，变量的值就被固定下来。
  - 延迟赋值（=）：变量的值在每次引用时根据当前的其他变量值重新计算。例如，“VAR1 = $(VAR2)”，如果 VAR2 的值在后面被修改，那么 VAR1 的值也会随之改变。

```
CC = g++
CFLAGS = -Wall -g

main.o: main.cpp
	$(CC) $(CFLAGS) -c main.cpp -o main.o # gcc编译相关指令，自行学习
```

3.3 变量

- $@：代表规则中的目标 target。
- $<：代表规则中的第一个依赖文件。
- $^：代表规则中所有的依赖文件。
- $?：代表规则中更新过的文件。
- 例：

```
mylib.so: obj1.o obj2.o obj3.o
        gcc -shared -o $@ $^
```

  - gcc -shared -o $@ $^ 会展开为 gcc -shared -o mylib.so obj1.o obj2.o obj3.o，调用gcc编译程序并传递这些参数，将这些目标文件构建成动态库 mylib.so。

3.4 通配符

- %：匹配任意字符串（包括空串）。在规则定义和函数操作中用于有规律的文件名转换和规则匹配，特别是在编译过程中为多个相似文件定义统一规则。
- *：匹配任意字符串（包括空串）。通常用于快速获取一组符合某种后缀或前缀等简单模式的文件列表，用于变量赋值或作为其他命令的参数。
- ?：匹配任意单个字符。%和*一般可以满足需求，这种通配符相对不常用。
- 通配符*只进行匹配，无法表达转换关系。例如，%.o：%.c 这种规则用于建立目标文件（.o）和源文件（.c）之间基于文件名的一对一匹配关系。如果写成 *.o：*.c ，Makefile将无法正确理解这种意图。因为*只是简单地匹配所有符合某种模式的文件，没有基于文件名部分相同的目标 - 依赖匹配机制。

3.5 伪目标

- （3.1节）工作原理（如何执行一个规则）：

  ① 检查目录下是否有和目标重名的文件，如果有，退出执行。 …

- Q1：如果一次执行生成后，想重新执行，需要手动删除产生的文件？

  - A1：可以编写规则，专门用于删除产生的文件。运行规则自动删除。

```
clean: # 只需要在命令行 make clean ，就可以删除相关文件
    rm -rf build/ # Linux命令，删除 build 目录及其内部所有文件
    rm -f *.o myapp # Linux命令，删除 myapp 可执行文件以及所有.o文件
# make存在一个内嵌隐含变量“RM = rm  -f”。
# 因此在书写clean规则的命令时可以使用变量$(RM)来代替rm。
```

- Q2：如果当前目录下有一个名为 clean 的文件，如何执行 clean ？

  - A2：可以使用伪目标（.PHONY）绕过 make 的文件重名检测。

3.5 伪目标

- 伪目标（phony targets）并不是实际要生成的文件名，而是用来执行一些命令或作为依赖关系的标签。设置了伪目标后， make 不会将对应的目标视作将要生成的文件，从而不进行文件是否存在的检测。
- 使用 .PHONY 关键字声明伪目标：.PHONY: 伪目标1 伪目标2...。
- 例：

```
.PHONY: cleanall cleanobj cleandiff

cleanall: cleanobj cleandiff
    rm program
cleanobj:
    rm *.o
cleandiff :
    rm *.diff
```

3.5 伪目标

- 通常会按照下面的说明进行相关的定义，并使用伪目标进行声明：

1. clean：通常用于清理项目构建过程中生成的文件。
   - 如目标文件（.o文件）、可执行文件、中间文件等。
2. all：通常作为总目标。
   - 依赖于其他实际的目标或者伪目标，用于构建整个项目。

```
all: myapp mylib # make all 会尝试生成所有依赖，即执行对应的命令
myapp: main.o util.o
    gcc -o myapp main.o util.o
mylib: file1.o file2.o
    ar -rcs mylib.a file1.o file2.o
.PHONY: all
```

3. test：用于执行项目的测试任务。

3.6 基本结构

- 循环结构 - foreach 函数
  - 基本语法：$(foreach var, list, text)，其中var是循环变量，list是值的列表，text是要对每个var的值执行的文本内容，通常包含对var的引用。
  - 例如，假设要将一组.c文件编译为.o文件，并且这些.c文件的文件名存储在变量SOURCE_FILES中，可以这样使用foreach函数：

```
SOURCE_FILES = file1.c file2.c file3.c
OBJECT_FILES = $(foreach src,$(SOURCE_FILES),$(patsubst %.c,%.o,$(src)))

all: $(OBJECT_FILES)
    gcc -o myapp $(OBJECT_FILES)
```

  - src会依次取SOURCE_FILES中的每个值（即每个.c文件名），然后$(patsubst %.c,%.o,$(src))会将每个.c文件名转换为对应的.o文件名，最终OBJECT_FILES变量就包含了所有转换后的.o文件名。

3.6 基本结构

- 条件选择结构 - ifeq 和 ifneq
  - 基本语法：ifeq (arg1,arg2)、ifneq (arg1,arg2)，跟着在条件满足时执行的命令块。
  - 例如，假设有一个变量BUILD_TYPE用于指定构建类型是debug还是release，可以根据不同的构建类型设置不同的编译选项：

```
CFLAGS = ...
BUILD_TYPE = ...

ifeq ($(BUILD_TYPE),debug)
    CFLAGS += -g
else
    CFLAGS += -O2
endif
```

3.7 综合案例

```
CC = gcc
SOURCE_FILES = main.c util.c math.c

# 根据源文件列表生成对应的目标文件列表
OBJECT_FILES = $(foreach src,$(SOURCE_FILES),$(patsubst %.c,%.o,$(src)))

# 定义构建类型变量
BUILD_TYPE = debug

# 根据不同构建类型设置不同的编译选项
ifeq ($(BUILD_TYPE),debug)
    CFLAGS += -g -Wall
else
    CFLAGS += -O2
endif

all: myapp
```

3.7 综合案例

```
# 编译规则，将所有目标文件链接成可执行文件
myapp: $(OBJECT_FILES)
    $(CC) $(CFLAGS) -o $@ $^

# 模式规则，用于将.c文件编译为.o文件
%.o: %.c
    $(CC) $(CFLAGS) -c $< -o $@

.PHONY: clean
clean:
    rm -f $(OBJECT_FILES) myapp

# 伪目标test，用于执行简单的测试
.PHONY: test
test: myapp
   ./myapp
```

总结

- 跨平台编译概述（了解）
- Cmake编译原理（了解）
- Makefile的使用（了解）